

DEVELOPER GUIDE

CloudLens

Complete Developer Documentation

React · Node.js · Express · MongoDB · AWS SDK v3

191

source files documented

9

database collections

4

scanned AWS services

Generated August 1, 2026



Table of Contents

1. Part 1 — Project Overview

2. Part 2 & 3 — Every File & Function, Explained

- Core Scan & Recommendation Engine
- Application Bootstrap
- API Layer — Controllers & Routes
- Data Layer — Repositories, Mappers, Pricing
- Cross-Cutting Services
- Infrastructure — Utils, Middleware, Config
- Frontend Pages
- Frontend Components (A–M)
- Frontend Components (N–Z)
- Resource Display Components
- Frontend Services, Hooks & Utils

3. Part 4 — Architecture Diagram

4. Part 5 — Scan Flow

5. Part 6 — Recommendation Engine Flow

6. Part 7 — API Flow

7. Part 8 — Database Documentation

8. Part 9 — How to Explain This Project (Interview Notes)

9. Part 10 — Code Execution: Run Scan Walkthrough

10. Part 11 — Folder Structure

PART 1



Project Overview

What CloudLens is, why it exists, the full tech stack, and how requests/scans/recommendations/the dashboard flow end to end.

Part 1 — Project Overview

What CloudLens Is

CloudLens is a full-stack MERN (MongoDB, Express, React, Node) SaaS dashboard that connects to a user's AWS account (via a set of IAM access keys the user provides), scans it across four services — **EC2, RDS, S3, and IAM** — and produces two things for every resource it finds:

1. **Cost-optimization recommendations** — idle/underutilized EC2 or RDS instances that could be stopped or downsized, S3 buckets with no lifecycle policy quietly accumulating storage cost, gp2 volumes that should be gp3, etc.
2. **Security findings** — publicly accessible S3 buckets or RDS instances, unencrypted EBS volumes or S3 buckets, IAM users with `AdministratorAccess` or console access with no MFA, wildcard IAM policies, stale/unused access keys and roles, and more.

Every finding is priced (using AWS's live Price List API where possible, with sane fallback rates where it isn't) and shown on a dashboard with severity levels, category tags, and one-click "Resolve" / "Ignore" tracking — plus a "Stop Instance" action directly from the app for the one finding type where the app can take a real remediation action itself.

Why It Was Built

CloudLens exists to answer a question every AWS account eventually needs answered: *"What am I actually paying for, is any of it wasted, and is any of it dangerously misconfigured?"* AWS's own tools that answer this (Cost Explorer, Trusted Advisor, Security Hub) are either paywalled behind support tiers, spread across multiple separate consoles, or require significant AWS expertise to interpret. CloudLens consolidates a focused subset of that value — the specific checks that most commonly cost people money or expose them to risk — into one connected, opinionated dashboard that a developer can stand up against their own account in minutes, run on a schedule, and get emailed about.

Technologies Used — Complete Tech Stack

BACKEND (`SERVER/`)

LAYER	TECHNOLOGY	VERSION	PURPOSE
Runtime	Node.js (CommonJS)	—	<code>"type": "commonjs"</code> — every backend file uses <code>require</code> / <code>module.exports</code> , not ESM
Web framework	Express	^5.2.1	REST API
Database	MongoDB via Mongoose	^9.7.4	Document store, 9 collections
Auth	jsonwebtoken + bcrypt	^9.0.3 / ^6.0.0	JWT issuance/verification, password hashing
AWS SDK	<code>@aws-sdk/client-ec2,rds,s3,iam,cloudwatch,pricing,sts</code> v3	^3.108x	Every AWS API call in the app
Scheduling	node-cron	^4.6.0	60-second tick driving scheduled scans
Email	Nodemailer	^9.0.3	Scan-completed / high-severity / weekly-digest emails (SMTP, with an auto-provisioned Ethereal test inbox when no SMTP env vars are set)
PDF generation	pdfkit	^0.19.1	Scan report export
Rate limiting	express-rate-limit	^8.6.1	
CORS	cors	^2.8.6	Restricted via a <code>CORS_ORIGIN</code> env-driven allowlist
Env config	dotenv	^17.4.2	
Dev tooling	nodemon	^3.1.14	Auto-restart in development

FRONTEND (CLIENT/)

LAYER	TECHNOLOGY	VERSION	PURPOSE
Framework	React	^19.2.7	
Build tool	Vite	^8.1.1	Dev server + production bundling
Routing	react-router-dom	^7.18.1	Client-side routing, ProtectedRoute -gated
HTTP client	Axios	^1.18.1	Wrapped in one shared instance (api.js) with a JWT-attaching interceptor
Styling	Tailwind CSS (@tailwindcss/vite)	^4.3.3	Utility-first CSS, no separate CSS files per component
Charts	Recharts	^3.9.2	Every chart on the Dashboard/Analytics pages
Animation	Framer Motion	^12.42.2	Page transitions, staggered list/row entrances, drawer slide-in
Icons	react-icons	^5.7.0	(Font Awesome 6 icon set, react-icons/fa6)
Toasts	react-hot-toast	^2.6.0	Wrapped in a shared toast.jsx utility + AppToaster component
Loading placeholders	react-loading-skeleton	^3.5.0	
Linting	oxlint	^1.71.0	

INFRASTRUCTURE / EXTERNAL SERVICES

- **MongoDB Atlas** — the app connects to a cloud-hosted MongoDB cluster (not a local instance) via MONGO_URI .
- **AWS** — the account being scanned; CloudLens never touches AWS resources it manages itself (no CloudFormation/Terraform), it only calls read (and one write: StopInstances) APIs against the connected account.
- **SMTP** — Gmail SMTP in this deployment (smtp.gmail.com:587), or Ethereal (a fake SMTP sandbox that generates preview URLs instead of really delivering mail) automatically in development when no SMTP credentials are configured.

Folder Structure (top level)

CloudLens/

- └ client/ React frontend (Vite)
- └ server/ Express backend (Node/MongoDB)
- └ docs/ Generated developer documentation (this guide)

See Part 11 for the full recursive folder tree with per-folder explanations.

Backend Architecture

The backend follows a strict layered architecture, and — with the sole exception of server.js / app.js / db.js config — no layer skips the one below it:

routes/ → thin Express Router definitions: path + middleware + controller, no logic
↓
controllers/ → parses req, calls exactly one service function, shapes the HTTP response
↓
services/ → business logic, organized by concern (scanner/, recommender/, analyzer/,
 dashboard/, auth/, scheduler/, notification/, report/, email/, ...)
↓
services/database/ → repositories – the ONLY layer that touches Mongoose models directly
↓
models/ → Mongoose schemas (9 collections)

Cutting across this vertical stack are three horizontal layers specific to the scanning pipeline:

- **services/aws/** — thin AWS SDK v3 wrappers (one file per AWS service), never contains analysis logic
- **mappers/** — normalize AWS's raw, deeply-nested response shapes into flat objects

- `services/analyzer/` and `services/recommender/` — pure functions turning normalized data into findings, then findings into recommendations (see Part 6 for the full pipeline diagram)

Every mutating or expensive endpoint runs behind the `protect` auth middleware (`middleware/auth.middleware.js`), which verifies the JWT and additionally checks the token's embedded `tokenVersion` against the user's current one (supporting "log out of all devices" without a server-side token blacklist). Every controller is wrapped in `asyncHandler` so a rejected promise reaches the centralized `errorHandler` middleware instead of crashing the process or hanging the request.

Frontend Architecture

```
main.jsx      → React root, mounts <App />
App.jsx       → react-router-dom route table, wraps protected routes in <ProtectedRoute>
↓
pages/        → one component per route – owns page-level state, data fetching (via
               services/), and composes components/ into a layout
↓
components/   → reusable, mostly presentational pieces (tables, charts, cards, badges,
               modals) – 39 of them, from generic (Badge, Card, IconButton) to
               domain-specific (RecommendationTable, ResourceDetailsDrawer)
↓
hooks/        → reusable stateful logic extracted out of pages (useAsyncData,
               useAwsConnection, useRecommendationActions, useScanTrigger, useTheme,
               useClickOutside)
↓
services/     → one file per backend resource, wraps Axios calls (dashboard.service.js,
               scan.service.js, recommendation.service.js, ...)
↓
api/api.js    → the single shared Axios instance every service.js imports
```

State management is deliberately simple: no Redux/Zustand/Context-based global store. Each page owns its own data via hooks (mostly `useAsyncData` , a generic fetch-on-mount-with-loading/error-state hook), and cross-cutting concerns (auth token, theme) live in `localStorage` read directly by the relevant hook/service rather than through React Context.

Request Lifecycle (generic API request)

```
User action in browser (click / page load)
→ React event handler or useEffect
→ services/*.service.js function call
→ Axios instance (api.js) – request interceptor attaches "Authorization: Bearer <JWT>"
→ HTTP request to Express
→ Express route matches path + method
→ protect middleware verifies JWT + tokenVersion → attaches req.user
→ asyncHandler-wrapped controller runs
→ controller calls exactly one service function
→ service calls one or more repositories
→ repository runs a Mongoose query/aggregation against MongoDB
→ result flows back up: repository → service → controller
→ controller sends res.json({...}) with a consistent shape (see apiResponse.js)
→ Axios response resolves in the frontend service function
→ calling hook/page updates React state
→ component re-renders with the new data
```

If anything throws anywhere in that chain, `asyncHandler` catches it and forwards to Express's error-handling middleware chain, which `errorHandler.middleware.js` turns into a consistent `{ success: false, message }` JSON error response with the correct HTTP status code (`ApiError` instances carry their own status code; anything else defaults to 500).

Scan Lifecycle

See Part 5 and Part 10 for the fully detailed diagram and click-through — summary:

```

User clicks "Run Scan" (or a schedule's nextRun comes due)
  → POST /api/scan
  → scan.controller.js → scanner.service.js's scanInfrastructure(userId)
  → AwsConnection looked up, AWS SDK clients built from decrypted stored credentials
  → Scan document created (status: RUNNING)
  → scanEC2() – fully awaited, NOT fault-tolerant (a failure here fails the whole scan)
  → scanS3(), scanRDS(), scanIAM() – each independently try/caught (a failure here just zeroes that service's contribution, the scan continues)
  → each scanner: fetch AWS data → analyze → recommend → merge into one Recommendation per resource → save Resource + Recommendation documents
  → S3's scanner additionally generates fleet-wide Insight documents
  → totals summed across all four services
  → Scan document updated (status: COMPLETED, totalResources, estimatedSavings)
  → (if a Notification preference doc exists and scanCompleted/highSeverity is true) an email is sent
  → an AuditLog entry is written
  → response returned to the frontend, which redirects to / refreshes the dashboard

```

Recommendation Lifecycle

See Part 6 for the full pipeline diagram — summary of a single recommendation's life:

```

Created:  a scanner runs its resource-type's analyzer + recommender functions, merges every check into ONE Recommendation document per resource, status defaults to OPEN
Displayed: the dashboard/Recommendations page reads OPEN (and RESOLVED/IGNORED, for history) recommendations; OPEN-only for "outstanding work" aggregations (severity counts, savings totals, category counts); status-UNfiltered for real infra spend (monthlyCost sums) and historical trend charts
Actioned: user clicks Resolve → status: RESOLVED (marks it handled, does NOT change AWS or the resource's real cost)
          user clicks Ignore → status: IGNORED (dismissed, same non-effect on AWS)
          user clicks Stop Instance (EC2 STOP action only) → actually calls AWS's StopInstances API, then (on the NEXT scan) the resource's isRunning becomes false and its recommendation naturally resolves to LOW/NONE
Superseded: the NEXT scan creates a brand new Recommendation document for the same resourceId – old Recommendation documents are never updated in place, they simply stop being "the latest scan's" data once a new scan completes

```

Dashboard Lifecycle

```

Dashboard.jsx mounts
  → useAsyncData-style hooks call dashboard.service.js functions
  → GET /api/dashboard (summary: severity counts, service breakdown, savings, cost breakdown, resource distribution, savings opportunities, AI insights)
  → dashboard.controller.js → dashboard.service.js + serviceAggregation.service.js
  → both read ONLY the latest Scan for this user (Scan.findOne({userId}).sort({createdAt: -1})) and that scan's Resources + Recommendations – nothing here re-scans or hits AWS live, it's all reading what the last scan already found
  → MongoDB aggregation pipelines ($facet/$match/$group) compute severity/category/savings numbers in one round trip; serviceAggregation.service.js's plain-JS reduction computes the per-service breakdown, cost breakdown, resource distribution, savings opportunities list, and rule-based AI insights
  → response assembled and returned
  → Dashboard.jsx distributes the response across ~8 chart/summary components
  → charts render (Recharts), each wrapped in a loading skeleton until data arrives

```

The dashboard is therefore always exactly as fresh as the most recent completed scan — there is no live "read AWS right now" path anywhere in the dashboard, which is what keeps it fast (no AWS round-trips on every page load) at the cost of requiring a fresh scan to see fresh data.

PART 2



Every File & Function, Explained

191 source files. Every one documented: purpose, callers, dependencies, every exported function's inputs/outputs/steps/DB & AWS calls/error handling, data flow, a worked example, and anything a new engineer would trip over.

A note on structure: the original brief asked for file-level documentation (Part 2) and function-level documentation (Part 3) as separate passes. In practice, every file section below already includes full per-function detail — purpose, input, output, internal steps, database calls, AWS calls, error handling, and every return-value shape — inline, right where that function is used. Splitting that into a second, separate pass would have meant either duplicating every function's documentation twice, or forcing the reader to flip between two disconnected sections to understand one file. So Part 3's requirements are fully satisfied inside Part 2, file by file, rather than repeated in a second pass.

CORE SCAN & RECOMMENDATION ENGINE

scanner/ · analyzer/ · recommender/ · calculator/ · aws/ — the pipeline that turns an AWS API response into a priced, actionable recommendation.

Part 2/3 — Core Engine: Scanners, Analyzers, Recommenders, Calculators, AWS SDK Layer

This is the heart of CloudLens — the subsystem that actually talks to AWS, turns raw API responses into findings, and produces the recommendations the rest of the app displays. Every one of the four resource types (EC2, RDS, S3, IAM) follows the **same four-layer pipeline**:

```
AWS SDK wrapper (services/aws/*.service.js)
  ↓ raw AWS response
Mapper (mappers/*.mapper.js)           [EC2/RDS/S3/IAM only — normalizes the AWS shape]
  ↓ normalized objects
Analyzer (services/analyzer/*.analyzer.js)
  ↓ derived booleans/metrics ("isIdle", "hasUnencrypted", "issues: [...]")
Recommender (services/recommender/*.recommender.js)
  ↓ one { severity, action, recommendation, reason, ... } per check
Scanner (services/scanner/*.scanner.js) — the orchestrator for this resource type
  ↓ merges all checks into ONE Recommendation doc per resource
  ↓ saves Resource + Recommendation to MongoDB
```

`scanner.service.js` sits one level above all four resource-type scanners and is what a scan actually calls.

`server/src/services/scanner/scanner.service.js`

PURPOSE

The top-level scan orchestrator. This is the single function the rest of the app calls to "run a scan" — it doesn't know anything about EC2/RDS/S3/IAM internals, it just creates a `Scan` record, builds AWS SDK clients for the user, calls each resource-type scanner in sequence, and rolls up the totals.

WHEN THIS FILE IS CALLED

Whenever a scan is triggered — either a user clicking "Run Scan" (via `scan.controller.js`) or the 60-second cron tick in `scheduler.service.js` picking up a due schedule.

WHO CALLS THIS FILE

- `server/src/controllers/scan.controller.js` — the manual "Run Scan" button's controller
- `server/src/services/scheduler/scheduler.service.js` — automatic scheduled scans

WHICH FILES IT DEPENDS ON

- `factories/awsClient.factory.js` — builds all 6 AWS SDK clients (EC2, CloudWatch, Pricing, S3, RDS, IAM) from the user's stored, decrypted credentials
- `models/AwsConnection.js` — checked directly (not through a repository) to fail fast with a clear error if the user hasn't connected AWS yet
- `services/database/scan.repository.js` — `createScan` / `completeScan`
- `services/scanner/ec2.scanner.js`, `rds.scanner.js`, `s3.scanner.js`, `iam.scanner.js` — the four resource-type scanners
- `utils/ApiError.js` — thrown when no AWS connection exists

EXPORTED FUNCTIONS

- `scanInfrastructure(userId)`
- Purpose:** Run a complete scan of a user's connected AWS account across all four resource types.

- **Input:** `userId` (MongoDB ObjectId of the authenticated user)
- **Output / Return value:** `{ scanId, totalResources, estimatedSavings }`
- **Internal steps:**
 1. Look up the user's `AwsConnection`. If none exists, throw `ApiError(400, "Please connect an AWS account before running a scan.")` immediately — no `Scan` document is even created.
 2. Create a `Scan` document via `createScan(userId)` (status `RUNNING`).
 3. Build all AWS SDK clients for this user via `getAwsClientsForUser(userId)` (this internally re-fetches and decrypts the stored credentials).
 4. Run `scanEC2(...)` — **awaited directly, not wrapped in try/catch**. If EC2 scanning throws, the whole `scanInfrastructure` call rejects and the scan is left in `RUNNING` status forever (see Important Notes).
 5. Run `scanS3(...)`, `scanRDS(...)`, `scanIAM(...)` — each **individually wrapped in its own try/catch**. If any of these three throws, the error is logged to the console and that service's result defaults to `{ totalResources: 0, estimatedSavings: 0 }` — the scan continues with the other services rather than failing outright.
 6. Sum `totalResources` and `estimatedSavings` across all four results.
 7. Call `completeScan(scan._id, totalResources, estimatedSavings)` — sets status to `COMPLETED`, `completedAt`, and the rolled-up totals.
 8. Return `{ scanId, totalResources, estimatedSavings }`.
- **Database calls:** `AwsConnection.findOne({ userId })`, `createScan` → `Scan.create(...)`, `completeScan` → `Scan.findByIdAndUpdate(...)`.
- **AWS calls:** none directly — delegated entirely to the four resource-type scanners.
- **Error handling:** Only EC2 is allowed to fail the whole scan (see Important Notes below — this is asymmetric and worth knowing). S3/RDS/IAM failures are swallowed and logged.
- **Return values:** Always `{ scanId, totalResources, estimatedSavings }` on success; throws `ApiError(400, ...)` if no AWS connection; throws (unhandled) whatever EC2 scanning throws.

DATA FLOW

`userId` in → AWS connection validated → `Scan` doc created (`RUNNING`) → AWS clients built → EC2 scanned (blocking, no fallback) → S3/RDS/IAM scanned (each independently fault-tolerant) → totals summed → `Scan` doc updated (`COMPLETED`) → `{ scanId, totalResources, estimatedSavings }` returned to the controller, which sends it to the frontend.

EXAMPLE EXECUTION

A user with 3 EC2 instances, 2 S3 buckets, 1 RDS instance, and IAM misconfigured such that `iamClient` lacks `iam:ListRoles` permission:

1. EC2 scan succeeds → `{ totalResources: 3, estimatedSavings: 22.50 }`
2. S3 scan succeeds → `{ totalResources: 2, estimatedSavings: 4.10 }`
3. RDS scan succeeds → `{ totalResources: 1, estimatedSavings: 0 }`
4. IAM scan throws an `AccessDenied` error inside `scanIAM` → caught, logged (`"IAM scan failed: AccessDenied: ..."`), `iamResult` defaults to `{ totalResources: 0, estimatedSavings: 0 }`
5. Final: `totalResources = 6`, `estimatedSavings = 26.60`, scan marked `COMPLETED` even though IAM contributed nothing.

IMPORTANT NOTES

- **Asymmetric fault tolerance:** EC2 is the only one of the four scanners NOT wrapped in a try/catch. If a user's AWS credentials lack EC2 permissions (or CloudWatch/Pricing permissions used inside `scanEC2`), the entire scan throws, the `Scan` document is left stuck at `status: "RUNNING"` forever (nothing ever calls `completeScan` or marks it `FAILED`), and the user sees a failed request with no completed-or-failed scan record to show for it. S3/RDS/IAM don't have this problem — a permissions gap there degrades gracefully to zero resources for that service instead of failing the scan.
 - There is a placeholder comment for a future `eks.scanner.js` — see that file's entry below, which is currently a completely empty file with zero content and is never imported anywhere (dead code).
 - EC2 runs before S3/RDS/IAM sequentially, not in parallel — a scan's wall-clock time is roughly the sum of all four scanners' time, not the max.
-

 `server/src/services/scanner/ebs.scanner.js`

PURPOSE

None — this file is completely empty (0 bytes). It was intended to become a dedicated EBS-volume scanner (there's a comment in `scanner.service.js` referencing "An EBS scanner will be added here"), but was never implemented. EBS volume data is currently only surfaced indirectly, as metadata attached to each EC2 instance's `Resource.metadata.volumeIds` / `volumeTypes` / `encrypted` fields (see `ec2.scanner.js`) — there is no standalone EBS resource or recommendation.

WHEN THIS FILE IS CALLED

Never — it has zero exports and is imported nowhere in the codebase (confirmed: zero importers in the full-repo dependency graph).

WHO CALLS THIS FILE

Nobody.

WHICH FILES IT DEPENDS ON

None (empty file).

EXPORTED FUNCTIONS

None.

IMPORTANT NOTES

This is dead code / an unfinished feature stub. It should either be implemented (a real, standalone EBS volume scanner producing its own `Resource` / `Recommendation` records — useful for unattached/orphaned volumes, which today are invisible to CloudLens since only volumes attached to a scanned EC2 instance are ever seen) or deleted. It was flagged as low-priority dead code in an earlier project audit and intentionally left as-is per that audit's scope (only Critical/High findings were fixed).

EC2 pipeline

 `server/src/services/scanner/ec2.scanner.js`

PURPOSE

Orchestrates a full EC2 scan: fetches every instance plus its EBS volumes and Elastic IPs, runs CPU/network/disk/security/storage checks per instance, prices each instance via the AWS Price List API, merges every check into one `Recommendation` document per instance, and persists both the `Resource` and `Recommendation` records.

WHEN THIS FILE IS CALLED

From `scanner.service.js`'s `scanInfrastructure`, once per scan, always first (before S3/RDS/IAM).

WHO CALLS THIS FILE

`server/src/services/scanner/scanner.service.js`

WHICH FILES IT DEPENDS ON

- `services/aws/ec2.service.js` — `getEC2Instances`, `getEBSVolumes`, `getElasticIPAddresses`
- `services/aws/cloudwatch.service.js` — `getCPUUtilization`, `getNetworkAndDiskMetrics`
- `services/aws/pricing.service.js` — `getEC2Pricing`
- `services/database/resource.repository.js` — `saveResources`
- `services/database/recommendation.repository.js` — `saveRecommendation`
- `services/analyzer/cpu.analyzer.js` — `calculateAverageCPU`
- `services/analyzer/ec2.analyzer.js` — `calculateInstanceAgeDays`, `analyzeNetworkUtilization`, `analyzeDiskIO`, `analyzeSecurityGroups`, `analyzeEBSVolumes`, `analyzeElasticIP`
- `services/recommender/ec2.recommender.js` — all 7 `get*Recommendation` functions + `mergeEC2Recommendations`
- `builders/pricing.filter.builder.js` — `buildEC2PricingFilters`
- `parser/pricing.parser.js` — `extractHourlyPrice`

- `utils/instanceType.util.js` — `getAdjacentInstanceType` (for UPSIZE/DOWNSIZE cost comparison)
- `calculator/savings.calculator.js` — `calculateMonthlyCost`, `calculateEstimatedSavings`, `calculateCostComparison`

EXPORTED FUNCTIONS

- `scanEC2(scanId, userId, clients)`
- **Purpose:** Entry point — scans every EC2 instance in the account.
- **Input:** `scanId` (Scan ObjectID), `userId`, `clients` ({ `ec2Client`, `cloudWatchClient`, `pricingClient` })
- **Output:** { `totalResources`, `estimatedSavings`, `resources` } (the raw AWS instance list, not the enriched Resource records)
- **Internal steps:**
 1. Fetch instances, all EBS volumes, and all Elastic IPs **in parallel** (`Promise.all`) — one `DescribeVolumes` / `DescribeAddresses` call total for the whole account, not per-instance.
 2. Group volumes by instance ID (`groupVolumesByInstance`) and build a `Set` of instance IDs that have a real Elastic IP (`getElasticIPInstanceIds`) — both done once, client-side, to avoid N+1 AWS calls.
 3. For each instance, call `processInstance` sequentially (not parallelized across instances — see Important Notes).
 4. Persist all instance `Resource` records in one batch via `saveResources(scanId, "EC2", instanceResources, userId)`.
 5. Return totals.
- **Database calls:** `saveResources` (once, batched), `saveRecommendation` (once per instance, inside `processInstance`)
- **AWS calls:** `DescribeInstances`, `DescribeVolumes`, `DescribeAddresses` (each once per scan)
- **Error handling:** none at this level — a thrown error propagates up to `scanner.service.js`, which does NOT catch EC2 errors (see that file's Important Notes).
- `processInstance(scanId, userId, clients, instance, volumesByInstance, elasticIpInstanceIds)` (*not exported — internal, documented because it's where all the real logic lives*)
- **Purpose:** Fully analyze and price one EC2 instance, save its recommendation, and return its enriched Resource record.
- **Internal steps:**
 1. Fetch CPU utilization (`getCPUUtilization`), extended network/disk metrics (`getNetworkAndDiskMetrics`), and pricing (`getEC2Pricing`) **in parallel**.
 2. Compute `averageCPU` via `calculateAverageCPU(cpuValues)` — returns `0` if `cpuValues` is empty (stopped instance).
 3. Compute `isRunning = instance.state === "running"` — this is the guard that prevents a stopped instance's `averageCPU === 0` from being misread as "idle, recommend STOP."
 4. Compute `monthlyCost = isRunning ? calculateMonthlyCost(hourlyPrice) : 0` — a stopped instance has **zero** compute cost, full stop (EC2, unlike RDS, has no separate storage-cost component to preserve).
 5. Run the base CPU recommendation (`getEC2Recommendation`) — passes `isRunning`; if false, short-circuits to a `LOW` / `NONE` / "already stopped" result without even looking at `averageCPU`.
 6. Compute `estimatedSavings = calculateEstimatedSavings(monthlyCost, baseRecommendation.action)` — `STOP` → full `monthlyCost`; `DOWNSIZE` → 50%; anything else → `0`.
 7. Run 5 more independent checks, each producing its own { `severity`, `action`, `recommendation`, `reason` } : network utilization, disk I/O, instance age, security groups, EBS volumes, Elastic IP. (Network and disk also take `isRunning` and short-circuit the same way as the base check; age/security-group/EBS/Elastic-IP checks are configuration-based and are **not** gated by running state — a stopped instance's unencrypted volume or default security group is still a real finding.)
 8. If the base action is `UPSIZE` / `DOWNSIZE`, look up pricing for the *adjacent* instance type (`getOptimizedMonthlyCost`) to build a before/after cost comparison; otherwise this is skipped (returns the current cost unchanged).
 9. Merge all 7 checks into ONE recommendation via `mergeEC2Recommendations` (see that function's own entry below for the merge algorithm).
 10. Save exactly one `Recommendation` document (`saveRecommendation`).
 11. Return the enriched `Resource` record (instance metadata + CPU/network/disk/security/EBS/Elastic-IP findings all folded into `metadata`).
- **Database calls:** `saveRecommendation` (one call per instance)
- **AWS calls:** `GetMetricData` (CPU) ×1, `GetMetricData` (network+disk, 6 metrics in one call) ×1, `GetProducts` (pricing) ×1, plus optionally one more `GetProducts` call for the adjacent instance type if the action is `UPSIZE/DOWNSIZE`
- **Error handling:** the adjacent-instance-type pricing lookup (`getOptimizedMonthlyCost`) is individually try/caught and falls back to the current cost on failure (a pricing API hiccup for the *comparison* shouldn't fail the whole instance's scan). Everything else is unguarded.

DATA FLOW

`instance` (raw AWS shape) in → CPU/network/disk/pricing fetched in parallel → `isRunning` derived from `instance.state` → 7 independent recommendation checks run → merged into one `Recommendation` → both `Resource` and `Recommendation` persisted → enriched `Resource` object returned to `scanEC2`'s caller loop.

EXAMPLE EXECUTION

A `t3.micro` instance, `state: "stopped"`, 1 unencrypted `gp2` EBS volume, no Elastic IP, 2 security groups (no default):

- `averageCPU = 0` (no CloudWatch data), `isRunning = false`
- `monthlyCost = 0` (stopped, so compute cost is zeroed regardless of hourly price)
- Base check: `{ severity: "LOW", action: "NONE", recommendation: "Instance is already stopped – no active compute cost." }`
- Network/disk checks: also `LOW / NONE` (`isRunning` guard)
- Age check: depends on launch date, say `LOW / NONE`
- Security group check: `LOW / NONE` (`≤5` groups, none default)
- **EBS check:** `{ severity: "HIGH", action: "REVIEW", recommendation: "One or more attached EBS volumes are unencrypted." }` — this one fires regardless of running state
- Elastic IP check: `LOW / NONE` (no public IP while stopped)
- Merged result: `severity: "HIGH"` (worst of all 7), `action: "REVIEW"` (base was NONE but an add-on flagged), `monthlyCost: 0`, `estimatedSavings: 0`

IMPORTANT NOTES

- Instances are processed **sequentially** in a `for...of` loop, not `Promise.all` — a scan of 50 instances makes 50 round-trips of (CPU + network/disk + pricing) one after another. This is a genuine performance bottleneck for large fleets (noted, not yet fixed, in an earlier project audit as a "Medium" finding — parallelizing this was one of the optional enhancements offered to but not selected by the project owner).
- The `isRunning` guard here is the fix for a specific, previously-real bug: `calculateAverageCPU([])` returns `0` for a stopped instance (no CloudWatch datapoints), which is indistinguishable from "running but completely idle" unless the actual instance state is checked first. Without this guard, every stopped instance would get a false "STOP this instance to save \$X/mo" recommendation quoting its full running cost as phantom savings.

```
server/src/services/recommender/ec2.recommender.js
```

PURPOSE

The most elaborate of the four recommenders — 7 independent pure check functions (base CPU, network, disk, instance age, security groups, EBS volumes, Elastic IP) plus the merge function that combines them into the single `Recommendation` actually saved for an EC2 instance.

WHEN THIS FILE IS CALLED

Inside `ec2.scanner.js`'s `processInstance`, once per EC2 instance.

WHO CALLS THIS FILE

```
server/src/services/scanner/ec2.scanner.js
```

WHICH FILES IT DEPENDS ON

`services/analyzer/ec2.analyzer.js` — imports only `calculateEC2ConfidenceScore`, the weighted 0–100 confidence scorer used by the base CPU check.

EXPORTED FUNCTIONS

- `getEC2Recommendation({ averageCPU, avgNetworkIn = 0, avgNetworkOut = 0, diskTotal = 0, instanceAgeDays = 0, isRunning = true })` — the base, cost-driving check.
- **Purpose:** Decide whether an instance should be stopped (idle), upsized (saturated), or left alone, and compute the confidence score.
- **Internal steps:** If `!isRunning`, return immediately: `{severity: "LOW", action: "NONE", confidence: 100, recommendation: "Instance is already stopped – no active compute cost.", reason: "Instance state is not running, so CPU utilization cannot be measured."}` — no CPU threshold logic even runs. Otherwise: compute `confidence` via `calculateEC2ConfidenceScore`; if `averageCPU < 5` (`IDLE_CPU_THRESHOLD`) → `{severity: "HIGH", action: "STOP", ...}`; if `averageCPU > 80` (`HIGH_CPU_THRESHOLD`) → `{severity: "MEDIUM", action: "UPSIZE", ...}`; otherwise → `{severity: "LOW", action: "NONE", recommendation: "Instance utilization is healthy."}`.

- **Return values:** always the 5-field shape above; `action` is one of `NONE` / `STOP` / `UPSIZE` (never `DOWNSIZE` from this specific function, despite `DOWNSIZE` being a valid action value elsewhere in the schema — this function's thresholds only ever produce `STOP` or `UPSIZE` as its actionable outcomes).
- **Error handling:** none needed — pure function, no I/O.
- `getNetworkRecommendation({ averageCPU, avgNetworkIn, avgNetworkOut, isRunning = true })` — a secondary idle signal layered on top of CPU. If `!isRunning` → `LOW` / `NONE` / "Network utilization is healthy." (stopped instance has nothing to measure). Otherwise: CPU idle AND network below 1MB/hr (`LOW_NETWORK_BYTES_THRESHOLD`) → `HIGH` / `REVIEW` , "Instance appears completely idle." CPU idle but network above 50MB/hr (`HIGH_NETWORK_BYTES_THRESHOLD`) → `MEDIUM` / `REVIEW` , "Consider a network optimized instance" (a CPU-idle instance that's still moving a lot of network traffic suggests a different bottleneck than compute). Otherwise → `LOW` / `NONE` .
- `getDiskRecommendation({ averageCPU, avgNetworkIn, avgNetworkOut, diskAnalysis, isRunning = true })` — similarly layered. `!isRunning` → `LOW` / `NONE` . CPU idle AND disk bytes below 1MB/hr AND network below 1MB/hr → `HIGH` / `REVIEW` , "Instance is almost unused" (all three signals negligible at once is a stronger idle signal than CPU alone). Disk ops above 1000/hr (`HIGH_DISK_OPS_THRESHOLD`) → `MEDIUM` / `REVIEW` , "Recommend storage optimized instances." Otherwise → `LOW` / `NONE` .
- `getInstanceAgeRecommendation(instanceAgeDays)` — no `isRunning` guard (age is state-independent). `>365 days` (`OLD_INSTANCE_AGE_DAYS`) → `LOW` / `REVIEW` , "review for modernization." Otherwise → `LOW` / `NONE` .
- `getSecurityGroupRecommendation({ count, hasDefault })` — config-based, no `isRunning` guard (a security group's exposure applies the instant the instance is running, stopped or not). `count > 5` (`MAX_SECURITY_GROUPS`) → `HIGH` / `REVIEW` . Else `hasDefault` → `MEDIUM` / `REVIEW` . Else → `LOW` / `NONE` .
- `getEBSRecommendation({ hasUnencrypted, hasGp2 })` — config-based, no `isRunning` guard (an unencrypted volume is a data-at-rest risk regardless of power state). `hasUnencrypted` → `HIGH` / `REVIEW` . Else `hasGp2` → `LOW` / `REVIEW` (migrate to gp3). Else → `LOW` / `NONE` .
- `getElasticIPRecommendation(elasticIpStatus)` — `"TEMPORARY"` → `LOW` / `REVIEW` , "Suggest allocating an Elastic IP." Else (`"NONE"` or `"ELASTIC"`) → `LOW` / `NONE` .
- `mergeEC2Recommendations({ baseRecommendation, networkRecommendation, diskRecommendation, ageRecommendation, securityGroupRecommendation, ebsRecommendation, elasticIpRecommendation, monthlyCost, estimatedSavings })`
- **Purpose:** Combine all 7 checks into the single Recommendation document actually persisted.
- **Internal steps:** `worst` = highest-severity of all 7 (rank `LOW`:1/`MEDIUM`:2/`HIGH`:3). `flagged` = every check with `action !== "NONE"` .
Action selection has a special rule, unlike RDS/S3's merge functions: the base CPU verdict (`STOP` / `UPSIZE`) always wins if it's non-`NONE` — the 6 add-on checks can only escalate `action` to a generic `REVIEW` when the base is `NONE` , they can never override or hijack a `STOP` / `UPSIZE` verdict. This is deliberate: the existing "Stop Instance" button in the UI is gated on `action === "STOP"` , so an add-on check accidentally taking over the action field would silently break that button.
- **Return value:** `{ severity: worst.severity, action, confidence: baseRecommendation.confidence, recommendation: <every flagged check's message joined with spaces, or a fixed "all healthy" sentence>, reason: <all 7 reasons joined with " | ">, monthlyCost, estimatedSavings }` .

DATA FLOW

7 independent check results in (one of which, the base check, also determines `estimatedSavings` upstream in the scanner) → worst-severity + base-priority-action out.

EXAMPLE EXECUTION

Running instance, 3% CPU, low network, low disk, 2 security groups (no default), all volumes encrypted, no Elastic IP, 100 days old: base → `HIGH` / `STOP` (idle) → this alone makes the merged action `STOP` regardless of what any add-on check says, merged severity `HIGH` .

IMPORTANT NOTES

This is the one recommender file across all four resource types where the "worst severity wins" rule and "which check controls the action" rule are **decoupled** — severity always follows the worst check, but action follows the base check specifically, not whichever check happened to be worst. RDS/S3 conflate the two (action becomes `REVIEW` if *anything* is flagged); IAM's `mergeFindings` ties both to the single worst flagged check. Three resource types, three different merge philosophies — worth knowing if asked to compare them.

 `server/src/services/analyzer/ec2.analyzer.js`

PURPOSE

Pure functions that turn raw AWS/CloudWatch signals for one EC2 instance into normalized, analysis-ready values — instance age in days, network/disk averages, security group summary, EBS volume summary, Elastic IP classification, and a weighted 0–100 confidence score for the

CPU-based recommendation.

WHEN THIS FILE IS CALLED

Inside `ec2.scanner.js`'s `processInstance`, once per instance, for every field it computes.

WHO CALLS THIS FILE

`server/src/services/scanner/ec2.scanner.js`, and indirectly `server/src/services/recommender/ec2.recommender.js` (imports just `calculateEC2ConfidenceScore`)

WHICH FILES IT DEPENDS ON

`services/analyzer/cpu.analyzer.js` — reuses `calculateAverageCPU` (a generic "mean of a numeric array" helper, despite the name) for network/disk averaging too.

EXPORTED FUNCTIONS

- `calculateInstanceAgeDays(launchTime)` — Purpose: age of the instance in whole days. Input: a `Date` /ISO string or falsy. Output: integer ≥ 0 , or `0` if `launchTime` is falsy. Steps: `Date.now() - new Date(launchTime).getTime()`, divided into days, floored, clamped to ≥ 0 . No DB/AWS calls. No error handling needed (pure math).
- `analyzeNetworkUtilization({ networkInValues, networkOutValues })` — Purpose: average NetworkIn/NetworkOut over the lookback window. Output: `{ avgNetworkIn, avgNetworkOut }`. Steps: delegates both to `calculateAverageCPU`.
- `analyzeDiskIO({ diskReadBytesValues, diskWriteBytesValues, diskReadOpsValues, diskWriteOpsValues })` — Output: `{ avgDiskReadBytes, avgDiskWriteBytes, avgDiskReadOps, avgDiskWriteOps }`, each averaged the same way. Note: EBS-backed instances (the vast majority) publish **no** CloudWatch datapoints for these metrics at all (they're instance-store-only metrics) — so this virtually always resolves to `0` for typical accounts, which is expected, not an error.
- `analyzeSecurityGroups(securityGroups = [])` — Output: `{ count, hasDefault, groups, groupNames, groupIds }`. Steps: extracts names/IDs, flags `hasDefault` if any group is literally named `"default"`.
- `analyzeEBSVolumes(volumes = [])` — Input: raw `DescribeVolumes` results already filtered to volumes attached to this instance. Output: `{ volumeCount, volumeIds, volumeTypes, encrypted, hasUnencrypted, hasGp2 }`. `hasUnencrypted` is true if **any** attached volume has `Encrypted: false`.
- `analyzeElasticIP({ publicIp, isAssociated })` — Output: `"NONE"` (no public IP), `"ELASTIC"` (public IP backed by a real allocated Elastic IP), or `"TEMPORARY"` (an ephemeral auto-assigned public IP not backed by an EIP allocation). A stopped instance almost always has `publicIp` unset (AWS releases the ephemeral IP on stop), so this naturally resolves to `"NONE"` for stopped instances.
- `calculateEC2ConfidenceScore({ averageCPU, avgNetworkTotal, avgDiskTotal, instanceAgeDays })` — Purpose: a weighted 0–100 confidence score for the base CPU recommendation (replaces older fixed 95/90/99 constants). Weights: CPU 40%, Network 20%, Disk 20%, Age 20%. Each signal is scored by how decisively it supports the CPU verdict (e.g. very low network traffic strongly supports an "idle" CPU verdict; high network traffic weakens it). Steps: `cpuSignal = |averageCPU - 50| × 2` (clamped 0–100 — far from the 50% midpoint = more decisive); `networkSignal / diskSignal` scaled against fixed 50MB reference constants, inverted when CPU is low (low network activity increases confidence in "idle"); `ageSignal` scaled against a 730-day reference. Final score is the weighted sum, rounded and clamped 0–100.

DATA FLOW

Raw CloudWatch value arrays + instance metadata in → averaged/classified/scored outputs out, all pure functions with no side effects.

EXAMPLE EXECUTION

`calculateEC2ConfidenceScore({ averageCPU: 2, avgNetworkTotal: 500000, avgDiskTotal: 0, instanceAgeDays: 400 })` → CPU is far from 50 (high signal), network is low (supports idle, high signal since CPU < 50), disk is 0 (max signal), age is over half the 730-day reference → a high confidence score (roughly 80–95) that this instance really is idle, not just momentarily quiet.

IMPORTANT NOTES

None of these functions touch the database or AWS directly — everything here is pure, synchronous, and easily unit-testable in isolation (which is how several of this session's fixes were verified, via standalone Node scripts calling these functions directly).

 `server/src/services/analyzer/cpu.analyzer.js`

PURPOSE

A single, tiny, generic "average of a numeric array" helper. Despite the name, it's reused for CPU, network, and disk averaging alike across EC2 and RDS.

WHEN THIS FILE IS CALLED

Inside `ec2.analyzer.js`, `ec2.scanner.js`, and `rds.scanner.js`, anywhere a CloudWatch metric's array of datapoints needs to become one number.

WHO CALLS THIS FILE

`server/src/services/analyzer/ec2.analyzer.js`, `server/src/services/scanner/ec2.scanner.js`,
`server/src/services/scanner/rds.scanner.js`

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `calculateAverageCPU(values = [])` — Input: array of numbers (or nothing). Output: the mean, rounded to 2 decimals via `Number(x.toFixed(2))`. **Returns 0 for an empty array** — this single line is the root cause of the "stopped resource looks idle" bug class that required explicit `isRunning` guards throughout the EC2 and RDS recommenders: a resource with no CloudWatch datapoints (because it isn't running) is mathematically indistinguishable from a resource that's running with 0% utilization unless the caller separately checks the actual instance/DB state.

IMPORTANT NOTES

This function's behavior is 100% correct and appropriate for its literal job ("average an array, or 0 if there's nothing to average"). The bug this caused elsewhere was never in this function — it was in callers that used the resulting `0` as a utilization signal without first confirming the resource was actually running. This is a good example of a correctness bug that lives at an integration boundary, not inside any single function.

RDS pipeline

 `server/src/services/scanner/rds.scanner.js`

PURPOSE

Orchestrates a full RDS scan: fetches every DB instance, checks its security/backup/Multi-AZ/storage/engine-version configuration, checks CPU utilization, prices it, merges every check into one `Recommendation` per instance, and persists both `Resource` and `Recommendation` records.

WHEN THIS FILE IS CALLED

From `scanner.service.js`, once per scan, wrapped in a try/catch there (a failure here does not fail the whole scan).

WHO CALLS THIS FILE

`server/src/services/scanner/scanner.service.js`

WHICH FILES IT DEPENDS ON

- `services/aws/rds.service.js` — `getDBInstances`
- `services/aws/cloudwatch.service.js` — `getRDSCPUUtilization`
- `services/aws/pricing.service.js` — `getRDSPricing`
- `services/database/resource.repository.js`, `services/database/recommendation.repository.js`
- `services/analyzer/cpu.analyzer.js` — `calculateAverageCPU`
- `services/analyzer/rds.analyzer.js` — `analyzeRDSConfiguration`
- `services/recommender/rds.recommender.js` — all 7 `get*Recommendation` functions + `mergeRDSRecommendations`
- `builders/pricing.filter.builder.js` — `buildRDSPricingFilters`
- `parser/pricing.parser.js` — `extractHourlyPrice`
- `calculator/rds.calculator.js` — `calculateRDSMonthlyCost`, `calculateRDSEstimatedSavings`

EXPORTED FUNCTIONS

- `scanRDS(scanId, userId, clients)`
- **Input:** `clients = { rdsClient, cloudWatchClient, pricingClient }`
- **Output:** `{ totalResources, estimatedSavings, resources }`
- **Internal steps:** fetch all DB instances → `processDBInstance` sequentially for each → batch-save all Resource records → return totals.
- `processDBInstance(scanId, userId, clients, instance)` (internal)
- **Internal steps:**
 1. Fetch CPU utilization and pricing in parallel.
 2. `averageCPU = calculateAverageCPU(cpuValues)`, `hourlyPrice = extractHourlyPrice(pricingResponse)`.
 3. `isRunning = instance.status === "available"` — RDS's equivalent of EC2's `state === "running"` check. (Note: RDS uses `status`, not `state`, and the "running" value is `"available"`, not `"running"` — a real, easy-to-get-wrong naming difference between the two AWS services that this codebase handles correctly.)
 4. `monthlyCost = calculateRDSMonthlyCost(isRunning ? hourlyPrice : 0, instance.allocatedStorage)` — **only the compute portion is zeroed for a stopped instance; storage cost is preserved**, because unlike EC2, a stopped RDS instance's storage is still provisioned and billed. This is the one meaningful difference from the EC2 pattern.
 5. Run `analyzeRDSConfiguration` (security/backup/Multi-AZ/storage-type/engine-version + CPU idle/high flags, all in one pass).
 6. Run 7 independent recommendation checks: public access, encryption, backup, Multi-AZ, CPU (passing `isRunning`), storage type (gp2→gp3), engine version.
 7. Merge via `mergeRDSRecommendations`, save, return the enriched Resource.
- **Database calls:** `saveRecommendation`
- **AWS calls:** `GetMetricData` (RDS CPU) × 1, `GetProducts` (RDS pricing) × 1

DATA FLOW

Same shape as EC2's pipeline, with the storage-cost-preservation difference noted above.

EXAMPLE EXECUTION

A stopped `db.t3.micro` MySQL instance, 100GB allocated storage, encrypted, private, Multi-AZ enabled, 7-day backups, gp3 storage, current engine version:

- `isRunning = false` → `monthlyCost = calculateRDSMonthlyCost(0, 100) = 0 + 100×0.115 = 11.50` (storage-only)
- CPU check: `{ severity: "LOW", action: "NONE", recommendation: "Database instance is stopped – no active compute cost." }` (isRunning guard)
- All other checks pass clean → merged: `severity: "LOW", action: "NONE", monthlyCost: 11.50, estimatedSavings: 0`

IMPORTANT NOTES

Same sequential-processing performance characteristic as EC2 (one instance at a time, not parallelized).

```
server/src/services/analyzer/rds.analyzer.js
```

PURPOSE

Single pure function that turns raw RDS instance config + CPU data into a normalized issues list and derived booleans, for the recommender to act on.

WHEN THIS FILE IS CALLED

Inside `rds.scanner.js`'s `processDBInstance`.

WHO CALLS THIS FILE

```
server/src/services/scanner/rds.scanner.js
```

WHICH FILES IT DEPENDS ON

None (self-contained, including its own engine-version baseline table).

EXPORTED FUNCTIONS

- `analyzeRDSConfiguration({ publiclyAccessible, storageEncrypted, backupRetentionPeriod, multiAZ, storageType, engine, engineVersion, averageCPU })`
- **Purpose:** One-pass analysis of every RDS configuration dimension the recommender cares about.
- **Output:** `{ issues: [...], publiclyAccessible, storageEncrypted, backupRetentionPeriod, backupDisabled, lowBackupRetention, multiAZ, storageType, isGp2, engineVersionIsOld, averageCPU, isIdle, isHighCPU }`
- **Internal steps:** builds a human-readable `issues` string array by checking, in order: public access, encryption disabled, backups disabled (`retentionPeriod === 0`) vs. low (`0 < retention < 7` days), Multi-AZ disabled, gp2 storage, old engine version (via a static per-engine-family minimum-version table: MySQL 8.0, Aurora-MySQL 8.0, Postgres 14.0, Aurora-Postgres 14.0, MariaDB 10.6, SQL Server 15.0, Oracle 19.0), CPU idle (`<5%`) or high (`>80%`).
- **Database/AWS calls:** none — pure function.
- **Error handling:** version parsing (`parseVersionTuple`) defensively returns `[0, 0]` for unparseable version strings rather than throwing; an engine family with no baseline in the table is never flagged as old (returns `false` rather than guessing).

IMPORTANT NOTES

The engine-version baseline is a **static, hardcoded heuristic**, not a live lookup against AWS's actual end-of-life/deprecation schedule — it will silently become outdated as AWS ships new major versions and should be treated as "clearly outdated" detection, not an authoritative EOL check.

```
server/src/services/recommender/rds.recommender.js
```

PURPOSE

7 independent pure functions, each producing one `{severity, action, confidence, recommendation, reason}` (plus `savingsRate` for the CPU check) for one dimension of an RDS instance, plus a merge function that combines all 7 into the single Recommendation actually saved.

WHEN THIS FILE IS CALLED

Inside `rds.scanner.js`'s `processDBInstance`, once per RDS instance.

WHO CALLS THIS FILE

```
server/src/services/scanner/rds.scanner.js
```

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `getRDSPublicAccessRecommendation(publiclyAccessible)` — `HIGH / REVIEW` if public, else `LOW / NONE`.
- `getRDSEncryptionRecommendation(storageEncrypted)` — `HIGH / REVIEW` if not encrypted, else `LOW / NONE`.
- `getRDSBackupRecommendation({ backupDisabled, lowBackupRetention, backupRetentionPeriod })` — `MEDIUM / REVIEW` if disabled (retention=0) or low (<7 days), else `LOW / NONE`.
- `getRDSMultiAZRecommendation(multiAZ)` — `MEDIUM / REVIEW` if disabled, else `LOW / NONE`.
- `getRDSCPURecommendation({ averageCPU, isIdle, isHighCPU, isRunning = true })` — **the `isRunning` -guarded check.** If `!isRunning`, returns `{severity: "LOW", action: "NONE", confidence: 100, recommendation: "Database instance is stopped – no active compute cost.", savingsRate: 0}` immediately, without even looking at `averageCPU / isIdle / isHighCPU`. Otherwise: idle (<5%) → `MEDIUM / REVIEW`, `savingsRate: 0.3` (downsizing suggestion); high (>80%) → `MEDIUM / REVIEW`, `savingsRate: 0`; healthy → `LOW / NONE`.
- `getRDSStorageTypeRecommendation(isGp2)` — `LOW / REVIEW` if gp2, else `LOW / NONE`.
- `getRDSEngineVersionRecommendation({ engineVersionIsOld, engine, engineVersion })` — `LOW / REVIEW` if old, else `LOW / NONE`.
- `mergeRDSRecommendations({ ...all 7 above, monthlyCost, estimatedSavings })` — Purpose: combine into one saved Recommendation. Steps: `worst` = the highest-severity of all 7 (rank `LOW:1, MEDIUM:2, HIGH:3`); `flagged` = every check whose `action !== "NONE"`; final `action` = `"REVIEW"` if anything is flagged, else `"NONE"`; `recommendation / reason` text = every flagged check's message joined with spaces (recommendation) or every check's reason joined with `" | "` (reason, includes healthy checks too, for full transparency).

DATA FLOW

7 independent check results in → one worst-of-all-severities, any-flagged action, concatenated message out.

EXAMPLE EXECUTION

See `rds.scanner.js`'s example above — a stopped, otherwise-clean instance merges to `LOW` / `NONE` /\$0 savings; a stopped instance with an unencrypted volume would merge to `HIGH` / `REVIEW` (encryption still fires) with `estimatedSavings: 0` (CPU check contributes no savings rate since it's stopped).

IMPORTANT NOTES

Unlike IAM's merge function (which picks only the single worst finding's action/message), EC2 and RDS's merge functions concatenate **every** flagged finding's text — a multi-issue instance's `recommendation` field can be a long sentence combining several distinct pieces of advice.

S3 pipeline

`server/src/services/scanner/s3.scanner.js`

PURPOSE

Orchestrates a full S3 scan: for every bucket, checks public access/versioning/lifecycle/encryption/policy/storage-class distribution, prices its storage, merges every check into one `Recommendation` per bucket, saves `Resource` records, and additionally generates fleet-wide `Insight` documents across all buckets scanned.

WHEN THIS FILE IS CALLED

From `scanner.service.js`, wrapped in try/catch there.

WHO CALLS THIS FILE

`server/src/services/scanner/scanner.service.js`

WHICH FILES IT DEPENDS ON

- `services/aws/s3.service.js` — 8 functions covering region/public-access/versioning/lifecycle/encryption/replication/policy
- `services/aws/cloudwatch.service.js` — `getS3StorageMetrics`
- `services/aws/pricing.service.js` — `getS3Pricing`
- `services/database/resource.repository.js`, `recommendation.repository.js`, `insight.repository.js`
- `services/analyzer/s3.analyzer.js` — `analyzeBucketConfiguration`, `analyzeBucketPolicy`, `analyzeLifecycleRules`, `analyzeStorageClassDistribution`
- `services/recommender/s3.recommender.js` — all 6 `get*Recommendation` functions + `mergeS3Recommendations`
- `services/insight/s3.insight.js` — `generateS3Insights` (fleet-wide, not per-bucket)
- `builders/pricing.filter.builder.js`, `parser/pricing.parser.js`, `calculator/storage.calculator.js`

EXPORTED FUNCTIONS

- `scanS3(scanId, userId, clients)`
- Input:** `clients = { s3Client, cloudWatchClient, pricingClient }`
- Internal steps:** fetch all buckets → `processBucket` sequentially for each, collecting a compact `insightInput` per bucket alongside its full `Resource` → batch-save Resources → **after all buckets are processed**, call `generateS3Insights(insightInputs)` once for the whole fleet and persist the results via `saveInsights` → return totals.
- This is the only one of the four scanners that writes to the `Insight` collection.
- `processBucket(scanId, userId, clients, bucket)` (*internal*)
- Internal steps:**
 - Fetch 8 signals **in parallel**: region, public status, versioning, lifecycle rules, encryption, replication, storage metrics (CloudWatch), bucket policy.
 - Compute `bucketSizeGB`, run `analyzeLifecycleRules` (inspects rule *content* — transitions/expiration/multipart cleanup — not just presence), `analyzeStorageClassDistribution` (STANDARD/IA/Glacier byte percentages).
 - Fetch price-per-GB for the bucket's region (independently try/caught — a pricing failure falls back to a hardcoded \$0.023/GB Standard-tier rate rather than failing the bucket).
 - Run `analyzeBucketConfiguration` (public/versioning/lifecycle-presence) and `analyzeBucketPolicy` (inspects policy *statement content* for wildcard principals/public grants — a bucket can be "not public" per Block Public Access settings but still have a policy granting public access, which this catches independently).

- Run 6 independent checks: config (public/versioning/lifecycle), encryption, storage (size/lifecycle/tiering/replication shape), policy content, lifecycle rule content, storage-class distribution.
 - Merge via `mergeS3Recommendations`, save.
 - Return both the enriched `Resource` and a compact `insightInput` object (just the primitives `generateS3Insights`'s rules need).
- AWS calls:** `GetBucketLocation`, `GetPublicAccessBlock`, `GetBucketVersioning`, `GetBucketLifecycleConfiguration`, `GetBucketEncryption`, `GetBucketReplication`, `GetMetricData` (storage), `GetBucketPolicy`, `GetProducts` (pricing) — 9 AWS calls per bucket.

DATA FLOW

`bucket` in → 8 signals fetched in parallel → 6 independent checks → merged Recommendation + Resource → **also** a compact insight-input contributed to a fleet-wide insight pass that runs once after every bucket is done.

EXAMPLE EXECUTION

A public bucket with no lifecycle policy, 150GB, Standard storage, no encryption:

- Config check: **HIGH** (public) — `getS3Recommendation` returns HIGH severity specifically because `isPublic` is true (any other issue alone would be **MEDIUM**)
- Encryption check: **HIGH** (`!encryptionEnabled`)
- Storage check: **HIGH** (`bucketSizeGB >= 100 && !hasLifecycleRules`), `savingsRate: 0.3`
- Lifecycle check: **HIGH** (`bucketSizeGB > 20 && !configured`)
- Merged: `severity: "HIGH"`, `action: "REVIEW"`, `estimatedSavings ≈ 30%` of monthly cost

IMPORTANT NOTES

S3's graceful-degradation philosophy is the most consistent of the four scanners: nearly every AWS call in `s3.service.js` is individually try/catched with a documented, deliberate fallback value for "can't tell" cases (see that file's entry below) — an S3 scan is very unlikely to fail outright even with a partially-permissioned IAM policy.

 `server/src/services/analyzer/s3.analyzer.js`

PURPOSE

4 pure functions turning raw S3 API responses into normalized findings: config issues list, bucket policy content analysis (public/wildcard grants), lifecycle rule content analysis, and storage-class byte-percentage breakdown.

WHEN THIS FILE IS CALLED

Inside `s3.scanner.js`'s `processBucket`.

WHO CALLS THIS FILE

`server/src/services/scanner/s3.scanner.js`

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `analyzeBucketConfiguration({ isPublic, versioningEnabled, hasLifecycleRules })` — Output: `{ issues, isPublic, versioningEnabled, hasLifecycleRules }`. Simple three-flag issues list.
- `analyzeLifecycleRules(rules)` — Input: array of raw lifecycle rules, or `null` (fetch failed/undeterminable — treated as "assume properly configured" to avoid a false alarm), or `[]` (definitively no lifecycle config — a real finding). Output: `{ configured, ruleCount, hasTransitionToIA, hasTransitionToGlacier, hasIntelligentTiering, hasExpirationRule, hasAbortIncompleteMultipartUpload, onlyExpirationConfigured }`. Inspects each rule's `Transitions` array for IA (`STANDARD_IA` / `ONEZONE_IA`) and Glacier (`GLACIER` / `DEEP_ARCHIVE` / `GLACIER_IR`) storage classes.
- `isWildcardPrincipal(principal)` (*internal helper, also useful standalone*) — a Principal is wildcard if it's literally `"*"` or `{"AWS": "*"}` / `{"AWS": ["*", ...]}`. Service principals (`{"Service": "..."}`) and specific ARNs are NOT flagged. Does **not** evaluate the statement's `Condition` block (e.g. a source-IP restriction narrowing a `"*"` principal) — a wildcard principal is always flagged regardless of any condition, the conservative default for a security scanner.

- `analyzeBucketPolicy(policy)` — Input: parsed policy JSON or `null`. Output: `{ policyExists, hasWildcardPrincipal, publicRead, publicWrite, fullAccess, wildcardPermissions, issues }`. Only inspects `Effect: "Allow"` statements (a `Deny` targeting `"*"` is a control, not a risk). `wildcardPermissions` (any wildcard action, e.g. `s3:*`) is flagged **regardless of Principal** — granting unrestricted actions to even one specific named account is still a least-privilege violation. `publicRead` / `publicWrite` / `fullAccess` additionally require the Principal to be wildcard.
- `analyzeStorageClassDistribution(storageClassBytes)` — Output: `{ totalBytes, bytes, percentages }` — percentage of total bytes in each of 5 tracked storage classes (STANDARD, STANDARD_IA, ONEZONE_IA, GLACIER, DEEP_ARCHIVE). Percentages are relative to the sum of just these 5 tracked classes, not literally 100% of every possible S3 storage class.

IMPORTANT NOTES

`analyzeLifecycleRules(null)` returning "assume everything is configured" (rather than "assume nothing is configured") is a deliberate false-alarm-avoidance choice — worth knowing because it means a permissions gap on `s3:GetLifecycleConfiguration` silently hides a real "no lifecycle policy" finding rather than surfacing it, which is the opposite failure mode from what a security-conscious reader might expect.

```
server/src/services/recommender/s3.recommender.js
```

PURPOSE

6 independent pure functions producing one recommendation each (config, encryption, storage shape, bucket policy, lifecycle content, storage-class distribution), plus the merge function.

WHEN THIS FILE IS CALLED

Inside `s3.scanner.js`'s `processBucket`.

WHO CALLS THIS FILE

```
server/src/services/scanner/s3.scanner.js
```

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `getS3Recommendation({ issues, isPublic })` — `HIGH` if any issues AND public; `MEDIUM` if issues but not public; `LOW` / `NONE` if no issues.
- `getS3EncryptionRecommendation({ encryptionEnabled, encryptionType })` — `HIGH` / `REVIEW` if disabled; `LOW` / `NONE` either way once enabled (KMS vs AES256 both count as healthy, just different reason text).
- `getS3StorageRecommendation({ bucketSizeGB, objectCount, hasLifecycleRules, hasIntelligentTiering, hasReplication })` — `HIGH` if $\geq 100\text{GB}$ with no lifecycle (`savingsRate: 0.3`); `MEDIUM` if $\geq 100\text{GB}$ with no Intelligent-Tiering (`savingsRate: 0.2`); `LOW` / `REVIEW` if $< 1\text{GB}$ but replication is configured (`savingsRate: 1` — replication roughly doubles storage cost for negligible benefit at that scale); else `LOW` / `NONE`.
- `getS3PolicyRecommendation({ policyExists, hasWildcardPrincipal, publicRead, publicWrite, fullAccess, wildcardPermissions, issues })` — `HIGH` / `REVIEW` if any public grant; `MEDIUM` / `REVIEW` if wildcard principal/permissions without being fully public; `LOW` / `NONE` otherwise (whether or not a policy exists at all).
- `getS3LifecycleRecommendation({ bucketSizeGB, lifecycleAnalysis })` — `HIGH` if unconfigured AND $> 20\text{GB}$; `MEDIUM` if configured but only expiration (no transitions/cleanup), or configured but does nothing useful at all; `LOW` / `NONE` if properly configured or small enough not to matter.
- `getS3StorageClassRecommendation({ percentages, totalBytes })` — `LOW` / `NONE` if $< 1\text{GB}$ tracked (too small to be meaningful); `MEDIUM` / `REVIEW` if $\geq 80\%$ still in STANDARD (suggests Intelligent-Tiering) or $\geq 10\%$ in an IA tier with 0% in Glacier (suggests a further cold-storage transition); `LOW` / `NONE` otherwise.
- `mergeS3Recommendations({ ...all 6, monthlyCost, estimatedSavings })` — same worst-severity/any-flagged-action/concatenated-text pattern as EC2/RDS's merge functions.

IMPORTANT NOTES

S3 is the only one of the four resource types where the base config check (`getS3Recommendation`) and the policy-content check (`getS3PolicyRecommendation`) are deliberately kept separate — a bucket can pass Block Public Access checks (what `getS3Recommendation` looks at) while its bucket *policy* still grants public access via a different mechanism, which only `getS3PolicyRecommendation` catches.

IAM pipeline

 `server/src/services/scanner/iam.scanner.js`

PURPOSE

Orchestrates a full IAM scan: every IAM user, every customer-managed IAM role (AWS-managed service-linked roles are filtered out entirely), plus one synthetic "root account" resource — each gets its own security analysis and merged Recommendation. IAM is the only scanner where **every** recommendation is a `SECURITY` category finding with `$0` cost/savings (there's no pricing dimension to IAM).

WHEN THIS FILE IS CALLED

From `scanner.service.js`, wrapped in try/catch there.

WHO CALLS THIS FILE

`server/src/services/scanner/scanner.service.js`

WHICH FILES IT DEPENDS ON

- `services/aws/iam.service.js` — 13 functions covering users/roles/MFA/console-access/policies/groups/access-keys/tags/root-account-summary
- `services/database/resource.repository.js`, `recommendation.repository.js`
- `services/analyzer/iam.analyzer.js` — `isServiceLinkedRole`, `analyzeUserSecurity`, `analyzeRoleSecurity`
- `services/recommender/iam.recommender.js` — 8 check functions + 3 merge functions (user/role/root — three different shapes)

EXPORTED FUNCTIONS

- `scanIAM(scanId, userId, clients)`
- Internal steps:**
 - Fetch all users and all roles in parallel.
 - Filter out AWS-managed service-linked roles (`isServiceLinkedRole`) — these are never analyzed, recommended on, or saved, so they never appear anywhere in the dashboard.
 - Process the root account (`processRootAccount`) — always exactly one synthetic resource, `resourceId: "root-account"`.
 - Process every user sequentially (`processUser`).
 - Process every remaining (non-service-linked) role sequentially (`processRole`).
 - Batch-save `IAM_USER` resources, `IAM_ROLE` resources, and the single `IAM_ROOT_ACCOUNT` resource as **three separate `saveResources` calls** (one per distinct `resourceType`) — the only scanner that saves multiple resource-type batches in one pass. User/role batches are skipped entirely (`if (userResources.length) ...`) if empty, to avoid an unnecessary empty-array DB write; the root account save always runs (there's always exactly one).
 - Return totals — `estimatedSavings` is always `0` for the whole IAM scan.
- `processUser(scanId, userId, clients, user)` (*internal*) — fetches MFA status, console access, attached/inline policies, groups, access keys (each with its own `GetAccessKeyLastUsed` lookup), tags, then customer-managed policy documents for the attached ones — all in parallel where independent. Runs `analyzeUserSecurity`, then 8 checks: console-MFA, admin-access, wildcard-permissions, access-key-age, login-activity, tags, age, long-name. Merges via `mergeIAMUserRecommendations`.
- `processRole(scanId, userId, clients, role)` (*internal*) — same shape minus MFA/console/access-key checks (roles don't have those), plus a role-usage check (never-used / not-used-recently, based on `lastUsedDate`). Merges via `mergeIAMRoleRecommendations`.
- `processRootAccount(scanId, userId, clients)` (*internal*) — just 2 checks: root MFA enabled, root has access keys present. Merges via `mergeRootAccountRecommendations`.
- Database calls (all three `process*` functions):** one `saveRecommendation` call each, always with `category: "SECURITY"`, `monthlyCost: 0`, `estimatedSavings: 0`.
- AWS calls:** up to 8 IAM API calls per user (MFA, login profile, attached policies, inline policies + one `GetUserPolicy` per inline policy, groups, access keys + one `GetAccessKeyLastUsed` per key, tags), up to 4 per role, 1 (`GetAccountSummary`) for root.

DATA FLOW

Users + roles fetched → service-linked roles filtered out → root account, then every user, then every role individually analyzed and merged → 3 separate batched saves (by `resourceType`) → totals returned (always `$0` savings).

EXAMPLE EXECUTION

A user with console access, no MFA, `AdministratorAccess` attached, one access key unused for 30 days:

- MFA check: `HIGH` / `ENABLE_MFA` (`consoleUserWithoutMFA`)
- Admin check: `HIGH` / `REMOVE_ADMIN_ACCESS`
- Access key check: `MEDIUM` / `DELETE_UNUSED_KEY` (unused > 14 days)
- Merged (via `mergeFindings`, which picks the SINGLE worst flagged finding, not a concatenation): `severity: "HIGH"`, `action: "REMOVE_ADMIN_ACCESS"` (whichever HIGH finding was evaluated as "worst" first in the reduce), recommendation text includes a `"(+2 other findings)"` suffix.

IMPORTANT NOTES

IAM's merge behavior (`mergeFindings` in the recommender, used by all 3 IAM merge functions) is meaningfully different from EC2/RDS/S3's merge functions: IAM picks **one single action** (the worst flagged finding's own action, e.g. `ENABLE_MFA` or `REMOVE_ADMIN_ACCESS`) rather than a generic `REVIEW`, because every IAM check has its own specific, real remediation action — unlike EC2/RDS/S3 where `REVIEW` is a reasonable universal catch-all.

```
server/src/services/analyzer/iam.analyzer.js
```

PURPOSE

Pure functions turning raw IAM data (policies, access keys, dates) into the specific booleans the IAM recommender needs: service-linked-role detection, access-key age/unused analysis, and full user/role security posture.

WHEN THIS FILE IS CALLED

Inside `iam.scanner.js`'s `processUser` / `processRole`.

WHO CALLS THIS FILE

`server/src/services/scanner/iam.scanner.js`

WHICH FILES IT DEPENDS ON

`utils/iamPolicy.util.js` — `hasWildcardAction`, `hasFullWildcardPermissions`

EXPORTED FUNCTIONS

- `calculateDaysSince(date)` — same pattern as `ec2.analyzer.js`'s age function, returns `null` (not `0`) for a falsy date — distinguishing "no date recorded" from "0 days old" matters for the never-used/never-logged-in checks downstream.
- `isServiceLinkedRole(role)` — true if `role.path` starts with `/aws-service-role/` OR `role.roleName` starts with `AWSServiceRoleFor` (belt-and-braces, since a handful of legacy service-linked roles don't use the path convention).
- `analyzeAccessKeys(accessKeys = [])` — Output: `{ keys (enriched with ageDays/isOld/isUnused), activeKeyCount, hasOldActiveKey, hasUnusedActiveKey }`. A key is `isOld` if active AND >90 days old; `isUnused` if active, never used, AND > 14 days old (a 14-day grace period before flagging a brand-new key as "unused").
- `analyzeUserSecurity({ hasConsoleAccess, mfaEnabled, attachedPolicies, inlinePolicies, customerManagedPolicies, accessKeys, passwordLastUsed, createDate, tags, userName })` — the full per-user analysis. Key derived field: `consoleUserWithoutMFA = hasConsoleAccess && !mfaEnabled` (an API-only user with no console access is never flagged for missing MFA). Also computes admin-access, wildcard/excessive-permission flags (inspecting only inline + customer-managed policy documents — AWS-managed policy *content* is never inspected, it's pre-vetted by AWS and excluded before this function is ever called), never-logged-in, age, no-tags, long-name (>40 chars).
- `analyzeRoleSecurity({ ... })` — same shape minus MFA/console/access-key fields, plus `neverUsed` / `notUsedRecently` (>90 days since `lastUsedDate`).

IMPORTANT NOTES

`hasFullWildcardPermissions` vs `hasExcessivePermissions` is a real, meaningful distinction: the former requires BOTH `Action: "*" AND Resource: "*" in the same statement (true god-mode); the latter is a wildcard action without a wildcard resource (still bad, but scoped) — these map to HIGH and MEDIUM severity respectively in the recommender.`

 `server/src/services/recommender/iam.recommender.js`

(Already fully read and understood earlier in this session — see the codebase itself at `server/src/services/recommender/iam.recommender.js` for the complete 11-check implementation: console-MFA, root-MFA, root-access-keys, admin-access, wildcard-permissions, access-key-age, login-activity, role-usage, tags, long-name, age. All 11 are pure functions with a consistent `{severity, action, confidence, recommendation, reason, flagged}` shape, combined by the shared `mergeFindings` helper described above.)

Calculators

 `server/src/calculator/savings.calculator.js`

PURPOSE

The foundational cost-math functions shared across EC2 (directly) and RDS (via `rds.calculator.js`, which wraps `calculateMonthlyCost`).

WHO CALLS THIS FILE

`server/src/calculator/rds.calculator.js`, `server/src/services/scanner/ec2.scanner.js`

EXPORTED FUNCTIONS

- `calculateMonthlyCost(hourlyPrice)` — $\text{hourlyPrice} \times 24 \times 30$ (a flat 30-day month approximation, not actual calendar days).
- `calculateEstimatedSavings(monthlyCost, action)` — `STOP` → the full `monthlyCost`; `DOWNSIZE` → 50% of it; anything else (`NONE` / `UPSIZE` / `REVIEW`) → `0`.
- `calculateCostComparison(action, currentMonthlyCost, optimizedMonthlyCost)` — Output: `{ currentMonthlyCost, optimizedMonthlyCost, monthlySavings }`. `STOP` → optimized is always `0`. `UPSIZE` / `DOWNSIZE` → uses the caller-supplied `optimizedMonthlyCost` (looked up via a second pricing call for the adjacent instance type). Anything else → optimized equals current, `monthlySavings: 0`.

IMPORTANT NOTES

`calculateEstimatedSavings`'s 50% `DOWNSIZE` assumption is a flat heuristic, not derived from actual pricing of a smaller instance type — the separate `calculateCostComparison` function (used only for the UI's cost-comparison display, not for the `estimatedSavings` field saved to the database) is what uses real adjacent-instance-type pricing. This means `Recommendation.estimatedSavings` for a `DOWNSIZE` action is always exactly half the current cost, regardless of what the actual next-size-down instance costs.

 `server/src/calculator/rds.calculator.js`

PURPOSE

RDS-specific cost math — adds a storage-cost component on top of `savings.calculator.js`'s compute-cost math, since RDS (unlike EC2) bills storage separately from compute.

EXPORTED FUNCTIONS

- `calculateRDSMonthlyCost(hourlyPrice, allocatedStorageGB)` — $\text{calculateMonthlyCost}(\text{hourlyPrice}) + \text{allocatedStorageGB} \times 0.115$. The `0.115` /GB figure is a **flat fallback constant** (`RDS_STORAGE_FALLBACK_PRICE_PER_GB`, approximate gp2/us-east-1 rate) — RDS storage pricing isn't queried live via the Pricing API at all (unlike EC2/S3), because it splits across storage-type/engine/deployment-option combinations in a way that's much less reliably filterable via `GetProducts`.
- `calculateRDSEstimatedSavings(monthlyCost, savingsRate)` — $\text{monthlyCost} \times \text{savingsRate}$, rounded to 2 decimals. `savingsRate` comes from the CPU recommendation (`0.3` for idle/oversized, `0` otherwise).

IMPORTANT NOTES

Because storage cost uses a hardcoded rate rather than live pricing, RDS's `monthlyCost` figure is always an approximation for the storage portion — accurate for gp2 in low-cost regions, increasingly inaccurate for io1/io2 storage or expensive regions.

 `server/src/calculator/storage.calculator.js`

PURPOSE

S3-specific cost math: byte→GB conversion and a fallback Standard-tier price.

EXPORTED FUNCTIONS

- `calculateBucketSizeGB(bucketSizeBytes)` — divides by `1024^3` (GiB, matching how AWS's own cost calculators convert `BucketSizeBytes`), not decimal 10^9 .
- `calculateS3MonthlyCost(bucketSizeGB, pricePerGB)` — uses the live-fetched `pricePerGB` if `>0`, else falls back to the hardcoded `S3_STANDARD_FALLBACK_PRICE_PER_GB = 0.023` (AWS's well-known first-50TB/month Standard rate in us-east-1) — so a bucket always gets a non-zero cost estimate even if the live Pricing API call fails or the region isn't mappable.
- `calculateS3EstimatedSavings(monthlyCost, savingsRate)` — same shape as RDS's version.

AWS SDK wrapper layer (services/aws/*.service.js)

Every file in this folder is a thin wrapper around one AWS SDK v3 client — it only knows how to build a `Command` object, `.send()` it, and shape the response. **None of these files contain any analysis or recommendation logic** — that's a strict separation maintained throughout the codebase (AWS wrappers → mappers → analyzers → recommenders, never skipping a layer).

 `server/src/services/aws/cloudwatch.service.js`

PURPOSE

All CloudWatch `GetMetricData` calls used anywhere in the app — EC2 CPU, EC2 network+disk (6 metrics in one call), S3 storage (object count + 5 storage-class byte metrics in one call), RDS CPU.

EXPORTED FUNCTIONS

- `getCPUUtilization(cloudWatchClient, instanceId)` — `AWS/EC2` namespace, `CPUUtilization` metric, 1-hour period, `Average` stat, 7-day lookback window (shared by all 4 functions via `getSevenDayWindow()`). Returns the raw `Values` array (or `[]`). **Not wrapped in try/catch** — a failure here propagates up and (for EC2) fails the whole scan.
- `getS3StorageMetrics(cloudWatchClient, bucketName)` — one `GetMetricData` call with 6 queries: object count (`AllStorageTypes` dimension, a genuine complete total) + 5 storage-class-specific `BucketSizeBytes` queries (`StandardStorage`, `StandardIAStorage`, `OneZoneIAStorage`, `GlacierStorage`, `DeepArchiveStorage`), 24-hour period (S3's daily storage metrics only publish once/day). **Is try/caught** — returns all-zero defaults on failure, logged as a warning.
- `getNetworkAndDiskMetrics(cloudWatchClient, instanceId)` — 6 queries in one call: `NetworkIn/Out`, `DiskRead/WriteBytes`, `DiskRead/WriteOps`, all `AWS/EC2` namespace, 1-hour period. Try/caught, returns all-empty-arrays on failure. Note: Disk metrics only exist for instance-store-backed instances — the vast majority of EBS-backed instances will simply have empty arrays here always, which is expected, not a failure.
- `getRDSCPUUtilization(cloudWatchClient, dbInstanceIdentifier)` — `AWS/RDS` namespace, `CPUUtilization`, same 1-hour/7-day window. Try/caught, returns `[]` on failure.

IMPORTANT NOTES

The asymmetry in error handling (`getCPUUtilization` unguarded vs. the other three guarded) directly explains why an EC2 permissions gap fails the whole scan while an S3/RDS permissions gap degrades gracefully.

 `server/src/services/aws/ec2.service.js`

PURPOSE

EC2 SDK wrapper: list instances, stop an instance, list all EBS volumes, list all Elastic IPs.

EXPORTED FUNCTIONS

- `getEC2Instances(ec2Client)` — `DescribeInstancesCommand({})` (no filters — returns every instance in every state), then `mapEC2Instances(response)` normalizes AWS's deeply-nested `Reservations[].Instances[]` shape into a flat array.

- `stopEC2Instance(ec2Client, instanceId)` — `StopInstancesCommand({ InstanceIds: [instanceId] })`. This is the only *mutating* AWS call anywhere in the scanning pipeline — everything else is read-only. Called from `recommendation.service.js` when a user clicks "Stop Instance," not from any scanner.
- `getEBSVolumes(ec2Client)` — `DescribeVolumesCommand({})`, all volumes account-wide, fetched once per scan (not per instance) and grouped client-side by `ec2.scanner.js`.
- `getElasticIPAddresses(ec2Client)` — `DescribeAddressesCommand({})`, all allocated EIPs account-wide.

 `server/src/services/aws/rds.service.js`

PURPOSE

RDS SDK wrapper — a single function.

EXPORTED FUNCTIONS

- `getDBInstances(rdsClient)` — `DescribeDBInstancesCommand({})`, then `mapRDSInstances(response)`.

 `server/src/services/aws/s3.service.js`

PURPOSE

S3 SDK wrapper — 8 functions, one per bucket-level signal, each independently error-handled with a documented, deliberate fallback for the "can't tell" case. This file is the clearest example in the codebase of the "undeterminable signal defaults to the safe/non-alarming answer" philosophy.

EXPORTED FUNCTIONS

- `getBuckets(s3Client)` — `ListBucketsCommand({})` → `mapS3Buckets`.
- `getBucketRegion(s3Client, bucketName)` — `GetBucketLocationCommand`; an empty `LocationConstraint` means `us-east-1` (AWS's own quirk, handled explicitly); on error, returns `"unknown"`.
- `isBucketPublic(s3Client, bucketName)` — `GetPublicAccessBlockCommand`; public unless **all four** Block Public Access flags are enabled; if no config exists at all (`NoSuchPublicAccessBlockConfiguration`), that itself means nothing is blocking public access → returns `true` (public) — the one function in this file where "no config" is treated as the *risky* answer, not the safe one, because that's the factually correct interpretation of that specific error.
- `isVersioningEnabled(s3Client, bucketName)` — on any other error, defaults to `true` (assume enabled, avoid false alarm).
- `getLifecycleRules(s3Client, bucketName)` — `NoSuchLifecycleConfiguration` → `[]` (definitive, not an error); any other failure → `null` (genuinely undeterminable, handled specially by `analyzeLifecycleRules`).
- `getBucketEncryption(s3Client, bucketName)` — `ServerSideEncryptionConfigurationNotFoundError` → `{ encryptionEnabled: false, encryptionType: "None" }` (definitive); any other failure → `{ encryptionEnabled: true, encryptionType: "AES256" }` (assumes AWS's account-wide default-since-2023 SSE-S3 applies, avoiding a false alarm).
- `hasReplicationConfigured(s3Client, bucketName)` — `ReplicationConfigurationNotFoundError` → `false`; any other failure → `false`.
- `getBucketPolicy(s3Client, bucketName)` — `NoSuchBucketPolicy` → `null`, silently (most buckets legitimately have none, not logged as a warning); any other failure → `null`, logged.

IMPORTANT NOTES

`isBucketPublic`'s "no Block Public Access config = public" default is the one place in this file that defaults to the *alarming* answer rather than the safe one — worth remembering specifically because every other function in this file (and most of the codebase) defaults the other way.

 `server/src/services/aws/iam.service.js`

PURPOSE

IAM SDK wrapper — 14 functions covering users, roles, MFA, console access, attached/inline policies (including resolving inline policy documents and customer-managed policy documents), groups, access keys (with per-key last-used lookups), tags, and root account summary.

EXPORTED FUNCTIONS (SELECTED HIGHLIGHTS — FULL LIST OF 14 IN THE FILE)

- `getUsers / getRoles` — `ListUsers / ListRoles`, mapped via `iam.mapper.js`.
- `getRootAccountSecuritySummary(iamClient)` — `GetAccountSummaryCommand`; `SummaryMap` values are `1 / 0`, not real booleans (explicitly converted); on error, defaults to `{ mfaEnabled: true, hasAccessKeys: false }` (assume fine, avoid a false root-account alarm).
- `hasConsoleAccess(iamClient, userName)` — `GetLoginProfileCommand`; `NoSuchEntityException` → `false` (definitive: no login profile = no console access, an expected case for API-only users, not an error); any other failure → `false` too (can't tell → assume no console access → avoid a false "no MFA" alarm for a user who may not even have console access).
- `hasUserMFA` — on error, defaults to `true` (assume MFA present — avoid a false alarm).
- `getInlineUserPolicies / getInlineRolePolicies` — two-level: list policy names, then fetch each one's document individually (`GetUserPolicyCommand / GetRolePolicyCommand`) in parallel via `Promise.all`; a single policy's fetch failure returns `{ policyName, document: null }` for just that one policy rather than failing the whole user/role.
- `getUserAccessKeys` — lists keys, then for each one makes a separate `GetAccessKeyLastUsedCommand` call in parallel; a failure on any individual key's last-used lookup leaves `lastUsedDate: null` for just that key.
- `getPolicyDocument(iamClient, policyArn)` (*internal*) — 2-call resolution: `GetPolicyCommand` to find the current default version ID, then `GetPolicyVersionCommand` to fetch that version's actual document. Only ever called for customer-managed policies (AWS-managed policy content is pre-vetted and skipped).
- `getCustomerManagedPolicyDocuments(iamClient, attachedPolicies)` — filters to just customer-managed ARNs (via `isCustomerManagedPolicyArn`), fetches each document in parallel.

IMPORTANT NOTES

This is the "chattiest" AWS wrapper in the app by far — analyzing one IAM user with 2 inline policies and 3 access keys makes on the order of 10+ separate IAM API calls. IAM's API has notably low default rate limits compared to EC2/S3, so a very large IAM footprint (hundreds of users) is the most likely place a scan would hit AWS API throttling, though no explicit retry/backoff/rate-limiting is implemented anywhere in this file.

 `server/src/services/aws/pricing.service.js`

PURPOSE

Wraps the AWS Price List API (`@aws-sdk/client-pricing`, which — unusually — is a **global/single-region** service, always queried via `us-east-1` regardless of the resource's actual region; see `factories/awsClient.factory.js` for where that's configured) for EC2, S3, and RDS.

EXPORTED FUNCTIONS

- `getEC2Pricing(pricingClient, pricingFilters)` — `GetProductsCommand({ ServiceCode: "AmazonEC2", Filters: [...] })` with 6 `TERM_MATCH` filters: `instanceType`, `location`, `operatingSystem`, `tenancy`, `capacitystatus`, `preInstalledSw`. `MaxResults: 1`.
- `getS3Pricing` — `ServiceCode: "AmazonS3"`, filtered by `productFamily=Storage`, `volumeType`, `location`.
- `getRDSPricing` — `ServiceCode: "AmazonRDS"`, filtered by `instanceType`, `location`, `databaseEngine`, `deploymentOption`.

All three are **not** individually try/catched in this file — failures propagate to the caller (each scanner handles this differently: EC2/RDS don't guard the pricing call itself, but `extractHourlyPrice` in `pricing.parser.js` gracefully returns `0` for an empty/malformed response rather than throwing; S3's `getS3PricePerGB` in `s3.scanner.js` DOES wrap its pricing call in try/catch).

IMPORTANT NOTES

Every filter value comes from `builders/pricing.filter.builder.js`, which translates AWS API region codes (`ap-south-1`) into the Pricing API's human-readable location names (`"Asia Pacific (Mumbai)"`) via `utils/region.util.js` — a mismatch here (an unmapped region) silently results in zero pricing results, not an error, which is why every calculator has a hardcoded fallback rate.

 `server/src/services/aws/sts.service.js`

PURPOSE

Validates a set of AWS credentials by calling STS `GetCallerIdentity` — the "does this access key actually work" check used when a user connects their AWS account.

WHEN THIS FILE IS CALLED

From `awsConnection.service.js`, when a user submits credentials on the Connect AWS page, before they're saved.

WHO CALLS THIS FILE

`server/src/services/awsConnection/awsConnection.service.js`

WHICH FILES IT DEPENDS ON

`config/aws.js` — `createSTSCliet`

EXPORTED FUNCTIONS

- `validateAwsCredentials({ accessKeyId, secretAccessKey, region })` — builds a throwaway STS client with the candidate credentials, calls `GetCallerIdentityCommand({})`. On success: `{ valid: true, accountId: response.Account, arn: response.Arn }`. On **any** failure (bad credentials, no permissions, network error): `{ valid: false }` — no error detail is surfaced to the caller, just a boolean.

IMPORTANT NOTES

This is the only AWS call in the entire app that happens **before** any credentials are persisted — it's the gate that decides whether

`awsConnection.service.js` proceeds to encrypt and save the secret key at all.

APPLICATION BOOTSTRAP

app.js and the one file behind the health-check endpoint.

 server/src/app.js

PURPOSE

Assembles the Express application: CORS policy, JSON body parsing, mounts every route module at its API prefix, and installs the final `notFound` + `errorHandler` middleware pair. Exports the configured `app` — does **not** call `.listen()` (that's `server.js`'s job), which is what makes the app importable/testable independently of actually binding a port.

WHEN THIS FILE IS CALLED

Once, at process startup, required by `server.js`.

WHO CALLS THIS FILE

`server/src/server.js`

WHICH FILES IT DEPENDS ON

- `middleware/auth.middleware.js` — the `protect` function, applied per-route-group
- `middlewares/errorHandler.middleware.js`, `middlewares/notFound.middleware.js` — the final two middleware in the chain
- All 12 files in `routes/` — `analytics`, `auditLog`, `auth`, `aws`, `dashboard`, `health`, `notification`, `recommendation`, `report`, `scan`, `scans`, `schedule`
- External: `cors`, `express`

INTERNAL BEHAVIOR (IN EXACT ORDER)

- `const app = express();`
- Build `corsOrigins` from `process.env.CORS_ORIGIN` (comma-separated, trimmed) or default to `["http://localhost:5173"]`.
- `app.use(cors({ origin: corsOrigins }));` — restricts cross-origin requests to the allowlist (hardened from a previously wide-open `cors()` call).
- `app.use(express.json());` — JSON body parsing for every route.
- `app.use("/api/auth", authRoutes);` — mounted **without** `protect` at the `app.use` level (signup/login must be reachable unauthenticated); `auth.routes.js` self-protects only its `logout-all` endpoint internally.
- Every other route module is mounted **with** `protect` applied at the `app.use` level: `/api/health`, `/api/scan`, `/api/scans`, `/api/dashboard`, `/api/recommendations`, `/api/aws`, `/api/schedules`, `/api/notifications`, `/api/analytics`, `/api/reports`, `/api/audit-logs` — all 11 of these require a valid JWT for every single route inside them, including the health check.
- `app.use(notFound);` — catches any request that matched no route.
- `app.use(errorHandler);` — the final error-handling middleware; catches anything thrown/rejected anywhere upstream (via `asyncHandler`) and shapes it into a consistent JSON error response.
- `module.exports = app;`

DATA FLOW

Every incoming HTTP request passes through this file's middleware chain top-to-bottom exactly once: CORS check → JSON parse → route-group match (with `protect` applied first for every group except `/api/auth`) → the matched router's own internal route handling → (if nothing matched) `notFound` → (if anything threw) `errorHandler`.

EXAMPLE EXECUTION

A request to `GET /api/dashboard` with no `Authorization` header: CORS passes (same-origin or allowlisted) → JSON parsing is a no-op for a GET with no body → matches `app.use("/api/dashboard", protect, dashboardRoutes)` → `protect` runs first, finds no/invalid token, throws/responds 401 before `dashboardRoutes` ever sees the request.

IMPORTANT NOTES

`/api/health` being behind `protect` is unusual for a liveness/health-check endpoint — most infrastructure health checks (load balancer probes, uptime monitors) expect an unauthenticated `GET /health` to return 200, and this design would make any such external monitor fail with a 401 unless it's also configured to send a valid JWT. This is a real, documented design quirk worth knowing rather than assuming health checks work the conventional way.

 `server/src/services/health.service.js`

PURPOSE

The simplest file in the entire codebase — returns a static liveness payload for the health-check endpoint. Not a database ping, not an AWS connectivity check, just confirmation that the Node process itself is up and responding.

WHEN THIS FILE IS CALLED

On every `GET /api/health` request (which, per the note above, still requires a valid JWT to reach).

WHO CALLS THIS FILE

`server/src/controllers/health.controller.js`

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `healthService()` (default export, not a named export — the only file in the codebase exported this way)
- **Purpose:** Return a fixed "I'm alive" payload.
- **Input:** none.
- **Output:** `{ message: "Server is running", timestamp: new Date() }`.
- **Internal steps:** none — a single object literal returned immediately.
- **Database calls:** none.
- **AWS calls:** none.
- **Error handling:** none needed — cannot throw.

DATA FLOW

Called synchronously, returns immediately; the controller wraps this in `sendSuccess(res, healthService())`.

EXAMPLE EXECUTION

`healthService()` → `{ message: "Server is running", timestamp: 2026-08-01T12:00:00.000Z }`

IMPORTANT NOTES

This does **not** verify MongoDB connectivity, AWS reachability, or any downstream dependency — a "healthy" response here only proves the Express process is running and able to handle a request, not that the app is fully functional. A more thorough health check (e.g. a lightweight `mongoose.connection.readyState` check) would be a reasonable enhancement if this endpoint is ever wired up to real uptime monitoring.

API LAYER — CONTROLLERS & ROUTES

Every Express route and its controller — 24 files, one section each.

Dossier B2 — Express API Layer (Controllers + Routes)

Covers all 12 controller files in `server/src/controllers/` and all 12 route files in `server/src/routes/`. Every file was read in full. Cross-reference data (importers/imports) comes from the pre-built dependency graph at `dep_graph.json`.

Global context established from `server/src/app.js` (relevant to every route file below): all routers are mounted with the `protect` auth middleware (`server/src/middleware/auth.middleware.js`) applied at the `app.use(mountPath, protect, router)` call site, **except** `auth.routes.js`, which is mounted bare (`app.use("/api/auth", authRoutes)`) because it contains the signup/login endpoints that must be reachable without a token. Mount table:

ROUTER FILE	MOUNT PATH	PROTECTED AT MOUNT?
<code>auth.routes.js</code>	<code>/api/auth</code>	No (self-protects <code>logout-all</code> only)
<code>health.routes.js</code>	<code>/api/health</code>	Yes
<code>scan.routes.js</code>	<code>/api/scan</code>	Yes
<code>scans.routes.js</code>	<code>/api/scans</code>	Yes
<code>dashboard.routes.js</code>	<code>/api/dashboard</code>	Yes
<code>recommendation.routes.js</code>	<code>/api/recommendations</code>	Yes
<code>aws.routes.js</code>	<code>/api/aws</code>	Yes
<code>schedule.routes.js</code>	<code>/api/schedules</code>	Yes
<code>notification.routes.js</code>	<code>/api/notifications</code>	Yes
<code>analytics.routes.js</code>	<code>/api/analytics</code>	Yes
<code>report.routes.js</code>	<code>/api/reports</code>	Yes
<code>auditLog.routes.js</code>	<code>/api/audit-logs</code>	Yes

`protect` (in `server/src/middleware/auth.middleware.js`) reads the `Authorization: Bearer <token>` header, verifies the JWT with `verifyToken` (`server/src/Utils/jwt.js`, HMAC via `jsonwebtoken` using `process.env.JWT_SECRET`), then re-fetches the user by `decoded.userId` and compares `decoded.tokenVersion` against the DB's current `user.tokenVersion` (so a "logout all devices" call, which increments `tokenVersion`, retroactively invalidates every previously-issued token). On success it sets `req.user = { userId: decoded.userId }` — every controller in this dossier reads `req.user.userId` for tenant scoping. On failure it calls `sendError(res, 401, ...)` directly (bypasses `asyncHandler` / `errorHandler` — it's synchronous error dispatch, not a thrown exception).

Every controller in this codebase is wrapped in `asyncHandler` (`server/src/Utils/asyncHandler.js`) — `(handler) => (req,res,next) => Promise.resolve(handler(req,res,next)).catch(next)` — so any thrown error (including `ApiError` instances) is forwarded to the centralized `errorHandler` middleware (`server/src/middlewares/errorHandler.middleware.js`), which maps Mongoose `ValidationError` → 400, `CastError` → 400, Mongo duplicate-key `code 11000` → 409, `ApiError.statusCode` → that code, and anything else → 500 with the generic message `"Internal Server Error"` (the real error is `console.error`'d server-side only). Success responses in most (not all — see `auth.controller.js`) controllers use `sendSuccess(res, data, status = 200)` from `server/src/Utils/apiResponse.js`, which sends `{ success: true, data }`.

 `server/src/controllers/analytics.controller.js`

PURPOSE

Thin HTTP adapter exposing four read-only trend endpoints (cost, savings, resource growth, high-severity alerts over time) that feed the dashboard's chart components. Contains no business logic itself — every handler is a one-line delegation to `analytics.service.js`.

WHEN THIS FILE IS CALLED

On every incoming `GET` request to `/api/analytics/cost-trend`, `/api/analytics/savings-trend`, `/api/analytics/resource-growth`, or `/api/analytics/high-alerts-trend`, after the `protect` middleware has already run.

WHO CALLS THIS FILE

`server/src/routes/analytics.routes.js` imports and binds all four exports as route handlers.

WHICH FILES IT DEPENDS ON

- `server/src/services/analytics/analytics.service.js` — supplies the actual trend-computation logic (`getCostTrend`, `getSavingsTrend`, `getResourceGrowth`, `getHighAlertsTrend`).
- `server/src/utlis/apiResponse.js` — `sendSuccess` for the uniform `{success,data}` response envelope.
- `server/src/utlis/asyncHandler.js` — wraps each handler so a rejected promise reaches the centralized error middleware.

EXPORTED FUNCTIONS

- `costTrend(req, res)`
 - Purpose: Return monthly-cost-per-scan time series for the current user.
 - Input: no body/query/params used; reads `req.user.userId`.
 - Output: `res.json({ success: true, data })` status 200, where `data` is an array of `{ date, monthlyCost }`.
 - Internal steps: 1) call `getCostTrend(req.user.userId)`; 2) `sendSuccess(res, data)`.
 - Database calls: none directly (delegated to service → `analytics.repository.js`).
 - AWS calls: none.
 - Error handling: no local try/catch; relies on `asyncHandler` → `errorHandler`. The service never throws `ApiError`, so failures here surface as generic 500s only if something unexpected (e.g. a DB outage) happens.
 - HTTP status codes: 200 (only success path exercised in normal operation).
- `savingsTrend(req, res)`
 - Purpose: Return estimated-savings-per-scan time series.
 - Input: `req.user.userId` only.
 - Output: `{ success: true, data }`, `data` = array of `{ date, estimatedSavings }`.
 - Internal steps: call `getSavingsTrend(req.user.userId)`; `sendSuccess(res, data)`.
 - Database/AWS calls: none directly.
 - Error handling: same as above.
 - HTTP status: 200.
- `resourceGrowth(req, res)`
 - Purpose: Return total-resource-count-per-scan time series.
 - Output: `data` = array of `{ date, totalResources }`.
 - Internal steps / error handling: same pattern.
 - HTTP status: 200.
- `highAlertsTrend(req, res)`
 - Purpose: Return count of HIGH-severity recommendations per scan over time.
 - Output: `data` = array of `{ date, highAlerts }`.
 - Internal steps / error handling: same pattern.
 - HTTP status: 200.

ROUTE TABLE (SEE ANALYTICS.ROUTES.JS BELOW)

DATA FLOW

Request → `protect` (attaches `req.user.userId`) → controller extracts `userId` → calls matching `analytics.service.js` function → service queries `analytics.repository.js` (which reads the `Scan` and `Recommendation` Mongo collections, filtered to `userId` and typically `status: "COMPLETED"`) → arrays are returned up the chain unmodified → `sendSuccess` wraps and serializes as JSON.

EXAMPLE EXECUTION

Request: `GET /api/analytics/savings-trend` with `Authorization: Bearer <jwt>`. Response: `200 { "success": true, "data": [ { "date": "2026-07-20T10:00:00.000Z", "estimatedSavings": 42.5 }, { "date": "2026-07-27T10:00:00.000Z", "estimatedSavings": 58.1 } ] }`.

IMPORTANT NOTES

- All four handlers are structurally identical; a new engineer adding a fifth trend only needs to add one line here plus the corresponding service/route entry.
- No pagination, date-range filtering, or query-param support — every completed scan for the user is returned every time.
- If the user has zero scans, `getCostTrend` / `getHighAlertsTrend` return `[]` explicitly (early-return guard in the service); `getSavingsTrend` / `getResourceGrowth` also naturally return `[]` via `.map()` over an empty array.

 `server/src/controllers/auth.controller.js`

PURPOSE

Handles the three unauthenticated-or-self-authenticated identity endpoints: account creation (`signup`), credential exchange for a JWT (`login`), and global session invalidation (`logoutAll`). Owns all request-body validation for these flows (email format, password strength, name length) before delegating to `auth.service.js`.

WHEN THIS FILE IS CALLED

`signup` / `login` on every `POST /api/auth/signup` / `POST /api/auth/login` (no auth required — `auth.routes.js` is mounted without `protect`). `logoutAll` on `POST /api/auth/logout-all`, which the route itself gates with `protect`.

WHO CALLS THIS FILE

`server/src/routes/auth.routes.js` imports `{ signup, login, logoutAll }` and binds them to routes.

WHICH FILES IT DEPENDS ON

- `server/src/constants/auditActions.js` — `AUDIT_ACTIONS.USER_LOGIN` constant used when recording the login audit entry.
- `server/src/services/auditLog/auditLog.service.js` — `recordAuditLog` to log successful logins.
- `server/src/services/auth/auth.service.js` — `registerUser`, `loginUser`, `logoutAllDevices` (the actual DB/bcrypt/JWT logic).
- `server/src/utils/ApiError.js` — thrown for every validation failure (all with status 400).
- `server/src/utils/apiResponse.js` — `sendSuccess`, used only by `logoutAll`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.
- `server/src/utils/validators.js` — `isValidEmail`, `isValidPassword`, `isValidFullName` regex-based checks.

EXPORTED FUNCTIONS

- `signup(req, res)`
- Purpose: Create a new user account.
- Input: `req.body = { fullName, email, password }`.
- Output: `res.status(201).json({ success: true, user })` — **note: not wrapped via `sendSuccess`**, so the envelope is `{success, user}`, not `{success, data}`.
- Internal steps: 1) destructure `fullName, email, password`; 2) if any is falsy, throw `ApiError(400, "Full name, email and password are required")`; 3) validate `fullName` via `isValidFullName` (2–100 chars after trim) else `ApiError(400, "Full name must be between 2 and 100 characters")`; 4) validate `email` via `isValidEmail` (regex `/^[^s@]+@[^s@]+\.[^s@]+$/` , ≤254 chars) else `ApiError(400, "Please provide a valid email address")`; 5) validate `password` via `isValidPassword` (regex requires ≥1 letter, ≥1 digit, 8–128 chars) else `ApiError(400, "Password must be at least 8 characters and include a letter and a number")`; 6) call `registerUser({ fullName: fullName.trim(), email: email.trim().toLowerCase(), password })`; 7) respond 201 with the created user.
- Database calls: `registerUser` → `findUserByEmail` (duplicate check) then `createUser` (both in `user.repository.js`, backed by the `User` model, which bcrypt-hashes the password with `SALT_ROUNDS = 10` before this call already applied — actually hashing happens in `auth.service.js` itself via `bcrypt.hash`).
- AWS calls: none.
- Error handling: `ApiError(400, ...)` for every validation branch above; `auth.service.registerUser` itself throws `ApiError(409, "Email is already registered")` if `findUserByEmail` finds a match. All forwarded via `asyncHandler`.
- HTTP status codes: 201 (success), 400 (missing/invalid fields), 409 (duplicate email).
- `login(req, res)`
- Purpose: Authenticate a user and issue a JWT.
- Input: `req.body = { email, password }`.

- Output: `res.status(200).json({ success: true, token, user })` — again **not** via `sendSuccess`; top-level `token` / `user` fields, no `data` wrapper.
- Internal steps: 1) destructure `email, password`; 2) if either missing, throw `ApiError(400, "Email and password are required")`; 3) validate email format else `ApiError(400, "Please provide a valid email address")` (password strength is *not* re-checked on login, only presence); 4) call `loginUser({ email: email.trim().toLowerCase(), password })` → `{ token, user }`; 5) `recordAuditLog(user._id, AUDIT_ACTIONS.USER_LOGIN, `${user.email} logged in`)`; 6) respond 200 with `{success, token, user}`.
- Database calls: `loginUser` → `findUserByEmail`; `recordAuditLog` → `createAuditLog` (fire-and-forget, swallows its own errors).
- AWS calls: none.
- Error handling: `ApiError(400, ...)` for missing/invalid input; `auth.service.loginUser` throws `ApiError(401, "Invalid email or password")` both when no user is found and when `bcrypt.compare` fails (deliberately identical message/status in both cases to avoid leaking which part was wrong).
- HTTP status codes: 200 (success), 400 (missing/invalid email), 401 (bad credentials), 429 (only via the `loginRateLimiter` middleware on the route, not this controller — 10 attempts/15 min per IP).
- **logoutAll(req, res)**
- Purpose: Invalidate every JWT previously issued to the current user (global logout).
- Input: no body; requires `req.user.userId` (route applies `protect`).
- Output: `sendSuccess(res, { loggedOutAllDevices: true })` → `{ success: true, data: { loggedOutAllDevices: true } }`, status 200.
- Internal steps: 1) call `logoutAllDevices(req.user.userId)`; 2) `sendSuccess`.
- Database calls: `logoutAllDevices` → `User.findByIdAndUpdate(userId, { $inc: { tokenVersion: 1 } }, { new: true })` directly on the `User` model (not through `user.repository.js`).
- AWS calls: none.
- Error handling: `logoutAllDevices` throws `ApiError(404, "User not found")` if the update returns null (practically unreachable since `protect` already resolved the user for this same request).
- HTTP status codes: 200 (success), 404 (user vanished mid-request — edge case).

ROUTE TABLE (SEE AUTH.ROUTES.JS BELOW)

DATA FLOW

`signup` / `login`: request body → controller-side field validation (throws `ApiError` on first failure) → `auth.service.js` (bcrypt hash/compare, JWT signing via `jwt.js`) → `user.repository.js` → `User` Mongo model → response built manually (not `sendSuccess`) and returned.

`logoutAll`: `req.user.userId` (from `protect`) → `auth.service.logoutAllDevices` → direct `User.findByIdAndUpdate` bump of `tokenVersion` → `sendSuccess`.

EXAMPLE EXECUTION

Request: `POST /api/auth/login` body `{ "email": "user@example.com", "password": "Password123" }`. Response (success): `200 { "success": true, "token": "eyJhbGciOi...", "user": { "_id": "...", "fullName": "...", "email": "user@example.com", ... } }` (password field excluded via the `User` model's `toJSON` transform). Response (bad password): `401 { "success": false, "message": "Invalid email or password" }`.

IMPORTANT NOTES

- **Response-shape inconsistency:** `signup` and `login` bypass `sendSuccess` and hand-roll `res.status(...).json(...)`, so their payloads are flat (`{success,user}` / `{success,token,user}`) while almost every other endpoint in the codebase returns `{success,data}`. A frontend client consuming this API needs to special-case these two calls.
- Password strength (`isValidPassword`) is only enforced at signup, not at login — a user whose password was valid when created will always be able to log in even if the policy regex changes later; this is expected but worth knowing.
- `login`'s rate limiting (`loginRateLimiter` , 10 requests/15 min/IP) is applied in the `route` file, not visible from this controller alone.
- Login and "invalid password" and "no such user" deliberately return the exact same `401` message — a timing/enumeration-safety choice, not a bug.

 `server/src/controllers/auditLog.controller.js`

PURPOSE

Single-endpoint controller that returns the authenticated user's audit log history (login, AWS connect/disconnect, scan start/complete, recommendation actions, etc.).

WHEN THIS FILE IS CALLED

On every `GET /api/audit-logs` request, after `protect`.

WHO CALLS THIS FILE

`server/src/routes/auditLog.routes.js` binds `getAuditLogs` to `GET /`.

WHICH FILES IT DEPENDS ON

- `server/src/services/auditLog/auditLog.service.js` — `fetchAuditLogs(userId)`, the actual DB read.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `getAuditLogs(req, res)`
- Purpose: Return all audit log entries for the current user.
- Input: `req.user.userId` only; no query params (no filtering/pagination supported).
- Output: `{ success: true, data: logs }`, `logs` an array of audit log documents (each with `userId`, `action`, `description`, timestamps per the `AuditLog` model).
- Internal steps: 1) `fetchAuditLogs(req.user.userId)`; 2) `sendSuccess(res, logs)`.
- Database calls: `auditLog.service.js` → `auditLog.repository.js` → `AuditLog` Mongoose model, `find`-style query scoped by `userId`.
- AWS calls: none.
- Error handling: `asyncHandler` only; no explicit throws in this path.
- HTTP status codes: 200 only.

ROUTE TABLE (SEE AUDITLOG.ROUTES.JS BELOW)

DATA FLOW

Request → `protect` → `getAuditLogs` reads `req.user.userId` → `fetchAuditLogs` queries Mongo → array returned → `sendSuccess` serializes.

EXAMPLE EXECUTION

Request: `GET /api/audit-logs`. Response: `200 { "success": true, "data": [ { "_id": "...", "userId": "...", "action": "SCAN_COMPLETED", "description": "Scan completed - 42 resources scanned, $12.50 potential savings identified", "createdAt": "2026-07-31T09:00:00.000Z" } ] }`.

IMPORTANT NOTES

- This is a strictly read-only file (14 lines) — there is no controller-side write path for audit logs; entries are created exclusively by `recordAuditLog` calls scattered through other controllers (auth, awsConnection, recommendation, scan).
- No date filtering or limit — every historical entry for the user comes back in one response, which could grow unbounded over the account's lifetime.

 `server/src/controllers/awsConnection.controller.js`

PURPOSE

Manages the single AWS account connection each user can have: validating raw credentials without saving them (`testConnection`), persisting a validated connection (`connect`), reading the current connection (`getAccount`), and removing it (`disconnect`). This is the credential-onboarding surface for the whole product — every scan depends on a connection existing.

WHEN THIS FILE IS CALLED

On `POST /api/aws/test-connection`, `POST /api/aws/connect`, `GET /api/aws/account`, `DELETE /api/aws/disconnect` — all behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/aws.routes.js` imports `{ testConnection, connect, getAccount, disconnect }` and binds them.

WHICH FILES IT DEPENDS ON

- `server/src/constants/auditActions.js` — `AUDIT_ACTIONS.AWS_CONNECTED` / `AWS_DISCONNECTED`.
- `server/src/services/auditLog/auditLog.service.js` — `recordAuditLog`.
- `server/src/services/awsConnection/awsConnection.service.js` — all the actual STS validation / persistence logic.
- `server/src/utils/ApiError.js` — thrown for missing required fields.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `testConnection(req, res)`
 - Purpose: Verify a set of AWS credentials works (via STS `GetCallerIdentity`) without persisting anything.
 - Input: `req.body = { region, accessKeyId, secretAccessKey }`.
 - Output: `sendSuccess(res, result) → { success: true, data: { accountId, arn, region } }`, status 200.
 - Internal steps: 1) destructure body fields; 2) if any of `region` / `accessKeyId` / `secretAccessKey` is falsy, throw `ApiError(400, "Region, Access Key ID and Secret Access Key are required")`; 3) call `testAwsConnection({ region, accessKeyId, secretAccessKey })`; 4) `sendSuccess`.
 - Database calls: none (this handler never touches Mongo).
 - AWS calls: `testAwsConnection` → `validateAwsCredentials` (`sts.service.js`) → `createSTSCliant` + `GetCallerIdentityCommand` (`@aws-sdk/client-sts`).
 - Error handling: `ApiError(400, ...)` for missing fields; `awsConnection.service.testAwsConnection` throws `ApiError(400, "Invalid AWS credentials. Please check your Access Key ID, Secret Access Key, and Region.")` if STS validation fails (caught internally by `validateAwsCredentials`, which returns `{valid:false}` rather than throwing, so this is a normal control-flow branch, not an exception bubbling from the SDK).
 - HTTP status codes: 200 (valid credentials), 400 (missing fields or invalid credentials).
- `connect(req, res)`
 - Purpose: Persist a validated AWS connection for the current user (one per user, enforced).
 - Input: `req.body = { region, accessKeyId, secretAccessKey, accountAlias }` (`accountAlias` optional).
 - Output: `sendSuccess(res, connection, 201) → { success: true, data: connection }`, status 201. `connection` is the created `AwsConnection` document (as a plain object) with `encryptedSecretKey` stripped.
 - Internal steps: 1) validate the three required fields, same `ApiError(400, ...)` as `testConnection`; 2) call `connectAwsAccount({ userId: req.user.userId, region, accessKeyId, secretAccessKey, accountAlias })`; 3) `recordAuditLog(req.user.userId, AUDIT_ACTIONS.AWS_CONNECTED, "AWS account ${connection.accountId} (${connection.region}) connected")`; 4) `sendSuccess(res, connection, 201)`.
 - Database calls: `connectAwsAccount` → `AwsConnection.findOne({userId})` (uniqueness check) then `AwsConnection.create({...})` with `accessKeyId` stored in **plaintext** and `secretAccessKey` stored **encrypted** (`encrypt()` from `utils/encryption.js`) as `encryptedSecretKey`.
 - AWS calls: `connectAwsAccount` re-validates via `validateAwsCredentials` (STS `GetCallerIdentityCommand`) before persisting.
 - Error handling: `ApiError(400, ...)` for missing fields; service throws `ApiError(409, "An AWS account is already connected. Disconnect it before connecting a new one.")` if a connection already exists for the user; `ApiError(400, "Invalid AWS credentials...")` if STS validation fails.
 - HTTP status codes: 201 (created), 400 (missing fields or invalid credentials), 409 (already connected).
- `getAccount(req, res)`
 - Purpose: Return the current user's AWS connection metadata (for the "Connected Account" UI).
 - Input: `req.user.userId` only.
 - Output: `sendSuccess(res, connection) → { success: true, data: connection }`. `connection` may be `null` if none exists.
 - Internal steps: 1) `getConnectionByUserId(req.user.userId)`; 2) `sendSuccess`.
 - Database calls: `AwsConnection.findOne({userId}).select("-encryptedSecretKey")`.
 - AWS calls: none.
 - Error handling: none — no null check, no 404. A user with no connection gets `200 {success:true, data:null}`.
 - HTTP status codes: 200 only (even when there is no connection).
- `disconnect(req, res)`
 - Purpose: Remove the current user's AWS connection.
 - Input: `req.user.userId` only.

- Output: `sendSuccess(res, { disconnected: true })`, status 200.
- Internal steps: 1) `disconnectAwsAccount(req.user.userId)` → deleted connection doc; 2) `recordAuditLog(req.user.userId, AUDIT_ACTIONS.AWS_DISCONNECTED, "AWS account ${connection.accountId} disconnected")`; 3) `sendSuccess(res, { disconnected: true })`.
- Database calls: `AwsConnection.findOneAndDelete({userId})`.
- AWS calls: none.
- Error handling: service throws `ApiError(404, "No AWS account is connected.")` if nothing was found to delete.
- HTTP status codes: 200 (deleted), 404 (nothing to disconnect).

ROUTE TABLE (SEE AWS.ROUTES.JS BELOW)

DATA FLOW

`testConnection` / `connect`: request body → field presence check → `awsConnection.service.js` → `sts.service.js` (AWS STS `GetCallerIdentity`, live network call to AWS) → on success, `connect` additionally writes to Mongo (secret encrypted at rest) → audit log recorded → response. `getAccount` / `disconnect`: `req.user.userId` → direct Mongo read/delete on `AwsConnection` → (disconnect only) audit log → response.

EXAMPLE EXECUTION

Request: `POST /api/aws/connect` body `{ "region": "us-east-1", "accessKeyId": "AKIA...", "secretAccessKey": "wJalr...", "accountAlias": "Prod" }`. Response: `201 { "success": true, "data": { "_id": "...", "userId": "...", "accountId": "123456789012", "accountAlias": "Prod", "region": "us-east-1", "accessKeyId": "AKIA...", "status": "CONNECTED", "connectedAt": "2026-08-01T...", "lastValidatedAt": "2026-08-01T..." } }` (no `encryptedSecretKey` field in the response).

IMPORTANT NOTES

- `getAccount` returns `200` with `data: null` rather than a `404` when no connection exists — inconsistent with `disconnect`, which does `404` in the equivalent "nothing there" case. A frontend must null-check the `data` field rather than relying on status code.
- `connect` calls STS **twice** in total across a typical "test then connect" UI flow (once in `testConnection`, once again inside `connectAwsAccount` itself) — `connect` does not trust a prior `testConnection` call and always re-validates.
- The raw `accessKeyId` is stored and returned in plaintext (only `secretAccessKey` is encrypted) — anyone with API/DB read access sees the access key ID.
- Enforces exactly one AWS connection per user (409 on a second `connect`); there is no "update connection" endpoint — the only way to change credentials is `disconnect` then `connect` again.

 `server/src/controllers/dashboard.controller.js`

PURPOSE

Serves the two data feeds behind the main dashboard page: an aggregated summary object (`getDashboard`) and the most recent set of recommendations (`getLatestRecommendations`).

WHEN THIS FILE IS CALLED

On `GET /api/dashboard/` and `GET /api/dashboard/recommendations`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/dashboard.routes.js` binds `{ getDashboard, getLatestRecommendations }`.

WHICH FILES IT DEPENDS ON

- `server/src/services/dashboard/dashboard.service.js` — `getDashboardData`, `getRecommendations` (the aggregation logic).
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `getDashboard(req, res)`
- Purpose: Return the full dashboard summary payload.
- Input: `req.user.userId` only.

- Output: `sendSuccess(res, dashboard)` status 200. `dashboard` is the merged object produced by `dashboard.service.getDashboardData` : the spread of `getDashboardSummary` (EC2-era resource/recommendation counts) plus `insights`, `ec2Stats`, `iamStats`, `categoryStats`, and the spread of `getGenericDashboardInsights` (`serviceSummary`, `severitySummary`, `costBreakdown`, `resourceDistribution`, `savingsOpportunities`, `aiInsights`).
- Internal steps: 1) `getDashboardData(req.user.userId)` ; 2) `sendSuccess` .
- Database calls: none directly — `getDashboardData` runs six queries in parallel via `Promise.all` against `dashboard.repository.js`, `insight.repository.js`, and `serviceAggregation.service.js` (which itself queries `Scan`, `Recommendation`, `Resource`).
- AWS calls: none (this reads previously-scanned/stored data only; no live AWS calls happen on dashboard load).
- Error handling: `asyncHandler` only; the service issues no `ApiError` in this path.
- HTTP status codes: 200 only.
- `getLatestRecommendations(req, res)`
- Purpose: Return the latest recommendation set for the current user (used for a dashboard widget, distinct from the full Recommendations page).
- Input: `req.user.userId` only.
- Output: `sendSuccess(res, recommendations)`, status 200.
- Internal steps: 1) `getRecommendations(req.user.userId)` (which just calls `getLatestRecommendations` from `recommendation.repository.js`); 2) `sendSuccess` .
- Database calls: `recommendation.repository.js` query scoped to `userId` .
- AWS calls: none.
- Error handling: `asyncHandler` only.
- HTTP status codes: 200 only.

ROUTE TABLE (SEE DASHBOARD.ROUTES.JS BELOW)

DATA FLOW

Request → `req.user.userId` → `dashboard.service.js` fans out to multiple repositories/services in parallel (`Promise.all`) → results merged into one object (or returned directly for the recommendations endpoint) → `sendSuccess` .

EXAMPLE EXECUTION

Request: `GET /api/dashboard/` . Response: `200 { "success": true, "data": { "totalResources": 42, "totalRecommendations": 7, "insights": [...], "ec2Stats": {...}, "iamStats": {...}, "categoryStats": {...}, "serviceSummary": {...}, "severitySummary": {...}, "costBreakdown": {...}, "resourceDistribution": {...}, "savingsOpportunities": {...}, "aiInsights": [...] } }` (exact summary fields come from `dashboard.repository.getDashboardSummary`, not read in this dossier's scope).

IMPORTANT NOTES

- Naming collision: the controller function `getLatestRecommendations` calls a service function also named `getRecommendations` (aliased on import as `{ getDashboardData, getRecommendations }`), which itself calls a repository function named `getLatestRecommendations` — three different functions across three files share overlapping names, which can be confusing when tracing a stack trace.
- No caching — every dashboard load re-runs all six queries/aggregations synchronously within the request.

 `server/src/controllers/health.controller.js`

PURPOSE

Minimal liveness-check endpoint returning a static "server is running" payload with a timestamp.

WHEN THIS FILE IS CALLED

On `GET /api/health/`, but note this route is mounted **behind** `protect` in `app.js` (`app.use("/api/health", protect, healthRoutes)`) — unlike a typical public health check.

WHO CALLS THIS FILE

`server/src/routes/health.routes.js` binds `checkHealth` to `GET /` .

WHICH FILES IT DEPENDS ON

- `server/src/services/health.service.js` — the synchronous function that builds the payload.
- `server/src/utils/apiResponse.js` — `sendSuccess` .

- `server/src/utils/asyncHandler.js` — error-forwarding wrapper (unnecessary here since `healthService()` cannot reject, but applied uniformly).

EXPORTED FUNCTIONS

- `checkHealth(req, res)`
- Purpose: Confirm the API process is up and responding.
- Input: none (no body/params/query; still requires a valid JWT due to route-level `protect`).
- Output: `sendSuccess(res, data) → { success: true, data: { message: "Server is running", timestamp: <Date> } }`, status 200.
- Internal steps: 1) call `healthService()` (synchronous, not awaited even though wrapped in an async handler — works fine since it returns a plain object, not a Promise); 2) `sendSuccess`.
- Database calls: none.
- AWS calls: none.
- Error handling: `asyncHandler` wrapper present but effectively a no-op since `healthService` never throws or rejects.
- HTTP status codes: 200 only.

ROUTE TABLE (SEE HEALTH.ROUTES.JS BELOW)

DATA FLOW

Request → (auth check via `protect`) → `checkHealth` → `healthService()` builds `{message, timestamp}` synchronously → `sendSuccess`.

EXAMPLE EXECUTION

Request: `GET /api/health/` with valid `Authorization` header. Response: `200 { "success": true, "data": { "message": "Server is running", "timestamp": "2026-08-01T12:00:00.000Z" } }`.

IMPORTANT NOTES

- **Requires authentication** — this is not a typical infra/load-balancer health check endpoint since it demands a valid JWT (and a DB lookup inside `protect` to check `tokenVersion`). A monitoring probe without a token gets `401`, not `200`, which is likely not what's intended for uptime monitoring; worth flagging to whoever owns infra health checks.
- Does not check DB connectivity, AWS reachability, or any dependency — it is purely "the Node process is alive and a token check passed."

`server/src/controllers/notification.controller.js`

PURPOSE

Read/update endpoints for a user's email-notification preferences (scan-completed alerts, high-severity alerts, weekly report opt-in/out).

WHEN THIS FILE IS CALLED

On `GET /api/notifications/settings` and `PATCH /api/notifications/settings`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/notification.routes.js` binds `{ getSettings, updateSettings }`.

WHICH FILES IT DEPENDS ON

- `server/src/services/notification/notification.service.js` — `getOrCreateSettings`, `editSettings`.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `getSettings(req, res)`
- Purpose: Return the current user's notification settings, auto-creating a default row if none exists yet.
- Input: `req.user.userId` only.
- Output: `sendSuccess(res, settings)`, status 200. `settings` includes at minimum `{userId, email, scanCompleted, highSeverity, weeklyReport}`.
- Internal steps: 1) `getOrCreateSettings(req.user.userId)` — internally: look up existing settings by `userId`; if found return as-is; otherwise look up the `User`, and if missing throw 404, else `createNotificationSettings` with defaults `scanCompleted: true, highSeverity: true, weeklyReport: true` and `email` copied from the user record; 2) `sendSuccess`.

- Database calls: `notification.repository.js` (`getNotificationSettingsByUserId` , `createNotificationSettings`) and `user.repository.js` (`findUserById` , only on first-ever call).
- AWS calls: none.
- Error handling: service throws `ApiError(404, "User not found")` if the user record is missing when auto-creating (practically unreachable given `protect` already resolved the same user).
- HTTP status codes: 200 (existing or freshly-created settings), 404 (edge case above).
- `updateSettings(req, res)`
- Purpose: Patch one or more notification preference fields.
- Input: `req.body = { email, scanCompleted, highSeverity, weeklyReport }` — all optional; only defined fields are applied.
- Output: `sendSuccess(res, settings)` , status 200, the updated settings document.
- Internal steps: 1) destructure the four possible fields from `req.body` ; 2) call `editSettings(req.user.userId, {email, scanCompleted, highSeverity, weeklyReport})` , which itself calls `getOrCreateSettings` first (so a `PATCH` before any `GET` still works and auto-provisions), then builds an `updates` object containing only the fields that are `!== undefined` , then calls `updateNotificationSettings` ; 3) `sendSuccess` .
- Database calls: same repository functions as above, plus `updateNotificationSettings` .
- AWS calls: none.
- Error handling: inherits the same 404 edge case from `getOrCreateSettings` ; no validation on the *values* of `email` /booleans (e.g. an invalid email string would be stored as-is, no format check here unlike signup).
- HTTP status codes: 200, 404 (edge case).

ROUTE TABLE (SEE NOTIFICATION.ROUTES.JS BELOW)

DATA FLOW

Request → `req.user.userId` (+ optional body for PATCH) → `notification.service.js` → `notification.repository.js` / `user.repository.js` → Mongo `NotificationSettings` -equivalent collection (backed by `Notification` model) → `sendSuccess` .

EXAMPLE EXECUTION

Request: `PATCH /api/notifications/settings` body `{ "weeklyReport": false }` . Response: `200 { "success": true, "data": { "userId": "...", "email": "user@example.com", "scanCompleted": true, "highSeverity": true, "weeklyReport": false } }` .

IMPORTANT NOTES

- `updateSettings` performs no validation on `email` format — unlike `auth.controller.js` 's `signup` , an arbitrary string could be saved as the notification email.
- The "auto-create on first access" behavior means there is no explicit `POST` for notification settings; both `GET` and `PATCH` transparently provision a row if one is missing.

 `server/src/controllers/recommendation.controller.js`

PURPOSE

Serves the Recommendations page: listing all recommendations for the user and handling the three actions a user can take on one — ignore it, mark it resolved, or (for EC2 right-sizing/idle recommendations) actually stop the underlying EC2 instance via a live AWS call and then resolve it.

WHEN THIS FILE IS CALLED

On `GET /api/recommendations/` , `PATCH /api/recommendations/:id/ignore` , `PATCH /api/recommendations/:id/resolve` , `POST /api/recommendations/:id/stop` , behind `protect` .

WHO CALLS THIS FILE

`server/src/routes/recommendation.routes.js` binds `{ getRecommendations, ignoreRecommendation, resolveRecommendation, stopRecommendation }` .

WHICH FILES IT DEPENDS ON

- `server/src/services/recommendation/recommendation.service.js` — `fetchRecommendations` , `ignoreRecommendationById` , `resolveRecommendationById` , `stopRecommendationById` (all the mutation/AWS logic).
- `server/src/services/auditLog/auditLog.service.js` — `recordAuditLog` .
- `server/src/constants/auditActions.js` — `AUDIT_ACTIONS.RECOMMENDATION_IGNORED` / `RECOMMENDATION_RESOLVED` / `EC2_STOPPED` .

- `server/src/utils/apiResponse.js` — `sendSuccess` .
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `describeRecommendation(recommendation)` (internal helper, not exported)
- Purpose: Build a human-readable label for audit-log descriptions.
- Returns `recommendation.resourceName || recommendation.resourceId` .
- `getRecommendations(req, res)`
- Purpose: Return all current recommendations for the user.
- Input: `req.user.userId` only.
- Output: `sendSuccess(res, recommendations)` , status 200, array of recommendation documents.
- Internal steps: `fetchRecommendations(req.user.userId)` → `sendSuccess` .
- Database calls: `recommendation.repository.getLatestRecommendations` .
- AWS calls: none.
- Error handling: `asyncHandler` only.
- HTTP status codes: 200.
- `ignoreRecommendation(req, res)`
- Purpose: Mark a recommendation as ignored (dismissed without action).
- Input: `req.params.id` (recommendation ID), `req.user.userId` .
- Output: `sendSuccess(res, recommendation)` , status 200, the updated/enriched recommendation.
- Internal steps: 1) `ignoreRecommendationById(req.params.id, req.user.userId)` — updates the doc's status and calls `enrichRecommendationWithResource` to attach live resource data (e.g. current `state`); 2) `recordAuditLog(req.user.userId, AUDIT_ACTIONS.RECOMMENDATION_IGNORED, "Recommendation ignored: ${describeRecommendation(recommendation)}")` ; 3) `sendSuccess` .
- Database calls: `recommendation.repository.ignoreRecommendation` (update scoped by `id` + `userId`), plus the enrichment lookup.
- AWS calls: none.
- Error handling: service throws `ApiError(404, "Recommendation not found")` if the update matched nothing (wrong ID or belongs to another user — `userId` scoping doubles as tenant isolation and existence check).
- HTTP status codes: 200, 404.
- `resolveRecommendation(req, res)`
- Purpose: Mark a recommendation as resolved (issue addressed).
- Input/Output/steps: identical pattern to `ignoreRecommendation` , using `resolveRecommendationById` and `AUDIT_ACTIONS.RECOMMENDATION_RESOLVED` with message `"Recommendation resolved: ${...}"` .
- Database calls: `recommendation.repository.resolveRecommendation` + enrichment.
- AWS calls: none.
- Error handling: `ApiError(404, "Recommendation not found")` .
- HTTP status codes: 200, 404.
- `stopRecommendation(req, res)`
- Purpose: For an EC2-related recommendation, actually stop the live EC2 instance via AWS, then resolve the recommendation.
- Input: `req.params.id` , `req.user.userId` .
- Output: `sendSuccess(res, recommendation)` , status 200, the resolved+enriched recommendation.
- Internal steps: 1) `stopRecommendationById(req.params.id, req.user.userId)` , which internally: looks up the recommendation (404 if missing), enriches it with live resource state and throws `ApiError(400, "This instance is already stopped.")` if already stopped, creates an EC2 client via `createEC2Client()` , calls `stopEC2Instance(ec2Client, recommendation.resourceId)` (AWS `StopInstancesCommand`), updates the local `Resource` doc's state to `"stopped"` via `updateResourceState` , then calls `resolveRecommendation` and re-enriches; 2) back in the controller, `recordAuditLog(req.user.userId, AUDIT_ACTIONS.EC2_STOPPED, "EC2 instance stopped: ${describeRecommendation(recommendation)}")` — a code comment explicitly notes only `EC2_STOPPED` is logged here, not also `RECOMMENDATION_RESOLVED` , so one click doesn't produce two audit entries; 3) `sendSuccess` .
- Database calls: `recommendation.repository` (`getRecommendationById` , `resolveRecommendation` , `enrichRecommendationWithResource`), `resource.repository.updateResourceState` .
- AWS calls: `@aws-sdk/client-ec2` `StopInstancesCommand` via `ec2.service.stopEC2Instance` , using a client built from the user's stored (decrypted) credentials via `createEC2Client()` .

- Error handling: `ApiError(404, "Recommendation not found");` `ApiError(400, "This instance is already stopped.");` any unhandled AWS SDK error (e.g. permissions failure, instance already terminated) propagates uncaught to the generic 500 path in `errorHandler` since it isn't wrapped in a try/catch or converted to an `ApiError`.
- HTTP status codes: 200, 400 (already stopped), 404 (not found), 500 (uncaught AWS SDK errors).

ROUTE TABLE (SEE RECOMMENDATION.ROUTES.JS BELOW)

DATA FLOW

`GET : userId` → repository read → response. `ignore / resolve : id + userId` → repository update (tenant-scoped) → enrichment with live resource snapshot → audit log → response. `stop : id + userId` → lookup + stale-state guard → live AWS `StopInstancesCommand` → local resource state sync → recommendation resolved → audit log → response.

EXAMPLE EXECUTION

Request: `POST /api/recommendations/64f.../stop`. Response (success): `200 { "success": true, "data": { "_id": "64f...", "resourceId": "i-0abc123", "resourceName": "web-server-1", "status": "RESOLVED", "state": "stopped", ... } }`. Response (already stopped): `400 { "success": false, "message": "This instance is already stopped." }`.

IMPORTANT NOTES

- `stopRecommendation` is the only endpoint in this dossier that performs a live, mutating AWS API call (stopping a real EC2 instance) — it is also the only recommendation action requiring live AWS credentials to be valid at call time (a revoked/expired key here surfaces as an unhandled 500, not a friendly error).
- The double-audit-log-prevention comment (EC2_STOPPED only, not also RECOMMENDATION_RESOLVED) is a deliberate design choice explicitly called out in the source comment — worth preserving if this logic is ever refactored.
- No explicit check that the recommendation is actually EC2-related before calling `stopEC2Instance` — if a non-EC2 recommendation's `resourceId` were passed through, `StopInstancesCommand` would simply fail against AWS with an invalid-instance-ID error, surfacing as a 500.

 `server/src/controllers/report.controller.js`

PURPOSE

Streams a generated PDF scan report directly to the HTTP response for download — the only controller in the codebase that returns a binary body instead of JSON.

WHEN THIS FILE IS CALLED

On `GET /api/reports/scan/:id`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/report.routes.js` binds `exportScanReport` to `GET /scan/:id`.

WHICH FILES IT DEPENDS ON

- `server/src/services/report/scanReport.service.js` — `getScanReportData` (gathers scan/resources/recommendations + computed rollups).
- `server/src/services/report/scanReportPdf.service.js` — `buildScanReportPdf` (renders a `pdfkit` `PDFDocument` from that data).
- `server/src/utils/ApiError.js` — thrown when the scan isn't found.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper. (Notably does **not** import `apiResponse.js` / `sendSuccess` — this controller never sends JSON on the success path.)

EXPORTED FUNCTIONS

- `exportScanReport(req, res)`
- Purpose: Build and stream a PDF report for one scan.
- Input: `req.params.id` (scan ID), `req.user.userId`.
- Output: **not JSON** — on success, sets `Content-Type: application/pdf` and `Content-Disposition: attachment; filename="cloudlens-scan-report-<id>.pdf"`, then pipes a `pdfkit` document stream directly to `res`. On failure (scan not found), throws `ApiError(404, "Scan not found")`, which the centralized handler turns into a normal JSON error response.
- Internal steps: 1) `getScanReportData(req.params.id, req.user.userId)` — internally calls `fetchScanDetails` (scan + resources + recommendations, tenant-scoped) and, if found, computes `severityCounts` (`{HIGH,MEDIUM,LOW}` tally from recommendations),

`monthlyCost` (sum of each recommendation's `monthlyCost`), and `resourceTypeCounts` (count of resources grouped by `resourceType`, defaulting to "UNKNOWN"); 2) if `reportData` is falsy, throw `ApiError(404, "Scan not found")`; 3) set the two response headers; 4) `buildScanReportPdf(reportData)` builds a multi-section `pdfkit` document (header banner, summary section, severity/resource breakdowns, etc.); 5) `doc.pipe(res)` then `doc.end()` to stream the PDF bytes.

- Database calls: transitively via `scanHistory.service.fetchScanDetails` → `scan.repository.getScanById`, `resource.repository.getResourcesByScanId`, `recommendation.repository.getRecommendationsByScanId`, all tenant-scoped by `userId`.
- AWS calls: none (PDF is built entirely from previously-scanned/stored data).
- Error handling: only the `ApiError(404, ...)` explicit check; any error thrown mid-stream by `pdfkit` after headers are already sent would not be catchable by the normal JSON error path (headers-already-sent problem), though this is not explicitly handled either way in the code.
- HTTP status codes: 200 (implicit — Express default status for a piped stream with no explicit `res.status()` call), 404 (scan not found or belongs to another user).

ROUTE TABLE (SEE REPORT.ROUTES.JS BELOW)

DATA FLOW

Request → `req.params.id` + `req.user.userId` → `scanReport.service.js` re-fetches the same tenant-scoped scan/resource/recommendation data the Scan Detail page uses, then derives rollups in plain JS (no new Mongo queries) → `scanReportPdf.service.js` renders those into a `pdfkit` document → headers set → document piped straight into the HTTP response stream.

EXAMPLE EXECUTION

Request: `GET /api/reports/scan/64f9e2.../` with valid auth. Response: binary 200 response, headers `Content-Type: application/pdf`, `Content-Disposition: attachment; filename="cloudlens-scan-report-64f9e2...pdf"`, body = PDF bytes (CloudLens-branded report with cost/severity/resource-type breakdowns). Response (bad ID / not this user's scan): `404 { "success": false, "message": "Scan not found" }`.

IMPORTANT NOTES

- Unlike every other controller, this one never calls `sendSuccess` — success is entirely implicit in the stream; there is no `{success:true}` envelope at all for the happy path (by necessity, since the body is binary PDF data, not JSON).
- 404 covers both "scan truly doesn't exist" and "scan belongs to a different user" — `fetchScanDetails`'s `userId` filter makes cross-tenant access indistinguishable from a nonexistent ID, which is the correct behavior for tenant isolation (doesn't leak existence of other users' scans).

 `server/src/controllers/scan.controller.js`

PURPOSE

Triggers a full, synchronous, on-demand infrastructure scan (EC2 + S3 + RDS + IAM) of the user's connected AWS account and returns a summary once complete. This is the single "Scan Now" button's backend.

WHEN THIS FILE IS CALLED

On `POST /api/scan/`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/scan.routes.js` binds `scan` to `POST /`.

WHICH FILES IT DEPENDS ON

- `server/src/services/scanner/scanner.service.js` — `scanInfrastructure`, the orchestrator that runs all four per-service scanners.
- `server/src/services/auditLog/auditLog.service.js` — `recordAuditLog`.
- `server/src/constants/auditActions.js` — `AUDIT_ACTIONS.SCAN_STARTED` / `SCAN_COMPLETED`.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `scan(req, res)`
- Purpose: Run a full AWS resource scan for the current user's connected account and persist the results.

- Input: no body; `req.user.userId` only.
- Output: `sendSuccess(res, result)`, status 200, `result = { scanId, totalResources, estimatedSavings }`.
- Internal steps: 1) `recordAuditLog(req.user.userId, AUDIT_ACTIONS.SCAN_STARTED, "Infrastructure scan started")` (recorded *before* the scan runs); 2) `scanInfrastructure(req.user.userId)` — internally: finds the user's `AwsConnection` (throws `ApiError(400,...)` if none), creates a `Scan` record, builds all six AWS SDK clients via `getAwsClientsForUser`, runs `scanEC2` (errors here propagate, aborting the whole request), then runs `scanS3`, `scanRDS`, `scanIAM` each wrapped in its own try/catch that only `console.error`s on failure and substitutes `{totalResources:0, estimatedSavings:0}` (so a missing S3/RDS/IAM permission degrades gracefully instead of failing the whole scan), sums all four services' `totalResources / estimatedSavings`, and calls `completeScan` to persist the totals onto the `Scan` record; 3) back in the controller, `recordAuditLog(req.user.userId, AUDIT_ACTIONS.SCAN_COMPLETED, "Scan completed – ${result.totalResources} resources scanned, ${result.estimatedSavings.toFixed(2)} potential savings identified")`; 4) `sendSuccess(res, result)`.
- Database calls: `AwsConnection.findOne`, `scan.repository.createScan`, `scan.repository.completeScan`, plus everything each per-service scanner writes (`Resource / Recommendation / Insight` documents) — out of scope for this file but triggered by it.
- AWS calls: live calls across EC2 (`DescribeInstances`, etc.), CloudWatch (CPU metrics), Pricing API, S3, RDS, and IAM — the entire scan pipeline, all transitively invoked from this one request.
- Error handling: no local try/catch in the controller; `scanInfrastructure` throws `ApiError(400, "Please connect an AWS account before running a scan.")` if no connection exists; a failure inside `scanEC2` (the only *unguarded* per-service call) propagates as an unhandled error → generic 500. S3/RDS/IAM failures are swallowed inside the service and do not fail the request.
- HTTP status codes: 200 (success, including partial success where some services failed silently), 400 (no AWS connection), 500 (EC2 scan failure or other unexpected error).

ROUTE TABLE (SEE SCAN.ROUTES.JS BELOW)

DATA FLOW

Request → audit log ("scan started") written *before* any AWS calls → `scanner.service.js` fans out sequentially (not parallel) across EC2 → S3 → RDS → IAM, each performing live AWS API calls, CloudWatch metric pulls, and Pricing API lookups, then persisting `Resource / Recommendation` documents per scanned item → totals aggregated → `Scan` record finalized → audit log ("scan completed", including totals) → JSON response.

EXAMPLE EXECUTION

Request: `POST /api/scan/` (no body) with valid auth and an existing AWS connection. Response: `200 { "success": true, "data": { "scanId": "64fa...", "totalResources": 37, "estimatedSavings": 128.42 } }`. Response (no AWS account connected): `400 { "success": false, "message": "Please connect an AWS account before running a scan." }`.

IMPORTANT NOTES

- This is a **synchronous, blocking** scan — the HTTP request stays open for the full duration of the AWS scan (potentially many seconds depending on account size), unlike a typical "kick off a background job and poll" pattern. There is no scan-progress endpoint visible in this dossier's file set.
- EC2 scan failure aborts the whole request (500), but S3/RDS/IAM scan failures are silently downgraded to a zero-contribution result and only logged server-side via `console.error` — the client has no way to tell from the response whether S3/RDS/IAM were actually scanned or silently skipped due to a permissions error.
- The `"SCAN_STARTED"` audit entry is written unconditionally, even if the scan subsequently fails with a 400/500 — there will be a `SCAN_STARTED` entry with no matching `SCAN_COMPLETED` entry in that case.
- Scheduled scans (via `scheduler.service.js` / node-cron) reuse the same `scanInfrastructure` function but are wired independently of this controller (see `server/src/services/scheduler/scheduler.service.js`, out of this dossier's scope).

 `server/src/controllers/scans.controller.js`

PURPOSE

Serves the Scan History page: a list of all past scans (`getScans`) and the full detail of one specific scan (`getScanById`), including its resources and recommendations. Distinct from `scan.controller.js`, which *triggers* a new scan — this file only *reads* past scans.

WHEN THIS FILE IS CALLED

On `GET /api/scans/` and `GET /api/scans/:id`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/scans.routes.js` binds `{ getScans, getScanById }`.

WHICH FILES IT DEPENDS ON

- `server/src/services/scan/scanHistory.service.js` — `fetchAllScans`, `fetchScanDetails`.
- `server/src/utils/ApiError.js` — thrown when a requested scan isn't found.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `getScans(req, res)`
 - Purpose: List every scan the user has run.
 - Input: `req.user.userId` only.
 - Output: `sendSuccess(res, scans)`, status 200, array of `Scan` documents.
 - Internal steps: `fetchAllScans(req.user.userId)` → `scan.repository.getAllScans(userId)` → `sendSuccess`.
 - Database calls: `scan.repository.getAllScans`.
 - AWS calls: none.
 - Error handling: `asyncHandler` only; no not-found case (an empty list is valid).
 - HTTP status codes: 200.
- `getScanById(req, res)`
 - Purpose: Return one scan's full detail: the scan record plus all resources and recommendations discovered in it.
 - Input: `req.params.id`, `req.user.userId`.
 - Output: `sendSuccess(res, details)`, status 200, `details = { scan, resources, recommendations }`.
 - Internal steps: 1) `fetchScanDetails(req.params.id, req.user.userId)` — internally: `getScanById(id, userId)` (tenant-scoped); if not found, returns `null` immediately (does not throw); otherwise runs `getResourcesByScanId` and `getRecommendationsByScanId` in parallel via `Promise.all` and returns the combined object; 2) if `details` is falsy, throw `ApiError(404, "Scan not found")`; 3) `sendSuccess`.
 - Database calls: `scan.repository.getScanById`, `resource.repository.getResourcesByScanId`, `recommendation.repository.getRecommendationsByScanId`.
 - AWS calls: none.
 - Error handling: explicit `ApiError(404, "Scan not found")` check in the controller itself (the service returns `null` rather than throwing, so the controller is responsible for the 404 translation — a different pattern from most other 404s in this dossier, which are thrown inside the service layer).
 - HTTP status codes: 200, 404.

ROUTE TABLE (SEE SCANS.ROUTES.JS BELOW)

DATA FLOW

`getScans`: `userId` → repository list query → response. `getScanById`: `id` + `userId` → tenant-scoped scan lookup → (if found) parallel resource + recommendation lookups scoped to the same scan → combined object → response; a miss short-circuits to a controller-level 404.

EXAMPLE EXECUTION

Request: `GET /api/scans/64fa1b2c3d4e5f6789012345`. Response (found): `200 { "success": true, "data": { "scan": { "_id": "64fa...", "status": "COMPLETED", "totalResources": 37, "estimatedSavings": 128.42, ... }, "resources": [ {...}, ... ], "recommendations": [ {...}, ... ] } }`. Response (not found / wrong user): `404 { "success": false, "message": "Scan not found" }`.

IMPORTANT NOTES

- `getScanById`'s 404 handling is done in the *controller*, not the service — the service intentionally returns `null` (see the same pattern in `report.controller.js`), which is the opposite convention from most other services in this codebase (e.g. `recommendation.service.js`, `schedule.service.js`) that throw `ApiError` themselves. A new engineer adding a similar "fetch by ID" endpoint should check which convention the sibling service already uses.
 - Tenant isolation for the detail endpoint is enforced three separate times (once per query — scan, resources, recommendations all independently filtered by `userId`), not just once at the top.
-

PURPOSE

CRUD-style endpoints for the user's recurring-scan schedule (create, list, update, delete). Business rule enforced at the service layer: exactly one schedule per user.

WHEN THIS FILE IS CALLED

On `POST /api/schedules/`, `GET /api/schedules/`, `PATCH /api/schedules/:id`, `DELETE /api/schedules/:id`, behind `protect`.

WHO CALLS THIS FILE

`server/src/routes/schedule.routes.js` binds `{ createSchedule, getSchedules, updateSchedule, deleteSchedule }`.

WHICH FILES IT DEPENDS ON

- `server/src/services/schedule/schedule.service.js` — `addSchedule`, `fetchSchedules`, `editSchedule`, `removeSchedule`.
- `server/src/utils/ApiError.js` — thrown for missing `frequency` on create.
- `server/src/utils/apiResponse.js` — `sendSuccess`.
- `server/src/utils/asyncHandler.js` — error-forwarding wrapper.

EXPORTED FUNCTIONS

- `createSchedule(req, res)`
 - Purpose: Create the user's recurring-scan schedule.
 - Input: `req.body = { frequency, enabled }` (`enabled` optional).
 - Output: `sendSuccess(res, schedule, 201)`, status 201, the created `Schedule` document.
 - Internal steps: 1) destructure `frequency, enabled`; 2) if `frequency` is falsy, throw `ApiError(400, "Frequency is required")`; 3) call `addSchedule({userId: req.user.userId, frequency, enabled})` — internally: validates `frequency` is one of `["hourly", "daily", "weekly"]` via `isValidFrequency` (else `ApiError(400, "Frequency must be one of: hourly, daily, weekly")`), checks no schedule already exists for the user (else `ApiError(409, "A scan schedule already exists. Update it instead of creating a new one.")`), computes `nextRun` via `computeNextRun(frequency)` (`now + FREQUENCY_MS[frequency]`, e.g. `+3600000` ms for `"hourly"`), then creates the record; 4) `sendSuccess(res, schedule, 201)`.
 - Database calls: `schedule.repository.getScheduleByUserId` (uniqueness check), `schedule.repository.createSchedule`.
 - AWS calls: none.
 - Error handling: `ApiError(400, "Frequency is required")` in the controller; `ApiError(400, "Frequency must be one of: hourly, daily, weekly")` and `ApiError(409, "A scan schedule already exists...")` in the service.
 - HTTP status codes: 201, 400, 409.
- `getSchedules(req, res)`
 - Purpose: List the user's schedule(s).
 - Input: `req.user.userId` only.
 - Output: `sendSuccess(res, schedules)`, status 200, `schedules` is an **array** containing zero or one schedule — `fetchSchedules` wraps the single record as `[schedule]` if it exists, or returns `[]` if not (there is no `getSchedule` singular endpoint).
 - Internal steps: `fetchSchedules(req.user.userId) → sendSuccess`.
 - Database calls: `schedule.repository.getScheduleByUserId`.
 - AWS calls: none.
 - Error handling: none needed — always returns an array, never throws for "no schedule."
 - HTTP status codes: 200.
- `updateSchedule(req, res)`
 - Purpose: Change the frequency and/or enabled state of the user's schedule.
 - Input: `req.params.id`, `req.body = { frequency, enabled }` (both optional).
 - Output: `sendSuccess(res, schedule)`, status 200, the updated document.
 - Internal steps: 1) destructure `frequency, enabled`; 2) `editSchedule(req.params.id, req.user.userId, {frequency, enabled})` — internally: if `frequency !== undefined` validates it via `isValidFrequency` (else `ApiError(400, ...)`), looks up the existing schedule by `id + userId` (else `ApiError(404, "Schedule not found")`), builds an `updates` object with only defined fields, and — only if the schedule is being (re-)enabled (`enabled === true && !current.enabled`) or the frequency is actually changing — recomputes `nextRun` via `computeNextRun`, so toggling other fields doesn't reset the timer; 3) `sendSuccess`.

- Database calls: `schedule.repository.getScheduleById` , `schedule.repository.updateSchedule` .
- AWS calls: none.
- Error handling: `ApiError(400, "Frequency must be one of: hourly, daily, weekly")` (invalid value), `ApiError(404, "Schedule not found")` .
- HTTP status codes: 200, 400, 404.
- `deleteSchedule(req, res)`
- Purpose: Remove the user's schedule.
- Input: `req.params.id` , `req.user.userId` .
- Output: `sendSuccess(res, {deleted: true})` , status 200.
- Internal steps: `removeSchedule(req.params.id, req.user.userId)` (throws `ApiError(404, "Schedule not found")` if nothing deleted) → `sendSuccess(res, {deleted:true})` .
- Database calls: `schedule.repository.deleteSchedule` .
- AWS calls: none.
- Error handling: `ApiError(404, "Schedule not found")` .
- HTTP status codes: 200, 404.

ROUTE TABLE (SEE SCHEDULE.ROUTES.JS BELOW)

DATA FLOW

`create` : body → controller presence check → service value/uniqueness validation → `nextRun` computed → Mongo insert → response. `get : userId` → single-record lookup wrapped as an array → response. `update` : `id` +body → service validation + conditional `nextRun` recompute → Mongo update → response. `delete` : `id` → Mongo delete (tenant-scoped) → response.

EXAMPLE EXECUTION

Request: `POST /api/schedules/` body `{ "frequency": "daily" }` . Response: `201 { "success": true, "data": { "_id": "...", "userId": "...", "frequency": "daily", "enabled": true, "nextRun": "2026-08-02T12:00:00.000Z", "createdAt": "2026-08-01T12:00:00.000Z" } }` . Response (second attempt, already exists): `409 { "success": false, "message": "A scan schedule already exists. Update it instead of creating a new one." }` .

IMPORTANT NOTES

- `frequency` validity is checked **twice** on `createSchedule` — once implicitly (falsy check in the controller) and once explicitly (`isValidFrequency` in the service) — but the controller's own check only catches `undefined` / `""` / `null` , not an invalid string like `"monthly"` ; that case is only caught by the service's `ApiError(400, "Frequency must be one of: hourly, daily, weekly")` .
- `getSchedules` always returns an array even though the underlying data model is "one schedule per user" — a new engineer might expect a single object and be surprised by the `[...]` wrapper.
- `nextRun` is only recomputed on enable or frequency change (not on every `PATCH`), so e.g. toggling nothing or just flipping `enabled` from `true` → `false` leaves the stale `nextRun` timestamp in place — this matters if `scheduler.service.js` (out of scope) reads `nextRun` directly rather than re-deriving it.

 `server/src/routes/analytics.routes.js`

PURPOSE

Declares the four analytics/trend GET endpoints and binds each to its `analytics.controller.js` handler. No route-local middleware — auth is applied once at the mount point in `app.js` .

WHEN THIS FILE IS CALLED

When `server/src/app.js` boots and calls `require("../routes/analytics.routes")` , the router is constructed once; individual routes then fire per matching incoming request.

WHO CALLS THIS FILE

`server/src/app.js` , which mounts it via `app.use("/api/analytics", protect, analyticsRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/analytics.controller.js` — supplies all four route handlers.
- `express` (external) — `express.Router()` .

EXPORTED FUNCTIONS

- Exports an Express Router instance (module.exports = router), not named functions.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	/cost-trend	(inherited: protect from app.js mount)	costTrend
GET	/savings-trend	(inherited: protect)	savingsTrend
GET	/resource-growth	(inherited: protect)	resourceGrowth
GET	/high-alerts-trend	(inherited: protect)	highAlertsTrend

DATA FLOW

Incoming request matched against one of the four static paths (full paths: /api/analytics/cost-trend , etc.) → routed straight to the corresponding controller function, no route-specific middleware or param parsing.

EXAMPLE EXECUTION

Request: GET /api/analytics/resource-growth . Routed to: resourceGrowth in analytics.controller.js → 200 {success:true,data:[...]}.

IMPORTANT NOTES

- All four routes are static (no :param), so there's no risk of path-matching ambiguity between them.

 server/src/routes/auditLog.routes.js

PURPOSE

Declares the single audit-log listing endpoint.

WHEN THIS FILE IS CALLED

At app boot (router construction) and per matching request thereafter.

WHO CALLS THIS FILE

server/src/app.js , mounted via app.use("/api/audit-logs", protect, auditLogRoutes) .

WHICH FILES IT DEPENDS ON

- server/src/controllers/auditLog.controller.js — getAuditLogs .
- express (external).

EXPORTED FUNCTIONS

- Exports an Express Router instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	/	(inherited: protect)	getAuditLogs

DATA FLOW

GET /api/audit-logs → getAuditLogs → user's audit trail returned as JSON.

EXAMPLE EXECUTION

Request: GET /api/audit-logs . Response: 200 {success:true,data:[{action:"SCAN_COMPLETED", ...}, ...]} .

IMPORTANT NOTES

- This is the shortest route file in the dossier (10 lines) — a single GET, no filtering options exposed at the routing layer.

PURPOSE

Declares the three identity endpoints and is the **only** router in the app mounted without a blanket `protect` at the mount point — it manages its own per-route authentication instead.

WHEN THIS FILE IS CALLED

At app boot, then per request to any `/api/auth/*` path.

WHO CALLS THIS FILE

`server/src/app.js`, mounted via `app.use("/api/auth", authRoutes)` — **no `protect` passed here**, unlike every other router in the app.

WHICH FILES IT DEPENDS ON

- `server/src/controllers/auth.controller.js` — `signup`, `login`, `logoutAll`.
- `server/src/middleware/auth.middleware.js` — `protect`, applied only to the `/logout-all` route.
- `server/src/middleware/rateLimiter.middleware.js` — `loginRateLimiter`, applied only to `/login`.
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
POST	<code>/signup</code>	none	<code>signup</code>
POST	<code>/login</code>	<code>loginRateLimiter</code> (10 req / 15 min / IP)	<code>login</code>
POST	<code>/logout-all</code>	<code>protect</code>	<code>logoutAll</code>

DATA FLOW

`/signup` and `/login` are reachable with no token at all (public). `/login` passes through `loginRateLimiter` first, which tracks attempts per IP and responds `429` directly (via `sendError`, not `next()` / `ApiError`) if the caller exceeds 10 attempts in a 15-minute window, otherwise calls `next()` into `login`. `/logout-all` requires a valid, current (non-stale- `tokenVersion`) JWT via `protect` before reaching `logoutAll`.

EXAMPLE EXECUTION

Request: `POST /api/auth/signup` body `{fullName, email, password}` → `signup` controller → `201 {success:true,user:{...}}`. Request: 11th login attempt within 15 minutes from the same IP → `429 {success:false,message:"Too many login attempts. Please try again in a few minutes."}` (short-circuited by `loginRateLimiter`, never reaches the controller).

IMPORTANT NOTES

- The route-level comment in the source explicitly explains the design: "Every API is authenticated except signup/login. Both live under `/api/auth` alongside `logout-all`, which protects itself per-route since it needs a valid token to know which user to log out." This is the one router where middleware placement genuinely matters for security and is worth calling out to new engineers touching this file.
- `loginRateLimiter` only guards `/login`, not `/signup` — repeated signup attempts (e.g. an email-enumeration or spam-account script) are not rate-limited at all in this file.

PURPOSE

Declares the four AWS-connection-management endpoints (test, connect, read, disconnect).

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/aws/*` request.

WHO CALLS THIS FILE

`server/src/app.js` , mounted via `app.use("/api/aws", protect, awsRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/awsConnection.controller.js` — `testConnection` , `connect` , `getAccount` , `disconnect` .
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
POST	<code>/test-connection</code>	(inherited: <code>protect</code>)	<code>testConnection</code>
POST	<code>/connect</code>	(inherited: <code>protect</code>)	<code>connect</code>
GET	<code>/account</code>	(inherited: <code>protect</code>)	<code>getAccount</code>
DELETE	<code>/disconnect</code>	(inherited: <code>protect</code>)	<code>disconnect</code>

DATA FLOW

Each path maps 1:1 to a controller function; no route-local middleware, params, or body parsing beyond what Express's global `express.json()` (registered in `app.js`) already provides.

EXAMPLE EXECUTION

Request: `DELETE /api/aws/disconnect` . Response: `200 {success:true,data:{disconnected:true}}` or `404 {success:false,message:"No AWS account is connected."}` .

IMPORTANT NOTES

- `/test-connection` requires a valid JWT (protected at mount) even though its purpose is unrelated to the user's own account state — it still needs `req.user.userId` -independent validation of arbitrary submitted credentials, but the endpoint itself doesn't use `req.user` at all inside the controller (see `testConnection` in `awsConnection.controller.js` , which never reads `req.user`).

 `server/src/routes/dashboard.routes.js`

PURPOSE

Declares the two dashboard read endpoints.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/dashboard/*` request.

WHO CALLS THIS FILE

`server/src/app.js` , mounted via `app.use("/api/dashboard", protect, dashboardRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/dashboard.controller.js` — `getDashboard` , `getLatestRecommendations` .
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	<code>/</code>	(inherited: <code>protect</code>)	<code>getDashboard</code>
GET	<code>/recommendations</code>	(inherited: <code>protect</code>)	<code>getLatestRecommendations</code>

DATA FLOW

`GET /api/dashboard/` → full aggregated dashboard payload. `GET /api/dashboard/recommendations` → latest recommendations only (a lighter-weight subset also independently available in full via `/api/recommendations/`).

EXAMPLE EXECUTION

Request: `GET /api/dashboard/recommendations`. Response: `200 {success:true,data:[{...recommendation...}, ...]}`.

IMPORTANT NOTES

- Source comments label the two routes ("`// Dashboard Summary`", "`// Latest Recommendations`") — this file is otherwise unremarkable, purely declarative.

 `server/src/routes/health.routes.js`

PURPOSE

Declares the single liveness-check route.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/health/*` request.

WHO CALLS THIS FILE

`server/src/app.js`, mounted via `app.use("/api/health", protect, healthRoutes)`.

WHICH FILES IT DEPENDS ON

- `server/src/controllers/health.controller.js` — `checkHealth`.
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	<code>/</code>	(inherited: <code>protect</code>)	<code>checkHealth</code>

DATA FLOW

`GET /api/health/` → `checkHealth` → static `{message,timestamp}` payload.

EXAMPLE EXECUTION

Request: `GET /api/health/` (with valid JWT). Response: `200 {success:true,data:{message:"Server is running",timestamp:"..."}}`.
Request without a token: `401 {success:false,message:"Unauthorized: No token provided"}` (from `protect`, never reaches this router's handler).

IMPORTANT NOTES

- Because `protect` is applied at the mount point in `app.js`, this "health check" cannot be used by an unauthenticated uptime monitor or load balancer — see the corresponding note in `health.controller.js`.

 `server/src/routes/notification.routes.js`

PURPOSE

Declares the notification-settings read/update endpoints.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/notifications/*` request.

WHO CALLS THIS FILE

server/src/app.js , mounted via app.use("/api/notifications", protect, notificationRoutes) .

WHICH FILES IT DEPENDS ON

- server/src/controllers/notification.controller.js — getSettings , updateSettings .
- express (external).

EXPORTED FUNCTIONS

- Exports an Express Router instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	/settings	(inherited: protect)	getSettings
PATCH	/settings	(inherited: protect)	updateSettings

DATA FLOW

Both verbs share the same /settings path (standard REST-ish pattern: GET to read, PATCH to partially update) → routed to the respective controller function.

EXAMPLE EXECUTION

Request: PATCH /api/notifications/settings body {"highSeverity": false} . Response: 200 {success:true,data:{...highSeverity:false}} .

IMPORTANT NOTES

- Uses PATCH , not PUT , correctly reflecting the controller's partial-update semantics (only defined fields in the body are applied).

server/src/routes/recommendation.routes.js

PURPOSE

Declares the recommendation listing and three action endpoints (ignore/resolve/stop).

WHEN THIS FILE IS CALLED

At app boot, then per matching /api/recommendations/* request.

WHO CALLS THIS FILE

server/src/app.js , mounted via app.use("/api/recommendations", protect, recommendationRoutes) .

WHICH FILES IT DEPENDS ON

- server/src/controllers/recommendation.controller.js — getRecommendations , ignoreRecommendation , resolveRecommendation , stopRecommendation .
- express (external).

EXPORTED FUNCTIONS

- Exports an Express Router instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	/	(inherited: protect)	getRecommendations
PATCH	/:id/ignore	(inherited: protect)	ignoreRecommendation
PATCH	/:id/resolve	(inherited: protect)	resolveRecommendation
POST	/:id/stop	(inherited: protect)	stopRecommendation

DATA FLOW

`:id` route param is parsed by Express and available as `req.params.id` in the controller, used together with `req.user.userId` for tenant-scoped lookups/updates in every action route.

EXAMPLE EXECUTION

Request: `PATCH /api/recommendations/64f.../ignore` . Response: `200 {success:true,data:{...},status:"IGNORED"}` or `404 {success:false,message:"Recommendation not found"}` .

IMPORTANT NOTES

- `/:id/stop` is deliberately `POST` , not `PATCH` like the other two actions — a reasonable REST convention choice since it triggers a side-effecting external action (stopping a real EC2 instance) rather than a pure state update, even though the net effect also resolves the recommendation.
- No `DELETE` route exists for recommendations — they can only be ignored or resolved, never removed from the collection via this API.

 `server/src/routes/report.routes.js`

PURPOSE

Declares the single PDF scan-report export endpoint.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/reports/*` request.

WHO CALLS THIS FILE

`server/src/app.js` , mounted via `app.use("/api/reports", protect, reportRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/report.controller.js` — `exportScanReport` .
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	<code>/scan/:id</code>	(inherited: <code>protect</code>)	<code>exportScanReport</code>

DATA FLOW

`:id` (scan ID) parsed into `req.params.id` → controller streams a PDF binary response instead of JSON.

EXAMPLE EXECUTION

Request: `GET /api/reports/scan/64fa1b2c3d4e5f6789012345` . Response: binary PDF stream (see `report.controller.js` for full detail) or `404 {success:false,message:"Scan not found"}` .

IMPORTANT NOTES

- Path is `/scan/:id` , not just `/:id` — namespacing that leaves room for future report types (e.g. `/api/reports/monthly/:id`) under the same router without a path collision.

 `server/src/routes/scan.routes.js`

PURPOSE

Declares the single scan-trigger endpoint.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/scan/*` request.

WHO CALLS THIS FILE

`server/src/app.js` , mounted via `app.use("/api/scan", protect, scanRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/scan.controller.js` — `scan` .
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
POST	/	(inherited: <code>protect</code>)	<code>scan</code>

DATA FLOW

`POST /api/scan/` with no body → controller runs the full blocking scan pipeline → JSON summary returned once complete.

EXAMPLE EXECUTION

Request: `POST /api/scan/` . Response: `200 {success:true,data:{scanId:"...",totalResources:37,estimatedSavings:128.42}}` .

IMPORTANT NOTES

- Nine lines total — the smallest route file alongside `health.routes.js` ; all the complexity lives downstream in `scanner.service.js` and the four per-service scanners, none of which are visible from this file.
- Note the singular `/api/scan` (this file, trigger-a-scan) versus plural `/api/scans` (`scans.routes.js` , list-past-scans) — an easy naming collision to mistype when adding a new client call.

 `server/src/routes/scans.routes.js`

PURPOSE

Declares the scan-history listing and detail endpoints.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/scans/*` request.

WHO CALLS THIS FILE

`server/src/app.js` , mounted via `app.use("/api/scans", protect, scansRoutes)` .

WHICH FILES IT DEPENDS ON

- `server/src/controllers/scans.controller.js` — `getScans` , `getScanById` .
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
GET	/	(inherited: <code>protect</code>)	<code>getScans</code>
GET	<code>/:id</code>	(inherited: <code>protect</code>)	<code>getScanById</code>

DATA FLOW

`GET /api/scans/` → all past scans for the user. `GET /api/scans/:id` → one scan's full detail (scan + resources + recommendations), 404 if not found or not owned by the requester.

EXAMPLE EXECUTION

Request: `GET /api/scans/`. Response: `200 {success:true,data:[{_id:"...",status:"COMPLETED",...}, ...]}`.

IMPORTANT NOTES

- See `scan.routes.js` note above regarding the singular/plural naming distinction between `/api/scan` (trigger) and `/api/scans` (history) — both exist simultaneously in this app and are easy to confuse.

 `server/src/routes/schedule.routes.js`

PURPOSE

Declares the full CRUD route set for the user's recurring-scan schedule.

WHEN THIS FILE IS CALLED

At app boot, then per matching `/api/schedules/*` request.

WHO CALLS THIS FILE

`server/src/app.js`, mounted via `app.use("/api/schedules", protect, scheduleRoutes)`.

WHICH FILES IT DEPENDS ON

- `server/src/controllers/schedule.controller.js` — `createSchedule`, `getSchedules`, `updateSchedule`, `deleteSchedule`.
- `express` (external).

EXPORTED FUNCTIONS

- Exports an Express `Router` instance.

ROUTE TABLE

METHOD	PATH	MIDDLEWARE	CONTROLLER
POST	<code>/</code>	(inherited: <code>protect</code>)	<code>createSchedule</code>
GET	<code>/</code>	(inherited: <code>protect</code>)	<code>getSchedules</code>
PATCH	<code>/:id</code>	(inherited: <code>protect</code>)	<code>updateSchedule</code>
DELETE	<code>/:id</code>	(inherited: <code>protect</code>)	<code>deleteSchedule</code>

DATA FLOW

Standard REST verb mapping onto a single router; `:id` routes parse `req.params.id`, used with `req.user.userId` for tenant-scoped update/delete.

EXAMPLE EXECUTION

Request: `DELETE /api/schedules/64fb...`. Response: `200 {success:true,data:{deleted:true}}` or `404 {success:false,message:"Schedule not found"}`.

IMPORTANT NOTES

- Despite the plural resource name (`schedules`) and full CRUD verb set, the underlying data model enforces at most one schedule per user (see `schedule.controller.js` / `schedule.service.js` notes) — the route shape suggests a one-to-many resource, but the service layer caps it at one.

DATA LAYER — REPOSITORIES, MAPPERS, PRICING

The only layer allowed to touch Mongoose models directly, plus the AWS response normalizers and pricing plumbing that feed it.

`server/src/services/database/analytics.repository.js`

PURPOSE

Read-only data-access layer supplying the historical trend data used by the Analytics page: the ordered list of a user's completed scans, plus per-scan monthly-cost sums and per-scan HIGH-severity alert counts across *all* of the user's scans (not just the latest one).

WHEN THIS FILE IS CALLED

Whenever `services/analytics/analytics.service.js` builds the trend-chart response for the Analytics page (`GET /api/analytics`).

WHO CALLS THIS FILE

`server/src/services/analytics/analytics.service.js` — the sole importer; it composes the trend series from these three functions.

WHICH FILES IT DEPENDS ON

- `server/src/models/Recommendation.js` — aggregated for `monthlyCost` sums and `severity: "HIGH"` counts per scan.
- `server/src/models/Scan.js` — queried for the ordered list of `COMPLETED` scans that anchors the trend timeline.

EXPORTED FUNCTIONS

- `getCompletedScansForUser(userId)`
 - Purpose: fetch every completed scan for a user, oldest first, to form the x-axis of a trend chart.
 - Input: `userId`
 - Output: array of `Scan` Mongoose documents, sorted ascending by `completedAt`.
 - Internal steps: 1. `Scan.find({ userId, status: "COMPLETED" })` 2. `.sort({ completedAt: 1 })`.
 - Database calls: `Scan.find({ userId, status: "COMPLETED" }).sort({ completedAt: 1 })`.
 - Error handling: none explicit; a DB error rejects the promise and propagates to the caller.
 - Return values: array, `[]` if the user has no completed scans.
- `getMonthlyCostByScan(scanIds)`
 - Purpose: sum `monthlyCost` across ALL recommendations (regardless of `status`) grouped by scan — the real infra spend at each historical snapshot.
 - Input: `scanIds` — array of Scan `_id` s (already user-scoped, taken from `getCompletedScansForUser` 's results).
 - Output: array of `{ _id: scanId, monthlyCost: number }`.
 - Internal steps: `Recommendation.aggregate([ { $match: { scanId: { $in: scanIds } } }, { $group: { _id: "$scanId", monthlyCost: { $sum: "$monthlyCost" } } } ])`.
 - Database calls: `Recommendation.aggregate([...])` (pipeline above).
 - Error handling: none explicit.
 - Return values: array, `[]` if none of the scanIds have recommendations.
- `getHighAlertsByScan(scanIds)`
 - Purpose: count HIGH-severity recommendations per scan (also unfiltered by status, per the same historical rule).
 - Input: `scanIds` — array of Scan `_id` s.
 - Output: array of `{ _id: scanId, highAlerts: number }`.
 - Internal steps: `Recommendation.aggregate([ { $match: { scanId: { $in: scanIds }, severity: "HIGH" } }, { $group: { _id: "$scanId", highAlerts: { $sum: 1 } } } ])`.
 - Database calls: `Recommendation.aggregate([...])` (pipeline above).
 - Error handling: none explicit.
 - Return values: array, `[]` if no HIGH-severity recommendations exist for those scans.

DATA FLOW

`analytics.service.js` calls `getCompletedScansForUser` to get the ordered scan list, extracts their `_id` s, then calls `getMonthlyCostByScan` and `getHighAlertsByScan` (in parallel) with that id list; it merges the two per-scanId result sets back onto the scan timeline to build the trend chart's data points.

EXAMPLE EXECUTION

For `userId = "64f0a1..."` with 3 completed scans `[s1, s2, s3]`: `getMonthlyCostByScan([s1,s2,s3])` runs `Recommendation.aggregate([{$match:{scanId:{$in:[s1,s2,s3]}}},{ $group: {_id:"$scanId", monthlyCost:{$sum:"$monthlyCost"}}}])` and might return `[{_id:s1, monthlyCost:210.40},{_id:s2, monthlyCost:245.30},{_id:s3, monthlyCost:198.10}]`.

IMPORTANT NOTES

Because `scanIds` passed into `getMonthlyCostByScan` / `getHighAlertsByScan` already belong to the requesting user (derived from `getCompletedScansForUser`), those two aggregations do not re-filter on `userId` themselves — correct only as long as callers never source `scanIds` from anywhere else. Unlike `dashboard.repository.js` and `recommendation.repository.js`, **neither aggregation here filters by `status`** — this is intentional and matches the domain rule that historical/trend queries reflect what was true at scan time, not just what's still outstanding today.

 `server/src/services/database/auditLog.repository.js`

PURPOSE

Thin two-function wrapper over the `AuditLog` model: write one audit event, and read a user's full audit history newest-first. Backs the Audit Log page and the fire-and-forget audit calls made from several controllers (auth, AWS connection, recommendation, scan).

WHEN THIS FILE IS CALLED

`createAuditLog` fires whenever `services/auditLog/auditLog.service.js`'s logging helper is invoked after a mutating action (login, connect AWS account, resolve/ignore a recommendation, run a scan, etc.). `getAuditLogsByUserId` fires on Audit Log page loads.

WHO CALLS THIS FILE

`server/src/services/auditLog/auditLog.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

- `server/src/models/AuditLog.js` — the model this wraps. Schema: `userId` (indexed), `action` (enum restricted to `AUDIT_ACTION_VALUES` from `constants/auditActions.js`), `description`; `timestamps`: { `createdAt`: true, `updatedAt`: false }; compound index { `userId`: 1, `createdAt`: -1 }.

EXPORTED FUNCTIONS

- `createAuditLog({ userId, action, description })`
 - Purpose: persist a single audit event.
 - Input: { `userId`, `action` (enum string), `description` (string) }.
 - Output: newly created `AuditLog` document.
 - Internal steps: `AuditLog.create({ userId, action, description })`.
 - Database calls: `AuditLog.create(...)`.
 - Error handling: Mongoose validation throws if `action` is outside the enum or `description` is missing — propagates to the caller.
 - Return values: the created `AuditLog` doc.
- `getAuditLogsByUserId(userId)`
 - Purpose: fetch a user's complete audit history, most recent first.
 - Input: `userId`.
 - Output: array of `AuditLog` documents.
 - Internal steps: `AuditLog.find({ userId }).sort({ createdAt: -1 })`.
 - Database calls: `AuditLog.find({ userId }).sort({ createdAt: -1 })` — this query pattern maps directly onto the model's { `userId`: 1, `createdAt`: -1 } compound index.
 - Error handling: none explicit.
 - Return values: array, `[]` if the user has no audit entries.

DATA FLOW

Controllers call `auditLog.service.js`'s logging helper immediately after a mutating action succeeds, which calls `createAuditLog`; the Audit Log page load path calls `getAuditLogsByUserId` through the same service.

EXAMPLE EXECUTION

`createAuditLog({ userId: "64f...", action: "SCAN_STARTED", description: "Started AWS scan" })` → `AuditLog.create({...})` → inserts a doc with `createdAt` set to now. `getAuditLogsByUserId("64f...")` → returns all matching docs sorted newest-first.

IMPORTANT NOTES

No pagination or limit is applied — `getAuditLogsByUserId` returns the user's entire history in one query, which could grow unbounded for long-lived accounts. Because the schema disables `updatedAt`, no `updatedAt` field exists on these documents.

 `server/src/services/database/dashboard.repository.js`

PURPOSE

The largest and most aggregation-heavy repository in the layer: builds every numeric summary card shown on the main Dashboard — resource/cost/severity totals from the user's latest scan, EC2 fleet-wide health stats, IAM security-posture stats, and recommendation counts by category. All four functions independently resolve "the user's latest scan" and scope every query to it.

WHEN THIS FILE IS CALLED

On every Dashboard page load (`GET /api/dashboard`), and by the weekly report cron (`weeklyReport.scheduler.js`) when composing the weekly email digest.

WHO CALLS THIS FILE

`server/src/services/dashboard/dashboard.service.js` and `server/src/services/notification/weeklyReport.scheduler.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/Recommendation.js` — aggregated for cost, savings, severity, and category counts.
- `server/src/models/Resource.js` — aggregated/queried for resource counts and EC2/IAM fleet metadata.
- `server/src/models/Scan.js` — queried once per function to resolve the user's latest scan.

EXPORTED FUNCTIONS

- `getDashboardSummary(userId)`
- Purpose: the primary Dashboard summary card — resource counts, monthly cost, estimated savings, and severity counts, scoped to the user's most recent scan.
- Input: `userId`.
- Output: `{ totalResources, runningResources, stoppedResources, monthlyCost, estimatedSavings, highRecommendations, mediumRecommendations, lowRecommendations }` — all numbers.
- Internal steps:
 - `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 })`.
 - If no `latestScan`, return the all-zero default object immediately (empty state).
 - `Resource.aggregate([ { $match: { scanId: latestScan._id, userId: latestScan.userId } }, { $facet: { totalResources: [ { $count: "count" } ], runningResources: [ { $match: { state: "running" } }, { $count: "count" } ], stoppedResources: [ { $match: { state: "stopped" } }, { $count: "count" } ] } } ])`.
 - `Recommendation.aggregate([{ $match: { scanId: latestScan._id, userId: latestScan.userId } }, { $facet: { totalMonthlyCost: [{ $group: { _id: null, total: { $sum: "$monthlyCost" } } }], totalEstimatedSavings: [{ $match: { status: "OPEN" } }, { $group: { _id: null, total: { $sum: "$estimatedSavings" } } }], highRecommendations: [{ $match: { severity: "HIGH", status: "OPEN" } }, { $count: "count" }], mediumRecommendations: [{ $match: { severity: "MEDIUM", status: "OPEN" } }, { $count: "count" }], lowRecommendations: [{ $match: { severity: "LOW", status: "OPEN" } }, { $count: "count" }] } }])`.
 - Destructure `resourceStats[0]` and `recommendationStats[0]`; each `$facet` bucket's `[0]?.count / [0]?.total` defaults to `0` via `|| 0`.
 - Return the composed object.

- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 }); Resource.aggregate([{$match, $facet}]; Recommendation.aggregate([{$match, $facet}])`.
 - Error handling: no try/catch; DB errors propagate. Empty state (no scan yet) returns all-zero defaults without querying Resource/Recommendation.
 - Return values: either the all-zero default object, or the fully computed object.
- getEC2FleetStats(userId)**
- Purpose: Dashboard's EC2 fleet-wide cards (avg CPU/network/disk, idle/old instance counts, encrypted volumes) — computed in plain JS over the latest scan's `EC2 Resource` docs' `metadata`, not via aggregation, since a scan's EC2 fleet is assumed small.
 - Input: `userId`.
 - Output: `{ avgCpu, avgNetwork, avgDisk, idleInstances, oldInstances, encryptedVolumes, totalVolumes }`.
 - Internal steps:
 - `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 });` if none, return `{...EMPTY_EC2_FLEET_STATS}`.
 - `ec2Resources = Resource.find({ scanId: latestScan._id, userId, resourceType: "EC2" });` if empty, return `{...EMPTY_EC2_FLEET_STATS}`.
 - Loop each resource's `metadata`: `cpu = metadata.averageCpu || 0; network = (networkIn||0) + (networkOut||0); disk = (diskReadBytes||0) + (diskWriteBytes||0);` accumulate totals; `idleInstances++` if `cpu < IDLE_CPU_THRESHOLD (5)`; `oldInstances++` if `(metadata.instanceAgeDays||0) > OLD_INSTANCE_AGE_DAYS (365)`; `encryptedFlags = metadata.encrypted || []`; `totalVolumes += encryptedFlags.length`; `encryptedVolumes += encryptedFlags.filter(Boolean).length`.
 - `count = ec2Resources.length`; return averages rounded via `Number((total/count).toFixed(2))`, plus the raw counts.
 - Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 }); Resource.find({ scanId, userId, resourceType: "EC2" })`.
 - Error handling: none explicit.
 - Return values: `EMPTY_EC2_FLEET_STATS = { avgCpu: 0, avgNetwork: 0, avgDisk: 0, idleInstances: 0, oldInstances: 0, encryptedVolumes: 0, totalVolumes: 0 }` (no scan, or no EC2 resources), or the computed stats object.
- getIAMFleetStats(userId)**
- Purpose: Dashboard's IAM Security Insights cards — user/role counts, MFA adoption, admin-user count, old access keys, high-risk users, from the latest scan's `IAM_USER / IAM_ROLE Resource` docs.
 - Input: `userId`.
 - Output: `{ totalUsers, totalRoles, mfaEnabledUsers, adminUsers, oldAccessKeys, highRiskUsers }`.
 - Internal steps:
 - `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 });` none → return `{...EMPTY_IAM_FLEET_STATS}`.
 - `Promise.all([Resource.find({ scanId, userId, resourceType: "IAM_USER" }), Resource.find({ scanId, userId, resourceType: "IAM_ROLE" })])` → `userResources, roleResources`.
 - If both empty, return `{...EMPTY_IAM_FLEET_STATS}`.
 - Loop `userResources`: `mfaEnabledUsers++` if `metadata.mfaEnabled`; `adminUsers++` if `metadata.hasAdminAccess`; `highRiskUsers++` if `metadata.riskLevel === "HIGH"`; `oldAccessKeys += (metadata.accessKeys||[]).filter(key => key.isOld).length`.
 - Return `totalUsers: userResources.length, totalRoles: roleResources.length`, plus the accumulated counts.
 - Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 });` two parallel `Resource.find(...)` calls.
 - Error handling: none explicit.
 - Return values: `EMPTY_IAM_FLEET_STATS = { totalUsers: 0, totalRoles: 0, mfaEnabledUsers: 0, adminUsers: 0, oldAccessKeys: 0, highRiskUsers: 0 }`, or the computed object.
- getRecommendationCategoryCounts(userId)**
- Purpose: Dashboard's per-category findings cards (Security/Cost/Performance/Reliability/Operational Excellence) — counts real findings (`action !== "NONE"`) that are still `OPEN`, scoped to the latest scan.
 - Input: `userId`.
 - Output: `{ securityFindings, costFindings, performanceFindings, reliabilityFindings, operationalExcellenceFindings }`.
 - Internal steps:
 - `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 });` none → return `{...EMPTY_CATEGORY_COUNTS}`.
 - `Recommendation.aggregate([{$match: { scanId: latestScan._id, userId: latestScan.userId, action: { $ne: "NONE" }, status: "OPEN" }, {$group: { _id: "$category", count: { $sum: 1 } } }])`.
 - `countByCategory = Object.fromEntries(counts.map(entry => [entry._id, entry.count]))`.

- Map `SECURITY→securityFindings` , `COST→costFindings` , `PERFORMANCE→performanceFindings` , `RELIABILITY→reliabilityFindings` , `OPERATIONAL_EXCELLENCE→operationalExcellenceFindings` , each defaulted with `|| 0` .
- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 })` ; `Recommendation.aggregate([$match, $group])` .
- Error handling: none explicit.
- Return values: `EMPTY_CATEGORY_COUNTS = { securityFindings: 0, costFindings: 0, performanceFindings: 0, reliabilityFindings: 0, operationalExcellenceFindings: 0 }` , or the computed mapping.

DATA FLOW

`dashboard.service.js` calls all four functions here (alongside `insight.repository.js` and `recommendation.repository.js`) to assemble the full `GET /api/dashboard` response. Each function independently re-resolves "latest scan" via its own `Scan.findOne` call rather than sharing one lookup across the file, then narrows to that scan's `Resource` / `Recommendation` documents.

EXAMPLE EXECUTION

For a user whose latest scan is `_id: "S1"` : `getDashboardSummary` runs the `Resource` facet, returning e.g. `totalResources:[{count:12}]` , `runningResources:[{count:9}]` , `stoppedResources:[{count:3}]` ; the `Recommendation` facet returns `totalMonthlyCost:[{_id:null,total:482.10}]` , `totalEstimatedSavings:[{_id:null,total:120.50}]` , `highRecommendations:[{count:2}]` , etc. Final result: `{ totalResources: 12, runningResources: 9, stoppedResources: 3, monthlyCost: 482.10, estimatedSavings: 120.50, highRecommendations: 2, mediumRecommendations: ..., lowRecommendations: ... }` .

IMPORTANT NOTES

This file is the clearest illustration of the status-filtering rule: in `getDashboardSummary` , `totalMonthlyCost` is **not** filtered by `status` (real infra spend keeps counting RESOLVED/IGNORED recommendations' cost — dismissing a recommendation doesn't change what AWS is billing), while `totalEstimatedSavings` and all three severity-count facets **are** filtered to `status: "OPEN"` (only outstanding, actionable recommendations count as "potential savings" or "alerts"). `getRecommendationCategoryCounts` applies the same `status: "OPEN"` filter. Each of the four exported functions performs its own separate `Scan.findOne({ userId }).sort({ createdAt: -1 })` — four round trips per dashboard load rather than one shared lookup. `getEC2FleetStats` / `getIAMFleetStats` intentionally use plain-JS reduction over `Resource.find()` results instead of an aggregation pipeline (per the inline comment, because a scan's EC2/IAM resource count is assumed small). The `Resource` model has no compound index covering `{ scanId, userId, resourceType }` (only `userId` is indexed) — these queries are not backed by a dedicated index.

`server/src/services/database/insight.repository.js`

PURPOSE

Persistence layer for S3-derived "insight" notes (lifecycle, cost, storage, security, general observations) generated during a scan and surfaced on the Dashboard's AI Insights card.

WHEN THIS FILE IS CALLED

`saveInsights` runs once per scan, at the end of `s3.scanner.js` , after its insight-generation step (`services/insight/s3.insight.js`) produces a batch of findings. `getInsightsByScanId` / `getLatestInsights` run on Dashboard loads and Scan Detail views.

WHO CALLS THIS FILE

`server/src/services/dashboard/dashboard.service.js` and `server/src/services/scanner/s3.scanner.js` .

WHICH FILES IT DEPENDS ON

- `server/src/models/Insight.js` — the model this wraps. Schema: `userId` (indexed), `scanId` , `category` (enum `SECURITY|COST|STORAGE|LIFECYCLE|GENERAL` , default `GENERAL`), `severity` (enum `INFO|LOW|MEDIUM|HIGH` , default `INFO`), `message` (required); `timestamps: true` .
- `server/src/models/Scan.js` — queried to resolve "the latest scan" for `getLatestInsights` .

EXPORTED FUNCTIONS

- `saveInsights(scanId, insights, userId)`
- Purpose: bulk-persist a batch of insight objects generated for one scan.
- Input: `scanId` , `insights` — array of plain objects (each shaped like `{ category, severity, message }`), `userId` .
- Output: array of inserted `Insight` documents, or `[]` .

- Internal steps: 1. If `insights.length === 0`, return `[]` immediately (no DB call). 2. `docs = insights.map(insight => ({ scanId, userId, ...insight })).` 3. `Insight.insertMany(docs)`.
- Database calls: `Insight.insertMany(docs)` (only when `insights` is non-empty).
- Error handling: Mongoose validation error (e.g. missing `message`, invalid `category` / `severity`) rejects `insertMany` and propagates.
- Return values: array of created docs, or `[]` if the input array was empty.
- `getInsightsByScanId(scanId, userId)`
- Purpose: fetch every insight tied to one scan.
- Input: `scanId`, `userId`.
- Output: array of `Insight` documents, sorted `createdAt` ascending (oldest first).
- Internal steps: `Insight.find({ scanId, userId }).sort({ createdAt: 1 })`.
- Database calls: `Insight.find({ scanId, userId }).sort({ createdAt: 1 })`.
- Error handling: none explicit.
- Return values: array, `[]` if none.
- `getLatestInsights(userId)`
- Purpose: fetch insights for the user's most recent scan (Dashboard AI Insights card).
- Input: `userId`.
- Output: array of `Insight` documents, or `[]`.
- Internal steps: 1. `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 })`. 2. If `!latestScan`, return `[]`. 3. Delegate to `getInsightsByScanId(latestScan._id, userId)`.
- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 })`, then `Insight.find({ scanId, userId }).sort({ createdAt: 1 })` via delegation.
- Error handling: none explicit.
- Return values: `[]` (no scan yet), or the array returned by `getInsightsByScanId`.

DATA FLOW

`s3.scanner.js` generates insight objects via `services/insight/s3.insight.js` during a scan, then calls `saveInsights` once to persist the whole batch. `dashboard.service.js` calls `getLatestInsights` on every Dashboard load to populate the AI Insights card with the newest scan's findings.

EXAMPLE EXECUTION

`saveInsights("scanId123", [{ category: "LIFECYCLE", severity: "MEDIUM", message: "Bucket X has no lifecycle policy" }], "userId456")` → `Insight.insertMany([{ scanId: "scanId123", userId: "userId456", category: "LIFECYCLE", severity: "MEDIUM", message: "Bucket X has no lifecycle policy" }])` → returns the 1 created doc.

IMPORTANT NOTES

`saveInsights` is exercised only by `s3.scanner.js` in practice (it's the sole caller among the scanners), even though the `Insight` model's `category` enum leaves room for other services. `getLatestInsights` performs two sequential DB round trips (`Scan.findOne` then `Insight.find`) rather than a single aggregation/lookup.

 `server/src/services/database/notification.repository.js`

PURPOSE

CRUD for per-user notification preferences (email opt-ins for scan-completed alerts, high-severity alerts, and the weekly report), plus one cross-tenant query used exclusively by the weekly report cron.

WHEN THIS FILE IS CALLED

`getNotificationSettingsByUserId` / `createNotificationSettings` / `updateNotificationSettings` run on the Notification Settings page's load/create/update flows. `getUsersWithWeeklyReportEnabled` runs once per weekly cron tick.

WHO CALLS THIS FILE

`server/src/services/notification/notification.service.js`, `server/src/services/notification/scanNotification.service.js`, `server/src/services/notification/weeklyReport.scheduler.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/Notification.js` — the model this wraps. Schema: `userId` (unique), `email` (required, trimmed, lowercased), `scanCompleted` (default `true`), `highSeverity` (default `true`), `weeklyReport` (default `true`); no `timestamps` option, so no `createdAt` / `updatedAt`.

EXPORTED FUNCTIONS

- `getNotificationSettingsByUserId(userId)`
 - Purpose: fetch one user's notification preferences.
 - Input: `userId`.
 - Output: `Notification` document or `null`.
 - Internal steps: `Notification.findOne({ userId })`.
 - Database calls: `Notification.findOne({ userId })`.
 - Return values: doc, or `null` if the user hasn't configured settings yet.
- `createNotificationSettings({ userId, email, scanCompleted, highSeverity, weeklyReport })`
 - Purpose: create a user's initial notification preferences.
 - Input: object with all five fields.
 - Output: newly created `Notification` document.
 - Internal steps: `Notification.create({ userId, email, scanCompleted, highSeverity, weeklyReport })`.
 - Database calls: `Notification.create(...)`.
 - Error handling: throws on validation error, or a duplicate-key error if a doc already exists for that `userId` (unique index).
 - Return values: the created doc.
- `updateNotificationSettings(userId, updates)`
 - Purpose: update an existing user's preferences.
 - Input: `userId`, `updates` (partial field object).
 - Output: updated `Notification` document, or `null`.
 - Internal steps: `Notification.findOneAndUpdate({ userId }, updates, { new: true })`.
 - Database calls: `Notification.findOneAndUpdate({ userId }, updates, { new: true })`.
 - Error handling: no `upsert` — if no document exists yet for that user, this resolves to `null` rather than creating one.
 - Return values: updated doc, or `null`.
- `getUsersWithWeeklyReportEnabled()`
 - Purpose: fetch every user across the whole system who has opted into the weekly report — used only by the weekly report cron.
 - Input: none.
 - Output: array of `Notification` documents.
 - Internal steps: `Notification.find({ weeklyReport: true })`.
 - Database calls: `Notification.find({ weeklyReport: true })` — deliberately **not** scoped by `userId` (per the inline comment: "it needs every opted-in user, not one").
 - Return values: array, `[]` if nobody has opted in.

DATA FLOW

The Notification Settings page load calls `getNotificationSettingsByUserId`; if `null`, the UI prompts the user through `createNotificationSettings`; subsequent edits go through `updateNotificationSettings`. Independently, `weeklyReport.scheduler.js`'s cron calls `getUsersWithWeeklyReportEnabled()` once per run to get every opted-in user across all tenants, then for each one builds and emails a weekly digest using data from `dashboard.repository.js`.

EXAMPLE EXECUTION

```
getUsersWithWeeklyReportEnabled() → Notification.find({ weeklyReport: true }) → [{ userId: "u1", email: "a@x.com", scanCompleted: true, highSeverity: true, weeklyReport: true }, { userId: "u2", ... }].
```

IMPORTANT NOTES

`getUsersWithWeeklyReportEnabled` is — like `schedule.repository.js`'s `getDueSchedules` — an intentional exception to the "every query scoped by `userId`" multi-tenancy rule: it is a system/cron-level query across all tenants, which is correct given its purpose but worth flagging explicitly. `updateNotificationSettings` has no upsert path, so calling it before a settings document exists silently returns `null` instead of creating one.

PURPOSE

The central data-access layer for `Recommendation` documents: creation (called by all 4 scanners), status transitions (ignore/resolve), and read paths that "enrich" bare recommendation docs with a joined snapshot of their related `Resource` fields (name, instance type, IPs, AZ, launch time, state) so the UI doesn't need a second round trip per row.

WHEN THIS FILE IS CALLED

`saveRecommendation` fires once per finding during every scanner's per-resource analysis step. The read/enrich functions fire on Dashboard loads, the Recommendations page, and Scan Detail views. `ignoreRecommendation` / `resolveRecommendation` fire when a user acts on a recommendation in the UI.

WHO CALLS THIS FILE

`server/src/services/dashboard/dashboard.service.js`, `server/src/services/dashboard/serviceAggregation.service.js`, `server/src/services/notification/scanNotification.service.js`, `server/src/services/recommendation/recommendation.service.js`, `server/src/services/scan/scanHistory.service.js`, and all four scanners: `server/src/services/scanner/ec2.scanner.js`, `iam.scanner.js`, `rds.scanner.js`, `s3.scanner.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/Recommendation.js` — the model this wraps.
- `server/src/models/Resource.js` — joined in application code (not via `$lookup`) to enrich recommendations with resource metadata.
- `server/src/models/Scan.js` — queried to resolve "the latest scan" for `getLatestRecommendations`.

EXPORTED FUNCTIONS

- `enrichRecommendationWithResource(recommendation)`
 - Purpose: single-recommendation convenience wrapper around the internal batch-enrichment helper.
 - Input: one `Recommendation` document, or `null`.
 - Output: enriched plain object, or `null`.
 - Internal steps: 1. If `!recommendation`, return `null`. 2. Call the internal `enrichRecommendationsWithResources([recommendation])`. 3. Return the single resulting element.
 - Database calls: delegates to the internal helper → `Resource.find({ scanId, userId, resourceId: { $in: [...] } })`.
 - Return values: enriched object, or `null`.
- `saveRecommendation(recommendationData)`
 - Purpose: persist one recommendation — called once per finding by each scanner.
 - Input: object matching the `Recommendation` schema (`userId`, `scanId`, `resourceId`, `category?`, `severity`, `action`, `confidence?`, `recommendation`, `reason`, `hourlyPrice?`, `monthlyCost?`, `estimatedSavings?`; `status` defaults to `"OPEN"`).
 - Output: the created `Recommendation` document.
 - Internal steps: `Recommendation.create(recommendationData)`.
 - Database calls: `Recommendation.create(recommendationData)`.
 - Error handling: Mongoose validation throws if required fields (`severity`, `action`, `recommendation`, `reason`) are missing — propagates to the scanner caller.
 - Return values: the created doc.
- `getRecommendationsByScanId(scanId, userId)`
 - Purpose: full, resource-enriched recommendation list for one specific scan (Scan Detail page).
 - Input: `scanId`, `userId`.
 - Output: array of enriched plain objects.
 - Internal steps: 1. `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })`. 2. Pass results through the internal `enrichRecommendationsWithResources`.
 - Database calls: `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })`, then `Resource.find({ scanId, userId, resourceId: { $in: [...] } })` via enrichment.
 - Return values: array, `[]` if the scan has no recommendations.
- `getLatestRecommendations(userId)`

- Purpose: resource-enriched recommendation list for the user's most recent scan — the default Recommendations page view.
- Input: `userId`.
- Output: array of enriched plain objects, or `[]`.
- Internal steps: 1. `latestScan = Scan.findOne({ userId }).sort({ createdAt: -1 })`. 2. If `!latestScan`, return `[]`. 3. `Recommendation.find({ scanId: latestScan._id, userId }).sort({ createdAt: -1 })`. 4. Pass results through `enrichRecommendationsWithResources`.
- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 })`; `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })`; `Resource.find(...)` via enrichment.
- Return values: `[]` (no scan yet), or the enriched array.
- **`getRecommendationById(id, userId)`**
- Purpose: fetch a single recommendation by its own `_id`, scoped to the owning user — used before ignore/resolve actions and for standalone lookups.
- Input: `id` (Recommendation `_id`), `userId`.
- Output: `Recommendation` document, or `null` — **not** enriched (raw doc, unlike the list functions above).
- Internal steps: `Recommendation.findOne({ _id: id, userId })`.
- Database calls: `Recommendation.findOne({ _id: id, userId })`.
- Return values: doc, or `null`.
- **`ignoreRecommendation(id, userId)`**
- Purpose: mark a recommendation as dismissed by the user — this removes it from every `status: "OPEN"`-filtered aggregation elsewhere (e.g. `dashboard.repository.js`'s severity/savings/category counts).
- Input: `id`, `userId`.
- Output: updated `Recommendation` document (`status: "IGNORED"`), or `null` if not found/not owned by the user.
- Internal steps: `Recommendation.findOneAndUpdate({ _id: id, userId }, { status: "IGNORED" }, { new: true })`.
- Database calls: `Recommendation.findOneAndUpdate(...)`.
- Return values: updated doc, or `null`.
- **`resolveRecommendation(id, userId)`**
- Purpose: mark a recommendation as acted-on/fixed.
- Input: `id`, `userId`.
- Output: updated `Recommendation` document (`status: "RESOLVED"`), or `null`.
- Internal steps: `Recommendation.findOneAndUpdate({ _id: id, userId }, { status: "RESOLVED" }, { new: true })`.
- Database calls: `Recommendation.findOneAndUpdate(...)`.
- Return values: updated doc, or `null`.

Internal (unexported) helper — `enrichRecommendationsWithResources(recommendations)`: an in-application "join" used by both `getRecommendationsByScanId` and `getLatestRecommendations`. Steps: 1. If `recommendations.length === 0`, return `[]`. 2. `scanId = recommendations[0].scanId`, `userId = recommendations[0].userId` — taken from **only the first** recommendation in the batch. 3. `resourceIds = recommendations.map(rec => rec.resourceId)`. 4. `Resource.find({ scanId, userId, resourceId: { $in: resourceIds } })`. 5. Build a `Map` keyed by `resource.resourceId`. 6. For each recommendation: convert to a plain object via `.toObject()` if available; look up the matching resource by `resourceId`; if none found, return the bare doc; otherwise spread the doc plus `resourceName`, `resourceType`, `instanceType` (`resource.metadata?.instanceType`), `state`, `publicIp`, `privateIp`, `availabilityZone`, `launchTime`, and the full `metadata` object.

DATA FLOW

Each of the 4 scanners calls `saveRecommendation` once per analyzed resource that produced a finding. The Recommendations page's default view calls `getLatestRecommendations`; drilling into a historical scan calls `getRecommendationsByScanId`; the "Ignore"/"Resolve" actions call `ignoreRecommendation` / `resolveRecommendation` via `recommendation.service.js` → `recommendation.controller.js`, typically followed by an audit-log entry in the controller layer.

EXAMPLE EXECUTION

`getLatestRecommendations("u1")`: `Scan.findOne({ userId: "u1" }).sort({ createdAt: -1 })` finds scan `"s9"` → `Recommendation.find({ scanId: "s9", userId: "u1" }).sort({ createdAt: -1 })` returns 5 docs → `enrichRecommendationsWithResources` runs `Resource.find({ scanId: "s9", userId: "u1", resourceId: { $in: [...5 ids] } })` → merges each → returns 5 enriched plain objects carrying `resourceName` / `instanceType` /etc.

IMPORTANT NOTES

`getRecommendationById` intentionally returns the raw Mongoose doc unenriched, while the two list functions return enriched plain objects — callers must not assume a consistent shape between single-fetch and list-fetch results. `enrichRecommendationsWithResources` derives `scanId` / `userId` only from `recommendations[0]` — if ever called with a batch spanning multiple scans or users, every resource lookup would incorrectly use just the first item's `scanId` / `userId`, silently under-enriching the rest; in practice this is safe today because both callers always pass a batch that shares one `scanId` and `userId`. Status transitions (`ignoreRecommendation` / `resolveRecommendation`) never touch `monthlyCost` — consistent with the domain rule that real infra spend is unaffected by dismissing a recommendation. No explicit try/catch anywhere in this file; all DB errors propagate to `asyncHandler` / the error-handler middleware.

 `server/src/services/database/resource.repository.js`

PURPOSE

Persistence layer for scanned AWS `Resource` documents: bulk-save a scanner's mapped resource list, update a single resource's state, and fetch all resources recorded for a scan.

WHEN THIS FILE IS CALLED

`saveResources` fires once per AWS service, per scan, from each of the 4 scanners. `updateResourceState` / `getResourcesByScanId` fire from reconciliation logic and scan-detail/service-breakdown views.

WHO CALLS THIS FILE

`server/src/services/dashboard/serviceAggregation.service.js`, `server/src/services/recommendation/recommendation.service.js`, `server/src/services/scan/scanHistory.service.js`, and all four scanners (`ec2.scanner.js`, `iam.scanner.js`, `rds.scanner.js`, `s3.scanner.js`).

WHICH FILES IT DEPENDS ON

- `server/src/models/Resource.js` — the model this wraps.

EXPORTED FUNCTIONS

- `saveResources(scanId, resourceType, resources, userId)`
- Purpose: bulk-insert every resource a scanner discovered for one AWS service, normalizing each into the generic `Resource` shape.
- Input: `scanId`; `resourceType` (string, e.g. "EC2", "RDS", "S3", "IAM_USER", "IAM_ROLE"); `resources` — array of `{ resourceId, name, state, metadata }` (the caller/scanner is responsible for producing this shape, per the inline comment); `userId`.
- Output: array of the locally-built plain objects that were passed to `insertMany` — **not** the documents `insertMany` resolves to (see notes).
- Internal steps: 1. `formattedResources = resources.map(resource => ({ userId, scanId, resourceType, resourceId: resource.resourceId, name: resource.name, state: resource.state, metadata: resource.metadata })).` 2. `Resource.insertMany(formattedResources)`. 3. Return `formattedResources` (the array built in step 1, discarding `insertMany`'s resolved value).
- Database calls: `Resource.insertMany(formattedResources)`.
- Error handling: none explicit; `insertMany` throws/rejects on validation error, propagating to the calling scanner.
- Return values: the pre-insert `formattedResources` array (without Mongo-assigned `_id` / `createdAt` / etc.).
- `updateResourceState(scanId, resourceId, state, userId)`
- Purpose: update just the `state` field of one resource.
- Input: `scanId`, `resourceId`, `state` (string), `userId`.
- Output: updated `Resource` document, or `null` if not found.
- Internal steps: `Resource.findOneAndUpdate({ scanId, resourceId, userId }, { state }, { new: true })`.
- Database calls: `Resource.findOneAndUpdate({ scanId, resourceId, userId }, { state }, { new: true })`.
- Return values: doc, or `null`.
- `getResourcesByScanId(scanId, userId)`
- Purpose: fetch every resource recorded for one scan, all resource types mixed together.
- Input: `scanId`, `userId`.
- Output: array of `Resource` documents, sorted `createdAt` descending.
- Internal steps: `Resource.find({ scanId, userId }).sort({ createdAt: -1 })`.
- Database calls: `Resource.find({ scanId, userId }).sort({ createdAt: -1 })`.

- Return values: array, `[]` if none.

DATA FLOW

Each scanner calls `saveResources` once per AWS service, after mapping raw AWS SDK responses into the generic `{ resourceId, name, state, metadata }` shape via its mapper. `scanHistory.service.js` and `serviceAggregation.service.js` call `getResourcesByScanId` to build Scan Detail and service-breakdown views. `recommendation.service.js` uses this repository (find/update) when reconciling a recommendation action against live resource state.

EXAMPLE EXECUTION

`saveResources("scan1", "EC2", [{ resourceId: "i-0123", name: "web-1", state: "running", metadata: { instanceType: "t3.micro", averageCpu: 12.4 } }], "user1")` → `Resource.insertMany([{ userId: "user1", scanId: "scan1", resourceType: "EC2", resourceId: "i-0123", name: "web-1", state: "running", metadata: { ... } }])` → the function returns that same array back to the scanner.

IMPORTANT NOTES

`saveResources`'s return value is the pre-insert array built locally, not what `insertMany` resolves to — a caller expecting Mongo-assigned `_id`s or schema defaults on the returned objects won't get them. The `Resource` model has no index beyond `userId` (`index: true`), so `{ scanId, userId }` and `{ scanId, resourceId, userId }` query patterns used throughout this file are not backed by a dedicated compound index.

 `server/src/services/database/scan.repository.js`

PURPOSE

Lifecycle management for `Scan` documents: start a new scan run, mark it completed with final totals, and list/fetch scans for a user.

WHEN THIS FILE IS CALLED

`createScan` fires at the start of every scan run (`scanner.service.js`). `completeScan` fires once all per-service scanners finish. `getAllScans` / `getScanById` fire on Scan History and Scan Detail page loads.

WHO CALLS THIS FILE

`server/src/services/scan/scanHistory.service.js` and `server/src/services/scanner/scanner.service.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/Scan.js` — the model this wraps. Schema: `userId` (indexed), `startedAt` (default `Date.now`), `completedAt`, `status` (enum `RUNNING|COMPLETED|FAILED`, default `RUNNING`), `totalResources` (default 0), `estimatedSavings` (default 0); `timestamps: true`.

EXPORTED FUNCTIONS

- `createScan(userId)`
 - Purpose: start a new scan run.
 - Input: `userId`.
 - Output: new `Scan` document with `status: "RUNNING"` (schema defaults apply automatically for `startedAt`, `totalResources`, `estimatedSavings`).
 - Internal steps: `Scan.create({ userId, status: "RUNNING" })`.
 - Database calls: `Scan.create({ userId, status: "RUNNING" })`.
 - Return values: the created doc.
- `completeScan(scanId, totalResources, estimatedSavings = 0)`
 - Purpose: finalize a scan once all 4 service scanners finish, recording totals.
 - Input: `scanId`; `totalResources` (number); `estimatedSavings` (number, defaults to 0 if omitted).
 - Output: updated `Scan` document, or `null` if `scanId` doesn't exist.
 - Internal steps: `Scan.findByIdAndUpdate(scanId, { status: "COMPLETED", completedAt: new Date(), totalResources, estimatedSavings }, { new: true })`.
 - Database calls: `Scan.findByIdAndUpdate(scanId, {...}, { new: true })` — note this uses `findByIdAndUpdate`, **not** scoped by `userId` at the query level (see notes).
 - Return values: doc, or `null`.

- `getAllScans(userId)`
- Purpose: scan-history list for a user.
- Input: `userId`.
- Output: array of `Scan` documents, sorted `createdAt` descending.
- Internal steps: `Scan.find({ userId }).sort({ createdAt: -1 })`.
- Database calls: `Scan.find({ userId }).sort({ createdAt: -1 })`.
- Return values: array, `[]` if none.
- `getScanById(id, userId)`
- Purpose: fetch one scan by id, scoped to its owner.
- Input: `id`, `userId`.
- Output: `Scan` document, or `null`.
- Internal steps: `Scan.findOne({ _id: id, userId })`.
- Database calls: `Scan.findOne({ _id: id, userId })`.
- Return values: doc, or `null`.

DATA FLOW

`scanner.service.js` calls `createScan` at the start of a scan run, runs all 4 service-specific scanners, then calls `completeScan` with the aggregate `totalResources` / `estimatedSavings` once done. `scanHistory.service.js` calls `getAllScans` / `getScanById` to populate the Scan History and Scan Detail pages.

EXAMPLE EXECUTION

`createScan("u1")` → `Scan.create({ userId: "u1", status: "RUNNING" })` → `{ _id: "s1", userId: "u1", status: "RUNNING", startedAt: <now>, totalResources: 0, estimatedSavings: 0 }`. Later: `completeScan("s1", 42, 310.5)` → `Scan.findByIdAndUpdate("s1", { status: "COMPLETED", completedAt: <now>, totalResources: 42, estimatedSavings: 310.5 }, { new: true })` → returns the updated doc.

IMPORTANT NOTES

`completeScan` uses `findByIdAndUpdate` rather than a `userId`-scoped `findOneAndUpdate` — the one write path in this file not defensively re-scoped to the requesting user at the query level; this is safe only because `scanId` is generated server-side during the same request flow and never accepted directly from client input for this call. This repository has no function to transition a scan to `"FAILED"`, even though the `Scan` model's `status` enum includes that value — if a scan run errors mid-flight, nothing in this file moves it out of `"RUNNING"`.

 `server/src/services/database/schedule.repository.js`

PURPOSE

CRUD for a user's recurring scan `Schedule` (one per user, enforced by a unique index), plus one cross-tenant query the scheduler cron uses to find schedules that are due to run.

WHEN THIS FILE IS CALLED

The CRUD functions fire from the Schedule Scan page's create/edit/delete flows. `getDueSchedules` fires once per scheduler cron tick.

WHO CALLS THIS FILE

`server/src/services/schedule/schedule.service.js` and `server/src/services/scheduler/scheduler.service.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/Schedule.js` — the model this wraps. Schema: `userId` (unique), `frequency` (enum `hourly|daily|weekly`, required), `enabled` (default `true`), `lastRun` (default `null`), `nextRun` (required), `createdAt` (default `Date.now`); no `timestamps` option (no `updatedAt`).

EXPORTED FUNCTIONS

- `createSchedule({ userId, frequency, enabled, nextRun })`
- Output: newly created `Schedule` document.
- Internal steps / DB calls: `Schedule.create({ userId, frequency, enabled, nextRun })`.
- Error handling: throws a duplicate-key error if the user already has a schedule (unique index on `userId`).

- `getScheduleByUserId(userId)`
- Output: `Schedule` document, or `null`.
- Internal steps / DB calls: `Schedule.findOne({ userId })`.
- `getScheduleById(id, userId)`
- Output: `Schedule` document, or `null`.
- Internal steps / DB calls: `Schedule.findOne({ _id: id, userId })`.
- `updateSchedule(id, userId, updates)`
- Output: updated `Schedule` document, or `null`.
- Internal steps / DB calls: `Schedule.findOneAndUpdate({ _id: id, userId }, updates, { new: true })`.
- `deleteSchedule(id, userId)`
- Output: the deleted `Schedule` document, or `null`.
- Internal steps / DB calls: `Schedule.findOneAndDelete({ _id: id, userId })`.
- `getDueSchedules(now)`
- Purpose: find every enabled schedule across all users whose `nextRun` has passed — used only by the scheduler cron tick.
- Input: `now` (a `Date`).
- Output: array of `Schedule` documents.
- Internal steps / DB calls: `Schedule.find({ enabled: true, nextRun: { $lte: now } })` — deliberately **not** scoped by `userId` (per the inline comment: "it needs every user's due schedules, not one user's").
- Return values: array, `[]` if nothing is due.

DATA FLOW

Schedule Scan page create/edit/delete actions go through `schedule.service.js` into these CRUD functions, each scoped to the requesting user (and effectively one-per-user, enforced by the unique `userId` index). Independently, `scheduler.service.js`'s cron tick calls `getDueSchedules(new Date())` to find every enabled, overdue schedule across all users, triggers `scanner.service.js` for each, and then updates `lastRun` / `nextRun` via `updateSchedule`.

EXAMPLE EXECUTION

```
getDueSchedules(new Date("2026-08-01T00:00:00Z")) → Schedule.find({ enabled: true, nextRun: { $lte: "2026-08-01T00:00:00Z" } }) → [{ userId: "u1", frequency: "daily", nextRun: "2026-07-31T23:00:00Z", ... }, ...].
```

IMPORTANT NOTES

`getDueSchedules` is, like `notification.repository.js`'s `getUsersWithWeeklyReportEnabled`, an intentional exception to the per-`userId` scoping rule — a system-level cron query spanning all tenants. The unique index on `Schedule.userId` means `createSchedule` throws a duplicate-key error if the user already has a schedule; this file has no upsert convenience, so callers must check `getScheduleByUserId` first and choose create vs. update themselves.

 `server/src/services/database/user.repository.js`

PURPOSE

A minimal 3-function wrapper over the `User` model, used exclusively by the auth flow and by notification lookups: find by email (login/signup duplicate check), find by id, and create.

WHEN THIS FILE IS CALLED

On every login and signup attempt, and wherever a user's own record needs to be resolved by id outside the auth middleware (e.g. notification email lookups).

WHO CALLS THIS FILE

`server/src/services/auth/auth.service.js` and `server/src/services/notification/notification.service.js`.

WHICH FILES IT DEPENDS ON

- `server/src/models/User.js` — the model this wraps. Schema: `fullName`, `email` (unique, trimmed, lowercased), `password` (hashed), `tokenVersion` (default `0`, bumped on "logout from all devices"); `timestamps: true`; `toJSON` transform strips `password`.

EXPORTED FUNCTIONS

- `findUserByEmail(email)`
 - Purpose: credential lookup for login, and duplicate-email check for signup.
 - Input: `email`.
 - Output: `User` document, or `null`.
 - Internal steps / DB calls: `User.findOne({ email })`.
 - Return values: doc, or `null`.
- `findUserById(userId)`
 - Purpose: resolve a user's own record by id.
 - Input: `userId`.
 - Output: `User` document, or `null`.
 - Internal steps / DB calls: `User.findById(userId)`.
 - Return values: doc, or `null`.
- `createUser(userData)`
 - Purpose: persist a new user at signup.
 - Input: `userData` — expected to already contain a hashed `password` (hashing happens in `auth.service.js`, not here).
 - Output: newly created `User` document.
 - Internal steps / DB calls: `User.create(userData)`.
 - Error handling: throws a duplicate-key error if `email` already exists (unique index).
 - Return values: the created doc.

DATA FLOW

`auth.service.js`'s login flow calls `findUserByEmail`, compares the bcrypt hash, and issues a JWT; its signup flow calls `findUserByEmail` first to reject duplicates, then `createUser`. Note: `server/src/middleware/auth.middleware.js` imports `models/User.js` directly (per the dependency graph) rather than going through this repository for its own per-request user lookup — `findUserById` here is used by other callers (e.g. `notification.service.js`), not by the auth middleware itself.

EXAMPLE EXECUTION

`findUserByEmail("akshatjohri03@gmail.com")` → `User.findOne({ email: "akshatjohri03@gmail.com" })` → the matching doc, or `null` if unregistered.

IMPORTANT NOTES

No password hashing, comparison, or JWT logic lives here — this file is pure data access; all business logic lives in `services/auth/auth.service.js`. Per the dependency graph, `server/src/middleware/auth.middleware.js` bypasses this repository and imports `models/User.js` directly — an inconsistency in how the `User` model is accessed across the codebase.

 `server/src/mappers/ec2.mapper.js`

PURPOSE

Pure transformation layer that converts a raw AWS SDK v3 `DescribeInstances` response into a flat array of plain instance objects that the rest of the app (analyzer/recommender/scanner) works with — decoupling downstream code from the SDK's nested `Reservations[].Instances[]` shape.

WHEN THIS FILE IS CALLED

Called once per `DescribeInstances` response, from `services/aws/ec2.service.js`'s instance-listing function, during `ec2.scanner.js`'s per-scan EC2 discovery step.

WHO CALLS THIS FILE

`server/src/services/aws/ec2.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

None (`imports: []` — a pure function with no internal dependencies).

EXPORTED FUNCTIONS

- `mapEC2Instances(response)`
- Purpose: flatten AWS's `Reservations[].Instances[]` nesting into one array, extracting the `Name` tag, security groups, and network/lifecycle fields.
- Input: `response` — raw `EC2Client DescribeInstancesCommand` output: `{ Reservations: [{ Instances: [...] }] }`.
- Output: array of plain objects: `{ instanceId, name, state, instanceType, availabilityZone, publicIp, privateIp, launchTime, securityGroups: [{ groupId, groupName }] }`.
- Internal steps: 1. `instances = []` . 2. For each `reservation` in `response.Reservations || []` . 3. For each `instance` in `reservation.Instances || []` , push `{ instanceId: instance.InstanceId, name: instance.Tags?.find(tag => tag.Key === "Name")?.Value || "N/A", state: instance.State.Name, instanceType: instance.InstanceType, availabilityZone: instance.Placement.AvailabilityZone, publicIp: instance.PublicIpAddress || null, privateIp: instance.PrivateIpAddress, launchTime: instance.LaunchTime, securityGroups: (instance.SecurityGroups || []).map(group => ({ groupId: group.GroupId, groupName: group.GroupName })) }` . 4. Return `instances` .
- AWS calls: none directly — this is a pure mapper; the actual `DescribeInstancesCommand` call happens in `ec2.service.js` , which hands its response to this function.
- Error handling: none — if `instance.State` or `instance.Placement` is missing/malformed, accessing `.Name` / `.AvailabilityZone` throws a `TypeError` that propagates uncaught.
- Return values: array, `[]` if `response.Reservations` is empty/undefined.

DATA FLOW

`ec2.service.js` calls `EC2Client.send(new DescribeInstancesCommand(...))` , then passes the raw response into `mapEC2Instances` to get a flat, analyzer-friendly array; `ec2.scanner.js` consumes that array for CPU analysis, pricing lookups, and recommendation generation.

EXAMPLE EXECUTION

Input: `{ Reservations: [{ Instances: [{ InstanceId: "i-0abc", Tags: [{ Key: "Name", Value: "web-1" }], State: { Name: "running" }, InstanceType: "t3.micro", Placement: { AvailabilityZone: "us-east-1a" }, PublicIpAddress: "1.2.3.4", PrivateIpAddress: "10.0.0.5", LaunchTime: "2025-01-01T00:00:00Z", SecurityGroups: [{ GroupId: "sg-1", GroupName: "default" }] }] }] }` . Output: `[{ instanceId: "i-0abc", name: "web-1", state: "running", instanceType: "t3.micro", availabilityZone: "us-east-1a", publicIp: "1.2.3.4", privateIp: "10.0.0.5", launchTime: "2025-01-01T00:00:00Z", securityGroups: [{ groupId: "sg-1", groupName: "default" }] }] }` .

IMPORTANT NOTES

An instance with no `"Name"` tag maps to the literal string `"N/A"` rather than `null` / `undefined` . The function assumes `instance.State` and `instance.Placement` are always present (true of AWS's real API contract, but not defensively coded against malformed/mock input).

 `server/src/mappers/iam.mapper.js`

PURPOSE

Converts raw IAM `ListUsers` / `ListRoles` SDK responses into plain objects, plus a shared helper (`extractTrustedServices`) that parses a role's trust policy to determine which AWS services can assume it.

WHEN THIS FILE IS CALLED

Called from `services/aws/iam.service.js` right after its `ListUsersCommand` / `ListRolesCommand` calls return, during `iam.scanner.js` 's per-scan IAM discovery step.

WHO CALLS THIS FILE

`server/src/services/aws/iam.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

- `server/src/utils/iamPolicy.util.js` — for `parsePolicyDocument` (URI-decodes and JSON-parses a raw policy document string, returning `null` on failure) and `toArray` (normalizes a single-value-or-array field into an array).

EXPORTED FUNCTIONS

- `mapIAMUsers(response)`
- Purpose: flatten a `ListUsers` response.

- Input: `response : { Users: [...] } .`
- Output: array of `{ userName, arn, createDate, passwordLastUsed } .`
- Internal steps: `(response.Users || []).map(user => ({ userName: user.UserName, arn: user.Arn, createDate: user.CreateDate, passwordLastUsed: user.PasswordLastUsed || null }))) .`
- Return values: array, `[]` if `response.Users` is empty/undefined.
- `mapIAMRoles(response)`
- Purpose: flatten a `ListRoles` response, attaching each role's trusted services.
- Input: `response : { Roles: [...] } .`
- Output: array of `{ roleName, arn, path, createDate, lastUsedDate, trustedServices } .`
- Internal steps: `(response.Roles || []).map(role => ({ roleName: role.RoleName, arn: role.Arn, path: role.Path, createDate: role.CreateDate, lastUsedDate: role.RoleLastUsed?.LastUsedDate || null, trustedServices: extractTrustedServices(role.AssumeRolePolicyDocument) })))` — calls the internal `extractTrustedServices` helper per role.
- Return values: array, `[]` if `response.Roles` is empty/undefined.

Internal (unexported) helper — `extractTrustedServices(assumeRolePolicyDocument)` : from a role's URI-encoded JSON trust policy, collects every AWS service principal named in an `Allow` statement (e.g. `"ec2.amazonaws.com"`). Steps: 1. `policy = parsePolicyDocument(assumeRolePolicyDocument)` ; if `null`, return `[]` . 2. `services = new Set()` . 3. For each `statement` in `toArray(policy.Statement)` : skip if `statement.Effect !== "Allow"` ; for each `service` in `toArray(statement.Principal?.Service)` , `services.add(service)` . 4. Return `Array.from(services)` (deduplicated).

DATA FLOW

`iam.service.js` calls `IAMClient.send(new ListUsersCommand(...)) / send(new ListRolesCommand(...))` , then passes each response through the corresponding mapper. `iam.scanner.js` subsequently filters out AWS-managed service-linked roles (identified via the mapped `path` field's `"/aws-service-role/"` prefix, per the inline comment referencing `iam.analyzer.js` 's `isServiceLinkedRole`) before analysis/recommendation.

EXAMPLE EXECUTION

Input: `{ Roles: [{ RoleName: "lambda-exec", Arn: "arn:aws:iam::123:role/lambda-exec", Path: "/", CreateDate: "2024-01-01", RoleLastUsed: { LastUsedDate: "2026-07-01" }, AssumeRolePolicyDocument: "%7B%22Statement%22... (encoded)..." } ] } .`
`mapIAMRoles(...)` → `[{ roleName: "lambda-exec", arn: "arn:aws:iam::123:role/lambda-exec", path: "/", createDate: "2024-01-01", lastUsedDate: "2026-07-01", trustedServices: ["lambda.amazonaws.com"] } ] .`

IMPORTANT NOTES

`path` is what `iam.analyzer.js` 's `isServiceLinkedRole` helper uses to distinguish AWS-managed service-linked roles (path starting `"/aws-service-role/"`) from customer-managed roles — this mapper's only job is to surface `path` faithfully; the filtering logic itself lives in the analyzer, not here. `extractTrustedServices` silently returns `[]` for an unparseable or missing trust policy rather than throwing (via `parsePolicyDocument` 's internal try/catch).

 `server/src/mappers/rds.mapper.js`

PURPOSE

Converts a raw `DescribeDBInstances` SDK response into a flat array of plain RDS instance objects, normalizing several optional/inconsistent AWS fields along the way.

WHEN THIS FILE IS CALLED

Called from `services/aws/rds.service.js` right after its `DescribeDBInstancesCommand` call returns, during `rds.scanner.js` 's per-scan RDS discovery step.

WHO CALLS THIS FILE

`server/src/services/aws/rds.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `mapRDSInstances(response)`

- Purpose: flatten a `DescribeDBInstances` response.
- Input: `response : { DBInstances: [...] }`.
- Output: array of `{ dbInstanceId: identifier, dbInstanceArn, engine, engineVersion, instanceClass, storageType, allocatedStorage, multiAZ, publiclyAccessible, backupRetentionPeriod, storageEncrypted, status, endpoint: { address, port } | null, availabilityZone }`.
- Internal steps: `(response.DBInstances || []).map(instance => ({ dbInstanceId: instance.DBInstanceId, dbInstanceArn: instance.DBInstanceArn, engine: instance.Engine, engineVersion: instance.EngineVersion, instanceClass: instance.DBInstanceClass, storageType: instance.StorageType, allocatedStorage: instance.AllocatedStorage || 0, multiAZ: Boolean(instance.MultiAZ), publiclyAccessible: Boolean(instance.PubliclyAccessible), backupRetentionPeriod: instance.BackupRetentionPeriod ?? 0, storageEncrypted: Boolean(instance.StorageEncrypted), status: instance.DBInstanceStatus, endpoint: instance.Endpoint ? { address: instance.Endpoint.Address, port: instance.Endpoint.Port } : null, availabilityZone: instance.AvailabilityZone })))`.
- Return values: array, `[]` if `response.DBInstances` is empty/undefined.

DATA FLOW

`rds.service.js` sends `DescribeDBInstancesCommand`, hands the response to `mapRDSInstances`, and `rds.scanner.js` consumes the flat array for CPU analysis (via CloudWatch), pricing lookups (`pricing.filter.builder.js`'s `buildRDSPrimingFilters` reads `instanceClass` / `availabilityZone` / `engine` / `multiAZ` directly from this shape), and recommendation generation.

EXAMPLE EXECUTION

Input: `{ DBInstances: [{ DBInstanceId: "prod-db", DBInstanceArn: "arn:...", Engine: "postgres", EngineVersion: "15.3", DBInstanceClass: "db.t3.medium", StorageType: "gp2", AllocatedStorage: 100, MultiAZ: true, PubliclyAccessible: false, StorageEncrypted: true, DBInstanceStatus: "available", Endpoint: { Address: "prod-db.abc.us-east-1.rds.amazonaws.com", Port: 5432 }, AvailabilityZone: "us-east-1a" }] }` (note: no `BackupRetentionPeriod` field present). Output: `[{ dbInstanceId: "prod-db", ..., backupRetentionPeriod: 0, endpoint: { address: "prod-db.abc.us-east-1.rds.amazonaws.com", port: 5432 }, ... }]`.

IMPORTANT NOTES

`backupRetentionPeriod` deliberately uses `??` instead of `||` — per the inline comment, AWS omits this field entirely (rather than sending an explicit `0`) when backups are disabled on some engine/version combinations, so `??` is used to map "field absent" to `0`. Functionally, given the only falsy possibilities here are `0` / `undefined` / `null`, `?? 0` and `|| 0` behave identically — the operator choice documents intent rather than changing behavior.

 `server/src/mappers/s3.mapper.js`

PURPOSE

The thinnest mapper in the layer — flattens a raw `ListBuckets` SDK response into a minimal array of `{ bucketName, creationDate }`. Richer S3 bucket detail (region, encryption, versioning, lifecycle, public access) is fetched per-bucket via separate SDK calls elsewhere and is not routed through this mapper.

WHEN THIS FILE IS CALLED

Called from `services/aws/s3.service.js` right after its `ListBucketsCommand` call returns, at the start of `s3.scanner.js`'s per-scan S3 discovery step.

WHO CALLS THIS FILE

`server/src/services/aws/s3.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `mapS3Buckets(response)`
- Purpose: flatten a `ListBuckets` response.
- Input: `response : { Buckets: [...] }`.
- Output: array of `{ bucketName, creationDate }`.

- Internal steps: 1. `buckets = []` . 2. For each `bucket` in `response.Buckets || []` , push `{ bucketName: bucket.Name, creationDate: bucket.CreationDate }` . 3. Return `buckets` .
- AWS calls: none — pure mapper; `s3.service.js` makes the actual `ListBucketsCommand` call and separately issues per-bucket detail calls (not routed through this mapper).
- Return values: array, `[]` if `response.Buckets` is empty/undefined.

DATA FLOW

`s3.service.js` calls `S3Client.send(new ListBucketsCommand(...))` , maps the response through `mapS3Buckets` to get the bucket name/creation-date list, then separately issues per-bucket `GetBucketLocation` / `GetBucketEncryption` /etc. calls that `s3.scanner.js` and `s3.analyzer.js` consume directly (outside this mapper).

EXAMPLE EXECUTION

Input: `{ Buckets: [{ Name: "my-app-logs", CreationDate: "2023-05-01T00:00:00Z" }] }` . Output: `[{ bucketName: "my-app-logs", creationDate: "2023-05-01T00:00:00Z" }]` .

IMPORTANT NOTES

At 17 lines, this is by far the smallest of the 4 mappers, capturing only 2 fields — unlike the EC2/RDS/IAM mappers, which flatten most of the resource's descriptive metadata in one pass. All richer S3 metadata is assembled downstream by `s3.service.js` / `s3.analyzer.js` from separate per-bucket API calls.

 `server/src/builders/pricing.filter.builder.js`

PURPOSE

Builds the AWS Price List API's filter key/value objects for EC2, S3, and RDS pricing lookups — translating each service's raw resource attributes (instance type, availability zone, engine, etc.) into the specific filter vocabulary the Pricing API expects.

WHEN THIS FILE IS CALLED

Called once per resource, during each scanner's per-resource pricing step, immediately before `services/aws/pricing.service.js` queries the AWS Price List API.

WHO CALLS THIS FILE

`server/src/services/scanner/ec2.scanner.js` , `server/src/services/scanner/rds.scanner.js` , `server/src/services/scanner/s3.scanner.js` .

WHICH FILES IT DEPENDS ON

- `server/src/utils/region.util.js` — for `getLocationFromAvailabilityZone` (converts an AZ like `"us-east-1a"` to the Pricing API's human-readable location string, e.g. `"US East (N. Virginia)"` , via the `REGIONS` map) and `getLocationFromRegion` (same lookup from a bare region code rather than an AZ).

EXPORTED FUNCTIONS

- `buildEC2PricingFilters(instance)`
- Purpose: build Pricing API filters for one EC2 instance.
- Input: `instance` — object with at least `{ instanceType, availabilityZone }` (the shape produced by `ec2.mapper.js`).
- Output: `{ instanceType, location, operatingSystem: "Linux", tenancy: "Shared", capacityStatus: "Used", preInstalledSw: "NA" }` .
- Internal steps: 1. `location = getLocationFromAvailabilityZone(instance.availabilityZone)` . 2. Return the object with `instance.instanceType` , the computed `location` , and 4 hardcoded constants.
- Return values: plain filter object. `operatingSystem` is hardcoded to `"Linux"` regardless of the instance's actual OS (see notes).
- `buildS3PricingFilters(region)`
- Purpose: build Pricing API filters for S3 Standard storage pricing in a given region.
- Input: `region` — bare AWS region code (e.g. `"us-east-1"`).
- Output: `{ location, volumeType: "Standard" }` .
- Internal steps: 1. `location = getLocationFromRegion(region)` . 2. Return `{ location, volumeType: "Standard" }` .
- Return values: plain filter object. `volumeType` is hardcoded to `"Standard"` regardless of the bucket's actual storage class.

- `buildRDSPricingFilters(instance)`
- Purpose: build Pricing API filters for one RDS instance.
- Input: `instance` — object with `{ instanceClass, availabilityZone, engine, multiAZ }` (the shape produced by `rds.mapper.js`).
- Output: `{ instanceType, location, databaseEngine, deploymentOption }`.
- Internal steps: 1. `location = getLocationFromAvailabilityZone(instance.availabilityZone)` 2. `databaseEngine = getPricingDatabaseEngine(instance.engine)` (internal helper, below). 3. `deploymentOption = instance.multiAZ ? "Multi-AZ" : "Single-AZ"` 4. Return `{ instanceType: instance.instanceClass, location, databaseEngine, deploymentOption }`.
- Return values: plain filter object.

Internal (unexported) helper — `getPricingDatabaseEngine(engine)` : maps a raw RDS engine code to the Pricing API's `databaseEngine` filter value via the `ENGINE_TO_PRICING_NAME` lookup table: `mysql→"MySQL"`, `postgres→"PostgreSQL"`, `mariadb→"MariaDB"`, `oracle-se2 / oracle-se2-cdb / oracle-ee / oracle-ee-cdb → "Oracle"`, `sqlserver-ex / sqlserver-web / sqlserver-se / sqlserver-ee → "SQL Server"`, `aurora-mysql→"Aurora MySQL"`, `aurora-postgresql→"Aurora PostgreSQL"`. If `engine` isn't in the table, falls back to capitalizing its first letter (`engine.charAt(0).toUpperCase() + engine.slice(1)`); if `engine` itself is falsy, it's returned as-is.

AWS calls: none directly — this file only builds filter parameter objects; the actual Pricing API `GetProducts` call happens in `services/aws/pricing.service.js`, which consumes these filter objects.

Error handling: none explicit anywhere in this file. If an availability zone/region isn't present in the `REGIONS` map (only 4 regions are defined: `ap-south-1`, `us-east-1`, `ap-southeast-1`, `ap-northeast-1`), `location` resolves to `undefined`, and the resulting filter object is passed on to `pricing.service.js` with an undefined `location` — not caught here, and likely to cause the downstream Pricing API lookup to fail or return no results.

DATA FLOW

A scanner maps raw AWS resource data via its mapper, then calls the matching `build*PricingFilters` function to translate that resource's attributes into Price List API filter terms, then passes the filters into `pricing.service.js`'s `GetProducts` call, then hands the raw pricing response to `pricing.parser.js` to extract a usable price number.

EXAMPLE EXECUTION

```
buildRDSPricingFilters({ instanceClass: "db.t3.medium", availabilityZone: "us-east-1a", engine: "aurora-postgresql",
multiAZ: true }): location = getLocationFromAvailabilityZone("us-east-1a") → REGIONS["us-east-1"] → "US East (N. Virginia)"; databaseEngine = getPricingDatabaseEngine("aurora-postgresql") → "Aurora PostgreSQL" (table lookup). Returns {
instanceType: "db.t3.medium", location: "US East (N. Virginia)", databaseEngine: "Aurora PostgreSQL", deploymentOption: "Multi-AZ" }.
```

IMPORTANT NOTES

`buildEC2PricingFilters` always sets `operatingSystem: "Linux"` — CloudLens does not price Windows EC2 instances differently even if the actual instance runs Windows; this is a hardcoded assumption, not derived from the instance's actual platform. The `REGIONS` map (`constants/regions.js`) only has 4 entries — any resource in a region outside this set produces `location: undefined` in the filters, silently, with no error raised in this file. `getLocationFromAvailabilityZone` does `availabilityZone.slice(0, -1)` to strip the trailing AZ letter (e.g. `"us-east-1a" → "us-east-1"`) — this throws if `availabilityZone` is `undefined` / `null` (not defensively checked).

 `server/src/parser/pricing.parser.js`

PURPOSE

Extracts a single usable USD price number out of the AWS Price List API's deeply-nested `GetProducts` response: one function for on-demand hourly compute pricing (EC2/RDS), one for tiered per-GB storage pricing (S3).

WHEN THIS FILE IS CALLED

Called immediately after `services/aws/pricing.service.js` receives a `GetProductsCommand` response, once per resource, during each scanner's pricing step.

WHO CALLS THIS FILE

`server/src/services/scanner/ec2.scanner.js`, `server/src/services/scanner/rds.scanner.js`, `server/src/services/scanner/s3.scanner.js`.

WHICH FILES IT DEPENDS ON

None.

EXPORTED FUNCTIONS

- `extractHourlyPrice(pricingResponse)`
- Purpose: pull the on-demand hourly USD price out of a Pricing API response for one SKU.
- Input: `pricingResponse` — raw `PricingClient GetProductsCommand` output: `{ PriceList: [stringifiedJSON, ...] }`.
- Output: `number` (`0` if no price data).
- Internal steps: 1. If `!pricingResponse?.PriceList?.length`, return `0`. 2. `product = JSON.parse(pricingResponse.PriceList[0])` — only the **first** `PriceList` entry is used. 3. `onDemand = Object.values(product.terms.OnDemand)[0]` — the first `OnDemand` term. 4. `priceDimension = Object.values(onDemand.priceDimensions)[0]` — the first price dimension. 5. Return `Number(priceDimension.pricePerUnit.USD)`.
- Error handling: only the top-level `PriceList.length` check guards against empty input; if `product.terms.OnDemand` is missing/malformed, or `PriceList[0]` isn't valid JSON, this throws uncaught (`JSON.parse` error or a `TypeError` on `undefined.terms`).
- Return values: a `number`; `0` is the explicit no-match fallback.
- `extractStoragePricePerGB(pricingResponse)`
- Purpose: pull the first-tier (0–X GB) per-GB storage price for S3, since S3 storage pricing is tiered by usage volume (e.g. first 50 TB, next 450 TB, over 500 TB), each exposed as its own `priceDimension` with a `beginRange`.
- Input: `pricingResponse` — same `PriceList` shape as above.
- Output: `number` (`0` if no price data).
- Internal steps: 1. If `!pricingResponse?.PriceList?.length`, return `0`. 2. `product = JSON.parse(pricingResponse.PriceList[0])`. 3. `onDemand = Object.values(product.terms.OnDemand)[0]`. 4. `priceDimensions = Object.values(onDemand.priceDimensions)` (all tiers, not just the first). 5. `firstTier = priceDimensions.find(dimension => dimension.beginRange === "0") || priceDimensions[0]` — explicitly looks for the tier starting at usage `0`, falling back to whichever tier is first in the object if none has `beginRange === "0"`. 6. Return `Number(firstTier.pricePerUnit.USD)`.
- Error handling: same as `extractHourlyPrice` — no protection beyond the initial length guard.
- Return values: a `number`; `0` as fallback.

AWS calls: none directly — pure parsing of an already-fetched Pricing API response; the actual `GetProductsCommand` call happens in `services/aws/pricing.service.js`.

DATA FLOW

`pricing.service.js` calls `PricingClient.send(new GetProductsCommand(filters))` using the filters built by `pricing.filter.builder.js`, then hands the raw response to whichever `extract*` function matches the resource type; the scanner feeds the resulting hourly/per-GB price into its calculator (`savings.calculator.js` / `rds.calculator.js` / `storage.calculator.js`) to derive `monthlyCost` and `estimatedSavings` for the eventual `Recommendation` document.

EXAMPLE EXECUTION

`pricingResponse = { PriceList: [ '{"terms":{"OnDemand":{"XYZ.JRTCKXETXF":{"priceDimensions":{"XYZ.JRTCKXETXF.6YS6EN2CT7":{"pricePerUnit":{"USD":{"0.0416000000"}}}}}}' ] } }` → `extractHourlyPrice(pricingResponse)`: `product.terms.OnDemand` 's first value → its `priceDimensions` ' first value → `Number("0.0416000000")` → `0.0416`.

IMPORTANT NOTES

Both functions blindly take `Object.values(...)[0]` / `PriceList[0]` — if the Pricing API ever returns multiple `PriceList` entries or multiple `OnDemand` terms for a query (possible if the filters aren't specific enough), only the first is used with no validation that it's the "right" one. `extractStoragePricePerGB` 's `beginRange === "0"` match is a **string** comparison, not numeric — it relies on the Pricing API's documented convention of using the literal string `"0"` for the first tier.

`server/src/factories/awsClient.factory.js`

PURPOSE

The single request-scoped factory for obtaining a full set of AWS SDK v3 clients (EC2, CloudWatch, Pricing, S3, RDS, IAM) for one user's connected AWS account. It looks up that user's `AwsConnection`, decrypts the stored secret key, and constructs fresh clients from those credentials — ensuring every scan uses per-tenant credentials rather than a shared global client or server-wide `.env` credentials.

WHEN THIS FILE IS CALLED

Once at the start of every scan run, before `scanner.service.js` dispatches to the 4 per-service scanners.

WHO CALLS THIS FILE

`server/src/services/scanner/scanner.service.js` — the sole importer.

WHICH FILES IT DEPENDS ON

- `server/src/config/aws.js` — for the `createEC2Client` / `createCloudWatchClient` / `createPricingClient` / `createS3Client` / `createRDSClient` / `createIAMClient` factory functions that actually instantiate each AWS SDK v3 client via a shared config-building helper (`buildClientConfig`).
- `server/src/models/AwsConnection.js` — queried to find the user's stored connection (`accessKeyId`, `encryptedSecretKey`, `region`).
- `server/src/utils/encryption.js` — for `decrypt`, to turn the stored `encryptedSecretKey` (AES-256-GCM, `iv:authTag:ciphertext` hex format) back into a usable plaintext secret access key.

EXPORTED FUNCTIONS

- `getAwsClientsForUser(userId)`
- Purpose: resolve one user's AWS credentials and build every AWS SDK client the scanner layer needs, in one call.
- Input: `userId`.
- Output: `{ ec2Client, cloudWatchClient, pricingClient, s3Client, rdsClient, iamClient }` — one instantiated AWS SDK v3 client per service.
- Internal steps: 1. `connection = AwsConnection.findOne({ userId })` 2. `credentials = connection ? { accessKeyId: connection.accessKeyId, secretAccessKey: decrypt(connection.encryptedSecretKey) } : undefined` 3. `region = connection?.region` 4. Return an object built by calling each `create*Client(credentials, region)` factory from `config/aws.js` — `pricingClient` is built via `createPricingClient(credentials)` only (no region argument), since `config/aws.js`'s `createPricingClient` always forces `"us-east-1"` internally (the Pricing API is only served from that endpoint).
- Database calls: `AwsConnection.findOne({ userId })` — the only DB call in this file.
- AWS calls (client construction, not live API calls):
 - `createEC2Client(credentials, region) → new EC2Client({ region, credentials? })`
 - `createCloudWatchClient(credentials, region) → new CloudWatchClient({ region, credentials? })`
 - `createPricingClient(credentials) → new PricingClient({ region: "us-east-1", credentials? })` — always `us-east-1` regardless of the connection's region
 - `createS3Client(credentials, region) → new S3Client({ region, credentials? })`
 - `createRDSClient(credentials, region) → new RDSClient({ region, credentials? })`
 - `createIAMClient(credentials, region) → new IAMClient({ region, credentials? })` — IAM is a global service but still takes the same per-region config shape
- (`config/aws.js`'s `buildClientConfig` helper: `{ region: region || process.env.AWS_REGION }`, plus `.credentials = { accessKeyId, secretAccessKey }` only if both are truthy)
- Error handling: none explicit in this file. If the user has no `AwsConnection`, `credentials` and `region` both resolve to `undefined`, and every `create*Client` call then falls back to the AWS SDK's own default provider chain (env vars/shared config) and `process.env.AWS_REGION`, rather than throwing. If `decrypt()` fails (e.g. malformed `encryptedSecretKey`, wrong `ENCRYPTION_KEY`), it throws synchronously and propagates uncaught to the caller.
- Return values: always the same 6-client object shape; never `null` / `undefined` itself — only the underlying `credentials` / `region` passed into each client can be `undefined` for a not-yet-connected user.

DATA FLOW

`scanner.service.js` calls `getAwsClientsForUser(userId)` once per scan run, then passes the returned client bundle down into `ec2.scanner.js` / `rds.scanner.js` / `s3.scanner.js` / `iam.scanner.js`, each of which uses only the client(s) it needs (e.g. `ec2.scanner.js` uses `ec2Client` + `cloudWatchClient` + `pricingClient`).

EXAMPLE EXECUTION

```
getAwsClientsForUser("64f0a1...") : AwsConnection.findOne({ userId: "64f0a1..." }) finds { accessKeyId: "AKIA...", encryptedSecretKey: "ab12:cd34:ef56...", region: "ap-south-1" } → decrypt(encryptedSecretKey) → "wJalrXUtnFEMI/K7MDENG/..." → credentials = { accessKeyId: "AKIA...", secretAccessKey: "wJalr..." } → returns { ec2Client: new EC2Client({ region: "ap-south-1", credentials }), ..., pricingClient: new PricingClient({ region: "us-east-1", credentials }), ... }.
```

IMPORTANT NOTES

This is the only file in the documented data-access layer that constructs AWS SDK clients rather than issuing Mongoose queries — it bridges MongoDB-stored, encrypted credentials into live AWS SDK client instances. A user who has never connected an AWS account still gets a full set

of clients back (not an error) — those clients rely entirely on the SDK's default credential provider chain, which will typically only fail once an actual AWS API call is attempted (not at construction time). Credentials are decrypted fresh on every call — there is no caching of the decrypted secret or of the constructed clients across calls.

CROSS-CUTTING SERVICES

Notifications, email, PDF reports, dashboard aggregation, the scheduler, auth, audit logging, analytics.

 `server/src/services/notification/notification.service.js`

PURPOSE

CRUD service for a user's per-account notification preferences (scan-completed emails, high-severity alerts, weekly report opt-in). It guarantees every user has a settings row by lazily creating one on first read, rather than treating "no settings yet" as a null/empty state the frontend has to special-case.

WHEN THIS FILE IS CALLED

On every load of the Notification Settings page (`GET /api/notifications/settings`) and whenever the user saves changes on that page (`PATCH /api/notifications/settings`).

WHO CALLS THIS FILE

- `server/src/controllers/notification.controller.js` (via `getSettings` and `updateSettings` handlers, reached from `server/src/routes/notification.routes.js` → `GET /api/notifications/settings`, `PATCH /api/notifications/settings`, both behind the global `protect` auth middleware)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/notification.repository.js` — `getNotificationSettingsByUserId`, `createNotificationSettings`, `updateNotificationSettings` (the actual Mongo reads/writes)
- `server/src/services/database/user.repository.js` — `findUserById` (needed to seed the default settings row with the user's real email)
- `server/src/utils/ApiError.js` — throws a structured 404 when the user record is missing

EXPORTED FUNCTIONS

- `getOrCreateSettings(userId)`
- Purpose: Returns the user's notification settings, auto-creating a default row the first time they're requested.
- Input: `userId` (ObjectId/string).
- Output: A `Notification` Mongoose document (existing or newly created).
- Internal steps:
 1. `existing = await getNotificationSettingsByUserId(userId)` ; if found, return it immediately.
 2. Otherwise `user = await findUserById(userId)` ; if no user, `throw new ApiError(404, "User not found")` .
 3. `return await createNotificationSettings({ userId, email: user.email, scanCompleted: true, highSeverity: true, weeklyReport: true })` .
- Database calls: `Notification.findOne({ userId })` (via `getNotificationSettingsByUserId`), `User.findById(userId)` (via `findUserById`), `Notification.create(...)` (via `createNotificationSettings`).
- AWS calls: none.
- Error handling: throws `ApiError(404, "User not found")` if `userId` doesn't resolve to a user (defensive — normally can't happen since `userId` comes from a verified JWT).
- Return values: existing `Notification` doc, or a newly created one with `scanCompleted: true, highSeverity: true, weeklyReport: true` defaults and `email` copied from the `User` record.
- `editSettings(userId, { email, scanCompleted, highSeverity, weeklyReport })`
- Purpose: Partially updates a user's notification settings, only touching fields that were actually provided.
- Input: `userId`, plus an options object where any of `email` / `scanCompleted` / `highSeverity` / `weeklyReport` may be `undefined` .
- Output: The updated `Notification` document.
- Internal steps:
 1. `await getOrCreateSettings(userId)` — ensures a row exists before updating (return value discarded).

2. Build `updates = {}`, adding each field only `if (field !== undefined)`.
 3. `return await updateNotificationSettings(userId, updates)`.
- Database calls: `Notification.findOne / create` (via step 1), then `Notification.findOneAndUpdate({ userId }, updates, { new: true })` (via `updateNotificationSettings`).
 - AWS calls: none.
 - Error handling: none directly (relies on `getOrCreateSettings`'s 404 if the user vanished); if `updates` is `{}` (nothing provided), `findOneAndUpdate` still runs and returns the unchanged document.
 - Return values: updated `Notification` document.

DATA FLOW

Controller → `getOrCreateSettings / editSettings` → `notification.repository.js` → MongoDB `notifications` collection. No AWS or email interaction happens here — this file only manages the *preferences* that `scanNotification.service.js` and `weeklyReport.scheduler.js` later read to decide whether to actually send mail.

EXAMPLE EXECUTION

A brand-new user opens Notification Settings for the first time: `GET /api/notifications/settings` → `getOrCreateSettings(userId)` finds no existing row → looks up the user's email (e.g. `akshatjohri03@gmail.com`) → creates `{ userId, email: "akshatjohri03@gmail.com", scanCompleted: true, highSeverity: true, weeklyReport: true }` → returns it to the frontend. The user then unchecks "Weekly Report" and saves: `PATCH /api/notifications/settings { weeklyReport: false }` → `editSettings` updates only `weeklyReport`, leaving `scanCompleted / highSeverity / email` untouched.

IMPORTANT NOTES

- The `Notification` schema enforces `userId` as `unique`, so `createNotificationSettings` will throw a Mongo duplicate-key error if called twice concurrently for the same brand-new user (unhandled here — a race is theoretically possible between the `findOne` and `create` in `getOrCreateSettings`, though unlikely given normal request patterns).
- `email` on the settings row is independent of the user's login email — editing it only changes where scan/weekly-report notifications go, not the account's login identity.

 `server/src/services/notification/scanNotification.service.js`

PURPOSE

Decides whether and what to email a user immediately after one of their scans completes — a "scan completed" summary email and/or a separate "high severity findings" email, gated by that user's notification preferences.

WHEN THIS FILE IS CALLED

After every scheduled scan run (i.e. inside `runDueSchedules`'s per-schedule loop in `scheduler.service.js`), immediately following `scanInfrastructure`. It is **not** called for scans triggered manually from the UI (`POST /api/scan` → `scan.controller.js` calls `scanInfrastructure` directly and never imports this service).

WHO CALLS THIS FILE

- `server/src/services/scheduler/scheduler.service.js` (`notifyScanCompletion`, called inside `runDueSchedules`'s try block, wrapped in its own try/catch so a notification failure can't affect scheduling state)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/recommendation.repository.js` — `getRecommendationsByScanId` (to know what the just-completed scan found)
- `server/src/services/database/notification.repository.js` — `getNotificationSettingsByUserId` (to check opt-in flags and destination email)
- `server/src/services/email/email.service.js` — `sendScanCompletedEmail`, `sendHighSeverityEmail` (actually dispatches the mail)

EXPORTED FUNCTIONS

- `notifyScanCompletion(userId, scanId, { totalResources, estimatedSavings })`
- Purpose: Sends up to two emails (scan-completed summary, high-severity alert) based on the user's saved preferences and the scan's findings.
- Input: `userId`, `scanId`, and a stats object `{ totalResources, estimatedSavings }` (the shape returned by `scanInfrastructure`).

- Output: `undefined` (fire-and-forget; nothing is returned to the caller).
- Internal steps:
 1. `settings = await getNotificationSettingsByUserId(userId)`.
 2. If `!settings || (!settings.scanCompleted && !settings.highSeverity)`, return immediately — no work needed.
 3. `recommendations = await getRecommendationsByScanId(scanId, userId)`.
 4. Bucket into `bySeverity = { HIGH: [], MEDIUM: [], LOW: [] }` by iterating `recommendations` and pushing each `rec` into `bySeverity[rec.severity]` (guarded by `if (bySeverity[rec.severity])`).
 5. If `settings.scanCompleted`, call `sendScanCompletedEmail(settings.email, { totalResources, highCount: bySeverity.HIGH.length, mediumCount: bySeverity.MEDIUM.length, lowCount: bySeverity.LOW.length, estimatedSavings })` inside a try/catch that only logs on failure (`console.error`).
 6. If `settings.highSeverity && bySeverity.HIGH.length > 0`, call `sendHighSeverityEmail(settings.email, { highCount: bySeverity.HIGH.length, recommendations: bySeverity.HIGH })`, also in its own try/catch.
- Database calls: `Notification.findOne({ userId })` (via `getNotificationSettingsByUserId`); `Recommendation.find({ scanId, userId })` + a `Resource.find({ scanId, userId, resourceId: { $in: ... } })` enrichment join (via `getRecommendationsByScanId`).
- AWS calls: none.
- Error handling: Each `sendMail` call is individually wrapped in try/catch — a failure sending the scan-completed email does not prevent the high-severity email from being attempted, and vice versa. Both failures are only logged (`console.error(...error.message)`), never thrown, so a broken SMTP config never fails the scan itself.
- Return values: none meaningful — always resolves to `undefined` whether it sent 0, 1, or 2 emails.

DATA FLOW

`scheduler.service.js` (post-scan) → `notifyScanCompletion` → reads `Notification` settings + `Recommendation` documents for the just-finished scan → `email.service.js` → SMTP/Ethereal transporter → recipient inbox (or Ethereal preview URL logged to console in dev).

EXAMPLE EXECUTION

A user has a weekly schedule with `scanCompleted: true`, `highSeverity: true`. The 08:00 Monday scan completes with `totalResources: 42`, `estimatedSavings: 137.50`, and the scan produced 3 HIGH, 5 MEDIUM, 2 LOW recommendations. `notifyScanCompletion` sends a "Scan Completed" email showing 42 resources / 3/5/2 alert counts / \$137.50 savings, then a separate "3 High Severity Findings" email listing up to the first 10 HIGH recommendations.

IMPORTANT NOTES

- If `settings` doesn't exist yet for a scheduled user (e.g. they never opened Notification Settings so `getOrCreateSettings` never ran), `notifyScanCompletion` silently no-ops — it does not create a default settings row itself, unlike `notification.service.js`'s `getOrCreateSettings`.
- `bySeverity[rec.severity]` relies on `rec.severity` always being exactly `"HIGH"` / `"MEDIUM"` / `"LOW"` per the `Recommendation` schema enum; anything else is silently dropped by the `if (bySeverity[rec.severity])` guard.
- Manual (non-scheduled) scans never trigger this file — a UI-triggered "Scan Now" never sends scan-completed or high-severity emails, only scheduled cron-driven scans do.

 `server/src/services/report/scanReport.service.js`

PURPOSE

Assembles the raw data model for a single scan's PDF report — reuses the exact scan/resource/recommendation lookup the Scan Detail page already uses, then derives the extra rollups (total monthly cost, severity counts, resource-type counts) the PDF renderer needs, all in plain JS with no new database queries.

WHEN THIS FILE IS CALLED

When a user clicks "Export PDF" / "Download Report" on a scan (`GET /api/reports/scan/:id`), before the PDF bytes are generated.

WHO CALLS THIS FILE

- `server/src/controllers/report.controller.js` (`exportScanReport` handler, reached from `server/src/routes/report.routes.js` → `GET /api/reports/scan/:id`)

WHICH FILES IT DEPENDS ON

- `server/src/services/scan/scanHistory.service.js` — `fetchScanDetails` (the single source of the scan's `{ scan, resources, recommendations }`)

EXPORTED FUNCTIONS

- `getScanReportData(scanId, userId)`
- Purpose: Produces the fully-shaped data object `scanReportPdf.service.js` needs to draw the report, or `null` if the scan doesn't exist / doesn't belong to the user.
- Input: `scanId`, `userId`.
- Output: `null`, or `{ scan, resources, recommendations, monthlyCost, severityCounts, resourceTypeCounts }`.
- Internal steps:
 1. `details = await fetchScanDetails(scanId, userId)`; if falsy, `return null`.
 2. Destructure `{ scan, resources, recommendations }` from `details`.
 3. Initialize `severityCounts = { HIGH: 0, MEDIUM: 0, LOW: 0 }` and `monthlyCost = 0`.
 4. `recommendations.forEach(rec => ...)`: if `rec.severity` in `severityCounts`, increment that bucket; unconditionally add `rec.monthlyCost || 0` to the running `monthlyCost` total.
 5. `resourceTypeCounts = resources.reduce(...)`: for each `resource`, `type = resource.resourceType || "UNKNOWN"`, increment `acc[type]`.
 6. Return `{ scan, resources, recommendations, monthlyCost, severityCounts, resourceTypeCounts }`.
- Database calls: none directly — `fetchScanDetails` performs `Scan.findOne({ _id: id, userId })`, `Resource.find({ scanId, userId })`, `Recommendation.find({ scanId, userId })` (enriched with resource data).
- AWS calls: none.
- Error handling: none thrown here — a missing/foreign scan simply produces `null`, which the controller turns into a 404 (`ApiError(404, "Scan not found")`).
- Return values: `null` (scan not found or not owned by `userId`), or the report-data object described above.

DATA FLOW

`report.controller.js` → `getScanReportData` → `scanHistory.service.js` (`fetchScanDetails`) → `scan.repository.js` / `resource.repository.js` / `recommendation.repository.js` → MongoDB. The returned object is handed straight to `scanReportPdf.service.js`'s `buildScanReportPdf`, which does no further database access.

EXAMPLE EXECUTION

`GET /api/reports/scan/64f0.../` for a scan with 3 recommendations (`monthlyCost` 10, 20, 5 respectively, severities HIGH/HIGH/LOW) and 12 resources (10 EC2, 2 S3): returns `monthlyCost: 35`, `severityCounts: { HIGH: 2, MEDIUM: 0, LOW: 1 }`, `resourceTypeCounts: { EC2: 10, S3: 2 }`.

IMPORTANT NOTES

- `monthlyCost` here is summed across **all** recommendations regardless of `status` (OPEN/RESOLVED/IGNORED) — unlike `getDashboardSummary`'s `estimatedSavings` (which filters to `status: "OPEN"`), this file does not filter by status at all, so a resolved or ignored recommendation's `monthlyCost` still counts toward the PDF's total.
- `resourceTypeCounts` groups by the raw `resourceType` string (e.g. `"IAM_USER"`, `"IAM_ROLE"` stay separate), unlike `serviceAggregation.service.js`'s `getServiceName` which collapses everything after the first underscore into one service bucket — the PDF's "Resources by Type" bar chart is therefore more granular than the dashboard's service breakdown.

 `server/src/services/report/scanReportPdf.service.js`

PURPOSE

Pure PDF-rendering module: turns the data object from `scanReport.service.js` into a multi-page PDFKit document (branded header, stat cards, a hand-drawn severity pie chart, a resource-type bar chart, and paginated resource/recommendation tables). Contains no database or business logic — it's a rendering layer only.

WHEN THIS FILE IS CALLED

Immediately after `getScanReportData` succeeds, inside the same `GET /api/reports/scan/:id` request that streams the resulting PDF back to the browser.

WHO CALLS THIS FILE

- `server/src/controllers/report.controller.js` (`exportScanReport` calls `buildScanReportPdf(reportData)` , then pipes the returned `PDFDocument` directly into the HTTP response)

WHICH FILES IT DEPENDS ON

- none internal — externalImports only: `pdfkit` (`PDFDocument`)

EXPORTED FUNCTIONS

- `buildScanReportPdf(reportData)`
- Purpose: Synchronously constructs and returns a streaming `PDFDocument` for the given scan report data.
- Input: `reportData = { scan, resources, recommendations, monthlyCost, severityCounts, resourceTypeCounts }` (exact shape from `getScanReportData`).
- Output: A `pdfkit PDFDocument` instance (`bufferPages: true` , `size: "A4"` , `margin: 40`), not yet ended — the caller (`report.controller.js`) is responsible for `doc.pipe(res); doc.end();` .
- Internal steps:
 1. Create `doc = new PDFDocument({ size: "A4", margin: PAGE_MARGIN, bufferPages: true })` (A4 = 595.28pt wide, 40pt margins, so `CONTENT_WIDTH = 515.28`).
 2. `drawHeader(doc, scan)` — fills a 90pt blue (`#2563eb`) banner with "CloudLens" + "Infrastructure Scan Report", then prints Scan ID, Status, Started/Completed timestamps (via `formatDate`), and a "Generated" timestamp.
 3. `sectionTitle(doc, "Scan Summary") + drawStatCards(doc, [...])` — a 3-column bordered card grid: Status, Total Resources, Monthly Cost, Estimated Savings (green), High Alerts (red), Medium/Low Alerts (amber).
 4. `sectionTitle(doc, "Severity Breakdown") + drawSeverityPieChart(doc, severityCounts)` — approximates a pie chart as a filled polygon per severity segment (colors: HIGH `#ef4444` , MEDIUM `#f59e0b` , LOW `#10b981`), with a side legend showing count and percentage; if `total === 0` , prints "No recommendations to chart." instead.
 5. `sectionTitle(doc, "Resources by Type") + drawResourceBarChart(doc, resourceTypeCounts)` — horizontal bars sized relative to `maxCount` , colored by cycling through an 8-color palette; empty state: "No resources found for this scan."
 6. `doc.addPage()` — forces the tables onto a fresh page.
 7. `drawTable(doc, { title: "Resources", columns: [Type, Name, State, Resource ID], rows: resources.map(...)} )` — striped rows, dark header re-drawn on every page break.
 8. `drawTable(doc, { title: "Recommendations", columns: [Resource, Severity (colored per-row), Recommendation, Monthly Cost, Savings], rows: recommendations.map(...)} )` .
 9. `drawFooter(doc)` — walks every buffered page (`doc.bufferedPageRange()`) and stamps "CloudLens · Page X of Y" centered at the bottom, temporarily zeroing `doc.page.margins.bottom` so pdfkit's own overflow check doesn't spawn an extra blank page for the footer text itself.
 10. `return doc` .
- Database calls: none.
- AWS calls: none.
- Error handling: none explicit — relies on `reportData` already being well-formed (guaranteed by `scanReport.service.js`); a malformed `reportData` (e.g. missing `severityCounts`) would throw a plain JS error inside `buildScanReportPdf` that propagates up through `asyncHandler` to the global error middleware.
- Return values: always a `PDFDocument` (this function never returns `null` / `undefined` — `getScanReportData` 's `null` case is checked by the controller *before* this function is even called).

DATA FLOW

`report.controller.js` → `buildScanReportPdf(reportData)` (pure, synchronous construction of drawing commands) → `doc.pipe(res)` streams PDF bytes directly to the HTTP response as pdfkit generates them → browser downloads `cloudlens-scan-report-<scanId>.pdf` .

EXAMPLE EXECUTION

For a scan with `severityCounts: { HIGH: 3, MEDIUM: 5, LOW: 2 }` and `resourceTypeCounts: { EC2: 10, S3: 2, RDS: 1 }` : page 1 shows the blue header, 6 stat cards, a 3-slice pie chart (HIGH 30%, MEDIUM 50%, LOW 20%) with a legend, and a 3-bar chart (EC2 longest bar). Page 2+ lists all resources in a striped table, followed by all recommendations, each page footer reading "CloudLens · Page 2 of 3" etc.

IMPORTANT NOTES

- `RESOURCE_TYPE_PALETTE` and `SEVERITY_COLORS` are a hardcoded duplicate of `client/src/Utils/chartColors.js` 's palette, kept as a separate copy specifically because the client bundle isn't reachable from the server process — the two must be kept in sync manually if the

dashboard's chart colors ever change.

- `drawTable` 's row-height calculation uses `doc.heightOfString` per column to support multi-line cell wrapping, so a very long `recommendation` string will make that row taller and can push later rows to a new page mid-table (the header re-draws automatically via `drawHeaderRow()` in that case).
- The pie chart's "many-sided polygon" approach (`steps = Math.max(2, Math.ceil((sweep / (Math.PI * 2)) * 120))`) is an approximation, not a true arc — visually indistinguishable at print resolution but technically a polygon, not a circle segment.
- `doc.page.margins.bottom = 0` inside `drawFooter` is a deliberate workaround for pdfkit auto-paginating when text is written inside the margin band; it's restored immediately after each footer write.

 `server/src/services/email/email.service.js`

PURPOSE

Thin, generic email-sending layer: wraps the shared Nodemailer transporter from `config/mailler.js`, applies a single `FROM_ADDRESS`, and exposes three purpose-specific senders (scan-completed, high-severity, weekly-report) that plug scan data into the corresponding HTML template.

WHEN THIS FILE IS CALLED

Whenever any of the three notification flows needs to actually dispatch mail: after a scheduled scan completes (`scanNotification.service.js`), and every Monday at 08:00 for weekly reports (`weeklyReport.scheduler.js`).

WHO CALLS THIS FILE

- `server/src/services/notification/scanNotification.service.js` (`sendScanCompletedEmail` , `sendHighSeverityEmail`)
- `server/src/services/notification/weeklyReport.scheduler.js` (`sendWeeklyReportEmail`)

WHICH FILES IT DEPENDS ON

- `server/src/config/mailler.js` — `getTransporter` (lazily builds/caches the Nodemailer transporter — real SMTP if `SMTP_HOST` is set, otherwise an auto-provisioned Ethereum test account)
- `server/src/services/email/emailTemplates.js` — `buildScanCompletedEmail` , `buildHighSeverityEmail` , `buildWeeklyReportEmail` (renders the HTML body for each mail)
- external: `nodemailer` (`getTestMessageUrl` , used only for the dev Ethereum preview link)

EXPORTED FUNCTIONS

- `sendMail({ to, subject, html })` (internal helper, not exported)
- Purpose: Shared low-level send used by all three exported functions.
- Input: `{ to, subject, html }`.
- Output: The Nodemailer `info` object returned by `transporter.sendMail(...)`.
- Internal steps: 1) `transporter = await getTransporter()` . 2) `info = await transporter.sendMail({ from: FROM_ADDRESS, to, subject, html })` . 3) `previewUrl = nodemailer.getTestMessageUrl(info)` ; if truthy (only true for the Ethereum dev transporter), `console.log` an inspectable preview link. 4) `return info` .
- Database calls: none.
- AWS calls: none — this is SMTP via Nodemailer, not AWS SES.
- Error handling: none caught here — any transporter/send failure propagates to the caller (which is where `scanNotification.service.js` and `weeklyReport.scheduler.js` each wrap the call in their own try/catch).
- `sendScanCompletedEmail(email, stats)`
- Purpose: Sends the "scan completed" summary email.
- Input: `email` (recipient string), `stats = { totalResources, highCount, mediumCount, lowCount, estimatedSavings }`.
- Output: Nodemailer `info` object (from `sendMail`).
- Internal steps: `sendMail({ to: email, subject: "CloudLens – Scan Completed", html: buildScanCompletedEmail(stats) })` .
- Database/AWS calls: none.
- Error handling: none (propagates).
- Return values: Nodemailer send-result object.
- `sendHighSeverityEmail(email, data)`
- Purpose: Sends the high-severity findings alert email.

- Input: `email`, `data = { highCount, recommendations }`.
- Output: Nodemailer `info` object.
- Internal steps: subject is pluralized dynamically: `` CloudLens — ${data.highCount} High Severity Finding${data.highCount === 1 ? "" : "s"} ``; body from `buildHighSeverityEmail(data)`.
- Database/AWS calls: none.
- Error handling: none (propagates).
- Return values: Nodemailer send-result object.
- `sendWeeklyReportEmail(email, data)`
- Purpose: Sends the weekly cost report email.
- Input: `email`, `data` = the object returned by `getDashboardSummary` (a superset of what `buildWeeklyReportEmail` reads).
- Output: Nodemailer `info` object.
- Internal steps: `sendMail({ to: email, subject: "CloudLens – Your Weekly Cost Report", html: buildWeeklyReportEmail(data) })`.
- Database/AWS calls: none.
- Error handling: none (propagates).
- Return values: Nodemailer send-result object.

DATA FLOW

Caller (scan/weekly notification service) → one of the three `send*Email` functions → `emailTemplates.js` builds the HTML string → `sendMail` → `config/mailer.js` 's cached transporter → real SMTP server (if `SMTP_HOST` env var set) or Ethereum test inbox (dev fallback, logged preview URL, never delivered anywhere real).

EXAMPLE EXECUTION

Dev environment, no `SMTP_HOST` set: `sendScanCompletedEmail("akshatjohri03@gmail.com", { totalResources: 42, highCount: 3, mediumCount: 5, lowCount: 2, estimatedSavings: 137.5 })` → transporter is Ethereum → console logs `Email preview ("CloudLens – Scan Completed" -> akshatjohri03@gmail.com): https://ethereal.email/message/...` — no real email arrives in the inbox.

IMPORTANT NOTES

- `FROM_ADDRESS` defaults to `"CloudLens <notifications@cloudlens.app>"` if `EMAIL_FROM` is unset — this is not a verified/deliverable domain, purely a display default.
- This is SMTP via Nodemailer end-to-end — there is no AWS SES integration anywhere in this codebase despite CloudLens being an AWS-focused product.
- None of the three exported senders catch errors themselves; every call site in `scanNotification.service.js` / `weeklyReport.scheduler.js` is individually try/caught so one failed send never cascades.

 `server/src/services/email/emailTemplates.js`

PURPOSE

Pure HTML-string builders for the three notification emails. Wraps each email's specific content in one shared branded layout (`baseLayout`) so the header/card/footer chrome is defined once instead of duplicated per template.

WHEN THIS FILE IS CALLED

Every time `email.service.js` 's `sendScanCompletedEmail`, `sendHighSeverityEmail`, or `sendWeeklyReportEmail` is invoked — these call the matching `build*Email` function synchronously to produce the `html` body before handing it to Nodemailer.

WHO CALLS THIS FILE

- `server/src/services/email/email.service.js` (`buildScanCompletedEmail`, `buildHighSeverityEmail`, `buildWeeklyReportEmail`)

WHICH FILES IT DEPENDS ON

- none (no internal imports, no external imports — pure template-literal string building)

EXPORTED FUNCTIONS

- `buildScanCompletedEmail({ totalResources, highCount, mediumCount, lowCount, estimatedSavings })`
- Purpose: Renders the "Scan completed" email body.

- Input: destructured stats object.
- Output: full HTML document string.
- Internal steps: builds an `<h1>Scan completed</h1>` + intro paragraph + a `statRow` table with rows "Resources Scanned" (`totalResources`), "High Alerts" (`highCount` , red `#dc2626`), "Medium Alerts" (`mediumCount` , amber `#d97706`), "Low Alerts" (`lowCount` , green `#16a34a`), "Estimated Monthly Savings" (`formatCurrency(estimatedSavings)` , green `#059669`); wraps in `baseLayout({ title: "Scan Completed – CloudLens", preheader: "Scan finished: ${totalResources} resources, ${formatCurrency(estimatedSavings)} potential savings.", bodyHtml })`.
- Database/AWS calls: none.
- Error handling: none.
- Return values: HTML string.
- `buildHighSeverityEmail({ highCount, recommendations })`
- Purpose: Renders the high-severity alert email body, listing up to the first 10 findings.
- Input: `highCount` , `recommendations` (array of enriched recommendation objects).
- Output: full HTML document string.
- Internal steps: 1) `items = recommendations.slice(0, 10).map(rec => <li>...)` — each list item shows `<strong>${rec.resourceId}</strong> – ${rec.recommendation}` plus `rec.reason` in smaller gray text. 2) `remaining = recommendations.length - 10`. 3) Body is `<h1>⚠ High severity findings</h1>` + intro ("Your latest scan found `<strong>${highCount}</strong>` high-severity recommendation(s) that need attention:") + the `<ul>` of items + (if `remaining > 0`) a " + `${remaining}` more – view them all in CloudLens." note. 4) Wraps in `baseLayout({ title: "High Severity Alert – CloudLens", preheader: "${highCount} high-severity recommendation(s) need your attention.", bodyHtml })`.
- Database/AWS calls: none.
- Error handling: none — if `recommendations` has fewer than 10 items, `remaining` is negative and the "+ N more" block is correctly suppressed by the `remaining > 0` check.
- Return values: HTML string.
- `buildWeeklyReportEmail({ totalResources, monthlyCost, estimatedSavings, highRecommendations, mediumRecommendations, lowRecommendations })`
- Purpose: Renders the weekly cost report email body.
- Input: destructured summary object (matches `getDashboardSummary` 's return shape field-for-field).
- Output: full HTML document string.
- Internal steps: builds `<h1>Your weekly cost report</h1>` + a `statRow` table: "Total Resources", "Monthly Cost" (`formatCurrency`), "Estimated Savings" (green), "High/Medium/Low Alerts" (red/amber/green); wraps in `baseLayout({ title: "Weekly Cost Report – CloudLens", preheader: "Weekly report: ${formatCurrency(monthlyCost)} monthly cost, ${formatCurrency(estimatedSavings)} potential savings.", bodyHtml })`.
- Database/AWS calls: none.
- Error handling: none.
- Return values: HTML string.

DATA FLOW

Called synchronously inside `email.service.js` 's three send functions; produces a self-contained HTML string (inline styles throughout, table-based layout for email-client compatibility) that becomes the `html` field of the Nodemailer `sendMail` call. No network or database access happens in this file at all.

EXAMPLE EXECUTION

`buildScanCompletedEmail({ totalResources: 42, highCount: 3, mediumCount: 5, lowCount: 2, estimatedSavings: 137.5 })` produces an HTML document whose title is "Scan Completed – CloudLens", with a stat table row "Estimated Monthly Savings" → "\$137.50" styled in `#059669`.

IMPORTANT NOTES

- `formatCurrency` is defined locally in this file (`` ${Number(value || 0).toFixed(2)} ``) – a separate, independent copy from any currency formatter in `client/src/utils/format.js`; keeping them numerically consistent (2 decimal places, \$ prefix, `Number(value || 0)` fallback for `null / undefined`) is a manual convention, not shared code.
- `buildHighSeverityEmail` truncates to the first 10 recommendations by array order as returned from `getRecommendationsByScanId` (sorted `createdAt: -1`, i.e. newest-created first) — not sorted by severity, cost, or estimated savings, so the 10 shown are simply the most recently created HIGH-severity recommendations for that scan.

- The preheader `<span>` (hidden preview text used by email clients) always duplicates the same summary numbers shown in the body — a common email-authoring pattern, not a bug.

 `server/src/services/dashboard/dashboard.service.js`

PURPOSE

Top-level aggregator for the main Dashboard page: fans out to six independent data sources in parallel (legacy EC2-era summary, insights, EC2 fleet stats, IAM fleet stats, recommendation category counts, and the newer generic service-agnostic insights) and merges them into one response object, without modifying any of the underlying aggregations.

WHEN THIS FILE IS CALLED

Every time the Dashboard page loads or refreshes (`GET /api/dashboard`), and separately whenever the dashboard's "Latest Recommendations" panel needs data (`GET /api/dashboard/recommendations`).

WHO CALLS THIS FILE

- `server/src/controllers/dashboard.controller.js` (`getDashboard` → `getDashboardData` ; `getLatestRecommendations` → `getRecommendations`), reached from `server/src/routes/dashboard.routes.js` → `GET /api/dashboard` and `GET /api/dashboard/recommendations`

WHICH FILES IT DEPENDS ON

- `server/src/services/dashboard/serviceAggregation.service.js` — `getGenericDashboardInsights` (adds the service-agnostic fields: `serviceSummary`, `severitySummary`, `costBreakdown`, `resourceDistribution`, `savingsOpportunities`, `aiInsights`)
- `server/src/services/database/dashboard.repository.js` — `getDashboardSummary` , `getEC2FleetStats` , `getIAMFleetStats` , `getRecommendationCategoryCounts`
- `server/src/services/database/insight.repository.js` — `getLatestInsights` (S3-specific insight sentences saved during the last scan)
- `server/src/services/database/recommendation.repository.js` — `getLatestRecommendations`

EXPORTED FUNCTIONS

- `getDashboardData(userId)`
- Purpose: Builds the complete payload the Dashboard page renders in one call.
- Input: `userId` .
- Output: A single merged object — spread of the legacy `summary` , plus `insights` , `ec2Stats` , `iamStats` , `categoryStats` , then spread of `genericInsights` (which itself contributes `serviceSummary` , `severitySummary` , `costBreakdown` , `resourceDistribution` , `savingsOpportunities` , `aiInsights`).
- Internal steps:
 1. `Promise.all([getDashboardSummary(userId), getLatestInsights(userId), getEC2FleetStats(userId), getIAMFleetStats(userId), getRecommendationCategoryCounts(userId), getGenericDashboardInsights(userId)])` — all six run concurrently.
 2. `return { ...summary, insights, ec2Stats, iamStats, categoryStats, ...genericInsights }` .
- Database calls: (indirectly, across the six parallel calls) `Scan.findOne` (multiple times, once per aggregator that needs the latest scan), `Resource.aggregate` / `Resource.find` , `Recommendation.aggregate` / `Recommendation.find` , `Insight.find` .
- AWS calls: none.
- Error handling: none explicit — if any of the six promises rejects, `Promise.all` rejects and the whole request fails via `asyncHandler` 's error propagation to the global error middleware (no partial-dashboard fallback).
- Return values: always the merged object described above (each sub-aggregator has its own "no scan yet" empty-default, e.g. `getDashboardSummary` returns zeroed fields, `getGenericDashboardInsights` returns `EMPTY_GENERIC_INSIGHTS` , `getEC2FleetStats` / `getIAMFleetStats` return their own empty-stat defaults) — so a user with zero scans gets a fully-shaped, all-zero/empty dashboard rather than a 404.
- `getRecommendations(userId)`
- Purpose: Returns the latest scan's recommendations (thin passthrough).
- Input: `userId` .
- Output: array of enriched recommendation objects (or `[]` if no scan exists).
- Internal steps: `return await getLatestRecommendations(userId)` .

- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 })`, then `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })`, enriched with matching `Resource` documents.
- AWS calls: none.
- Error handling: none.
- Return values: array (possibly empty).

DATA FLOW

`dashboard.controller.js` → `getDashboardData` → six parallel repository calls (all reading from the user's latest `Scan` and its associated `Resource` / `Recommendation` / `Insight` documents) → merged into one JSON response → rendered by the React Dashboard page's cards, tables, and charts.

EXAMPLE EXECUTION

A user with one completed scan loads `/dashboard`: `getDashboardSummary` returns `{ totalResources: 20, monthlyCost: 150, estimatedSavings: 40, highRecommendations: 2, ... }`; `getGenericDashboardInsights` independently re-derives `serviceSummary`, `aiInsights`, etc. from the same latest scan's resources/recommendations; the final response merges both without either overwriting the other's fields (they use disjoint field names).

IMPORTANT NOTES

- Six calls each independently look up "the latest scan" (`Scan.findOne({ userId }).sort({ createdAt: -1 })`) rather than sharing one lookup — functionally consistent (same scan every time, since scans don't change mid-request) but means the "latest scan" query runs multiple times per dashboard load rather than once.
- Comment in the source explicitly states this design intent: `getDashboardSummary` 's own aggregation is left "completely untouched" — new fields are additive merges, never overwrites, so older frontend code reading only the original fields continues to work unchanged.

`server/src/services/dashboard/serviceAggregation.service.js`

PURPOSE

Derives every "service-agnostic" dashboard field (per-service summary, severity summary, cost breakdown, resource distribution, top savings opportunities, and the AI Insights panel's rule-based messages) from a single fetch of the latest scan's resources and recommendations — designed so a brand-new resource type/service automatically participates in every one of these views with zero code changes, via a `getServiceName` convention plus a generic fallback in the insight rules.

WHEN THIS FILE IS CALLED

Every dashboard load, as one of the six parallel branches inside `dashboard.service.js` 's `getDashboardData`.

WHO CALLS THIS FILE

- `server/src/services/dashboard/dashboard.service.js` (`getGenericDashboardInsights`)

WHICH FILES IT DEPENDS ON

- `server/src/models/Scan.js` — `Scan.findOne` (direct Mongoose model access, not via a repository, to find the latest scan)
- `server/src/services/database/recommendation.repository.js` — `getRecommendationsByScanId`
- `server/src/services/database/resource.repository.js` — `getResourcesByScanId`

EXPORTED FUNCTIONS

- `getGenericDashboardInsights(userId)`
- Purpose: Single entry point producing all six generic dashboard fields from one shared resources+recommendations dataset.
- Input: `userId`.
- Output: `EMPTY_GENERIC_INSIGHTS` copy (all-empty defaults) if no scan exists yet, otherwise `{ serviceSummary, severitySummary, costBreakdown, resourceDistribution, savingsOpportunities, aiInsights }`.
- Internal steps:
 1. `latestScan = await Scan.findOne({ userId }).sort({ createdAt: -1 })`; if none, return `{ ...EMPTY_GENERIC_INSIGHTS }`.
 2. `[resources, recommendations] = await Promise.all([getResourcesByScanId(latestScan._id, userId), getRecommendationsByScanId(latestScan._id, userId)])`.
 3. `serviceSummary = buildServiceSummary(resources, recommendations)`.

4. Return `{ serviceSummary, severitySummary: buildSeveritySummary(recommendations), costBreakdown: buildCostBreakdown(resources, recommendations), resourceDistribution: buildResourceDistribution(resources), savingsOpportunities: buildSavingsOpportunities(recommendations), aiInsights: buildAiInsights(resources, recommendations, serviceSummary) }`.
- Database calls: `Scan.findOne({ userId }).sort({ createdAt: -1 })`; `Resource.find({ scanId, userId }).sort({ createdAt: -1 })` (via `getResourcesByScanId`); `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })` + resource-enrichment `Resource.find({ scanId, userId, resourceId: { $in: [...] } })` (via `getRecommendationsByScanId`).
- AWS calls: none.
- Error handling: none explicit; relies on the "no scan" early return to avoid null-pointer issues downstream.
- Return values: `EMPTY_GENERIC_INSIGHTS` shape `( { serviceSummary: [], severitySummary: {critical:0,high:0,medium:0,low:0,healthy:0}, costBreakdown: [], resourceDistribution: [], savingsOpportunities: [], aiInsights: [] } )` or the populated equivalent.
- `getServiceName(resourceType)` (also exported, used elsewhere for consistent grouping)
- Purpose: Normalizes any `resourceType` string (e.g. `"EC2"`, `"IAM_USER"`, `"IAM_ROLE"`, `"IAM_ROOT_ACCOUNT"`) down to its owning service (`"EC2"`, `"IAM"`, `"IAM"`, `"IAM"`).
- Input: `resourceType` (string or falsy).
- Output: string — text before the first underscore, or `"UNKNOWN"` if `resourceType` is falsy.
- Internal steps: `(resourceType || "UNKNOWN").split("_")[0]`.
- Database/AWS calls: none.
- Error handling: none needed (pure string function).
- Return values: a service name string.

INTERNAL (NON-EXPORTED) HELPER FUNCTIONS OF NOTE

- `buildServiceSummary(resources, recommendations)` — groups by `getServiceName`, counting `resourceCount` per service from `resources`, and `recommendationCount` / `estimatedSavings` per service from `recommendations` filtered to `rec.action !== "NONE" && rec.status === "OPEN"`; sorted descending by `resourceCount`.
- `buildSeveritySummary(recommendations)` — filters to `status === "OPEN"`, then buckets each into `healthy` (action === "NONE"), `critical` (severity HIGH and category SECURITY), `high` (severity HIGH, any other category), `medium`, or `low`. There is no `CRITICAL` value in the `Recommendation.severity` schema enum (`LOW` / `MEDIUM` / `HIGH` only) — "critical" here is purely a derived UI bucket, not a stored severity.
- `buildCostBreakdown(resources, recommendations)` — every service present in `resources` gets a cost entry (defaulting to 0) even with no priced recommendations (e.g. IAM), then `rec.monthlyCost` is summed in; sorted descending by cost.
- `buildResourceDistribution(resources)` — simple per-service resource counts, sorted descending.
- `buildSavingsOpportunities(recommendations, limit = 10)` — filters to `action !== "NONE" && status === "OPEN"`, sorts descending by `estimatedSavings`, takes the top 10, maps to `{ resource, service, savings, recommendation }`.
- `buildAiInsights(resources, recommendations, serviceSummary)` — runs the declarative `INSIGHT_RULES` array (below), then a generic fallback loop over `serviceSummary` for any service not already covered by a rule but with `recommendationCount > 0`; always appends at least the "No issues detected" fallback (if nothing else matched) and always appends a final `"Estimated monthly savings: $X.XX."` info line.

INSIGHT_RULES (EXACT THRESHOLDS)

- `IDLE_CPU_THRESHOLD = 5` (percent).
- **EC2:** `match: resource.state === "running" && (resource.metadata?.averageCpu ?? 100) < 5` → `"${count} underutilized EC2 instance(s) detected."`, severity `MEDIUM`. The `state === "running"` guard was specifically added because a stopped instance has no CloudWatch datapoints, so `averageCpu` reads as `0` and would otherwise always look idle.
- **IAM:** `match: Boolean(resource.metadata?.hasAdminAccess)` → `"${count} IAM user(s) has/have excessive permissions."`, severity `HIGH`.
- **RDS:** `match: resource.state === "available" && (resource.metadata?.averageCpu ?? 100) < 5` → `"${count} RDS instance(s) has/have low utilization."`, severity `MEDIUM`. Same `state === "available"` guard as EC2, same false-idle rationale.
- **S3:** `match: resource.metadata?.hasLifecycleRules === false` → `"${count} S3 bucket(s) is/are missing lifecycle rules."`, severity `LOW`.

DATA FLOW

`dashboard.service.js` → `getGenericDashboardInsights` → `Scan` model (latest scan) + `resource.repository.js` / `recommendation.repository.js` (that scan's resources/recommendations, fetched once) → six pure-JS reducer

functions derive every generic dashboard field from that one shared dataset → merged back into the dashboard payload by `dashboard.service.js`.

EXAMPLE EXECUTION

Latest scan has 5 EC2 instances (2 `running` with `averageCpu: 2`, 3 `stopped`), 1 IAM user with `hasAdminAccess: true`, 2 S3 buckets (1 missing lifecycle rules). `buildAiInsights` produces: "2 underutilized EC2 instances detected." (MEDIUM) — the 3 stopped instances are correctly excluded even though their `averageCpu` also reads as 0/undefined; "1 IAM user has excessive permissions." (HIGH); "1 S3 bucket is missing lifecycle rules." (LOW); plus the always-present "Estimated monthly savings: \$X.XX." (INFO) line.

IMPORTANT NOTES

- This is the file containing the EC2/RDS "underutilized" bug fix referenced in project context: `ec2.scanner.js` stores `metadata.averageCpu` for every instance regardless of state, and for a stopped instance `calculateAverageCPU` receives an empty CloudWatch datapoint array and returns `0` — before the `resource.state === "running" / "available"` guards were added here, that `0` was indistinguishable from a genuinely idle `running` instance, so every stopped EC2/unavailable RDS resource would have permanently shown up as "underutilized" in AI Insights. `ec2.scanner.js`'s own recommender logic already had the equivalent `isRunning` guard before this dashboard-side fix was applied.
- `INSIGHT_RULES` is intentionally declarative — adding a rule for a new service is a single new array entry, and even a service with **no** dedicated rule still surfaces via the generic fallback loop as long as it has open, actionable recommendations (`entry.recommendationCount > 0`), which is what keeps a future scanner (e.g. Lambda) automatically visible in AI Insights without code changes here.
- `buildServiceSummary` / `buildSeveritySummary` / `buildSavingsOpportunities` all filter to `status === "OPEN"` — resolved or ignored recommendations never inflate these dashboard counts, matching the same convention used in `getDashboardSummary`'s `estimatedSavings` field (but notably *not* the same convention `scanReport.service.js`'s PDF `monthlyCost` uses, which doesn't filter by status).

`server/src/services/scheduler/scheduler.service.js`

PURPOSE

The recurring-scan engine: a single `node-cron` job ticks every 60 seconds, asks MongoDB which per-user schedules are currently due, and runs `scanInfrastructure` for each — advancing that schedule's `nextRun` / `lastRun` afterward and firing post-scan notifications. Uses one shared in-memory `Set` of in-flight schedule IDs to guarantee a schedule is never scanned twice concurrently.

WHEN THIS FILE IS CALLED

`startScheduler()` runs once at process boot (`server.js`). Its registered job (`runDueSchedules`) then fires automatically every minute for the lifetime of the process.

WHO CALLS THIS FILE

- `server/src/server.js` (`startScheduler()` called once on startup)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/schedule.repository.js` — `getDueSchedules`, `updateSchedule`
- `server/src/services/scanner/scanner.service.js` — `scanInfrastructure` (runs the actual EC2/S3/RDS/IAM scan)
- `server/src/utils/schedule.util.js` — `computeNextRun`
- `server/src/services/notification/scanNotification.service.js` — `notifyScanCompletion`
- external: `node-cron` — schedules `runDueSchedules` on `"* * * * *"` (every minute)

EXPORTED FUNCTIONS

- `runDueSchedules()`
- Purpose: Processes every schedule that is currently due, sequentially, each protected by the in-flight guard.
- Input: none (reads `now = new Date()` and asks the DB which schedules qualify).
- Output: `undefined`.
- Internal steps:
 - `now = new Date()`; `dueSchedules = await getDueSchedules(now)` — every `Schedule` with `enabled: true` and `nextRun <= now`.
 - For each `schedule` in `dueSchedules` (sequential `for...of`):

a. `scheduleId = schedule._id.toString()` . b. If `inFlightScheduleIds.has(scheduleId)` , `continue` — skip it this tick (still being processed from a previous tick). c. `inFlightScheduleIds.add(scheduleId)` . d. `try : result = await scanInfrastructure(schedule.userId); lastRun = new Date(); await updateSchedule(schedule._id, schedule.userId, { lastRun, nextRun: computeNextRun(schedule.frequency, lastRun) })` ; then, in a nested try/catch, `await notifyScanCompletion(schedule.userId, result.scanId, result)` — a notify failure is only logged (`console.error`), never rethrown. e. `catch (error) : logs ` Scheduled scan failed for user ${schedule.userId}: + error.message ; still advances nextRun (via updateSchedule(schedule._id, schedule.userId, { nextRun: computeNextRun(schedule.frequency, now) })) but deliberately does **not** update lastRun on failure.` f. `finally : inFlightScheduleIds.delete(scheduleId)` — always releases the lock, success or failure.

- Database calls: `Schedule.find({ enabled: true, nextRun: { $lte: now } })` (via `getDueSchedules`); `Schedule.findOneAndUpdate({ _id, userId }, updates, { new: true })` (via `updateSchedule` , once per schedule per tick, success or failure path); indirectly, everything `scanInfrastructure` does (`Scan.create` , per-service `Resource` / `Recommendation` writes) and everything `notifyScanCompletion` does (`Notification.findOne` , `Recommendation.find`).
- AWS calls: none directly — but transitively, `scanInfrastructure` makes the full EC2/S3/RDS/IAM/CloudWatch/Pricing/STS SDK calls for the scan itself.
- Error handling: Two layers — (1) the whole per-schedule scan+update block is wrapped in try/catch/finally so one user's scan failure doesn't stop the loop from processing the next due schedule; (2) `notifyScanCompletion` is wrapped in its own inner try/catch so a notification failure never affects the already-recorded `lastRun` / `nextRun` state.
- Return values: none meaningful — always resolves to `undefined` after the loop finishes.

• `startScheduler()`

- Purpose: Registers the per-minute cron job and logs startup.
- Input: none.
- Output: `undefined` .
- Internal steps: 1) `cron.schedule("* * * * *", runDueSchedules)` . 2) `console.log("Scheduled scan cron started (checking every minute)")` .
- Database/AWS calls: none directly.
- Error handling: none.
- Return values: none.

DATA FLOW

`server.js` boot → `startScheduler()` registers the job → every 60s, `node-cron` fires `runDueSchedules` → `Schedule` collection (which users are due) → for each due user, `scanner.service.js`'s `scanInfrastructure` (full AWS scan + `Scan` / `Resource` / `Recommendation` writes) → `Schedule.findOneAndUpdate` (advance `lastRun` / `nextRun`) → `scanNotification.service.js` (best-effort email).

EXAMPLE EXECUTION

A user has an `hourly` schedule with `nextRun` in the past. At the next minute tick, `getDueSchedules(now)` returns that schedule. `runDueSchedules` adds its ID to `inFlightScheduleIds` , calls `scanInfrastructure(userId)` (takes, say, 90 seconds), and while that's running a second tick 60 seconds later sees the same schedule ID already in `inFlightScheduleIds` and skips it via `continue` . Once the scan finishes, `lastRun` is set to the completion time and `nextRun = computeNextRun("hourly", lastRun)` (one hour later), `notifyScanCompletion` fires the scan-completed/high-severity emails, and the ID is removed from `inFlightScheduleIds` in the `finally` block.

IMPORTANT NOTES

- **Concurrency guard:** `inFlightScheduleIds` is a plain in-memory `Set` , scoped to this single Node process — it protects against a slow scan overlapping with the *next* cron tick within the same process, but provides no protection at all across multiple server instances/replicas, since each process would have its own independent `Set` .
- **Failure semantics:** on scan failure, `nextRun` still advances (via `computeNextRun(schedule.frequency, now)` , using the *current* time as the base rather than the stale prior `nextRun`) but `lastRun` does not update — this is a deliberate design so a persistently-failing account (e.g. AWS disconnected) is retried on its normal interval instead of being hammered every minute, while `lastRun` continues to reflect the last time a scan actually succeeded.
- A single-collection-poll design (one cron job querying `Schedule` for anyone due) was chosen over per-user cron jobs specifically to avoid start/stop/restart bookkeeping on every schedule create/update/delete and to survive server restarts without re-registering anything.

PURPOSE

CRUD service for a user's recurring scan schedule. Enforces a strict one-schedule-per-user model (mirroring the one-AWS-connection-per-user design) and recomputes `nextRun` only when the interval actually changes or the schedule transitions from disabled to enabled.

WHEN THIS FILE IS CALLED

Whenever the user creates, views, edits, or deletes their scan schedule from the Schedule Scan page.

WHO CALLS THIS FILE

- `server/src/controllers/schedule.controller.js` (`createSchedule` → `addSchedule` ; `getSchedules` → `fetchSchedules` ; `updateSchedule` → `editSchedule` ; `deleteSchedule` → `removeSchedule`), reached from `server/src/routes/schedule.routes.js` → `POST /api/schedules` , `GET /api/schedules` , `PATCH /api/schedules/:id` , `DELETE /api/schedules/:id`

WHICH FILES IT DEPENDS ON

- `server/src/services/database/schedule.repository.js` — `createSchedule` , `getScheduleByUserId` , `getScheduleById` , `updateSchedule` , `deleteSchedule` (imported with local aliases `createScheduleRecord` / `updateScheduleRecord` / `deleteScheduleRecord`)
- `server/src/utills/ApiError.js` — structured 400/404/409 errors
- `server/src/utills/schedule.util.js` — `isValidFrequency` , `computeNextRun`

EXPORTED FUNCTIONS

- `addSchedule({ userId, frequency, enabled = true })`
 - Purpose: Creates the user's (single) scan schedule.
 - Input: `userId` , `frequency` (must be `"hourly"` / `"daily"` / `"weekly"`), `enabled` (defaults `true`).
 - Output: newly created `Schedule` document.
 - Internal steps: 1) `if (!isValidFrequency(frequency)) throw new ApiError(400, "Frequency must be one of: hourly, daily, weekly")` . 2) `existing = await getScheduleByUserId(userId)` ; if found, `throw new ApiError(409, "A scan schedule already exists. Update it instead of creating a new one.")` . 3) `nextRun = computeNextRun(frequency)` (from now). 4) `return await createScheduleRecord({ userId, frequency, enabled, nextRun })` .
 - Database calls: `Schedule.findOne({ userId })` , `Schedule.create(...)` .
 - AWS calls: none.
 - Error handling: `ApiError(400, ...)` for invalid frequency; `ApiError(409, ...)` if a schedule already exists for this user.
 - Return values: the created `Schedule` document.
- `fetchSchedules(userId)`
 - Purpose: Returns the user's schedule wrapped in an array (API shape consistency with a hypothetical multi-schedule list), or an empty array if none exists.
 - Input: `userId` .
 - Output: `[]` or `[schedule]` .
 - Internal steps: `schedule = await getScheduleByUserId(userId)` ; `return schedule ? [schedule] : []` .
 - Database calls: `Schedule.findOne({ userId })` .
 - AWS calls: none.
 - Error handling: none — absence is a normal, non-error state.
 - Return values: `[]` or a single-element array.
- `editSchedule(id, userId, { frequency, enabled })`
 - Purpose: Partially updates the schedule, recomputing `nextRun` only when necessary.
 - Input: `id` (schedule ID), `userId` , optional `frequency` / `enabled` .
 - Output: updated `Schedule` document.
 - Internal steps:
 - `if (frequency !== undefined && !isValidFrequency(frequency)) throw new ApiError(400, ...)` .
 - `current = await getScheduleById(id, userId)` ; if not found, `throw new ApiError(404, "Schedule not found")` .
 - Build `updates = {}` from provided `frequency` / `enabled` .

4. `isBeingEnabled = enabled === true && !current.enabled ; isFrequencyChanging = frequency !== undefined && frequency !== current.frequency .`
 5. If either is true, `updates.nextRun = computeNextRun(frequency !== undefined ? frequency : current.frequency)` — recomputed from *now*, not from the old `nextRun` .
 6. `return await updateScheduleRecord(id, userId, updates) .`
- Database calls: `Schedule.findOne({ _id: id, userId })` (via `getScheduleById`), `Schedule.findOneAndUpdate({ _id: id, userId }, updates, { new: true })` .
 - AWS calls: none.
 - Error handling: `ApiError(400, ...)` for invalid frequency; `ApiError(404, "Schedule not found")` if the schedule doesn't exist or doesn't belong to `userId` .
 - Return values: updated `Schedule` document.
- `removeSchedule(id, userId)`
 - Purpose: Deletes the user's schedule.
 - Input: `id` , `userId` .
 - Output: the deleted `Schedule` document.
 - Internal steps: `deleted = await deleteScheduleRecord(id, userId)` ; if falsy, `throw new ApiError(404, "Schedule not found") ; return deleted .`
 - Database calls: `Schedule.findOneAndDelete({ _id: id, userId })` .
 - AWS calls: none.
 - Error handling: `ApiError(404, "Schedule not found")` .
 - Return values: the deleted document.

DATA FLOW

`schedule.controller.js` → this service → `schedule.repository.js` → MongoDB `schedules` collection. `scheduler.service.js` 's cron tick later reads/writes the same `Schedule` collection independently (via `getDueSchedules` / `updateSchedule`), so a user editing their schedule mid-tick and the cron job updating `lastRun` / `nextRun` after a scan both operate on the same document without any cross-file coordination beyond MongoDB's own atomic per-document updates.

EXAMPLE EXECUTION

A user with no existing schedule calls `POST /api/schedules { frequency: "daily" }`: `addSchedule` validates `"daily"` is valid, confirms no existing schedule, computes `nextRun = now + 24h`, and creates the `Schedule` document. Later they call `PATCH /api/schedules/:id { enabled: false }` to pause it — `isBeingEnabled` is `false` (enabled isn't `true`) and `isFrequencyChanging` is `false`, so `nextRun` is left untouched; only `enabled: false` is written. Re-enabling later (`{ enabled: true }`) recomputes `nextRun` fresh from "now" rather than resuming the stale pre-pause timestamp.

IMPORTANT NOTES

- The one-schedule-per-user constraint is enforced at the service layer (an explicit `existing` check plus a `409`), backed by the `Schedule` model's own `userId: { unique: true }` index as a second line of defense.
- `computeNextRun` always computes relative to the current time (or an explicit `from` argument) — there's no concept of "catch up from where it left off"; pausing and re-enabling a schedule effectively resets its cycle.

 `server/src/services/scan/scanHistory.service.js`

PURPOSE

Read-only service for browsing past scans — lists all scans for a user and fetches one scan's full detail (the scan record plus every resource and recommendation it produced). Powers both the Scan History page and (indirectly, via reuse) the PDF report data builder.

WHEN THIS FILE IS CALLED

When the Scan History page loads (list) and when a user opens a specific scan's detail view or exports its PDF report.

WHO CALLS THIS FILE

- `server/src/controllers/scans.controller.js` (`getScans` → `fetchAllScans` ; `getScanById` → `fetchScanDetails`), reached from `server/src/routes/scans.routes.js` → `GET /api/scans` , `GET /api/scans/:id`
- `server/src/services/report/scanReport.service.js` (`fetchScanDetails` , reused for PDF generation)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/scan.repository.js` — `getAllScans` , `getScanById`
- `server/src/services/database/resource.repository.js` — `getResourcesByScanId`
- `server/src/services/database/recommendation.repository.js` — `getRecommendationsByScanId`

EXPORTED FUNCTIONS

- `fetchAllScans(userId)`
 - Purpose: Lists every scan the user has ever run, newest first.
 - Input: `userId` .
 - Output: array of `Scan` documents.
 - Internal steps: `return await getAllScans(userId)` .
 - Database calls: `Scan.find({ userId }).sort({ createdAt: -1 })` .
 - AWS calls: none.
 - Error handling: none.
 - Return values: array (possibly empty).
- `fetchScanDetails(id, userId)`
 - Purpose: Fetches one scan plus its full resource and recommendation sets.
 - Input: `id` (scan ID), `userId` .
 - Output: `null` if not found/not owned, otherwise `{ scan, resources, recommendations }` .
 - Internal steps: 1) `scan = await getScanById(id, userId)` ; if falsy, `return null` . 2) `[resources, recommendations] = await Promise.all([getResourcesByScanId(id, userId), getRecommendationsByScanId(id, userId)])` (run concurrently). 3) `return { scan, resources, recommendations }` .
 - Database calls: `Scan.findOne({ _id: id, userId })` ; `Resource.find({ scanId, userId }).sort({ createdAt: -1 })` ; `Recommendation.find({ scanId, userId }).sort({ createdAt: -1 })` plus its resource-enrichment join.
 - AWS calls: none.
 - Error handling: none thrown — a missing/foreign scan simply yields `null` , and it's up to each caller to translate that into a 404 (both `scans.controller.js` and `report.controller.js` do so independently).
 - Return values: `null` , or `{ scan, resources, recommendations }` .

DATA FLOW

Controller (or `scanReport.service.js`) → `fetchScanDetails` / `fetchAllScans` → `scan.repository.js` / `resource.repository.js` / `recommendation.repository.js` → MongoDB. This is the single shared source of "everything about one scan" reused by both the Scan Detail page's API and the PDF export pipeline.

EXAMPLE EXECUTION

`GET /api/scans/64f0.../` → `fetchScanDetails` returns the `Scan` doc (`status: "COMPLETED"` , `totalResources: 20`) plus its 20 `Resource` documents and however many `Recommendation` documents were generated, each recommendation enriched with `resourceName` / `instanceType` / `state` /etc. pulled from the matching `Resource` .

IMPORTANT NOTES

- `resources` and `recommendations` are fetched in parallel via `Promise.all` , not sequentially — slightly faster than awaiting them one at a time, with no ordering dependency between the two queries.
- Ownership is enforced at the query level (`{ _id: id, userId }` on the scan lookup) — a scan ID belonging to a different user simply doesn't match, so `getScanById` returns `null` and the whole function short-circuits to `null` rather than leaking another user's data.

 `server/src/services/recommendation/recommendation.service.js`

PURPOSE

Manages the lifecycle of individual recommendations from the Recommendations page: listing the latest scan's recommendations, marking one ignored or resolved, and — uniquely among these four actions — actually reaching into AWS to stop an EC2 instance when the user clicks "Stop Instance" on an oversized/idle-instance recommendation.

WHEN THIS FILE IS CALLED

Whenever the Recommendations page loads, or a user clicks Ignore / Resolve / Stop Instance on a specific recommendation row.

WHO CALLS THIS FILE

- `server/src/controllers/recommendation.controller.js` (`getRecommendations` → `fetchRecommendations` ; `ignoreRecommendation` → `ignoreRecommendationById` ; `resolveRecommendation` → `resolveRecommendationById` ; `stopRecommendation` → `stopRecommendationById`), reached from `server/src/routes/recommendation.routes.js` → `GET /api/recommendations` , `PATCH /api/recommendations/:id/ignore` , `PATCH /api/recommendations/:id/resolve` , `POST /api/recommendations/:id/stop`

WHICH FILES IT DEPENDS ON

- `server/src/config/aws.js` — `createEC2Client` (builds an EC2 SDK client using the server's own AWS credential chain, not the user's stored/decrypted `AwsConnection` credentials)
- `server/src/services/aws/ec2.service.js` — `stopEC2Instance` (wraps `StopInstancesCommand`)
- `server/src/services/database/recommendation.repository.js` — `getLatestRecommendations` , `getRecommendationById` , `ignoreRecommendation` , `resolveRecommendation` , `enrichRecommendationWithResource`
- `server/src/services/database/resource.repository.js` — `updateResourceState`
- `server/src/utils/ApiError.js` — structured 404/400 errors

EXPORTED FUNCTIONS

- `fetchRecommendations(userId)`
 - Purpose: Returns the latest scan's recommendations, resource-enriched.
 - Input: `userId` .
 - Output: array of enriched recommendation objects.
 - Internal steps: `return await getLatestRecommendations(userId)` .
 - Database calls: `Scan.findOne({ userId })` .sort({ `createdAt: -1` }) , `Recommendation.find({ scanId, userId })` .sort({ `createdAt: -1` }) , plus resource-enrichment `Resource.find({ scanId, userId, resourceId: { $in: [...] } })` .
 - AWS calls: none.
 - Error handling: none.
 - Return values: array (possibly empty if no scan exists).
- `ignoreRecommendationById(id, userId)`
 - Purpose: Marks a recommendation as `IGNORED` .
 - Input: `id` , `userId` .
 - Output: the updated, resource-enriched recommendation.
 - Internal steps: 1) `updated = await ignoreRecommendation(id, userId)` ; if falsy, `throw new ApiError(404, "Recommendation not found")` . 2) `return await enrichRecommendationWithResource(updated)` .
 - Database calls: `Recommendation.findOneAndUpdate({ _id: id, userId }, { status: "IGNORED" }, { new: true })` ; then a `Resource.find` enrichment lookup.
 - AWS calls: none.
 - Error handling: `ApiError(404, "Recommendation not found")` if the ID doesn't match a recommendation owned by `userId` .
 - Return values: enriched recommendation object.
- `resolveRecommendationById(id, userId)`
 - Purpose: Marks a recommendation as `RESOLVED` .
 - Input: `id` , `userId` .
 - Output: the updated, resource-enriched recommendation.
 - Internal steps: 1) `updated = await resolveRecommendation(id, userId)` ; if falsy, `throw new ApiError(404, "Recommendation not found")` . 2) `return await enrichRecommendationWithResource(updated)` .
 - Database calls: `Recommendation.findOneAndUpdate({ _id: id, userId }, { status: "RESOLVED" }, { new: true })` ; resource-enrichment lookup.
 - AWS calls: none.
 - Error handling: `ApiError(404, "Recommendation not found")` .
 - Return values: enriched recommendation object.
- `stopRecommendationById(id, userId)`

- Purpose: Actually stops the underlying EC2 instance via AWS, then marks the recommendation resolved. The only function in this file with a real AWS side effect.
- Input: `id`, `userId`.
- Output: the resolved, resource-enriched recommendation.
- Internal steps:
 1. `recommendation = await getRecommendationById(id, userId)`; if falsy, `throw new ApiError(404, "Recommendation not found")`.
 2. `enriched = await enrichRecommendationWithResource(recommendation)`; if `enriched?.state?.toLowerCase() === "stopped"`, `throw new ApiError(400, "This instance is already stopped.")` — a fresh server-side re-check in case the frontend's cached state is stale.
 3. `ec2Client = createEC2Client()` — built with **no** explicit credentials/region argument, so it falls back to the AWS SDK's default provider chain (environment variables / shared AWS config on the server), **not** the specific `AwsConnection`'s stored access key.
 4. `await stopEC2Instance(ec2Client, recommendation.resourceId)` — sends `StopInstancesCommand({ InstanceIds: [recommendation.resourceId] })`.
 5. `await updateResourceState(recommendation.scanId, recommendation.resourceId, "stopped", userId)` — updates the cached `Resource` document's `state` field so the dashboard reflects the change without waiting for the next scan.
 6. `updated = await resolveRecommendation(id, userId)` — marks the recommendation `RESOLVED` since the recommended action was carried out.
 7. `return await enrichRecommendationWithResource(updated)`.
- Database calls: `Recommendation.findOne({ _id: id, userId })` (via `getRecommendationById`); `Resource.find` enrichment (twice — once for the pre-check, once for the final return); `Resource.findOneAndUpdate({ scanId, resourceId, userId }, { state: "stopped" }, { new: true })` (via `updateResourceState`); `Recommendation.findOneAndUpdate({ _id: id, userId }, { status: "RESOLVED" }, { new: true })`.
- AWS calls: `EC2Client.send(new StopInstancesCommand({ InstanceIds: [instanceId] }))` via `stopEC2Instance`.
- Error handling: `ApiError(404, "Recommendation not found")` if the recommendation doesn't exist/isn't owned by the user; `ApiError(400, "This instance is already stopped.")` if the resource's cached state already reads `"stopped"` (case-insensitive check); any AWS SDK error from `stopEC2Instance` (e.g. instance already stopped on AWS's side, permission denied) propagates unhandled up through `asyncHandler`.
- Return values: the resolved, resource-enriched recommendation object.

DATA FLOW

`recommendation.controller.js` → this service → for `stopRecommendationById`: `config/aws.js` (`createEC2Client`) → `aws/ec2.service.js` (`stopEC2Instance`) → live AWS EC2 API → back to `resource.repository.js` / `recommendation.repository.js` to persist the state change and resolution in MongoDB.

EXAMPLE EXECUTION

A user has a HIGH-severity "STOP" recommendation for an idle `i-0abc123` instance. They click "Stop Instance": `stopRecommendationById` re-checks the resource isn't already stopped, calls AWS to stop `i-0abc123`, flips the cached `Resource.state` to `"stopped"`, marks the recommendation `RESOLVED`, and returns the updated recommendation (now showing `state: "stopped"`) to the UI. The controller then records an `EC2_STOPPED` audit log entry.

IMPORTANT NOTES

- `createEC2Client()` here is called with no credentials argument, which means it always uses the **server's own** AWS identity (default SDK credential chain), not the specific user's connected AWS account credentials stored (encrypted) in `AwsConnection` — this differs from how scans authenticate, which use `factories/awsClient.factory.js` to build clients from the user's own decrypted credentials. In a deployment where the server process has no AWS credentials of its own, this call would fail; in a deployment where it does, it stops the instance using whatever account the server itself is authorized against, which may or may not be the same AWS account the user connected.
- The re-check in step 2 (`enriched?.state?.toLowerCase() === "stopped"`) exists specifically because the frontend hides the "Stop Instance" button once a resource shows as stopped, but that UI snapshot can go stale (another scan ran, another browser tab acted, a direct API call was made) — this turns what would otherwise be a raw AWS SDK rejection into a clean, user-readable 400.
- `stopRecommendationById` calls `resolveRecommendation`, not `ignoreRecommendation` — stopping the instance is treated as *acting on* the recommendation, which is why the controller logs only `EC2_STOPPED` (not also `RECOMMENDATION_RESOLVED`) to avoid a double audit-log entry for one user action.

PURPOSE

Rule-based (no LLM), deterministic sentence generator that turns a scan's per-bucket S3 signals (already computed by `s3.scanner.js`) into short human-readable insight messages — mirrors the threshold/count style of the S3 recommender rather than doing any new AWS or database work itself.

WHEN THIS FILE IS CALLED

During every S3 portion of a scan (`scanS3` , inside `scanner.service.js` 's `scanInfrastructure`), after all buckets have been processed into per-bucket signal objects.

WHO CALLS THIS FILE

- `server/src/services/scanner/s3.scanner.js` (`generateS3Insights`)

WHICH FILES IT DEPENDS ON

- none (no internal or external imports — pure function over a plain JS array)

EXPORTED FUNCTIONS

- `generateS3Insights(buckets)`
- Purpose: Produces an array of `{ category, severity, message }` insight objects summarizing the S3 fleet's security, lifecycle, storage, and cost posture.
- Input: `buckets` — array of per-bucket signal objects with fields including `isPublic` , `publicPolicyGrant` , `encryptionEnabled` , `bucketSizeGB` , `hasLifecycleRules` , `standardPercent` , `versioningEnabled` , `monthlyCost` , `estimatedSavings` .
- Output: array of insight objects, always non-empty.
- Internal steps:
 1. If `!buckets.length` , return a single `[{ category: "GENERAL", severity: "INFO", message: "No S3 buckets were found in this scan." }]` .
 2. **Security — public buckets:** `publicBuckets = buckets.filter(b => b.isPublic || b.publicPolicyGrant)` ; if any, push `{ category: "SECURITY", severity: "HIGH", message: "${count} bucket(s) is/are public." }` .
 3. **Security — missing encryption:** `unencryptedBuckets = buckets.filter(b => !b.encryptionEnabled)` ; if any, push `{ category: "SECURITY", severity: "HIGH", message: "${count} bucket(s) is/are missing encryption." }` .
 4. **Lifecycle — large + unmanaged:** `largeBucketsWithoutLifecycle = buckets.filter(b => b.bucketSizeGB > 100 && !b.hasLifecycleRules)` (`LARGE_BUCKET_GB_THRESHOLD = 100`); pushes a singular- or plural-worded `{ category: "LIFECYCLE", severity: "MEDIUM", ... }` message depending on count (exactly 1 vs. more than 1 — handled as two separate branches, not via the shared `pluralize` helper).
 5. **Storage — STANDARD-heavy:** `standardHeavyBuckets = buckets.filter(b => b.bucketSizeGB >= 1 && (b.standardPercent || 0) >= 80)` (`STANDARD_HEAVY_PERCENT_THRESHOLD = 80`); if any, push `{ category: "STORAGE", severity: "MEDIUM", message: "...over 80% of its data in STANDARD storage – consider Intelligent-Tiering." }` .
 6. **Storage — versioning disabled:** `versioningDisabledBuckets = buckets.filter(b => !b.versioningEnabled)` ; if any, push `{ category: "STORAGE", severity: "LOW", message: "${count} bucket(s) has/have versioning disabled." }` .
 7. **Cost — current spend** (always included): `totalMonthlyCost = buckets.reduce((sum, b) => sum + (b.monthlyCost || 0), 0)` ; push `{ category: "COST", severity: "INFO", message: "Estimated storage cost is $X.XX/month." }` .
 8. **Cost — potential savings** (only if `> 0`): `totalEstimatedSavings = buckets.reduce((sum, b) => sum + (b.estimatedSavings || 0), 0)` ; if `> 0` , push `{ category: "COST", severity: "LOW", message: "Up to $X.XX/month could be saved by acting on the flagged recommendations." }` .
 9. **General — healthy signal:** if none of `publicBuckets` , `unencryptedBuckets` , `largeBucketsWithoutLifecycle` had any entries, push `{ category: "GENERAL", severity: "INFO", message: "No public buckets, missing encryption, or unmanaged large buckets were found – S3 posture looks healthy." }` (note: this healthy check does **not** consider `standardHeavyBuckets` or `versioningDisabledBuckets`).
 10. `return insights` .
- Database calls: none.
- AWS calls: none.
- Error handling: none — assumes `buckets` is a well-formed array from `s3.scanner.js` 's own `processBucket` step.

- Return values: always a non-empty array of `{ category, severity, message }` objects — the `buckets.length === 0` branch guarantees at least the "No S3 buckets" message, and every populated-buckets path guarantees at least the COST/INFO current-spend message.

DATA FLOW

`s3.scanner.js` (after computing per-bucket signals for every scanned bucket) → `generateS3Insights(buckets)` → returned array is persisted via `insight.repository.js`'s `saveInsights(scanId, insights, userId)` → later read back by `dashboard.service.js` via `getLatestInsights` for display on the Dashboard's S3 insights panel.

EXAMPLE EXECUTION

A scan finds 4 buckets: 1 public, 2 missing encryption, 1 at 250GB with no lifecycle rules, total monthly cost \$45.20, no savings opportunities.

`generateS3Insights` returns 5 insights: "1 bucket is public." (SECURITY/HIGH), "2 buckets are missing encryption." (SECURITY/HIGH), "One bucket larger than 100 GB has no lifecycle." (LIFECYCLE/MEDIUM), "Estimated storage cost is \$45.20/month." (COST/INFO) — plus whatever the versioning/STANDARD-heavy checks add if applicable. The "S3 posture looks healthy" message is suppressed because public/unencrypted/unmanaged-large buckets were found.

IMPORTANT NOTES

- All thresholds are hardcoded constants in this file: `LARGE_BUCKET_GB_THRESHOLD = 100`, `STANDARD_HEAVY_PERCENT_THRESHOLD = 80` — changing S3 insight sensitivity means editing this file directly, there's no external config.
- The "General — healthy" fallback message only checks 3 of the 5 possible negative signals (public, unencrypted, large-without-lifecycle) — a bucket that's otherwise clean but has versioning disabled or is STANDARD-heavy would get both its own LOW/MEDIUM insight *and* still not qualify for the healthy message, which is consistent (it isn't fully healthy) but means "no HIGH/MEDIUM findings" and "gets the healthy label" aren't perfectly the same condition.
- Output objects use `category` values (`SECURITY`, `LIFECYCLE`, `STORAGE`, `COST`, `GENERAL`) that are S3-specific and distinct from the `Recommendation` model's `category` enum (`COST`, `SECURITY`, `PERFORMANCE`, `RELIABILITY`, `OPERATIONAL_EXCELLENCE`) — these are `Insight` documents, a separate collection/model from `Recommendation`.

 `server/src/services/awsConnection/awsConnection.service.js`

PURPOSE

Manages the single AWS account connection per user: testing credentials without persisting them, connecting (validating + encrypting + storing) a new account, reading the current connection, and disconnecting. This is the gate that determines whether `scanInfrastructure` can even run for a given user.

WHEN THIS FILE IS CALLED

When the user visits the Connect AWS page and clicks "Test Connection" or "Connect", when the dashboard/navbar needs to know if an account is connected, and when the user disconnects their account.

WHO CALLS THIS FILE

- `server/src/controllers/awsConnection.controller.js` (`testConnection` → `testAwsConnection`; `connect` → `connectAwsAccount`; `getAccount` → `getConnectionByUserId`; `disconnect` → `disconnectAwsAccount`), reached from `server/src/routes/aws.routes.js` → `POST /api/aws/test-connection`, `POST /api/aws/connect`, `GET /api/aws/account`, `DELETE /api/aws/disconnect`

WHICH FILES IT DEPENDS ON

- `server/src/models/AwsConnection.js` — direct Mongoose model access (no repository layer for this model)
- `server/src/services/aws/sts.service.js` — `validateAwsCredentials` (confirms credentials work via STS `GetCallerIdentity`)
- `server/src/utils/ApiError.js` — structured 400/404/409 errors
- `server/src/utils/encryption.js` — `encrypt` (AES-256-GCM encryption of the secret access key before storage)

EXPORTED FUNCTIONS

- `getConnectionByUserId(userId)`
- Purpose: Returns the user's current AWS connection, if any, with the encrypted secret stripped.
- Input: `userId`.
- Output: `AwsConnection` document (or `null`) with `encryptedSecretKey` excluded via `.select("-encryptedSecretKey")`.
- Internal steps: `return await AwsConnection.findOne({ userId }).select("-encryptedSecretKey")`.
- Database calls: `AwsConnection.findOne({ userId })`.

- AWS calls: none.
- Error handling: none — no connection is a normal `null` result, not an error.
- Return values: `AwsConnection` document or `null`.
- `testAwsConnection({ region, accessKeyId, secretAccessKey })`
- Purpose: Validates a set of credentials via STS without persisting anything, letting the frontend confirm they work before the user commits to saving them.
- Input: `{ region, accessKeyId, secretAccessKey }`.
- Output: `{ accountId, arn, region }`.
- Internal steps: 1) `validation = await validateAwsCredentials({ accessKeyId, secretAccessKey, region })`. 2) If `!validation.valid`, throw new `ApiError(400, "Invalid AWS credentials. Please check your Access Key ID, Secret Access Key, and Region.")`. 3) `return { accountId: validation.accountId, arn: validation.arn, region }`.
- Database calls: none.
- AWS calls: `STSClient.send(new GetCallerIdentityCommand({}))` (via `validateAwsCredentials`).
- Error handling: `ApiError(400, ...)` if STS rejects the credentials (any STS error is caught inside `validateAwsCredentials` itself and turned into `{ valid: false }`).
- Return values: `{ accountId, arn, region }` on success.
- `connectAwsAccount({ userId, region, accessKeyId, secretAccessKey, accountAlias })`
- Purpose: Validates and persists a new AWS connection for the user, enforcing one connection per user.
- Input: `{ userId, region, accessKeyId, secretAccessKey, accountAlias }`.
- Output: the created `AwsConnection` as a plain object with `encryptedSecretKey` deleted.
- Internal steps:
 1. `existing = await AwsConnection.findOne({ userId })`; if found, throw new `ApiError(409, "An AWS account is already connected. Disconnect it before connecting a new one.")`.
 2. `validation = await validateAwsCredentials({ accessKeyId, secretAccessKey, region })`; if `!validation.valid`, throw new `ApiError(400, "Invalid AWS credentials. Please check your Access Key ID, Secret Access Key, and Region.")`.
 3. `now = new Date()`.
 4. `connection = await AwsConnection.create({ userId, accountId: validation.accountId, accountAlias, region, accessKeyId, encryptedSecretKey: encrypt(secretAccessKey), status: "CONNECTED", connectedAt: now, lastValidatedAt: now })`.
 5. `connectionObject = connection.toObject(); delete connectionObject.encryptedSecretKey; return connectionObject`.
- Database calls: `AwsConnection.findOne({ userId })`, `AwsConnection.create(...)`.
- AWS calls: `STSClient.send(new GetCallerIdentityCommand({}))` (via `validateAwsCredentials`).
- Error handling: `ApiError(409, ...)` if a connection already exists; `ApiError(400, ...)` if credentials don't validate.
- Return values: plain object representing the new connection (secret stripped).
- `disconnectAwsAccount(userId)`
- Purpose: Removes the user's AWS connection entirely.
- Input: `userId`.
- Output: the deleted `AwsConnection` document.
- Internal steps: 1) `connection = await AwsConnection.findOneAndDelete({ userId })`; if falsy, throw new `ApiError(404, "No AWS account is connected.")`. 2) `return connection`.
- Database calls: `AwsConnection.findOneAndDelete({ userId })`.
- AWS calls: none (does not revoke/rotate the credentials on the AWS side — only removes CloudLens's stored record of them).
- Error handling: `ApiError(404, "No AWS account is connected.")`.
- Return values: the deleted document.

DATA FLOW

`awsConnection.controller.js` → this service → `sts.service.js` (live AWS STS call to prove the credentials are real) → `utils/encryption.js` (AES-256-GCM encrypt the secret key before it ever touches the database) → `AwsConnection` MongoDB document. This connection is later read (and decrypted) by `factories/awsClient.factory.js` whenever `scanner.service.js` needs to build per-user AWS SDK clients for an actual scan.

EXAMPLE EXECUTION

A user pastes an access key/secret/region and clicks "Connect": `connectAwsAccount` confirms no existing connection, calls STS `GetCallerIdentity` (succeeds, returning `accountId: "123456789012"`), encrypts the secret with `crypto.createCipheriv("aes-256-gcm", ...)`, and stores a new `AwsConnection` document with `status: "CONNECTED"`. The controller then records an `AWS_CONNECTED` audit log entry and returns the stripped connection object to the frontend.

IMPORTANT NOTES

- The `AwsConnection` schema itself also strips `encryptedSecretKey` via a `toJSON` transform (defense in depth) — so even if a future code path forgot to manually delete it (as `connectAwsAccount` does), any `res.json()` / `JSON.stringify()` of the raw document could never leak it. `getConnectionByUserId` additionally excludes it at the query level via `.select("-encryptedSecretKey")`.
- `disconnectAwsAccount` only deletes CloudLens's own stored credential record — it never calls AWS to deactivate/rotate the underlying IAM access key, so the credentials remain valid on the AWS side until the user manually revokes them there.
- `testAwsConnection` and `connectAwsAccount` both call `validateAwsCredentials` independently — testing a connection and then connecting it triggers two separate live STS calls, not a cached result from the test.

 `server/src/services/auth/auth.service.js`

PURPOSE

Core authentication service: user registration (with bcrypt password hashing), login (credential verification + JWT issuance), and a "logout from all devices" mechanism implemented via a per-user token-version counter rather than a server-side token blacklist.

WHEN THIS FILE IS CALLED

On every signup, login, and "logout all devices" request.

WHO CALLS THIS FILE

- `server/src/controllers/auth.controller.js` (`signup` → `registerUser`; `login` → `loginUser`; `logoutAll` → `logoutAllDevices`), reached from `server/src/routes/auth.routes.js` → `POST /api/auth/signup`, `POST /api/auth/login` (behind `LoginRateLimiter`), `POST /api/auth/logout-all` (behind `protect`, i.e. requires a currently-valid JWT)

WHICH FILES IT DEPENDS ON

- `server/src/models/User.js` — direct Mongoose model access (only for `logoutAllDevices`'s `findByIdAndUpdate`; reads go through the repository)
- `server/src/services/database/user.repository.js` — `findUserByEmail`, `createUser`
- `server/src/utils/jwt.js` — `generateToken`
- `server/src/utils/ApiError.js` — structured 401/404/409 errors
- external: `bcrypt` (hashing/comparison, `SALT_ROUNDS = 10`)

EXPORTED FUNCTIONS

- `registerUser({ fullName, email, password })`
- Purpose: Creates a new user account with a bcrypt-hashed password.
- Input: `{ fullName, email, password }` (plaintext password).
- Output: the created user, via `user.toJSON()` (password stripped by the `User` schema's `toJSON` transform).
- Internal steps: 1) `existingUser = await findUserByEmail(email)`; if found, `throw new ApiError(409, "Email is already registered")`. 2) `hashedPassword = await bcrypt.hash(password, 10)`. 3) `user = await createUser({ fullName, email, password: hashedPassword })`. 4) `return user.toJSON()`.
- Database calls: `User.findOne({ email })` (via `findUserByEmail`), `User.create({ fullName, email, password })` (via `createUser`).
- AWS calls: none.
- Error handling: `ApiError(409, "Email is already registered")` if the email is already taken.
- Return values: plain user object (password field removed by `toJSON`).
- `loginUser({ email, password })`
- Purpose: Verifies credentials and issues a JWT.
- Input: `{ email, password }`.
- Output: `{ token, user }`.

- Internal steps: 1) `user = await findUserByEmail(email)`; if not found, `throw new ApiError(401, "Invalid email or password")`. 2) `isPasswordValid = await bcrypt.compare(password, user.password)`; if `false`, `throw new ApiError(401, "Invalid email or password")` (same message as the "user not found" case, deliberately not distinguishing the two to avoid leaking which emails are registered). 3) `token = generateToken(user._id, user.tokenVersion)` — JWT payload is `{ userId, tokenVersion }`, signed with `process.env.JWT_SECRET`, expiring per `process.env.JWT_EXPIRES_IN` (default `"7d"`). 4) `return { token, user: user.toJSON() }`.
- Database calls: `User.findOne({ email })`.
- AWS calls: none.
- Error handling: `ApiError(401, "Invalid email or password")` for both a nonexistent email and a wrong password — identical error message/status in both cases.
- Return values: `{ token, user }` (user object has password stripped).
- `logoutAllDevices(userId)`
- Purpose: Invalidates every JWT previously issued to this user by bumping their `tokenVersion`.
- Input: `userId`.
- Output: `{ loggedOutAllDevices: true }`.
- Internal steps: 1) `user = await User.findByIdAndUpdate(userId, { $inc: { tokenVersion: 1 } }, { new: true })`. 2) If falsy, `throw new ApiError(404, "User not found")`. 3) `return { loggedOutAllDevices: true }`.
- Database calls: `User.findByIdAndUpdate(userId, { $inc: { tokenVersion: 1 } }, { new: true })`.
- AWS calls: none.
- Error handling: `ApiError(404, "User not found")` (defensive — `userId` normally comes from a verified JWT so the user should exist).
- Return values: `{ loggedOutAllDevices: true }`.

DATA FLOW

`auth.controller.js` → this service → `user.repository.js` / direct `User` model access → MongoDB `users` collection;
`generateToken` /JWT signing via `utils/jwt.js`; the `protect` auth middleware (not in this file, but the consumer of `tokenVersion`) rejects any incoming JWT whose `tokenVersion` claim doesn't match the user's current stored value, which is what makes `logoutAllDevices` effective without a token blacklist.

EXAMPLE EXECUTION

`POST /api/auth/login { email: "akshatjohri03@gmail.com", password: "..." }` → `loginUser` finds the user, `bcrypt.compare` succeeds, `generateToken(userId, 0)` produces a JWT valid for 7 days. Later the user clicks "Log out everywhere": `POST /api/auth/logout-all` → `logoutAllDevices` bumps `tokenVersion` from `0` to `1` — the JWT issued at login (which embeds `tokenVersion: 0`) is now rejected by `protect` on its next use, even though it hasn't expired.

IMPORTANT NOTES

- Login intentionally returns the identical `ApiError(401, "Invalid email or password")` for both "no such user" and "wrong password" — this is a standard anti-enumeration practice, not an oversight.
- `logoutAllDevices` also logs out the device that *made* the logout-all request (there's no exception carve-out for the current session) — the auth controller's comment on the route (`auth.routes.js`) explicitly notes this is expected behavior.
- Password hashing uses `SALT_ROUNDS = 10`, a bcrypt-library constant local to this file.

 `server/src/services/auditLog/auditLog.service.js`

PURPOSE

Records security-relevant user actions (login, AWS connect/disconnect, scan start/complete, recommendation ignore/resolve, EC2 stop) as an immutable audit trail, and lists a user's own audit history. Deliberately designed so a logging failure can never break the action being logged.

WHEN THIS FILE IS CALLED

`recordAuditLog` is called by five different controllers immediately after their primary action succeeds: login, AWS connect/disconnect, scan start/complete, recommendation ignore/resolve/stop. `fetchAuditLogs` is called when the Audit Logs page loads.

WHO CALLS THIS FILE

- `server/src/controllers/auditLog.controller.js` (`getAuditLogs` → `fetchAuditLogs`)
- `server/src/controllers/auth.controller.js` (`recordAuditLog` after successful login — `USER_LOGIN`)

- `server/src/controllers/awsConnection.controller.js` (`recordAuditLog` after connect/disconnect — `AWS_CONNECTED` / `AWS_DISCONNECTED`)
- `server/src/controllers/recommendation.controller.js` (`recordAuditLog` after ignore/resolve/stop — `RECOMMENDATION_IGNORED` / `RECOMMENDATION_RESOLVED` / `EC2_STOPPED`)
- `server/src/controllers/scan.controller.js` (`recordAuditLog` before and after a manual scan — `SCAN_STARTED` / `SCAN_COMPLETED`)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/auditLog.repository.js` — `createAuditLog` , `getAuditLogsByUserId`

EXPORTED FUNCTIONS

- `recordAuditLog(userId, action, description)`
- Purpose: Fire-and-forget write of one audit entry; a failure here must never break the action being audited.
- Input: `userId` , `action` (must be one of the `AUDIT_ACTION_VALUES` enum from `constants/auditActions.js` : `USER_LOGIN` , `AWS_CONNECTED` , `AWS_DISCONNECTED` , `SCAN_STARTED` , `SCAN_COMPLETED` , `RECOMMENDATION_IGNORED` , `RECOMMENDATION_RESOLVED` , `EC2_STOPPED`), `description` (free-text string).
- Output: `undefined` .
- Internal steps: 1) `try { await createAuditLog({ userId, action, description }) } . 2) catch (error) { console.error("Failed to record audit log (${action}):", error.message) }` — the error is swallowed, never rethrown.
- Database calls: `AuditLog.create({ userId, action, description })` .
- AWS calls: none.
- Error handling: fully swallows any error (validation failure, DB connectivity issue, invalid `action` not matching the schema enum, etc.) — logs to console only.
- Return values: none meaningful — always resolves to `undefined` , whether the write succeeded or failed.
- `fetchAuditLogs(userId)`
- Purpose: Lists a user's own audit history, newest first.
- Input: `userId` .
- Output: array of `AuditLog` documents.
- Internal steps: `return await getAuditLogsByUserId(userId)` .
- Database calls: `AuditLog.find({ userId }).sort({ createdAt: -1 })` .
- AWS calls: none.
- Error handling: none — an unhandled DB error would propagate up through `asyncHandler` .
- Return values: array (possibly empty).

DATA FLOW

Various controllers → `recordAuditLog` (fire-and-forget, awaited but errors swallowed) → `auditLog.repository.js` → `AuditLog` MongoDB collection. Separately, `auditLog.controller.js` → `fetchAuditLogs` → same collection, scoped and sorted for display on the Audit Logs page.

EXAMPLE EXECUTION

A user connects an AWS account: `awsConnection.controller.js` 's `connect` handler calls `connectAwsAccount` (succeeds), then `await recordAuditLog(userId, "AWS_CONNECTED", "AWS account 123456789012 (us-east-1) connected")` . If MongoDB were briefly unreachable at that instant, `createAuditLog` would throw, `recordAuditLog` 's catch block logs `"Failed to record audit log (AWS_CONNECTED): <error message>"` to the server console, and the HTTP response still returns the successful connection to the user — the audit-log failure is invisible to the end user.

IMPORTANT NOTES

- Comment in the source explicitly draws the parallel: this fire-and-forget pattern "mirrors how `notifyScanCompletion` is wrapped around scan completion elsewhere" — both files share the design principle that a secondary side-effect (email or audit logging) must never fail the primary user-facing action.
- `recordAuditLog` is still `await` -ed by every caller (not truly detached/backgrounded) — "fire-and-forget" here means errors are swallowed, not that the call is non-blocking; the controller's response is still delayed by however long the `AuditLog.create` write (or its failure) takes.
- The `action` value is validated against the `AuditLog` schema's `enum: AUDIT_ACTION_VALUES` at the Mongoose layer — passing an action string not in `constants/auditActions.js` 's `AUDIT_ACTIONS` would cause `createAuditLog` to throw a validation error, which `recordAuditLog` would then silently swallow (the entry simply wouldn't be recorded, with no visible failure to any caller).

PURPOSE

Powers the Analytics page's four trend charts (cost over time, savings over time, resource count growth, high-alert count over time) by reading every one of a user's completed scans in chronological order and mapping each scan to one data point per chart.

WHEN THIS FILE IS CALLED

Whenever the Analytics page loads any of its four charts.

WHO CALLS THIS FILE

- `server/src/controllers/analytics.controller.js` (`costTrend` → `getCostTrend` ; `savingsTrend` → `getSavingsTrend` ; `resourceGrowth` → `getResourceGrowth` ; `highAlertsTrend` → `getHighAlertsTrend`), reached from `server/src/routes/analytics.routes.js` → `GET /api/analytics/cost-trend` , `GET /api/analytics/savings-trend` , `GET /api/analytics/resource-growth` , `GET /api/analytics/high-alerts-trend`

WHICH FILES IT DEPENDS ON

- `server/src/services/database/analytics.repository.js` — `getCompletedScansForUser` , `getMonthlyCostByScan` , `getHighAlertsByScan`

EXPORTED FUNCTIONS

- `getCostTrend(userId)`
- Purpose: One `{ date, monthlyCost }` point per completed scan.
- Input: `userId`.
- Output: array of `{ date, monthlyCost }` (`[]` if no completed scans).
- Internal steps: 1) `scans = await getCompletedScansForUser(userId)` ; if empty, `return []` . 2) `scanIds = scans.map(s => s._id)` . 3) `costByScan = await getMonthlyCostByScan(scanIds)` — a `Recommendation.aggregate` `$group` by `scanId` summing `monthlyCost` . 4) `costMap = new Map(costByScan.map(entry => [entry._id.toString(), entry.monthlyCost]))` . 5) `return scans.map(scan => ({ date: scanDate(scan), monthlyCost: costMap.get(scan._id.toString()) || 0 })), where scanDate(scan) = scan.completedAt || scan.startedAt || scan.createdAt` .
- Database calls: `Scan.find({ userId, status: "COMPLETED" }).sort({ completedAt: 1 })` (via `getCompletedScansForUser`) ; `Recommendation.aggregate([ { $match: { scanId: { $in: scanIds } } }, { $group: { _id: "$scanId", monthlyCost: { $sum: "$monthlyCost" } } }])` (via `getMonthlyCostByScan`) .
- AWS calls: none.
- Error handling: none.
- Return values: `[]` , or array of `{ date, monthlyCost }` sorted ascending by `completedAt` (oldest first, matching the repository's sort).
- `getSavingsTrend(userId)`
- Purpose: One `{ date, estimatedSavings }` point per completed scan.
- Input: `userId`.
- Output: array of `{ date, estimatedSavings }` .
- Internal steps: `scans = await getCompletedScansForUser(userId)` ; `return scans.map(scan => ({ date: scanDate(scan), estimatedSavings: scan.estimatedSavings || 0 }))` — reads directly off each `Scan` document's own `estimatedSavings` field (no separate aggregation needed, unlike `getCostTrend`) .
- Database calls: `Scan.find({ userId, status: "COMPLETED" }).sort({ completedAt: 1 })` .
- AWS calls: none.
- Error handling: none (note: unlike `getCostTrend` / `getHighAlertsTrend` , there's no explicit `if (!scans.length) return []` short-circuit here — but `scans.map` on an empty array naturally returns `[]` anyway, so the behavior is identical).
- Return values: array (possibly empty).
- `getResourceGrowth(userId)`
- Purpose: One `{ date, totalResources }` point per completed scan.
- Input: `userId`.
- Output: array of `{ date, totalResources }` .
- Internal steps: `scans = await getCompletedScansForUser(userId)` ; `return scans.map(scan => ({ date: scanDate(scan), totalResources: scan.totalResources || 0 }))` — reads directly off each `Scan` document.

- Database calls: `Scan.find({ userId, status: "COMPLETED" }).sort({ completedAt: 1 })`.
- AWS calls: none.
- Error handling: none (same no-explicit-guard note as `getSavingsTrend`).
- Return values: array (possibly empty).
- `getHighAlertsTrend(userId)`
- Purpose: One `{ date, highAlerts }` point per completed scan — the 4th chart, added to match the other three's pattern rather than being one of an originally fixed three-chart set.
- Input: `userId`.
- Output: array of `{ date, highAlerts }`.
- Internal steps: 1) `scans = await getCompletedScansForUser(userId)` ; if empty, `return []` . 2) `scanIds = scans.map(s => s._id)` . 3) `alertsByScan = await getHighAlertsByScan(scanIds)` — `Recommendation.aggregate` `$match` ing `severity: "HIGH"` then `$group` ing by `scanId` , counting. 4) `alertsMap = new Map(alertsByScan.map(entry => [entry._id.toString(), entry.highAlerts]))` . 5) `return scans.map(scan => ({ date: scanDate(scan), highAlerts: alertsMap.get(scan._id.toString()) || 0 }))` .
- Database calls: `Scan.find({ userId, status: "COMPLETED" })` ; `Recommendation.aggregate` (`[{ $match: { scanId: { $in: scanIds }, severity: "HIGH" } }, { $group: { _id: "$scanId", highAlerts: { $sum: 1 } } }]`) .
- AWS calls: none.
- Error handling: none.
- Return values: `[]` , or array of `{ date, highAlerts }` .

DATA FLOW

`analytics.controller.js` → one of the four trend functions → `analytics.repository.js` (`getCompletedScansForUser` always first, then a per-chart aggregation for cost/high-alerts, or a direct field read for savings/resource-growth) → MongoDB `scans` / `recommendations` collections → chart-ready arrays sorted oldest-to-newest by `completedAt` .

EXAMPLE EXECUTION

A user with 3 completed scans (`completedAt` on Jan 1, Jan 8, Jan 15) calls `GET /api/analytics/cost-trend` : `getCostTrend` returns `[{ date: <Jan 1>, monthlyCost: 120 }, { date: <Jan 8>, monthlyCost: 95 }, { date: <Jan 15>, monthlyCost: 110 }]` , letting the frontend plot a 3-point line chart of monthly cost over time.

IMPORTANT NOTES

- `getCostTrend` and `getHighAlertsTrend` re-derive their numbers via fresh `Recommendation.aggregate` calls (respecting *all* recommendations regardless of `status` , since neither aggregation filters by `status`), while `getSavingsTrend` / `getResourceGrowth` simply read the already-stored `Scan.estimatedSavings` / `Scan.totalResources` fields written once at scan completion time (`scanner.service.js` 's `completeScan`) — two different data-freshness models for what look like parallel functions: the cost/high-alert numbers reflect the recommendations' *current* state at chart-load time, while savings/resource-count reflect a snapshot frozen at scan-completion time.
- `scanDate(scan) = scan.completedAt || scan.startedAt || scan.createdAt` is a shared fallback chain — since these functions only ever operate on `status: "COMPLETED"` scans (which always have `completedAt` set by `completeScan`), the `startedAt` / `createdAt` fallbacks are effectively dead code paths under current usage, existing only as defensive fallbacks.
- All four functions only consider `status: "COMPLETED"` scans — a scan currently `RUNNING` or one that ended `FAILED` never contributes a data point to any of these four charts.

 `server/src/services/notification/weeklyReport.scheduler.js`

PURPOSE

Registers and runs the weekly cost-report cron job: every Monday at 08:00 it emails every user who has opted into `weeklyReport` a summary of their latest scan's dashboard numbers.

WHEN THIS FILE IS CALLED

`startWeeklyReportScheduler()` runs once at process boot (called from `server.js`). Its registered cron job (`sendWeeklyReports`) then fires automatically every Monday at 08:00 server time.

WHO CALLS THIS FILE

- `server/src/server.js` (calls `startWeeklyReportScheduler()` once on startup, alongside `startScheduler()` for the scan scheduler)

WHICH FILES IT DEPENDS ON

- `server/src/services/database/notification.repository.js` — `getUsersWithWeeklyReportEnabled` (finds every subscriber, not scoped to one user)
- `server/src/services/database/dashboard.repository.js` — `getDashboardSummary` (reuses the same latest-scan aggregation the dashboard page itself uses)
- `server/src/services/email/email.service.js` — `sendWeeklyReportEmail` (dispatches the email)
- external: `node-cron` — schedules `sendWeeklyReports` on the cron expression `"0 8 * * 1"`

EXPORTED FUNCTIONS

- `sendWeeklyReports()`
- Purpose: Iterates every weekly-report subscriber and emails each one their current dashboard summary.
- Input: none (reads all subscribers from the DB).
- Output: `undefined`.
- Internal steps:
 1. `subscribers = await getUsersWithWeeklyReportEnabled()` — all `Notification` docs where `weeklyReport: true`.
 2. For each `settings` in `subscribers` (sequential `for...of`, not parallelized):
 - a. `summary = await getDashboardSummary(settings.userId)`
 - b. `await sendWeeklyReportEmail(settings.email, summary)`
 - c. Both wrapped in one try/catch per subscriber — failure is logged (`console.error("Weekly report failed for user ${settings.userId}:", error.message)`) and the loop continues to the next subscriber.
- Database calls: `Notification.find({ weeklyReport: true })` (via `getUsersWithWeeklyReportEnabled`); per subscriber, `Scan.findOne({ userId }).sort({ createdAt: -1 })` plus `Resource.aggregate(...)` and `Recommendation.aggregate(...)$facet` pipelines (via `getDashboardSummary`).
- AWS calls: none.
- Error handling: per-subscriber try/catch — one user's failure (e.g. bad email, transporter error, no scan yet returning the empty-summary default) never stops the remaining subscribers from being processed.
- Return values: none meaningful; resolves to `undefined` after processing every subscriber.
- `startWeeklyReportScheduler()`
- Purpose: Registers the recurring cron job and logs that it started.
- Input: none.
- Output: `undefined`.
- Internal steps: 1) `cron.schedule("0 8 * * 1", sendWeeklyReports)` . 2) `console.log("Weekly cost report cron started (Mondays at 08:00)")` .
- Database calls: none directly (the scheduled callback makes them later).
- AWS calls: none.
- Error handling: none — a `node-cron` scheduling failure would throw synchronously at boot.
- Return values: none.

DATA FLOW

`server.js` boot → `startWeeklyReportScheduler()` registers the job → every Monday 08:00, `node-cron` fires `sendWeeklyReports` → `Notification` collection (subscriber list) → `dashboard.repository.js` (per-user summary from their latest `Scan` / `Resource` / `Recommendation` documents) → `email.service.js` → SMTP/Ethereal.

EXAMPLE EXECUTION

It's Monday, 08:00. Three users have `weeklyReport: true`. For the first, `getDashboardSummary` returns `{ totalResources: 58, monthlyCost: 412.10, estimatedSavings: 89.30, highRecommendations: 2, mediumRecommendations: 6, lowRecommendations: 1, ... }`; `sendWeeklyReportEmail` sends that data through `buildWeeklyReportEmail`, producing subject `"CloudLens – Your Weekly Cost Report"`. The loop then moves to subscriber two even if subscriber one's send throws.

IMPORTANT NOTES

- `getDashboardSummary` 's return object includes `runningResources` / `stoppedResources` fields that `buildWeeklyReportEmail` never reads — only `totalResources` , `monthlyCost` , `estimatedSavings` , `highRecommendations` , `mediumRecommendations` , `lowRecommendations` are used by the template; the extra fields are silently ignored, not an error.
 - If a subscriber has never run a scan, `getDashboardSummary` returns the zeroed default object (`totalResources: 0` , etc.) rather than throwing — so that user still receives a (mostly zero) weekly email rather than being skipped.
 - The cron schedule is server-local time; there's no per-user timezone handling — "Monday 08:00" means 08:00 in whatever timezone the Node process runs in.
 - This scheduler is entirely independent of `scheduler.service.js` 's per-minute scan scheduler — it does not trigger a new scan, it only reports on whatever the latest existing scan already found.
-

INFRASTRUCTURE — UTILS, MIDDLEWARE, CONFIG

Cross-cutting helpers with no service-specific home: error handling, JWT, encryption, validation, rate limiting, config, the process entry point.

CloudLens Backend Infrastructure Dossier (Batch 5: utils, middleware, config, constants, server.js)

 `server/src/utils/ApiError.js`

PURPOSE

Defines a custom `ApiError` class that extends the native `Error`, carrying an explicit HTTP `statusCode` alongside the error `message`. It lets services and controllers throw an error that already knows what HTTP status it should produce, instead of every catch site having to work that out.

WHEN THIS FILE IS CALLED

Whenever a service or controller function needs to abort processing with a specific HTTP error (e.g. "409 Email already registered", "404 No AWS account is connected"). It is `throw`n, not called directly as a function.

WHO CALLS THIS FILE

`server/src/controllers/auth.controller.js`, `server/src/controllers/awsConnection.controller.js`,
`server/src/controllers/report.controller.js`, `server/src/controllers/scans.controller.js`,
`server/src/controllers/schedule.controller.js`, `server/src/services/auth/auth.service.js`,
`server/src/services/awsConnection/awsConnection.service.js`, `server/src/services/notification/notification.service.js`,
`server/src/services/recommendation/recommendation.service.js`, `server/src/services/scanner/scanner.service.js`,
`server/src/services/schedule/schedule.service.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `ApiError` (default export, a class)
- Purpose: Represent an application error that already carries the HTTP status it should map to.
- Input: `constructor(statusCode, message)` — `statusCode` (Number, e.g. 400/401/404/409), `message` (String).
- Output / Return value: A new `ApiError` instance; `super(message)` sets `this.message`, and `this.statusCode` is assigned directly.
- Internal steps: 1) Calls `super(message)` to initialize the base `Error` (sets `.message`, `.stack`). 2) Assigns `this.statusCode = statusCode`.
- Database calls: none
- AWS calls: none
- Error handling: N/A — this is the error type.
- Return values: N/A (constructor).

DATA FLOW

A controller/service constructs `new ApiError(409, "Email is already registered")` and throws it. Because controllers are wrapped in `asyncHandler`, the rejected promise is forwarded to Express's `next(err)`, which routes it to `errorHandler.middleware.js`. That middleware reads `err.statusCode` (falling back to 500 if absent) and `err.message` to build the JSON error response via `sendError`.

EXAMPLE EXECUTION

```
throw new ApiError(404, "No AWS account is connected.");
// -> caught by errorHandler.middleware.js
// -> statusCode = 404, message = "No AWS account is connected."
// -> HTTP response: { success: false, message: "No AWS account is connected." }
```

IMPORTANT NOTES

Very small (11 lines), no validation on `statusCode` / `message` types — callers are trusted to pass a numeric HTTP status. Does not set `err.name`, so downstream code distinguishes it from Mongoose errors purely via the presence of `err.statusCode`.

 `server/src/utils/apiResponse.js`

PURPOSE

Provides two small helper functions, `sendSuccess` and `sendError`, that standardize the JSON envelope shape (`{ success, data }` or `{ success, message }`) for every HTTP response the API sends. This keeps response formatting consistent across all controllers and middleware without each one hand-rolling `res.json(...)`.

WHEN THIS FILE IS CALLED

On every successful controller response and every error response (validation failures, 401s, 404s, 500s) throughout the app.

WHO CALLS THIS FILE

`server/src/controllers/analytics.controller.js`, `auditLog.controller.js`, `auth.controller.js`, `awsConnection.controller.js`, `dashboard.controller.js`, `health.controller.js`, `notification.controller.js`, `recommendation.controller.js`, `scan.controller.js`, `scans.controller.js`, `schedule.controller.js`, `server/src/middleware/auth.middleware.js`, `server/src/middleware/ratelimiter.middleware.js`, `server/src/middlewares/errorHandler.middleware.js`, `server/src/middlewares/notFound.middleware.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `sendSuccess(res, data, status = 200)`
- Purpose: Send a standardized success response.
- Input: `res` (Express response object), `data` (any JSON-serializable payload), `status` (Number, optional, defaults to 200).
- Output / Return value: Returns the result of `res.status(status).json({ success: true, data })` (the Express response object, enabling chaining).
- Internal steps: 1) Calls `res.status(status)`. 2) Calls `.json({ success: true, data })`.
- Database calls: none
- AWS calls: none
- Error handling: none (assumes `res` is valid).
- Return values: the Express `res` object after `.json()`.
- `sendError(res, status, message)`
- Purpose: Send a standardized error response.
- Input: `res` (Express response object), `status` (Number HTTP status), `message` (String).
- Output / Return value: Returns `res.status(status).json({ success: false, message })`.
- Internal steps: 1) Calls `res.status(status)`. 2) Calls `.json({ success: false, message })`.
- Database calls: none
- AWS calls: none
- Error handling: none.
- Return values: the Express `res` object after `.json()`.

DATA FLOW

Every controller ends its success path with `sendSuccess(res, result)`; every error path (validation, auth, not-found, centralized error handler) ends with `sendError(res, status, message)`. This is the single point where the API's JSON envelope shape is defined, so client code (`client/src/api/api.js` and services) can assume `{ success, data }` or `{ success, message }` uniformly.

EXAMPLE EXECUTION

```
sendSuccess(res, { id: "abc123", email: "user@example.com" });
// HTTP 200: { "success": true, "data": { "id": "abc123", "email": "user@example.com" } }

sendError(res, 401, "Unauthorized: No token provided");
// HTTP 401: { "success": false, "message": "Unauthorized: No token provided" }
```

IMPORTANT NOTES

No `status` validation; a caller could pass a non-numeric status and Express would throw. `data` is passed through as-is with no sanitization — callers are responsible for excluding sensitive fields (though `User` and `AwsConnection` mongoose schemas also strip secrets in their own `toJSON` transforms as defense in depth).

 `server/src/utils/asyncHandler.js`

PURPOSE

A higher-order function that wraps an async Express route handler so any rejected promise (thrown error) is automatically forwarded to `next(err)`, removing the need for a `try/catch` block in every controller.

WHEN THIS FILE IS CALLED

At controller-definition time — every controller function exported from `server/src/controllers/*.js` is wrapped in `asyncHandler(...)` before being attached to a route. The wrapper itself then runs on every incoming HTTP request to that route.

WHO CALLS THIS FILE

`server/src/controllers/analytics.controller.js`, `auditLog.controller.js`, `auth.controller.js`, `awsConnection.controller.js`, `dashboard.controller.js`, `health.controller.js`, `notification.controller.js`, `recommendation.controller.js`, `report.controller.js`, `scan.controller.js`, `scans.controller.js`, `schedule.controller.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `asyncHandler(handler)` (default export)
- Purpose: Convert an async route handler into one that reports rejected promises to Express's error pipeline instead of causing an unhandled rejection.
- Input: `handler` — a function `(req, res, next) => Promise`.
- Output / Return value: A new function `(req, res, next) => void` suitable for direct use as an Express route handler/middleware.
- Internal steps: 1) Returns an arrow function `(req, res, next) => { ... }`. 2) Inside, calls `Promise.resolve(handler(req, res, next))` (so even a non-async handler that returns a plain value is safely wrapped). 3) Chains `.catch(next)` so any rejection/thrown error is passed to Express's `next`, routing it to the centralized error middleware.
- Database calls: none
- AWS calls: none
- Error handling: Delegates all error handling to whatever `next(err)` triggers downstream (`errorHandler.middleware.js`).
- Return values: The wrapped handler function.

DATA FLOW

Controllers are defined as `exports.someAction = asyncHandler(async (req, res) => { ... })`. If the inner async function throws (including an unhandled `await` rejection, e.g. a database error or an `ApiError` throw), `asyncHandler` catches it via `.catch(next)` and forwards it to Express, which routes it to `errorHandler.middleware.js`.

EXAMPLE EXECUTION

```
const getScan = asyncHandler(async (req, res) => {
  const scan = await scanRepository.findById(req.params.id); // throws if DB fails
  sendSuccess(res, scan);
});
// If findById rejects, asyncHandler's .catch(next) forwards the error to
// errorHandler.middleware.js instead of crashing the process.
```

IMPORTANT NOTES

Only 8 lines; extremely simple but load-bearing — without it, every controller would need its own `try/catch` or unhandled promise rejections could crash the Node process (depending on Node version/handling).

 `server/src/utils/encryption.js`

PURPOSE

Provides `encrypt` / `decrypt` helpers used to protect the AWS secret access key at rest. It implements symmetric authenticated encryption using Node's built-in `crypto` module so that `AwsConnection.encryptedSecretKey` is never stored or transmitted as plaintext.

WHEN THIS FILE IS CALLED

`encrypt` is called when a user connects a new AWS account (before saving `encryptedSecretKey` to MongoDB). `decrypt` is called every time the backend needs to actually use the stored AWS secret key to build AWS SDK clients (i.e., on every scan / AWS API call for that user).

WHO CALLS THIS FILE

`server/src/factories/awsClient.factory.js` (calls `decrypt`), `server/src/services/awsConnection/awsConnection.service.js` (calls `encrypt`)

WHICH FILES IT DEPENDS ON

External only: Node's built-in `crypto` module (for `randomBytes`, `createCipheriv`, `createDecipheriv`).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `encrypt(text)`
 - Purpose: Encrypt a plaintext string (the AWS secret access key) for storage.
 - Input: `text` (String) — plaintext, e.g. the raw AWS secret access key.
 - Output / Return value: A String of the form `"<ivHex>:<authTagHex>:<encryptedHex>"`.
 - Internal steps:
 1. Generate a random 12-byte initialization vector: `crypto.randomBytes(IV_LENGTH)` where `IV_LENGTH = 12`.
 2. Fetch the 256-bit key via `getKey()`, which reads `process.env.ENCRYPTION_KEY` (a hex string) and converts it to a `Buffer` with `Buffer.from(key, "hex")`. Throws `Error("ENCRYPTION_KEY is not set")` if the env var is missing.
 3. Create a cipher: `crypto.createCipheriv("aes-256-gcm", key, iv)`.
 4. Encrypt: `Buffer.concat([cipher.update(text, "utf8"), cipher.final()])`.
 5. Retrieve the GCM authentication tag: `cipher.getAuthTag()`.
 6. Join `iv`, `authTag`, and `encrypted` — each hex-encoded — with `:` as the separator, e.g. `[iv.toString("hex"), authTag.toString("hex"), encrypted.toString("hex")].join(":")`.
 - Database calls: none
 - AWS calls: none
 - Error handling: Throws a plain `Error` if `ENCRYPTION_KEY` is unset. No try/catch inside — propagates to the caller.
 - Return values: The colon-delimited `iv:authTag:ciphertext` hex string, stored directly in `AwsConnection.encryptedSecretKey`.
- `decrypt(encryptedText)`
 - Purpose: Recover the plaintext AWS secret access key from the stored encrypted string.
 - Input: `encryptedText` (String) — the `"ivHex:authTagHex:dataHex"` format produced by `encrypt`.
 - Output / Return value: The original plaintext String.
 - Internal steps:

1. Split the input on `:` into `[ivHex, authTagHex, dataHex]`.
 2. Create a decipher: `crypto.createDecipheriv("aes-256-gcm", getKey(), Buffer.from(ivHex, "hex"))`.
 3. Set the expected auth tag: `decipherer.setAuthTag(Buffer.from(authTagHex, "hex"))`.
 4. Decrypt: `Buffer.concat([decipherer.update(Buffer.from(dataHex, "hex")), decipherer.final()])`. GCM mode causes `decipherer.final()` to throw if the auth tag doesn't match (tamper/integrity check).
 5. Return `decrypted.toString("utf8")`.
- Database calls: none
 - AWS calls: none
 - Error handling: No explicit try/catch — if the ciphertext or auth tag was tampered with, or the key is wrong, `decipherer.final()` throws (propagates to caller, e.g. `getAwsClientsForUser` in `awsClient.factory.js`, which does not catch it either).
 - Return values: The decrypted plaintext string (the AWS secret access key).

DATA FLOW

`awsConnection.service.js`'s `connectAwsAccount` calls `encrypt(secretAccessKey)` and stores the result in `AwsConnection.encryptedSecretKey` via `AwsConnection.create(...)`. Later, `awsClient.factory.js`'s `getAwsClientsForUser(userId)` loads the `AwsConnection` document, calls `decrypt(connection.encryptedSecretKey)` to recover the plaintext secret key, and uses it (paired with the stored plaintext `accessKeyId`) to build AWS SDK client credentials for that request.

EXAMPLE EXECUTION

```
// ENCRYPTION_KEY env var (hex-encoded 32-byte key), e.g.:
// ENCRYPTION_KEY=0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcd

encrypt("my-secret-key");
// -> "a1b2c3d4e5f6a1b2c3d4e5f6:9f8e7d6c5b4a39281706f5e4:7c3f9a..." (shape: <24-hex-char IV>:<32-hex-char authTag>:<hex ciphertext>)

decrypt("a1b2c3d4e5f6a1b2c3d4e5f6:9f8e7d6c5b4a39281706f5e4:7c3f9a...");
// -> "my-secret-key"
```

IMPORTANT NOTES

Algorithm is **AES-256-GCM** (authenticated encryption), not plain AES-CBC — this means tampering with any part of the stored ciphertext (IV, auth tag, or data) causes `decrypt` to throw rather than silently returning corrupted plaintext. The IV is random per-encryption (12 bytes, the standard/recommended size for GCM) and is stored inline as the first colon-delimited hex segment of `encryptedSecretKey`, not in a separate DB field — so `encrypt` + `decrypt` are the only code that ever needs to parse that combined format. The encryption key itself (`ENCRYPTION_KEY`) is expected to be a hex string decoded to exactly 32 bytes (256 bits) for AES-256; a wrong-length key would cause `createCipheriv` / `createDecipheriv` to throw. There is no key-rotation or versioning scheme — changing `ENCRYPTION_KEY` would make all previously stored `encryptedSecretKey` values undecryptable.

 `server/src/utils/iamPolicy.util.js`

PURPOSE

A shared set of helpers for working with AWS IAM policy documents: normalizing values to arrays, safely parsing the URI-encoded JSON policy documents the IAM API returns, distinguishing AWS-managed vs. customer-managed policy ARNs, and detecting overly-broad ("wildcard") `Allow` statements.

WHEN THIS FILE IS CALLED

During an IAM scan: when mapping raw IAM API responses into internal shapes (trust policy parsing) and when analyzing attached/inline policies for security findings (wildcard action/resource detection), and when the IAM service decides whether a policy is worth fetching its document for.

WHO CALLS THIS FILE

`server/src/mappers/iam.mapper.js`, `server/src/services/analyzer/iam.analyzer.js`, `server/src/services/aws/iam.service.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports) — pure logic over plain JS objects/strings.

EXPORTED FUNCTIONS / EXPORTED VALUES

- **toArray(value)**
 - Purpose: Normalize a value that may be `undefined`, a single item, or an array (IAM policy `Action` / `Resource` / `Statement` fields can legally be either a single string/object or an array) into a consistent array.
 - Input: `value` — anything, but conceptually a policy field like `Action` or `Statement`.
 - Output / Return value: `[]` if `value` is `null` / `undefined`; `value` unchanged if already an `Array`; otherwise `[value]`.
 - Internal steps: 1) Null/undefined check returns `[]`. 2) `Array.isArray(value)` ternary wraps non-arrays in a single-element array.
 - Database calls: none. AWS calls: none.
 - Error handling: none needed (no throwing paths).
 - Return values: an `Array`.
- **parsePolicyDocument(document)**
 - Purpose: Safely parse an IAM policy document string, which AWS returns URI-encoded.
 - Input: `document` (String or falsy) — a URI-encoded JSON policy document as returned by IAM APIs (e.g. `GetPolicyVersion`, `GetRole`'s `AssumeRolePolicyDocument`).
 - Output / Return value: The parsed JS object, or `null` if `document` is falsy or parsing fails.
 - Internal steps: 1) Return `null` immediately if `!document`. 2) `try`: `JSON.parse(decodeURIComponent(document))`. 3) `catch`: return `null` (swallows the error, does not log or rethrow).
 - Database calls: none. AWS calls: none.
 - Error handling: Catches and silently swallows any decode/parse error, returning `null` — callers must handle a `null` result.
 - Return values: parsed policy document object, or `null`.
- **isCustomerManagedPolicyArn(policyArn = "")**
 - Purpose: Determine whether a policy ARN belongs to a customer-managed policy (worth inspecting) vs. an AWS-managed policy (pre-vetted by AWS, `arn:aws:iam::aws:policy/...`).
 - Input: `policyArn` (String, default `""`).
 - Output / Return value: `true` if the ARN does NOT start with `"arn:aws:iam::aws:policy/"`; `false` otherwise.
 - Internal steps: 1) `!policyArn.startsWith(AWS_MANAGED_POLICY_ARN_PREFIX)`.
 - Database calls: none. AWS calls: none.
 - Error handling: none.
 - Return values: Boolean.
- **hasWildcardAction(policyDocument)**
 - Purpose: Detect an `Allow` statement whose `Action` includes a bare `"*"` (any action), regardless of what `Resource` it's scoped to.
 - Input: `policyDocument` — a parsed policy document object (e.g. from `parsePolicyDocument`).
 - Output / Return value: Boolean — `true` if any `Allow` statement's `Action` array (via `toArray`) includes `"*"`.
 - Internal steps: 1) Delegates to internal `allowStatementsMatch(policyDocument, predicate)`, which returns `false` immediately if `!policyDocument`, otherwise checks `toArray(policyDocument.Statement).some(stmt => stmt.Effect === "Allow" && predicate(stmt))`. 2) The predicate here is `toArray(statement.Action).includes("*")`.
 - Database calls: none. AWS calls: none.
 - Error handling: none (returns `false` for falsy/malformed input rather than throwing).
 - Return values: Boolean.
- **hasFullWildcardPermissions(policyDocument)**
 - Purpose: Detect the most severe case — an `Allow` statement granting `"*"` action on `"*"` resource (equivalent to AWS's `AdministratorAccess`, but expressed via a custom/inline policy).
 - Input: `policyDocument` — parsed policy document object.
 - Output / Return value: Boolean — `true` if any `Allow` statement has both `Action` including `"*"` and `Resource` including `"*"`.
 - Internal steps: Same `allowStatementsMatch` helper, predicate: `toArray(statement.Action).includes("*") && toArray(statement.Resource).includes("*")`.
 - Database calls: none. AWS calls: none.
 - Error handling: none.
 - Return values: Boolean.

DATA FLOW

`iam.service.js` calls `isCustomerManagedPolicyArn` to decide which attached policies are worth an extra `GetPolicy` + `GetPolicyVersion` AWS API round trip. `iam.mapper.js` uses `parsePolicyDocument` to turn a role's trust policy (URI-encoded JSON string from the IAM API) into a usable object for the internal resource shape. `iam.analyzer.js` uses `hasWildcardAction` / `hasFullWildcardPermissions` (and `toArray` / `parsePolicyDocument`) to flag over-permissive IAM policies as security findings during a scan, which eventually become `Recommendation` documents.

EXAMPLE EXECUTION

```
const doc = parsePolicyDocument("%7B%22Statement%22%3A%5B%7B%22Effect%22%3A%22Allow%22%2C%22Action%22%3A%22*%22%2C%22Resource%22%3A%22*%22%7D%22%5D%22%7D");
// doc = { Statement: [{ Effect: "Allow", Action: "*", Resource: "*" }] }

hasFullWildcardPermissions(doc); // -> true
hasWildcardAction(doc); // -> true (full wildcard implies action wildcard too)
isCustomerManagedPolicyArn("arn:aws:iam::123456789012:policy/MyCustomPolicy"); // -> true
isCustomerManagedPolicyArn("arn:aws:iam::aws:policy/ReadOnlyAccess"); // -> false
```

IMPORTANT NOTES

`parsePolicyDocument` swallows parse errors silently (returns `null`) with no logging — a malformed policy document from AWS would be treated identically to a missing one by downstream code. `allowStatementsMatch` is an internal (non-exported) helper shared by both wildcard-detection functions.

 `server/src/utils/instanceType.util.js`

PURPOSE

Provides `getAdjacentInstanceType`, which estimates the "next size up" or "next size down" EC2 instance type (e.g. `t3.medium` -> `t3.large`) for cost-comparison purposes in upsize/downsize recommendations. It keeps the instance family fixed and only steps the size suffix along a fixed ladder.

WHEN THIS FILE IS CALLED

During EC2 scanning/recommendation generation, when the scanner needs to estimate the cost of resizing an instance up or down (e.g. an UPSIZE recommendation for a CPU-saturated instance, or a DOWNSIZE recommendation for an underutilized one).

WHO CALLS THIS FILE

`server/src/services/scanner/ec2.scanner.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `SIZE_LADDER` — internal constant (not exported), an ordered array of EC2 size suffixes from `"nano"` up to `"metal"`: `["nano", "micro", "small", "medium", "large", "xlarge", "2xlarge", "4xlarge", "8xlarge", "9xlarge", "10xlarge", "12xlarge", "16xlarge", "18xlarge", "24xlarge", "32xlarge", "metal"]`.
- `getAdjacentInstanceType(instanceType, direction)`
 - Purpose: Compute the instance type one step up or down the size ladder within the same family.
 - Input: `instanceType` (String, e.g. `"t3.medium"`), `direction` (String, `"up"` or anything else treated as "down").
 - Output / Return value: A String like `"t3.large"`, or `null` if the type can't be parsed or is already at the top/bottom of the ladder.
 - Internal steps:
 - Split `instanceType` on `"."` into `[family, size]`; if either is falsy, return `null`.
 - Find `size`'s index in `SIZE_LADDER` via `indexOf`; return `null` if not found (`-1`).
 - Compute `targetIndex = direction === "up" ? index + 1 : index - 1`.
 - If `targetIndex` is out of bounds (`< 0` or `>= SIZE_LADDER.length`), return `null`.
 - Return `` ${family}.${SIZE_LADDER[targetIndex]} ``.
 - Database calls: none. AWS calls: none.

- Error handling: No exceptions thrown; unparseable/edge-of-ladder input returns `null`, and callers are expected to treat `null` as "no size-change estimate available" rather than crash.
- Return values: String instance type, or `null`.

DATA FLOW

`ec2.scanner.js` calls `getAdjacentInstanceType(currentType, "up")` or `"down"` when building an upsize/downsize recommendation, then feeds the resulting instance type string into the AWS Pricing API lookup (via

`pricing.service.js` / `pricing.filter.builder.js` / `pricing.parser.js`) to estimate the cost delta of the resize.

EXAMPLE EXECUTION

```
getAdjacentInstanceType("t3.medium", "up"); // -> "t3.large"
getAdjacentInstanceType("t3.medium", "down"); // -> "t3.small"
getAdjacentInstanceType("t3.nano", "down"); // -> null (already at bottom of ladder)
getAdjacentInstanceType("badtype", "up"); // -> null (no "." separator to split family/size)
```

IMPORTANT NOTES

The ladder is generic across all instance families — it does not validate that a given family actually offers every size (e.g. not all families have a `"metal"` size), so a computed "adjacent" type could theoretically be a size AWS doesn't actually sell for that family; the pricing lookup downstream would simply fail to find a price for it.

 `server/src/utils/jwt.js`

PURPOSE

Wraps the `jsonwebtoken` library to provide two functions — `generateToken` and `verifyToken` — for issuing and validating the JWTs CloudLens uses for authentication. The token payload embeds both the user's ID and their current `tokenVersion`, which is what makes "logout from all devices" possible without a server-side token blacklist.

WHEN THIS FILE IS CALLED

`generateToken` is called on successful login/registration (`auth.service.js`). `verifyToken` is called on every incoming request to a protected route, inside `auth.middleware.js`'s `protect` function.

WHO CALLS THIS FILE

`server/src/middleware/auth.middleware.js`, `server/src/services/auth/auth.service.js`

WHICH FILES IT DEPENDS ON

External only: `jsonwebtoken`.

EXPORTED FUNCTIONS / EXPORTED VALUES

- `generateToken(userId, tokenVersion = 0)`
- Purpose: Issue a signed JWT for a user, embedding both their ID and the token-version they were signed under.
- Input: `userId` — the user's Mongo `_id` (ObjectId or string), `tokenVersion` (Number, defaults to `0`).
- Output / Return value: A signed JWT string.
- Internal steps:
 1. Calls `jwt.sign({ userId, tokenVersion }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN || "7d" })`.
 2. Payload is exactly `{ userId, tokenVersion }` (plus JWT's own standard claims like `iat` / `exp` added by `jsonwebtoken`).
 3. Expiry defaults to `"7d"` if `JWT_EXPIRES_IN` is not set in the environment.
- Database calls: none. AWS calls: none.
- Error handling: None explicit — `jwt.sign` throws synchronously if `JWT_SECRET` is undefined/invalid; not caught here, propagates to the caller.
- Return values: JWT string.
- `verifyToken(token)`
- Purpose: Verify a JWT's signature and expiry, and decode its payload.
- Input: `token` (String) — the raw JWT (already stripped of the `"Bearer "` prefix by the caller).

- Output / Return value: The decoded payload object, e.g. `{ userId, tokenVersion, iat, exp }`.
- Internal steps: 1) Calls `jwt.verify(token, process.env.JWT_SECRET)`, which validates the signature against `JWT_SECRET` and checks the `exp` claim.
- Database calls: none. AWS calls: none.
- Error handling: None explicit here — `jwt.verify` throws (`JsonWebTokenError`, `TokenExpiredError`, etc.) on invalid/expired/malformed tokens; the caller (`auth.middleware.js`'s `protect`) wraps its call in a `try/catch` and converts any thrown error into a 401 response.
- Return values: Decoded payload object (throws instead of returning on failure).

DATA FLOW

On login, `auth.service.js`'s `loginUser` calls `generateToken(user._id, user.tokenVersion)` and returns the token to the client, which stores it (in `localStorage`, per `app.js`'s comments) and sends it as `Authorization: Bearer <token>` on subsequent requests. `auth.middleware.js`'s `protect` extracts the token, calls `verifyToken(token)` to get `{ userId, tokenVersion }`, then loads the user from MongoDB (`User.findById(decoded.userId).select("tokenVersion")`) and compares `decoded.tokenVersion` against the *current* `user.tokenVersion` in the database — confirmed by reading `auth.middleware.js` lines 25-29: `if (!user || (decoded.tokenVersion || 0) !== user.tokenVersion) { return sendError(res, 401, ...) }`. This means a structurally valid, unexpired, correctly-signed JWT is still rejected if the user's `tokenVersion` has since been incremented (see `auth.service.js`'s `logoutAllDevices`, which does `User.findByIdAndUpdate(userId, { $inc: { tokenVersion: 1 } })`), which is exactly the "logout from all devices" mechanism — it invalidates every previously issued token in one atomic DB update, without maintaining any blacklist.

EXAMPLE EXECUTION

Decoded JWT payload shape (as returned by `verifyToken`):

```
{
  "userId": "64f1a2b3c4d5e6f7a8b9c0d1",
  "tokenVersion": 2,
  "iat": 1754000000,
  "exp": 1754604800
}
```

If the corresponding `User` document in MongoDB currently has `tokenVersion: 3` (because the user triggered "logout from all devices" after this token was issued), `protect` rejects this token with `401 Unauthorized: Session has expired, please log in again` even though the signature and expiry are both valid.

IMPORTANT NOTES

`JWT_SECRET` is read directly from `process.env` with no presence check in this file — an unset secret would make `jwt.sign` / `jwt.verify` behave unpredictably (signing with `undefined` throws). Default expiry is 7 days if `JWT_EXPIRES_IN` is unset. The `tokenVersion` check that enables "logout from all devices" is NOT implemented in this file — `jwt.js` only embeds/decodes `tokenVersion`; the actual comparison against the user's current DB value lives in `server/src/middleware/auth.middleware.js`.

`server/src/utils/region.util.js`

PURPOSE

Translates AWS availability-zone codes and region codes into human-readable AWS Pricing API "location" strings (e.g. `"us-east-1"` -> `"US East (N. Virginia)"`), which the Pricing API's `location` filter requires rather than accepting raw region codes.

WHEN THIS FILE IS CALLED

When building AWS Pricing API filter requests for EC2, RDS, or S3 resources — the region/AZ of a scanned resource must be converted to its Pricing-API-friendly location name before querying prices.

WHO CALLS THIS FILE

`server/src/builders/pricing.filter.builder.js`

WHICH FILES IT DEPENDS ON

`server/src/constants/regions.js` — imports the `REGIONS` map (region-code -> human-readable location name) that both functions look values up in.

EXPORTED FUNCTIONS / EXPORTED VALUES

- `getLocationFromAvailabilityZone(availabilityZone)`
- Purpose: Derive a Pricing API location string from an EC2 availability zone (which is not itself a valid region code).
- Input: `availabilityZone` (String, e.g. `"us-east-1a"`).
- Output / Return value: The mapped location String from `REGIONS`, or `undefined` if the derived region isn't in the map.
- Internal steps: 1) `region = availabilityZone.slice(0, -1)` — strips the trailing AZ letter (e.g. `"us-east-1a"` -> `"us-east-1"`). 2) Returns `REGIONS[region]`.
- Database calls: none. AWS calls: none.
- Error handling: None — if `availabilityZone` is `undefined` /not a string, `.slice` throws a `TypeError` (not caught here).
- Return values: String location name, or `undefined`.
- `getLocationFromRegion(region)`
- Purpose: Directly map a region code (as returned by S3's `GetBucketLocation`, which exposes a region rather than an AZ) to its Pricing API location name.
- Input: `region` (String, e.g. `"ap-south-1"`).
- Output / Return value: `REGIONS[region]`, or `undefined` if not present.
- Internal steps: Single lookup, no transformation.
- Database calls: none. AWS calls: none.
- Error handling: none.
- Return values: String location name, or `undefined`.

DATA FLOW

`pricing.filter.builder.js` calls one of these two functions (AZ-based for EC2/RDS, region-based for S3) to convert a scanned resource's location into the `location` filter value used in the AWS Pricing API `GetProducts` call, which is then parsed by `pricing.parser.js` to get an hourly/monthly price.

EXAMPLE EXECUTION

```
getLocationFromAvailabilityZone("ap-south-1a"); // -> "Asia Pacific (Mumbai)"
getLocationFromRegion("us-east-1");           // -> "US East (N. Virginia)"
getLocationFromRegion("eu-west-1");           // -> undefined (not present in REGIONS)
```

IMPORTANT NOTES

`REGIONS` (see `constants/regions.js`) only contains 4 region entries (`ap-south-1`, `us-east-1`, `ap-southeast-1`, `ap-northeast-1`) — any other AWS region resolves to `undefined`, which would make downstream Pricing API filtering silently fail to match a location. No fallback/default is applied.

 `server/src/utils/schedule.util.js`

PURPOSE

Small set of helpers for the recurring-scan scheduling feature: the list of valid scan frequencies, a validator for a submitted frequency, and a function to compute a schedule's next run time from its frequency.

WHEN THIS FILE IS CALLED

When a user creates/updates a scheduled scan (frequency validation) and whenever the scheduler cron tick processes a due schedule and needs to compute its next run time.

WHO CALLS THIS FILE

`server/src/services/schedule/schedule.service.js`, `server/src/services/scheduler/scheduler.service.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `FREQUENCY_OPTIONS`

- Purpose: The canonical list of valid schedule frequencies.
- Value: `["hourly", "daily", "weekly"]`.
- `isValidFrequency(frequency)`
- Purpose: Validate a user-submitted frequency string.
- Input: `frequency` (String, expected to be one of `FREQUENCY_OPTIONS`).
- Output / Return value: Boolean — `FREQUENCY_OPTIONS.includes(frequency)`.
- Internal steps: single `Array.includes` check.
- Database calls: none. AWS calls: none.
- Error handling: none.
- Return values: Boolean.
- `computeNextRun(frequency, from = new Date())`
- Purpose: Compute the next run `Date` for a given frequency, offset from a base time.
- Input: `frequency` (String, one of `"hourly"` / `"daily"` / `"weekly"`), `from` (Date, defaults to now).
- Output / Return value: A new `Date` equal to `from` plus the interval for that frequency.
- Internal steps: 1) Looks up `FREQUENCY_MS[frequency]` — an internal (non-exported) map: `{ hourly: 3600000, daily: 86400000, weekly: 604800000 }` (i.e. `60*60*1000`, `24*60*60*1000`, `7*24*60*60*1000`). 2) Returns `new Date(from.getTime() + FREQUENCY_MS[frequency])`.
- Database calls: none. AWS calls: none.
- Error handling: None — an invalid `frequency` not present in `FREQUENCY_MS` yields `undefined`, and `from.getTime() + undefined` produces `NaN`, resulting in an Invalid Date (no throw, no validation guard).
- Return values: `Date` object (or an Invalid Date for an unrecognized frequency).

DATA FLOW

`schedule.service.js` calls `isValidFrequency` when a user creates/updates a `Schedule` document, rejecting the request (via `ApiError`) if the frequency isn't one of the three valid options. `scheduler.service.js`'s cron tick (`runDueSchedules`, in `scheduler.service.js`) calls `computeNextRun(schedule.frequency, lastRun)` after a scheduled scan completes (success or failure) to set the `Schedule.nextRun` field for the following run.

EXAMPLE EXECUTION

```
isValidFrequency("daily"); // -> true
isValidFrequency("monthly"); // -> false (not in FREQUENCY_OPTIONS)

computeNextRun("daily", new Date("2026-08-01T08:00:00Z"));
// -> Date("2026-08-02T08:00:00.000Z")
```

IMPORTANT NOTES

`computeNextRun` performs no validation of `frequency` itself — it trusts callers to have already validated via `isValidFrequency` (or that the value came from the DB, where it was validated on write). Passing an invalid frequency silently produces an Invalid Date rather than throwing.

 `server/src/utils/validators.js`

PURPOSE

Plain-function input validators for user registration/login fields: email format, password strength, and full name length — used by the auth controller before delegating to `auth.service.js`.

WHEN THIS FILE IS CALLED

On every signup request (email, password, full name all validated) and login request (email format validated) — before any database work happens, in `auth.controller.js`.

WHO CALLS THIS FILE

`server/src/controllers/auth.controller.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports) — pure regex/string logic.

EXPORTED FUNCTIONS / EXPORTED VALUES

- `isValidEmail(email)`
 - Purpose: Validate an email address's basic shape and length.
 - Input: `email` (expected String).
 - Output / Return value: Boolean.
 - Internal steps: 1) Checks `typeof email === "string"`. 2) Checks `email.trim().length <= 254` (RFC 5321 practical max email length). 3) Tests against `EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/` (no whitespace/ `@` before the `@`, then an `@`, then no whitespace/ `@` and at least one `.` before the final segment) applied to the trimmed value.
 - Database calls: none. AWS calls: none.
 - Error handling: none — non-string input simply fails the `typeof` check and returns `false` rather than throwing.
 - Return values: Boolean.
- `isValidPassword(password)`
 - Purpose: Enforce a minimum password strength/length policy.
 - Input: `password` (expected String).
 - Output / Return value: Boolean.
 - Internal steps: 1) Checks `typeof password === "string"`. 2) Tests against `PASSWORD_REGEX = /^(?=.*[A-Za-z])(?=.*\d){8,128}$/` — requires at least one letter (lookahead `(?=.*[A-Za-z])`), at least one digit (lookahead `(?=.*\d)`), and a total length of 8-128 characters.
 - Database calls: none. AWS calls: none.
 - Error handling: none.
 - Return values: Boolean.
- `isValidFullName(fullName)`
 - Purpose: Enforce a reasonable length range for a user's display name.
 - Input: `fullName` (expected String).
 - Output / Return value: Boolean.
 - Internal steps: 1) Checks `typeof fullName === "string"`. 2) Checks `fullName.trim().length >= 2 && fullName.trim().length <= 100`.
 - Database calls: none. AWS calls: none.
 - Error handling: none.
 - Return values: Boolean.

DATA FLOW

`auth.controller.js`'s signup handler calls all three validators against the request body before calling `auth.service.js`'s `registerUser`; login calls `isValidEmail` (and presumably a presence check on password, not a strength check, since an existing password shouldn't be rejected for not meeting the *current* strength policy). Any failed validation causes the controller to respond with a 400-level error (via `ApiError` / `sendError`) before any database or bcrypt work happens.

EXAMPLE EXECUTION

```
isValidEmail("user@example.com"); // -> true
isValidEmail("not-an-email");     // -> false (no "@")

isValidPassword("my-secret-key1"); // -> true (has letters + a digit, length 15)
isValidPassword("alllettersnodigit"); // -> false (no digit)
isValidPassword("short1");         // -> false (only 6 chars, needs >= 8)

isValidFullName("Al");             // -> true (exactly 2 chars)
isValidFullName("A");              // -> false (below minimum of 2)
```

IMPORTANT NOTES

None of these validators sanitize/transform their input (e.g. no email normalization here — lowercasing happens separately in the `User` mongoose schema's `lowercase: true` option). `isValidPassword` has no upper-bound complexity requirement beyond "1 letter + 1 digit" — no special character or mixed-case requirement.

PURPOSE

Express middleware (`protect`) that authenticates incoming requests by validating the `Authorization: Bearer <token>` header's JWT, then re-checking the token's embedded `tokenVersion` against the user's current `tokenVersion` in MongoDB — the mechanism that makes "logout from all devices" actually invalidate previously issued tokens.

WHEN THIS FILE IS CALLED

On every request to a route mounted with `protect` in `app.js` (all routes except `/api/auth/*`), and additionally on specific routes within `auth.routes.js` (e.g. `logout-all`, which needs to know the current user).

WHO CALLS THIS FILE

`server/src/app.js` , `server/src/routes/auth.routes.js`

WHICH FILES IT DEPENDS ON

`server/src/models/User.js` (looks up the user's current `tokenVersion` by ID), `server/src/utils/apiResponse.js` (`sendError` for all rejection responses), `server/src/utils/jwt.js` (`verifyToken` to validate/decode the JWT)

EXPORTED FUNCTIONS / EXPORTED VALUES

- `protect` (default export, an async Express middleware function (`req, res, next`) => ...)
- Purpose: Gate access to protected routes to requests bearing a valid, still-current JWT, and attach `req.user` for downstream handlers.
- Input: Express `req` (reads `req.headers.authorization`), `res` , `next` .
- Output / Return value: Either calls `next()` (request proceeds, `req.user` is set) or sends a `401` JSON error response and does not call `next()` .
- Internal steps:
 1. Reads `authHeader = req.headers.authorization` . If missing or doesn't start with `"Bearer "` , responds `401 "Unauthorized: No token provided"` .
 2. Extracts `token = authHeader.split(" ")[1]` . If empty/falsy, responds the same 401 message again (redundant secondary guard).
 3. `try` : calls `decoded = verifyToken(token)` (from `utils/jwt.js`) — validates signature and expiry, decodes `{ userId, tokenVersion, iat, exp }` .
 4. Looks up the user: `const user = await User.findById(decoded.userId).select("tokenVersion")` — a targeted query that only fetches the `tokenVersion` field (plus `_id`), not the whole document.
 5. Checks `if (!user || (decoded.tokenVersion || 0) !== user.tokenVersion)` — rejects with `401 "Unauthorized: Session has expired, please log in again"` if the user no longer exists OR the token's embedded `tokenVersion` doesn't match the user's current DB value. `decoded.tokenVersion || 0` treats a missing/falsy claim as version `0` .
 6. On success, sets `req.user = { userId: decoded.userId }` (note: only `userId` , not the full decoded payload) and calls `next()` .
 7. `catch` : any error from `verifyToken` (bad signature, expired, malformed) or the DB lookup is caught and responds `401 "Unauthorized: Invalid or expired token"` .
- Database calls: `User.findById(decoded.userId).select("tokenVersion")` — one read per authenticated request.
- AWS calls: none.
- Error handling: Two explicit early-return 401s for missing/malformed header, one 401 for stale tokenVersion/deleted user, and a catch-all 401 for any thrown error (invalid signature, expired token, DB error).
- Return values: N/A (middleware side effects: either `next()` or an HTTP response).

DATA FLOW

This is the authentication gate for nearly the entire API surface (mounted in `app.js` on every route group except `/api/auth`). It converts a raw bearer token into a trusted `req.userId` -equivalent (`req.user.userId`) that every downstream controller/service uses to scope database queries (e.g. `AwsConnection.findOne({ userId })` , `Schedule` queries, etc.) to the authenticated user. The `tokenVersion` re-check against the live DB value is what makes `auth.service.js` 's `logoutAllDevices` (`$inc: { tokenVersion: 1 }`) immediately invalidate all previously issued JWTs for that user on their very next request, without any server-side token blacklist/store.

EXAMPLE EXECUTION

Request: `GET /api/dashboard` with header `Authorization: Bearer eyJhbGciOi...`

- Decoded payload: `{ userId: "64f1a2b3c4d5e6f7a8b9c0d1", tokenVersion: 1, iat: ..., exp: ... }`
- DB user document: `{ _id: "64f1a2b3c4d5e6f7a8b9c0d1", tokenVersion: 1 }` -> versions match -> `req.user = { userId: "64f1a2b3c4d5e6f7a8b9c0d1" }`, `next()` called, request proceeds to `dashboard.controller.js`.
- If the DB user's `tokenVersion` were `2` instead (bumped by a "logout from all devices" call), the same token would now fail: `401 { success: false, message: "Unauthorized: Session has expired, please log in again" }`.

IMPORTANT NOTES

This file lives in `server/src/middleware/` (singular), while the two other middleware files documented in this batch (`errorHandler.middleware.js`, `notFound.middleware.js`) live in `server/src/middlewares/` (plural) — a genuine, slightly inconsistent folder-naming split in this codebase (confirmed by directly listing both directories), not a typo introduced in this documentation. `req.user` only carries `userId`, not the full JWT payload or user document — any handler needing e.g. the user's email must re-fetch it. The DB lookup on every request (`User.findById`) trades a small per-request latency/DB-load cost for immediate revocation of logged-out sessions, versus a stateless JWT-only check that couldn't support "logout from all devices" at all.

 `server/src/middleware/rateLimiter.middleware.js`

PURPOSE

Defines `loginRateLimiter`, an `express-rate-limit`-based middleware that throttles repeated requests (login attempts) from the same IP address to slow down brute-force/credential-stuffing attacks.

WHEN THIS FILE IS CALLED

Applied to the login route (and potentially other auth routes) in `auth.routes.js`; runs on every request to that route before the controller executes.

WHO CALLS THIS FILE

`server/src/routes/auth.routes.js`

WHICH FILES IT DEPENDS ON

`server/src/utils/apiResponse.js` (`sendError`, used in the rate-limit `handler` callback). External: `express-rate-limit`.

EXPORTED FUNCTIONS / EXPORTED VALUES

- `loginRateLimiter`
- Purpose: Rate-limit middleware instance restricting each IP to a fixed number of attempts within a time window.
- Input: Configured via `rateLimit({...})` options, not a function signature per se — used as Express middleware (`req, res, next`).
- Output / Return value: When under the limit, calls `next()` normally (via `express-rate-limit` internals). When the limit is exceeded, invokes the custom `handler` instead.
- Internal steps / configuration:
 - `windowMs: 15 * 60 * 1000` — 15-minute sliding window.
 - `max: 10` — up to 10 requests per IP per window.
 - `standardHeaders: true` — sends `RateLimit-*` headers (draft standard).
 - `legacyHeaders: false` — suppresses the older `X-RateLimit-*` headers.
 - `handler: (req, res) => sendError(res, 429, "Too many login attempts. Please try again in a few minutes.")` — overrides the default rate-limit-exceeded response with the app's standard error envelope.
- Database calls: none. AWS calls: none.
- Error handling: N/A — this IS the "too many requests" handling for the wrapped route; on exceeding the limit it short-circuits with a 429 rather than reaching the controller.
- Return values: N/A (middleware).

DATA FLOW

Sits in front of the login controller action in `auth.routes.js`'s route chain. Each request increments an in-memory (default `express-rate-limit` store) counter keyed by IP; once 10 requests from the same IP occur within 15 minutes, subsequent requests get an immediate `429` via `sendError` instead of reaching `auth.controller.js`'s login handler (and therefore never reach `auth.service.js`'s `loginUser` /bcrypt comparison).

EXAMPLE EXECUTION

11th login attempt from the same IP within a 15-minute window:

```
HTTP 429
{ "success": false, "message": "Too many login attempts. Please try again in a few minutes." }
```

IMPORTANT NOTES

Uses `express-rate-limit`'s default in-memory store, which is per-process — in a multi-instance/horizontally-scaled deployment, each instance would track its own counter independently (rate limit isn't shared across instances) unless a shared store (e.g. Redis) were configured, which it is not here. Only exported as `{ loginRateLimiter }`, a named export (not default), unlike `auth.middleware.js`'s `protect` which is a default export — another small stylistic inconsistency between the two files in this folder.

 `server/src/middlewares/errorHandler.middleware.js`

PURPOSE

The centralized Express error-handling middleware (4-argument signature, so Express recognizes it as an error handler). Normalizes several known error shapes (Mongoose `ValidationError`, `CastError`, MongoDB duplicate-key error `11000`) into consistent 4xx JSON responses, and falls back to a generic `500 Internal Server Error` for anything else — while logging the raw error server-side either way.

WHEN THIS FILE IS CALLED

Whenever any route handler or middleware calls `next(err)` (including automatically via `asyncHandler`'s `.catch(next)`), or throws synchronously within Express's request cycle. Registered last in `app.js`'s middleware chain, after `notFound`.

WHO CALLS THIS FILE

`server/src/app.js`

WHICH FILES IT DEPENDS ON

`server/src/utils/apiResponse.js` (`sendError`, used for every response path).

EXPORTED FUNCTIONS / EXPORTED VALUES

- **`errorHandler` (default export, a 4-arg Express error middleware `(err, req, res, next)`)**
- Purpose: Convert any error reaching Express's error pipeline into a consistent JSON error response with an appropriate status code.
- Input: `err` (Error or ApiError or Mongoose error), `req`, `res`, `next` (unused — required by Express's 4-arg error-middleware signature convention, flagged with `// eslint-disable-next-line no-unused-vars`).
- Output / Return value: Sends a JSON error response via `sendError`; no return value used by callers.
- Internal steps:
 1. `console.error(err)` — logs the full error server-side unconditionally, before any classification.
 2. If `err.name === "ValidationError"` (Mongoose schema validation failure): builds `message` by mapping `Object.values(err.errors)` to each `fieldError.message` and joining with `", "`; responds `400`.
 3. Else if `err.name === "CastError"` (e.g. malformed ObjectId in a route param): responds `400` with ``Invalid ${err.path}``.
 4. Else if `err.code === 11000` (MongoDB duplicate-key error): extracts the conflicting field name via `Object.keys(err.keyPattern || {})[0] || "value"`; responds `409` with ``A record with this ${field} already exists``.
 5. Otherwise: `statusCode = err.statusCode || 500` (this is where `ApiError`'s `statusCode` is honored). `message = statusCode === 500 ? "Internal Server Error" : err.message` — i.e., for any 500, the actual error message/details are NOT sent to the client (only logged via step 1); for any other explicit status (from an `ApiError`), the real `err.message` is sent.
 6. Calls `sendError(res, statusCode, message)`.
- Database calls: none. AWS calls: none.
- Error handling: This IS the error handler — it does not itself throw; any error object shape reaches step 5's fallback if it doesn't match the three specific `name` / `code` checks.
- Return values: N/A (side effect: HTTP response).

DATA FLOW

Sits at the very end of `app.js`'s middleware chain (`app.use(notFound); app.use(errorHandler);`). Receives errors from: `ApiError` throws inside `asyncHandler` -wrapped controllers/services (mapped via `err.statusCode`), raw Mongoose errors bubbling up uncaught from model operations (`ValidationError` , `CastError` , duplicate key `11000`), and any other uncaught exception (mapped to a generic 500 with no leaked internal detail).

EXAMPLE EXECUTION

```
// A Mongoose validation failure, e.g. creating a User without a required field:
// err.name === "ValidationError", err.errors = { fullName: { message: "Path `fullName` is required." } }
// -> HTTP 400: { success: false, message: "Path `fullName` is required." }

// A duplicate email on signup (race condition past the pre-check in auth.service.js):
// err.code === 11000, err.keyPattern = { email: 1 }
// -> HTTP 409: { success: false, message: "A record with this email already exists" }

// An ApiError(404, "No AWS account is connected."):
// -> HTTP 404: { success: false, message: "No AWS account is connected." }

// An unclassified error (e.g. a TypeError from a bug):
// -> HTTP 500: { success: false, message: "Internal Server Error" } (real message only in server logs)
```

IMPORTANT NOTES

This file lives in `server/src/middlewares/` (plural), while `auth.middleware.js` and `rateLimiter.middleware.js` live in `server/src/middleware/` (singular) — confirmed by directly listing both directories; this is a real, pre-existing naming inconsistency in the codebase, not introduced by this documentation. All errors are logged via `console.error` (no structured/external logging), including expected 400/409 validation errors, not just unexpected 500s. The `next` parameter is intentionally unused but required for Express to recognize this as error-handling middleware (arity-based dispatch), hence the eslint-disable comment.

 `server/src/middlewares/notFound.middleware.js`

PURPOSE

A catch-all middleware that responds with a `404` for any request that didn't match any registered route.

WHEN THIS FILE IS CALLED

On any request whose path/method doesn't match one of the route groups mounted in `app.js` ; registered just before `errorHandler` and after all route mounts.

WHO CALLS THIS FILE

`server/src/app.js`

WHICH FILES IT DEPENDS ON

`server/src/utils/apiResponse.js` (`sendError`).

EXPORTED FUNCTIONS / EXPORTED VALUES

- `notFound` (default export, an Express middleware (`req, res`) => `void`)
- Purpose: Return a consistent 404 JSON response for unmatched routes.
- Input: `req` (reads `req.method` , `req.originalUrl`), `res` .
- Output / Return value: Calls `sendError(res, 404, \ Route not found: ${req.method} ${req.originalUrl}\ )` .
- Internal steps: Single call, no branching.
- Database calls: none. AWS calls: none.
- Error handling: N/A — this is itself the terminal handler for unmatched routes; it does not call `next()` .
- Return values: N/A (side effect: HTTP response).

DATA FLOW

Positioned in `app.js` right after all `/api/*` route mounts and right before `errorHandler` : `app.use(notFound); app.use(errorHandler);` . Because Express matches middleware/routes in registration order, `notFound` only executes if no earlier route handler matched the incoming

request.

EXAMPLE EXECUTION

```
GET /api/nonexistent-route
-> HTTP 404
{ "success": false, "message": "Route not found: GET /api/nonexistent-route" }
```

IMPORTANT NOTES

Lives in `server/src/middlewares/` (plural) alongside `errorHandler.middleware.js`, while `auth.middleware.js` and `ratelimiter.middleware.js` live in the singular `server/src/middleware/` folder — the same singular/plural folder-naming inconsistency noted for the other three middleware files in this batch. This is the smallest file in the entire batch (8 lines).

 `server/src/config/aws.js`

PURPOSE

A factory module providing one `create*Client` function per AWS SDK v3 client used by CloudLens (EC2, S3, CloudWatch, RDS, IAM, STS, Pricing). Centralizes the logic for building each client's config object from optional per-user decrypted credentials and a region, so no other file constructs an AWS SDK client directly.

WHEN THIS FILE IS CALLED

Whenever the backend needs a fresh AWS SDK client scoped to a specific user's connected account — during a scan (all resource-type clients) and during AWS credential validation (STS) and recommendation generation (EC2).

WHO CALLS THIS FILE

`server/src/factories/awsClient.factory.js`, `server/src/services/aws/sts.service.js`,
`server/src/services/recommendation/recommendation.service.js`

WHICH FILES IT DEPENDS ON

External only: `@aws-sdk/client-cloudwatch`, `@aws-sdk/client-ec2`, `@aws-sdk/client-iam`, `@aws-sdk/client-pricing`, `@aws-sdk/client-rds`, `@aws-sdk/client-s3`, `@aws-sdk/client-sts`.

EXPORTED FUNCTIONS / EXPORTED VALUES

- `buildClientConfig(credentials, region)` — internal (not exported) helper.
- Purpose: Build the shared AWS SDK client config object.
- Input: `credentials` (optional `{ accessKeyId, secretAccessKey }` or `undefined`), `region` (optional String).
- Output / Return value: `{ region }` (falling back to `process.env.AWS_REGION` if `region` is falsy), plus a `credentials` key only if both `credentials?.accessKeyId` and `credentials?.secretAccessKey` are truthy.
- Internal steps: 1) `config = { region: region || process.env.AWS_REGION }`. 2) If both credential fields are present, sets `config.credentials = { accessKeyId, secretAccessKey }`. 3) Returns `config`.
- Database/AWS calls: none.
- `createEC2Client(credentials, region)` — Purpose: Build an `EC2Client`. Returns `new EC2Client(buildClientConfig(credentials, region))`.
- `createS3Client(credentials, region)` — Purpose: Build an `S3Client`. Same pattern.
- `createCloudWatchClient(credentials, region)` — Purpose: Build a `CloudWatchClient`. Same pattern.
- `createRDSClient(credentials, region)` — Purpose: Build an `RDSClient`. Same pattern.
- `createIAMClient(credentials, region)` — Purpose: Build an `IAMClient`. Same pattern (IAM is a global AWS service, but the client still takes a config with `region`).
- `createSTSClient(credentials, region)` — Purpose: Build an `STSClient`. Same pattern; used to validate credentials (`GetCallerIdentity`).
- `createPricingClient(credentials)` — Purpose: Build a `PricingClient`. Always hardcodes region to `"us-east-1"` regardless of the caller's `region` argument, since the AWS Pricing API is only served from that endpoint: `new PricingClient(buildClientConfig(credentials, "us-east-1"))`.

For all seven functions: Input is `(credentials, region)` (a `{ accessKeyId, secretAccessKey }` object or `undefined`, and a region string) — `createPricingClient` only takes `credentials`. Output is a ready-to-use AWS SDK v3 client instance. No internal branching beyond `buildClientConfig`. Database calls: none. AWS calls: none directly (client construction only — no API calls are made here). Error handling: none — the AWS SDK constructors themselves would throw on truly malformed config, uncaught. Return value: the client instance.

DATA FLOW

`awsClient.factory.js` 's `getAwsClientsForUser(userId)` loads the user's `AwsConnection`, decrypts the secret key (`utils/encryption.js`), and calls six of these seven functions (all except `createSTSCClient`) to build the full set of clients used during a scan. `sts.service.js` calls `createSTSCClient` directly to validate a candidate set of credentials (via `GetCallerIdentity`) before a connection is even saved. `recommendation.service.js` also imports from here directly for EC2 client construction in a recommendation-application flow (e.g. stopping an idle instance).

EXAMPLE EXECUTION

```
createEC2Client({ accessKeyId: "AKIA...", secretAccessKey: "wJal..." }, "ap-south-1");
// -> new EC2Client({ region: "ap-south-1", credentials: { accessKeyId: "AKIA...", secretAccessKey: "wJal..." } })

createEC2Client(undefined, undefined);
// -> new EC2Client({ region: process.env.AWS_REGION }) - SDK falls back to its own default credential provider chain
```

IMPORTANT NOTES

`createPricingClient` always pins to `us-east-1` regardless of the `region` argument passed by callers — this is intentional AWS API behavior (Pricing API is single-region), documented in the source comments. No client caching/reuse — a new client instance is created on every call, per the design comment in `awsClient.factory.js` ("never a shared global client").

 `server/src/config/db.js`

PURPOSE

Establishes the MongoDB connection via Mongoose, with a DNS-resolver workaround applied before connecting so `mongodb+srv://` SRV-record lookups succeed even on networks whose default DNS resolver doesn't support them well.

WHEN THIS FILE IS CALLED

Once, at process startup — invoked by `server.js`.

WHO CALLS THIS FILE

`server/src/server.js`

WHICH FILES IT DEPENDS ON

External only: `dns` (Node built-in), `mongoose`.

EXPORTED FUNCTIONS / EXPORTED VALUES

- **`connectDB` (default export, an async function, no parameters)**
- Purpose: Connect Mongoose to the MongoDB deployment specified by `MONGO_URI`, exiting the process if the connection fails.
- Input: none (reads `process.env.MONGO_URI` internally).
- Output / Return value: Resolves with no explicit value on success (implicitly `undefined`); does not return the connection object.
- Internal steps:
 1. (Module load time, not inside the function) `dns.setServers(["8.8.8.8", "1.1.1.1"])` — repoints Node's DNS resolution to Google's and Cloudflare's public resolvers, specifically to make `mongodb+srv://` SRV lookups reliable.
 2. `try : await mongoose.connect(process.env.MONGO_URI)`.
 3. On success: `console.log("MongoDB Connected")`.
 4. `catch (error) : console.error(error.message)` then `process.exit(1)` — terminates the entire Node process immediately on a connection failure.
- Database calls: `mongoose.connect(...)` — establishes the connection pool used by every Mongoose model in the app thereafter.
- AWS calls: none.

- Error handling: Catches any connection error, logs `error.message` only (not the full stack), and calls `process.exit(1)` — a hard, unrecoverable exit rather than a retry.
- Return values: none meaningful (a resolved Promise, or the process exits before returning on failure).

DATA FLOW

Called once by `server.js` at the very start of the startup sequence (before `app.listen`). Its success/failure determines whether the rest of the app can function at all, since every model (`User` , `AwsConnection` , `Scan` , `Resource` , `Recommendation` , `Insight` , `Notification` , `Schedule` , `AuditLog`) depends on this connection.

EXAMPLE EXECUTION

```
// .env: MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/cloudlens
connectDB();
// console: "MongoDB Connected"

// If MONGO_URI is unreachable/invalid:
// console.error: "<mongoose connection error message>"
// process.exit(1) -> entire Node process terminates
```

IMPORTANT NOTES

`connectDB()` is called in `server.js` without `await` or a `.then()` / `.catch()` — it's fired and not awaited before `app.listen(PORT, ...)` runs, meaning the HTTP server starts listening (and could start accepting requests) even before the MongoDB connection has actually resolved; requests arriving in that brief window would hit Mongoose operations queued against a not-yet-open connection (Mongoose buffers commands by default until connected, so this is typically benign but not guaranteed instant). The `process.exit(1)` on failure means there is no automatic retry/backoff — an operator must restart the process. The DNS override to public resolvers (`8.8.8.8` , `1.1.1.1`) is applied globally to the Node process's DNS resolution, not scoped to just the Mongo connection, which could affect any other DNS-dependent code in the process (e.g. outbound calls to AWS or SMTP) since `dns.setServers` changes Node's default resolver process-wide.

 `server/src/config/mailler.js`

PURPOSE

Provides a cached, lazily-initialized `nodemailer` transporter: uses real SMTP settings from environment variables if `SMTP_HOST` is configured, otherwise falls back to an auto-provisioned Ethereum test inbox for local/dev use.

WHEN THIS FILE IS CALLED

Whenever the app needs to send an email (scan notifications, weekly cost reports) — the transporter is created on first use and reused thereafter.

WHO CALLS THIS FILE

`server/src/services/email/email.service.js`

WHICH FILES IT DEPENDS ON

External only: `nodemailer` .

EXPORTED FUNCTIONS / EXPORTED VALUES

- `getTransporter()`
- Purpose: Return a cached (or newly built, on first call) `nodemailer` transporter, shared across all callers.
- Input: none.
- Output / Return value: A Promise resolving to a `nodemailer` transporter object.
- Internal steps:
 1. If module-level `transporterPromise` is not yet set (`null`), assigns it `buildTransporter()` (an unresolved Promise, cached immediately so concurrent callers share the same in-flight build rather than provisioning multiple Ethereum accounts).
 2. Returns `transporterPromise` .
- Database calls: none. AWS calls: none.
- Error handling: none directly — errors from `buildTransporter()` 's internals (e.g. `nodemailer.createTestAccount()` failing) propagate to the caller via the rejected Promise.

- Return values: Promise<Transporter>.
- `buildTransporter()` — internal (not exported) async helper.
- Purpose: Decide between real SMTP and the Ethereum dev fallback, and construct the transporter.
- Input: none (reads env vars).
- Output / Return value: Promise<Transporter>.
- Internal steps:
 1. If `process.env.SMTP_HOST` is set: returns `nodemailer.createTransport({ host: SMTP_HOST, port: Number(SMTP_PORT) || 587, secure: SMTP_SECURE === "true", auth: SMTP_USER ? { user: SMTP_USER, pass: SMTP_PASS } : undefined })`.
 2. Otherwise (dev fallback): calls `await nodemailer.createTestAccount()` to auto-provision an Ethereum test inbox, logs a `console.warn` that emails will go to Ethereum (dev only) rather than real recipients, and returns `nodemailer.createTransport({ host: "smtp.ethereal.email", port: 587, secure: false, auth: { user: testAccount.user, pass: testAccount.pass } })`.
- Database calls: none. AWS calls: none (network call to Ethereum's account-provisioning service in the fallback path only).
- Error handling: none explicit — any failure in `createTestAccount()` or `createTransport()` propagates up through `getTransporter()`'s Promise.
- Return values: Promise<Transporter>.

DATA FLOW

`email.service.js` calls `getTransporter()` whenever it needs to send an email (e.g. `sendWeeklyReportEmail`, scan-completion notification emails), then calls `.sendMail(...)` on the resolved transporter. Because `transporterPromise` is cached at module scope, the SMTP connection/Ethereum account is only set up once per process lifetime, not once per email.

EXAMPLE EXECUTION

```
// With SMTP_HOST set:
// SMTP_HOST=smtp.example.com SMTP_PORT=587 SMTP_SECURE=false SMTP_USER=user SMTP_PASS=pass
getTransporter(); // -> real nodemailer transporter targeting smtp.example.com:587

// Without SMTP_HOST (dev):
getTransporter();
// console.warn: "SMTP_HOST not set - emails will go to an Ethereum test inbox (dev only), not real recipients."
// -> transporter targeting smtp.ethereal.email:587 with an auto-generated test account
```

IMPORTANT NOTES

`transporterPromise` is a module-level singleton — if the very first call to `getTransporter()` fails (e.g. Ethereum account creation fails due to no network), the rejected Promise is cached and would need the module to be re-required (or the cache manually cleared) to retry, since `transporterPromise` is never reset to `null` on failure. In the dev/Ethereum fallback, emails are not actually delivered to real inboxes; they're only viewable via `nodemailer.getTestMessageUrl(info)`, per the code comment (that preview-URL usage lives in `email.service.js`, not this file).

 `server/src/constants/auditActions.js`

PURPOSE

The single source of truth for audit-log action type strings (e.g. `"USER_LOGIN"`, `"SCAN_COMPLETED"`), used both as the Mongoose schema enum for `AuditLog.action` and as the set of values controllers pass when recording an audit entry — so a new action type only needs to be added in one place.

WHEN THIS FILE IS CALLED

Imported at module-load time by the `AuditLog` model (to build its schema enum) and by every controller that records an audit-log entry (login, AWS connect/disconnect, scan start/complete, recommendation ignore/resolve).

WHO CALLS THIS FILE

`server/src/controllers/auth.controller.js`, `server/src/controllers/awsConnection.controller.js`,
`server/src/controllers/recommendation.controller.js`, `server/src/controllers/scan.controller.js`,
`server/src/models/AuditLog.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- **AUDIT_ACTIONS**
- Purpose: A key-to-value map (both key and value are identical strings) enumerating every valid audit action type.
- Value: `{ USER_LOGIN: "USER_LOGIN", AWS_CONNECTED: "AWS_CONNECTED", AWS_DISCONNECTED: "AWS_DISCONNECTED", SCAN_STARTED: "SCAN_STARTED", SCAN_COMPLETED: "SCAN_COMPLETED", RECOMMENDATION_IGNORED: "RECOMMENDATION_IGNORED", RECOMMENDATION_RESOLVED: "RECOMMENDATION_RESOLVED", EC2_STOPPED: "EC2_STOPPED" }`.
- Used by controllers as e.g. `AUDIT_ACTIONS.SCAN_STARTED` when calling the audit log service.
- **AUDIT_ACTION_VALUES**
- Purpose: A flat array of just the values, derived via `Object.values(AUDIT_ACTIONS)` — convenient for use directly as a Mongoose schema `enum` array.
- Value: `["USER_LOGIN", "AWS_CONNECTED", "AWS_DISCONNECTED", "SCAN_STARTED", "SCAN_COMPLETED", "RECOMMENDATION_IGNORED", "RECOMMENDATION_RESOLVED", "EC2_STOPPED"]`.

DATA FLOW

`models/AuditLog.js` imports `AUDIT_ACTION_VALUES` and uses it as the `enum` constraint on the `action` field of the `AuditLog` schema, so Mongoose rejects any document with an action string not in this list. Controllers (`auth`, `awsConnection`, `recommendation`, `scan`) import `AUDIT_ACTIONS` and pass e.g. `AUDIT_ACTIONS.AWS_CONNECTED` to `auditLog.service.js` when recording that a user connected an AWS account.

EXAMPLE EXECUTION

```
const { AUDIT_ACTIONS } = require("../constants/auditActions");
auditLogService.record({ userId, action: AUDIT_ACTIONS.SCAN_COMPLETED, ... });
// action stored: "SCAN_COMPLETED"
```

IMPORTANT NOTES

Every key and its value are the same string (e.g. `USER_LOGIN: "USER_LOGIN"`) — a pattern that makes the constant self-documenting/typo-resistant when used as `AUDIT_ACTIONS.SCAN_STARTED` rather than a raw string literal.

 `server/src/constants/regions.js`

PURPOSE

A small static map from AWS region codes to the human-readable "location" strings the AWS Pricing API expects in its `location` filter (e.g. `GetProducts`).

WHEN THIS FILE IS CALLED

Imported at module load by `region.util.js`, which performs the actual lookups during pricing-filter construction for scans.

WHO CALLS THIS FILE

`server/src/utils/region.util.js`

WHICH FILES IT DEPENDS ON

None (no internal or external imports).

EXPORTED FUNCTIONS / EXPORTED VALUES

- **REGIONS**
- Purpose: Region-code -> Pricing-API-location-name lookup table.
- Value:

```
``js { "ap-south-1": "Asia Pacific (Mumbai)", "us-east-1": "US East (N. Virginia)", "ap-southeast-1": "Asia Pacific (Singapore)", "ap-northeast-1": "Asia Pacific (Tokyo)", }
```

DATA FLOW

`region.util.js` 's `getLocationFromAvailabilityZone` and `getLocationFromRegion` both index directly into this object to translate a scanned resource's AWS region/AZ into the location string used when querying the AWS Pricing API for that resource's price.

EXAMPLE EXECUTION

```
REGIONS["us-east-1"]; // -> "US East (N. Virginia)"
REGIONS["eu-west-1"]; // -> undefined (not a supported region in this app)
```

IMPORTANT NOTES

Only 4 regions are supported (`ap-south-1` , `us-east-1` , `ap-southeast-1` , `ap-northeast-1`) — this is a hard-coded allowlist, not a complete AWS region list; a resource in any other AWS region would resolve to `undefined` when looked up, silently breaking Pricing API filtering for that resource (no error is thrown by this file itself — `region.util.js` simply returns `undefined`).

 `server/src/server.js`

PURPOSE

The process entry point for the backend: loads environment variables, wires up the Express app, connects to MongoDB, starts both cron-based background schedulers (per-user scheduled scans and the weekly cost-report email), and starts the HTTP listener.

WHEN THIS FILE IS CALLED

Executed once when the Node process starts (e.g. `node server.js` / `npm start` / `npm run dev`).

WHO CALLS THIS FILE

None — `importedBy` is empty in the dependency graph; this is the top-level entry point, never imported by other modules.

WHICH FILES IT DEPENDS ON

`server/src/app.js` (the configured Express app instance), `server/src/config/db.js` (`connectDB`), `server/src/services/notification/weeklyReport.scheduler.js` (`startWeeklyReportScheduler`), `server/src/services/scheduler/scheduler.service.js` (`startScheduler`). External: `dotenv` .

EXPORTED FUNCTIONS / EXPORTED VALUES

None — `server.js` exports nothing (`module.exports` is never set); it is a script, not a library module.

DATA FLOW

This file is the composition root: it does not implement any business logic itself, only sequences the startup of the other subsystems (config loading, DB, schedulers, HTTP server) documented elsewhere in this dossier and in the app.js batch.

EXAMPLE EXECUTION

Running `node server.js` with a valid `.env` produces, in order, console output similar to:

```
MongoDB Connected
Scheduled scan cron started (checking every minute)
Weekly cost report cron started (Mondays at 08:00)
Server running on 5000
```

IMPORTANT NOTES — STARTUP SEQUENCE, ERROR BEHAVIOR, AND ENV VARS

Exact startup sequence (in source order, `server.js` lines 1-16):

1. `require("dotenv").config()` — loads `.env` into `process.env` before anything else runs, so every subsequent `require` (including `app.js` and its route/controller tree) sees populated env vars at their own module-load time.
2. `const app = require("./app")` — synchronously builds the full Express app (CORS config, JSON body parser, all routes, `protect` middleware, `notFound` , `errorHandler`) but does not start listening yet.
3. `const connectDB = require("./config/db")` and `const { startScheduler } = require("./services/scheduler/scheduler.service")` and `const { startWeeklyReportScheduler } = require("./services/notification/weeklyReport.scheduler")` — import statements (no side effects yet beyond `db.js` 's module-level `dns.setServers(...)` call, which runs at `require` time).

4. `connectDB()` — called but **not awaited**. This kicks off the async `mongoose.connect(process.env.MONGO_URI)` in the background; execution of `server.js` continues immediately without waiting for the connection to establish.
5. `startScheduler()` — registers the per-minute `node-cron` job (`"* * * * *` ") that checks for and runs due scheduled scans (`scheduler.service.js` 's `runDueSchedules`). Logs `"Scheduled scan cron started (checking every minute)"`.
6. `startWeeklyReportScheduler()` — registers the weekly `node-cron` job (`"0 8 * * 1"` , every Monday at 08:00) that emails cost-summary reports to subscribed users (`weeklyReport.scheduler.js` 's `sendWeeklyReports`). Logs `"Weekly cost report cron started (Mondays at 08:00)"`.
7. `const PORT = process.env.PORT || 5000` — resolves the listen port.
8. `app.listen(PORT, () => console.log(\ Server running on ${PORT}\ ))` — starts the HTTP server.

What happens on an uncaught error during startup:

- If `MONGO_URI` is invalid/unreachable, `connectDB()` 's internal `try/catch` catches the error, logs `error.message` , and calls `process.exit(1)` — but because step 4 is not awaited, this exit can happen *after* `app.listen` has already started accepting connections (steps 5-8 run synchronously and immediately, regardless of whether the Mongo connection promise has resolved or rejected yet). In practice this means there is a window where the server is listening on `PORT` before MongoDB is confirmed connected, and if the connection ultimately fails, the entire process terminates via `process.exit(1)` , closing the HTTP server that had just started.
- `startScheduler()` and `startWeeklyReportScheduler()` themselves only synchronously register cron jobs (`cron.schedule(...)`) and do not throw during registration; any error inside their cron callbacks (`runDueSchedules` , `sendWeeklyReports`) is caught internally by those functions' own try/catch blocks (per-schedule and per-subscriber respectively) and logged via `console.error` , not propagated — a failure in one scheduled scan or one weekly report email does not crash the process or stop the cron job from running on its next tick.
- There is no top-level `process.on("uncaughtException"/"unhandledRejection")` handler anywhere in `server.js` — an uncaught synchronous error during the sequence above (e.g. `require("./app")` throwing due to a misconfigured route) would crash the process with Node's default uncaught-exception behavior (print stack trace, exit non-zero).

Port / env vars read (directly in `server.js` , plus indirectly via the modules it starts):

- `PORT` (default `5000`) — HTTP listen port.
- `MONGO_URI` — read inside `config/db.js` 's `connectDB` .
- Indirectly triggered at startup (via `app.js` , loaded in step 2): `CORS_ORIGIN` (comma-separated allowed origins, default `["http://localhost:5173"]`).
- All other env vars (`JWT_SECRET` , `JWT_EXPIRES_IN` , `ENCRYPTION_KEY` , `SMTP_*` , `AWS_REGION`) are read lazily by their respective modules at call time, not eagerly during the `server.js` startup sequence itself.

IMPORTANT NOTES

`connectDB()` is fire-and-forget (not awaited) in `server.js` , meaning `app.listen` and both scheduler registrations proceed regardless of MongoDB connection state — Mongoose's default command buffering generally masks this in normal operation (queries issued before the connection is open are queued rather than immediately failing), but a hard DB connection failure still terminates the whole process via `connectDB` 's internal `process.exit(1)` , abruptly closing the just-started HTTP listener. There is no graceful shutdown handling (no `SIGTERM` / `SIGINT` listener to close the HTTP server or Mongoose connection cleanly). The two cron schedulers (`startScheduler` , `startWeeklyReportScheduler`) are started unconditionally on every process start — there is no env-var flag to disable them (e.g. for a read-only replica or a one-off script context).

FRONTEND PAGES

One component per route — owns page-level state, data fetching, and layout composition.

`client/src/pages/Analytics.jsx`

PURPOSE

Displays four trend charts (monthly cost, estimated savings, resource growth, high-severity alerts) computed across all of the user's completed scans over time, giving a longitudinal view of cost/security trends rather than a single-scan snapshot.

ROUTE

`/analytics`, wrapped in `ProtectedRoute` (nested under the `<Route element={<ProtectedRoute />>` block in `App.jsx`).

WHEN THIS FILE IS CALLED

When an authenticated user navigates to `/analytics` (e.g. via the Navbar link).

WHO CALLS THIS FILE

- `client/src/App.jsx` (route registration at `/analytics`)

WHICH FILES IT DEPENDS ON

- `components/Navbar.jsx` — top navigation bar + global "Run Scan" trigger
- `components/PageTransition.jsx` — page enter/exit animation wrapper
- `components/ErrorBanner.jsx` — displays fetch/scan error banner
- `components/TrendChart.jsx` — the chart component, rendered 4 times with different data/config
- `hooks/useAsyncData.js` — generic fetch/loading/error hook
- `hooks/useScanTrigger.js` — drives the global "Run Scan" button and its step-progress UI
- `services/analytics.service.js` — `getCostTrend`, `getSavingsTrend`, `getResourceGrowth`, `getHighAlertsTrend`
- `utils/chartColors.js` — `CHART_COLORS` palette constants
- `utils/format.js` — `formatCurrency`

STATE MANAGEMENT

- `useAsyncData(fetchAnalyticsBundle, [], { initialValue: EMPTY_ANALYTICS, errorMessage: "Failed to load analytics data." })` → `data` (object `{costTrend, savingsTrend, resourceGrowth, highAlertsTrend}`), `loading` (starts `true`), `error`. The hook's internal `useEffect` fires `fetchAnalyticsBundle` once on mount (deps `[]`).
- `useScanTrigger()` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`. Called with **no** `onSuccess` argument — unlike most other pages, Analytics does not refetch its trend data after a scan completes.
- No local `useState` in the component body itself; all state lives in the two hooks.

API CALLS MADE

`fetchAnalyticsBundle` (module-level helper) runs 4 calls in parallel via `Promise.all` on mount:

- `getCostTrend()` → `GET /api/analytics/cost-trend`
- `getSavingsTrend()` → `GET /api/analytics/savings-trend`
- `getResourceGrowth()` → `GET /api/analytics/resource-growth`
- `getHighAlertsTrend()` → `GET /api/analytics/high-alerts-trend`

Clicking "Run Scan" in Navbar also fires `scanInfrastructure()` → `POST /api/scan` (via `useScanTrigger`), but this does not re-trigger the analytics fetch.

COMPONENT TREE

`PageTransition` > `Navbar` (`onScan`, `scanning`, `stepIndex`, `scanPhase`) + heading block + `ErrorBanner` + a 2-column grid of 4× `TrendChart` ("Monthly Cost Trend", "Estimated Savings Trend", "Resources Growth" [`variant="area"`], "High Alerts Trend").

DATA FLOW

On mount, `useAsyncData`'s effect calls `fetchAnalyticsBundle`, firing all 4 analytics endpoints concurrently; while pending, `loading=true` is passed into every `TrendChart`, which renders its own skeleton. When `Promise.all` resolves, `data` is set and each chart re-renders with real series data. If any of the 4 calls rejects, the whole bundle fails, `error` is set to "Failed to load analytics data." and shown via `ErrorBanner`, and `data` stays at `EMPTY_ANALYTICS` (all charts render empty state). Running a scan via Navbar POSTs `/api/scan` and shows success/failure toasts, but does not refresh this page's trend data automatically.

EXAMPLE EXECUTION

User clicks "Analytics" nav link → navigates to `/analytics` → `Analytics` mounts → `useAsyncData`'s internal `useEffect` fires → `fetchAnalyticsBundle()` → 4 parallel GETs to `/api/analytics/*` → responses combined into `{costTrend, savingsTrend, resourceGrowth, highAlertsTrend}` → `setData` → `loading` flips to `false` → 4 `TrendChart` components each render a line/area chart from their respective array.

IMPORTANT NOTES

- Notable inconsistency: `useScanTrigger()` is called without a `refetch` callback here, so running a scan from this page does **not** refresh the visible trends (unlike AuditLogs, Dashboard, Recommendations, ScanHistory, ScanDetail, which all pass `refetch`).
- `error` and `scanError` are combined into a single `ErrorBanner` via `error || scanError`, so if both occur, only the async-data error message is shown.
- `initialValue: EMPTY_ANALYTICS` guarantees `data.costTrend` etc. are always arrays, so `TrendChart` never receives `undefined`.
- `formatCount` is a small local helper (`` ${value ?? 0} ``) used only for the two count-based charts (resource growth, high alerts); the two currency charts use `formatCurrency`.

 `client/src/pages/AuditLogs.jsx`

PURPOSE

Displays a searchable, filterable, paginated table of the account's audit log entries (login, AWS connect/disconnect, scan started/completed, recommendation ignored/resolved, EC2 stopped, etc.) for security/compliance visibility.

ROUTE

`/audit-logs`, `ProtectedRoute`-wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/audit-logs`.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/Badge.jsx` — colored pill per audit action type
- `components/Card.jsx` — container
- `components/ErrorBanner.jsx`
- `components/FilterSelect.jsx` — action-type dropdown
- `components/Navbar.jsx`
- `components/NoResultsState.jsx` — empty-filtered-results message + clear button
- `components/PageTransition.jsx`
- `components/TablePagination.jsx`
- `components/TableSkeleton.jsx` — loading skeleton rows
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `services/auditLog.service.js` — `getAuditLogs`
- `utils/badge.js` — `getAuditActionBadgeStyle`
- `utils/format.js` — `formatDateTime`, `formatAuditActionLabel`

STATE MANAGEMENT

- `useAsyncData(getAuditLogs, [], { initialValue: [], errorMessage: "Failed to load audit logs." })` → `data` aliased `logs`, `loading`, `error`, `refetch`.
- `useScanTrigger(refetch)` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; `refetch` is passed as the success callback, so a completed scan reloads the log list (new `SCAN_STARTED` / `SCAN_COMPLETED` entries appear).
- `useState search` (string, `""`) — free-text query; resets `currentPage` to 1 on change.
- `useState actionFilter` (string, `"ALL"`) — selected action-type filter; resets `currentPage` to 1 on change.
- `useState currentPage` (number, `1`).
- `useMemo filtered` — derived from `logs` / `actionFilter` / `search`: applies the action filter then `matchesSearch` (case-insensitive match against `description` + the formatted action label).
- Derived per render (not state): `totalPages`, `safePage` (clamped), `paginated` (page slice of `filtered`).

API CALLS MADE

- `getAuditLogs()` → `GET /api/audit-logs` — called once on mount, and again after every successful scan (via `refetch` passed into `useScanTrigger`). No mutation endpoints are called from this page.

COMPONENT TREE

`PageTransition` > `Navbar` + heading + `ErrorBanner` + `Card` containing: `TableSkeleton` (while loading) OR "No activity has been recorded yet." text (if `logs` empty) OR [search input + `FilterSelect`] + (`NoResultsState` if `filtered` is empty, else an animated `<table>` of `paginated` rows [Timestamp, `Badge` (Action), Description] using `AnimatePresence` / `motion.tr`) + `TablePagination`.

DATA FLOW

On mount, `useAsyncData` calls `getAuditLogs` → `GET /api/audit-logs` → `logs` stored in state, `loading` flips to `false`, table renders. Typing in the search box or changing the `FilterSelect` recomputes `filtered` entirely client-side (no new network call) and resets to page 1. Clicking "Run Scan" in Navbar triggers `scanInfrastructure` via `useScanTrigger`; on success it calls `refetch`, re-GETting `/api/audit-logs` so newly created `SCAN_STARTED` / `SCAN_COMPLETED` entries appear without a full page reload.

EXAMPLE EXECUTION

User navigates to `/audit-logs` → `useAsyncData` fires `getAuditLogs()` → `GET /api/audit-logs` → returns array of log objects → `TableSkeleton` replaced by the real table → user types "scan" in search → `filtered` recomputed client-side to entries whose description/action label contains "scan" → paginated table re-renders with matches only, `currentPage` reset to 1.

IMPORTANT NOTES

- Filtering/search/sorting is entirely client-side over the single full `logs` array returned by one GET — there is no server-side search/pagination endpoint, so all audit logs are fetched at once regardless of volume.
- Two distinct empty states: "No activity has been recorded yet." (`logs.length === 0`) vs. `NoResultsState` (`filtered.length === 0` but `logs.length > 0`, i.e. filters are too narrow).
- `COLUMNS` / `ACTION_OPTIONS` / animation variant objects are module-level constants, not recreated per render.

 `client/src/pages/ConnectAws.jsx`

PURPOSE

A form letting the user link an AWS account by entering a region, access key, secret key, and an optional friendly alias; the credentials must be successfully "tested" against AWS before they can be saved, at which point CloudLens can start scanning that account.

ROUTE

`/connect-aws`, `ProtectedRoute`-wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/connect-aws` (e.g. to link a first AWS account or replace an existing connection).

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/ErrorBanner.jsx`
- `components/PageTransition.jsx`
- `services/awsConnection.service.js` — `testAwsConnection` , `connectAwsAccount`

Notably, `ConnectAws` does **not** import `Navbar` — unlike every other protected page, it renders without the shared top navigation bar, presented as a centered full-screen card similar to Login/Signup.

STATE MANAGEMENT

- `useState formData` (`{region: "ap-south-1", accessKeyId: "", secretAccessKey: "", accountAlias: ""}`) — the form fields.
- `useState showSecret` (bool, `false`) — toggles the secret-key input between `password` / `text` .
- `useState error` (string|null) — form-level error message.
- `useState testing` (bool) — in-flight flag for "Test Connection".
- `useState testResult` (object|null) — result of a successful test (`accountId` , `region` , `arn`); reset to `null` whenever any of the three `CREDENTIAL_FIELDS` (`region` , `accessKeyId` , `secretAccessKey`) is edited, forcing re-testing before saving. Editing `accountAlias` does **not** invalidate it.
- `useState saving` (bool) — in-flight flag for "Save & Connect".
- No `useEffect` ; navigation on success is imperative via `useNavigate` .

API CALLS MADE

- `testAwsConnection(formData)` → `POST /api/aws/test-connection` — called from `handleTestConnection` when "Test Connection" is clicked. On success sets `testResult` ; on failure clears `testResult` and sets `error` from `err.response?.data?.message` .
- `connectAwsAccount(formData)` → `POST /api/aws/connect` — called from `handleSave` on form submit, guarded by `if (!testResult)` `return` ; (and the Save button is disabled without a passing test). On success, `navigate("/")` .

COMPONENT TREE

`PageTransition` (centered flex layout) > `motion.div` card containing: header (cloud icon, "Connect AWS") + `ErrorBanner` + conditional green success summary (Account ID, Region, ARN — shown when `testResult` is set) + `<form>` with region `<select>` , `accessKeyId` input, `secretAccessKey` input (with show/hide toggle button), `accountAlias` input, and two buttons ("Test Connection", "Save & Connect").

DATA FLOW

User fills in region/access key/secret key → clicks "Test Connection" → `handleTestConnection` sets `testing=true` , calls `testAwsConnection` → `POST /api/aws/test-connection` with current credentials → backend performs a server-side AWS STS call and returns `{accountId, region, arn}` → `testResult` is set and the "Connected Successfully" panel renders, enabling "Save & Connect". User clicks "Save & Connect" → `handleSave` calls `connectAwsAccount` → `POST /api/aws/connect` persisting the credentials server-side → on success, `navigate("/")` sends the user to the Dashboard.

EXAMPLE EXECUTION

User enters `region=us-east-1` , `accessKeyId=AKIA...` , `secretAccessKey=...` → clicks Test Connection → `POST /api/aws/test-connection` → `200 {accountId:"123456789012", region:"us-east-1", arn:"arn:aws:iam::..."}` → `testResult` set → "Save & Connect" enabled → user clicks it → `POST /api/aws/connect` → success → `navigate("/")` → Dashboard mounts.

IMPORTANT NOTES

- Editing region/accessKeyld/secretAccessKey after a successful test silently invalidates `testResult` , re-disabling "Save & Connect" until re-tested.
- The Save button carries a `title` tooltip ("Test the connection successfully before saving") while disabled due to a missing test result.
- Secret key input defaults to `type="password"` , toggleable to plaintext via an eye icon.
- No loading/skeleton state on mount because there is no initial GET — the form always starts empty. `getAwsConnection` is exported from `awsConnection.service.js` but is **not called anywhere in this file**, so an existing connection's data is never pre-filled here.

 `client/src/pages/Dashboard.jsx`

PURPOSE

The main landing page after login — an overview dashboard showing hero stats (resources, monthly cost, savings), AI-generated insights, a grid of AWS services scanned, roughly 20 summary metric cards spanning cost/security/EC2/IAM categories, four charts (resource health, cost

breakdown, resource distribution, savings opportunity), and the full recommendation table with inline actions.

ROUTE

/ (root), `ProtectedRoute` -wrapped.

WHEN THIS FILE IS CALLED

On the redirect after a successful login (`Login.jsx` calls `navigate("/")`), or whenever the user navigates to the root path.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/AIInsightsCard.jsx` — renders `dashboardData.aiInsights`
- `components/CostBreakdownDonutChart.jsx`
- `components/ErrorBanner.jsx`
- `components/HeroSection.jsx` — headline stats
- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/RecommendationTable.jsx`
- `components/ResourceDistributionBarChart.jsx`
- `components/ResourceHealthPieChart.jsx`
- `components/SavingsOpportunityBarChart.jsx`
- `components/ServiceOverviewGrid.jsx`
- `components/SummaryCard.jsx` — reused ~17 times for individual metric tiles
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `services/dashboard.service.js` — `getDashboard`
- `services/recommendation.service.js` — `getRecommendations`
- `services/scan.service.js` — `getScans`
- `utils/format.js` — `formatCurrency`, `formatPercent`, `formatBytes`

STATE MANAGEMENT

- `useAsyncData(fetchDashboardBundle, [], { errorMessage: "Failed to load dashboard data. Is the backend server running?" })` → `data` (default `null` — this page's `useAsyncData` call omits `initialValue`), `setData`, `error`, `loading`, `refetch`. Fires `fetchDashboardBundle` on mount.
- `useScanTrigger(refetch)` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; `refetch` wired as the success callback so a completed scan reloads the entire dashboard bundle.
- No other local `useState`. `handleRecommendationUpdate(updated)` is a plain function (not a hook) that calls `setData` to splice an updated recommendation into `data.recommendations` by `_id`; passed as `onRecommendationUpdate` to `RecommendationTable` so ignoring/resolving/stopping a recommendation updates the dashboard in place without a full refetch.

API CALLS MADE

`fetchDashboardBundle` (module-level helper) runs 3 calls in parallel via `Promise.all` on mount:

- `getDashboard()` → `GET /api/dashboard`
- `getRecommendations()` → `GET /api/recommendations`
- `getScans()` → `GET /api/scans` (only `scans[0]`, the most recent, is kept as `lastScan`)

All three re-run together after a successful scan (via `refetch`). Recommendation mutation endpoints (ignore/resolve/stop) are invoked inside the `RecommendationTable` child component, not directly here; results flow back via `onRecommendationUpdate`.

COMPONENT TREE

`PageTransition` > `Navbar` + `ErrorBanner` + `HeroSection` (`totalResources`, `monthlyCost`, `estimatedSavings`, `lastScan`) + `AIInsightsCard` + "Resources by Service" heading + `ServiceOverviewGrid` + grid of 8 `SummaryCard` (Resources/Running/Stopped/Monthly Cost/Estimated Savings/High/Medium/Low Alerts) + "Recommendation Categories" heading + 3 `SummaryCard` (Security/Cost/Performance Findings) + "EC2 Fleet Insights" heading + 6 `SummaryCard` (Avg CPU/Network/Disk, Idle/Old Instances, Encrypted Volumes) + "IAM Security

Insights" heading + 6 `SummaryCard` (IAM Users/Roles, MFA Enabled, Admin Users, Old Access Keys, High Risk Users) + 2×2 chart grid (`ResourceHealthPieChart` , `CostBreakdownDonutChart` , `ResourceDistributionBarChart` , `SavingsOpportunityBarChart`) + `RecommendationTable` .

DATA FLOW

On mount, `useAsyncData` fires `fetchDashboardBundle` , issuing 3 parallel GETs; while pending, `isLoading = loading || !data` is threaded into every child's `loading` prop so each renders its own skeleton. Once resolved, `dashboardData` / `recommendations` / `lastScan` are derived from `data` and passed to ~25 child components. If the *initial* fetch fails entirely (`error && !data`), the whole page is replaced by a centered error card with a "Retry" button calling `refetch` directly. If a later refetch fails while stale data still exists, the error shows via `ErrorBanner` on top of the still-visible dashboard. Running a scan (Navbar) calls `scanInfrastructure` via `useScanTrigger` ; on success `refetch()` re-runs all 3 GETs. Acting on a recommendation row inside `RecommendationTable` calls a mutation endpoint from within that child and reports the updated object back via `onRecommendationUpdate` , patching `data.recommendations` locally without a network round trip.

EXAMPLE EXECUTION

User logs in → `Login` navigates to `/` → `Dashboard` mounts → `useAsyncData` 's effect fires `fetchDashboardBundle` → `Promise.all([GET /api/dashboard, GET /api/recommendations, GET /api/scans])` → all resolve → `data = {dashboard, recommendations, lastScan}` → `isLoading` becomes `false` → `HeroSection` shows `totalResources` / `monthlyCost` / `estimatedSavings` , ~20 `SummaryCard` s populate, 4 charts render, `RecommendationTable` lists all recommendation rows.

IMPORTANT NOTES

- Full-page error fallback only triggers when there is **no** cached data at all (`error && !data`); once data has loaded successfully once, subsequent errors degrade to a banner instead of blanking the page.
- Uses optional chaining extensively (`dashboardData.categoryStats?.securityFindings` , `dashboardData.ec2Stats?.avgCpu` , `dashboardData.iamStats?.totalUsers` , etc.) since nested stat objects may be `undefined` before first load or if the backend omits them.
- This is the only page whose `useAsyncData` call omits `initialValue` , so `data` is `null` until the first successful response — relevant to the error-fallback branch's `!data` check.

 `client/src/pages/Login.jsx`

PURPOSE

Email/password sign-in form; on success stores the JWT in `localStorage` and redirects to the dashboard.

ROUTE

`/login` , **not** wrapped in `ProtectedRoute` — it's a sibling top-level `<Route>` outside the `ProtectedRoute` element in `App.jsx`, so it must be reachable while unauthenticated (it's what grants the token in the first place).

WHEN THIS FILE IS CALLED

User navigates to `/login` directly, is redirected here by `ProtectedRoute` when no token exists (`<Navigate to="/login" replace />`), or clicks the "Login" link from Signup's footer.

WHO CALLS THIS FILE

- `client/src/App.jsx` (route registration). `ProtectedRoute.jsx` triggers the redirect to this URL when unauthenticated, though it does so via React Router's `Navigate` , not a direct import of `Login` .

WHICH FILES IT DEPENDS ON

- `components/ErrorBanner.jsx`
- `components/PageTransition.jsx`
- `services/auth.service.js` — `login`

STATE MANAGEMENT

- `useState email` (string, `""`).
- `useState password` (string, `""`).
- `useState error` (string|null) — login failure message.
- `useState loading` (bool, `false`) — submit in-flight flag; disables the button and shows a spinner.
- No `useEffect` .

API CALLS MADE

- `login(email, password)` → `POST /api/auth/login` — called only from `handleSubmit` on form submission. On success, the response is destructured for `token`, persisted via `localStorage.setItem("token", token)`, then `navigate("/")`. On failure, `error` is set from `err.response?.data?.message` (falls back to "Invalid email or password").

COMPONENT TREE

`PageTransition` (centered) > `motion.div` card > header (cloud icon, "Welcome back") + `ErrorBanner` + `<form>` (email input, password input, submit button with spinner) + footer link to `/signup`.

DATA FLOW

User types email/password → submits → `handleSubmit` calls `login()` → `POST /api/auth/login` with `{email, password}` → backend validates credentials and returns `{token, ...}` → token persisted to `localStorage` under key `"token"` → `navigate("/")` → React Router re-renders; `ProtectedRoute` now sees a token and renders the matched route's `<Outlet/>` → `Dashboard` mounts and starts its own data fetch, with every subsequent request carrying `Authorization: Bearer <token>` via the Axios interceptor in `api/api.js`.

EXAMPLE EXECUTION

User enters valid credentials → clicks Login → `loading=true`, spinner shown → `POST /api/auth/login` → `200 {token: "eyJ..."}` → `localStorage.token` set → `navigate("/")` → Dashboard renders.

IMPORTANT NOTES

- No client-side email/password format validation beyond native `required / type="email"` HTML attributes.
- `loading` is always reset in the `finally` block regardless of outcome.
- No "remember me" or refresh-token flow is visible here — the JWT persists indefinitely in `localStorage` until a `401` clears it (handled globally by `api/api.js`'s response interceptor) or the user logs out (logout logic is not present in this file — presumably lives in `Navbar.jsx`, which is outside the 13 documented files).

 `client/src/pages/NotificationSettings.jsx`

PURPOSE

Lets the user configure which email notifications they receive (scan completed, high-severity recommendation, weekly cost report) and set or change the destination notification email address.

ROUTE

`/notifications`, `ProtectedRoute`-wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/notifications`.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/Card.jsx`
- `components/ErrorBanner.jsx`
- `components/ToggleSwitch.jsx` — the on/off switch for each notification type
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `services/notification.service.js` — `getNotificationSettings`, `updateNotificationSettings`
- `utils/toast.jsx` — `showSuccessToast`, `showErrorToast`

STATE MANAGEMENT

- `useAsyncData(getNotificationSettings, [], { errorMessage: "Failed to load notification settings." })` → `data` aliased `settings` (default `null`), `setData`, `loading`, `error`. Fires on mount.

- `useScanTrigger()` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; called with `no onSuccess` — a completed scan does not refetch notification settings here (reasonable, since a scan doesn't change these settings).
- `useState email` (string, `""`) — editable email field, kept separate from `settings.email` so it can be edited before saving.
- `useState savingEmail` (bool) — in-flight flag for the email Save button.
- `useState togglingKey` (string|null) — tracks which specific toggle (`scanCompleted` / `highSeverity` / `weeklyReport`) is mid-request, so only that one `ToggleSwitch` shows a disabled state while the others stay interactive.
- `useEffect([settings])` — when `settings` loads/changes, copies `settings.email` into local `email` state so the input starts pre-filled with the server value.

API CALLS MADE

- `getNotificationSettings()` → `GET /api/notifications/settings` — on mount.
- `updateNotificationSettings({ [key]: !settings[key] })` → `PATCH /api/notifications/settings` — called from `handleToggle` whenever a `ToggleSwitch` is flipped (per-key partial update).
- `updateNotificationSettings({ email })` → `PATCH /api/notifications/settings` — called from `handleSaveEmail` when the Save button next to the email input is clicked (only enabled if `email` differs from `settings.email`).

COMPONENT TREE

`PageTransition` > `Navbar` + heading + `ErrorBanner` + `Card` (`max-w-xl`) containing: loading text (while `loading` / `!settings`) OR ["Email Notifications" header + email input & Save button + 3× toggle rows (icon, title, description, `ToggleSwitch`) driven by `NOTIFICATION_OPTIONS`].

DATA FLOW

On mount, `GET /api/notifications/settings` populates `settings` and the local `email` field via the sync `useEffect` . Toggling a switch calls `handleToggle` → `PATCH` with just that one key flipped → server returns the full updated settings object → `setData(updated)` replaces `settings` wholesale. Saving the email similarly `PATCH` es `{email}` , replaces `settings` , and fires a success toast.

EXAMPLE EXECUTION

User loads `/notifications` → `GET /api/notifications/settings` → `{email:"user@x.com", scanCompleted:true, highSeverity:false, weeklyReport:true}` → `settings` populates, email field shows `"user@x.com"` → user flips "High Severity Recommendation" toggle → `handleToggle("highSeverity")` → `PATCH /api/notifications/settings {highSeverity:true}` → updated settings returned → `ToggleSwitch` now shows enabled.

IMPORTANT NOTES

- Toggling and email-saving both replace the entire `settings` object from the server response rather than optimistically updating locally, so the UI waits for the round trip (`togglingKey` disables just that row's switch meanwhile).
- The email Save button is disabled both while saving and whenever the field already matches the saved value, preventing redundant `PATCH` calls.
- Toggle/save failures surface via toast (`showErrorToast`) rather than the page-level `ErrorBanner` , which is reserved for the initial-load error and `scanError` .

 `client/src/pages/Recommendations.jsx`

PURPOSE

A dedicated page listing every cost/security/performance recommendation for the account (the same `RecommendationTable` component the Dashboard embeds), without the rest of the dashboard's stat cards/charts — a focused, recommendations-only view.

ROUTE

`/recommendations` , `ProtectedRoute` -wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/recommendations` .

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/ErrorBanner.jsx`
- `components/RecommendationTable.jsx`
- `services/recommendation.service.js` — `getRecommendations`
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`

STATE MANAGEMENT

- `useAsyncData(getRecommendations, [], { initialValue: [], errorMessage: "Failed to load recommendations." })` → `data` aliased `recommendations`, `setData`, `loading`, `error`, `refetch`. Fires on mount.
- `useScanTrigger(refetch)` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; `refetch` wired so a completed scan reloads the recommendation list.
- `handleRecommendationUpdate(updated)` — plain function (not a hook); calls `setData` to map over `recommendations` and replace the entry whose `_id` matches `updated._id`; passed as `onRecommendationUpdate` to `RecommendationTable` so ignore/resolve/stop actions reflect immediately without a refetch.

API CALLS MADE

- `getRecommendations()` → `GET /api/recommendations` — on mount and again after a successful scan (`refetch`). Mutation calls (ignore/resolve/stop) happen inside `RecommendationTable`, not directly in this file; results return via `onRecommendationUpdate`.

COMPONENT TREE

`PageTransition` > `Navbar` + heading + `ErrorBanner` + `RecommendationTable` (`recommendations`, `onRecommendationUpdate`, `refresh={refetch}`, `loading`).

DATA FLOW

On mount, `GET /api/recommendations` populates the table. Acting on a row inside `RecommendationTable` (ignore/resolve/stop) triggers a mutation request from within that child; the returned updated recommendation is merged into local state via `handleRecommendationUpdate` — no full refetch needed for a single-row action, though `RecommendationTable` is also given `refresh={refetch}` (likely used for bulk actions or a manual-refresh control inside that component). Running a scan repopulates the whole list via `refetch` once the scan finishes.

EXAMPLE EXECUTION

User clicks the "Recommendations" nav link → `GET /api/recommendations` → array of recommendation objects rendered as rows → user clicks "Ignore" on a row inside `RecommendationTable` → `PATCH /api/recommendations/:id/ignore` → updated recommendation returned → `handleRecommendationUpdate` splices it into the local array → that row's badge/status updates in place.

IMPORTANT NOTES

- This is the simplest of the three "recommendations-consuming" pages (Dashboard, Recommendations, ScanDetail) — no additional stats, filters, or charts, just the table.

 `client/src/pages/ScanDetail.jsx`

PURPOSE

Shows the full detail of a single scan by ID — status, timestamp, resource/savings stat cards, the complete resource inventory table, and the recommendations generated by that specific scan — plus lets the user export a PDF report of that scan.

ROUTE

`/scan-history/:scanId`, `ProtectedRoute` -wrapped.

WHEN THIS FILE IS CALLED

When the user clicks "View Details" on a row in `ScanHistory.jsx` (`navigate(\ /scan-history/${scanId}\ )`), or navigates to the URL directly.

WHO CALLS THIS FILE

- `client/src/App.jsx` (route registration). `client/src/pages/ScanHistory.jsx` navigates here programmatically.

WHICH FILES IT DEPENDS ON

- `components/ActionButton.jsx` — "Export Report" button with its own loading state
- `components/Card.jsx`
- `components/ErrorBanner.jsx`
- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/RecommendationTable.jsx`
- `components/ResourceTable.jsx`
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `services/report.service.js` — `downloadScanReport`
- `services/scan.service.js` — `getScanById`
- `utils/badge.js` — `getScanStatusBadgeClass`
- `utils/format.js` — `formatScanTime`, `formatCurrency`
- `utils/toast.js` — `showErrorToast`
- External: `react-router-dom` 's `useParams` / `Link`, `react-loading-skeleton` 's `Skeleton` for inline loading placeholders.

STATE MANAGEMENT

- `useParams()` → `scanId` (string from the URL; not React state).
- `useAsyncData()` → `getScanById(scanId)`, `[scanId]`, `{ errorMessage: "Failed to load scan details." }` → `data` aliased `scanData` (default `null`), `setData` aliased `setScanData`, `loading`, `error`, `refetch`. Deps `[scanId]` means the fetch re-runs if `scanId` changes.
- `useScanTrigger(refetch)` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; `refetch` wired so a completed scan reloads this scan's data.
- `useState exporting` (bool, `false`) — in-flight flag for the PDF export button.
- `handleRecommendationUpdate(updated)` — plain function; patches `scanData.recommendations` by `_id` via `setScanData`, passed as `onRecommendationUpdate` to `RecommendationTable`.

API CALLS MADE

- `getScanById(scanId)` → `GET /api/scans/:id` — on mount and whenever `scanId` changes.
- `downloadScanReport(scanId)` → `GET /api/reports/scan/:scanId ( responseType: "blob" )` — called from `handleExportReport` when "Export Report" is clicked; on success it programmatically creates and clicks a download `<a>` for the returned PDF blob (no navigation, no state mutation — the "result" is a file download). On failure, the error is logged to console and shown via `showErrorToast`.
- Recommendation mutation endpoints (ignore/resolve/stop) are called inside `RecommendationTable`, not this file directly.

COMPONENT TREE

`PageTransition` > `Navbar` (only `onScan` / `scanning` passed — see note below) + `ErrorBanner(scanError)` + "Back to Scan History" `Link` + header row (title, formatted scan time or `Skeleton`, status badge or `Skeleton`, `ActionButton` "Export Report") + 2-card stat grid (Resources count, Estimated Savings) + `Card` containing `ResourceTable(resources)` + `RecommendationTable(recommendations)`.

DATA FLOW

On mount (or `scanId` change), `GET /api/scans/:id` returns `{scan, resources, recommendations}` which becomes `scanData`; while loading, stat values and the header render `Skeleton` placeholders. If the fetch finishes and there's an error or no `scanData`, the component early-returns a minimal "Scan not found" view instead of the full layout. Once loaded, scan-level fields populate the header/stat cards, `resources` populate `ResourceTable`, and `recommendations` populate `RecommendationTable`. Clicking "Export Report" downloads a PDF via a blob response and otherwise does not change page state. Acting on a recommendation row updates `scanData.recommendations` locally via `handleRecommendationUpdate`.

EXAMPLE EXECUTION

User is on `/scan-history`, clicks "View Details" for a completed scan → `navigate(\ /scan-history/${scan_id}\ )` → `ScanDetail` mounts → `useParams` reads `scanId` → `useAsyncData` calls `getScanById(scanId)` → `GET /api/scans/64f.../` → `200 {data: {scan:{...}, resources:[14 items], recommendations:[6 items]}}` → `scanData` set → `isLoading` false → header shows formatted date + status `Badge` → stat cards show "14" resources, "\$132.50" estimated savings → `ResourceTable` renders 14 rows → `RecommendationTable` renders 6 rows. User clicks "Export Report" → `exporting=true` → `GET /api/reports/scan/64f.../` (blob) → browser downloads `cloudlens-scan-report-64f....pdf` → `exporting=false`.

IMPORTANT NOTES

- **Inconsistency:** the `<Navbar onScan={runScan} scanning={scanning} />` render used in the main return omits the `stepIndex` / `scanPhase` props that every other page passes to `Navbar` — the multi-step scan-progress UI likely won't animate through its steps correctly when a scan is triggered from this specific page.
- Two different error displays exist: an early-return full-page "Scan not found." message (when `!loading && (error || !scanData)`) versus an `ErrorBanner` fed only by `scanError` in the main render path — the async-data `error` itself is not shown via `ErrorBanner` in the success-path render.
- `resources` and `recommendations` both default to `[]` via destructuring (`const { scan, resources = [], recommendations = [] } = scanData || {}`), guarding against a scan whose sub-collections are missing.

 `client/src/pages/ScanHistory.jsx`

PURPOSE

Lists all past scans in a searchable, filterable, sortable, paginated table (date, resource count, savings, duration, status), with each row clickable to open that scan's `ScanDetail` page.

ROUTE

`/scan-history`, `ProtectedRoute` -wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/scan-history`.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/Card.jsx`
- `components/ErrorBanner.jsx`
- `components/TableSkeleton.jsx`
- `components/Badge.jsx`
- `components/FilterSelect.jsx`
- `components/NoResultsState.jsx`
- `components/SortableTh.jsx` — clickable/sortable header cell
- `components/TablePagination.jsx`
- `services/scan.service.js` — `getScans`
- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `utils/format.js` — `formatScanTime`, `formatCurrency`, `formatScanDuration`, `getScanDurationMs`
- `utils/badge.js` — `getScanStatusBadgeStyle`
- `utils/sort.js` — `compareValues`
- External: `react-router-dom`'s `useNavigate` / `Link`.

STATE MANAGEMENT

- `useAsyncData(getScans, [], { initialValue: [], errorMessage: "Failed to load scan history." })` → `data` aliased `scans`, `loading`, `error`, `refetch`. Fires on mount.
- `useScanTrigger(refetch)` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; `refetch` wired so a completed scan adds/updates rows.
- `useState search` (string, "").
- `useState statusFilter` (string, "ALL").
- `useState sortConfig` (`{key:"date", direction:"desc"} | null`) — current sort column/direction; clicking the same column cycles asc → desc → null (unsorted); clicking a different column starts at asc.

- `useState currentPage` (number, 1).
- `useMemo filteredAndSorted` — filters `scans` by `statusFilter` + `matchesSearch`, then sorts by `sortConfig` using `getSortValue` + `compareValues` (recomputed when `scans` / `statusFilter` / `search` / `sortConfig` change).
- Derived per render: `totalPages`, `safePage`, `paginated`.

API CALLS MADE

- `getScans()` → `GET /api/scans` — on mount and after each successful scan (via `refetch`). No per-row mutation calls — clicking a row only navigates.

COMPONENT TREE

`PageTransition` > `Navbar` + heading + `ErrorBanner` + `Card` containing: `TableSkeleton` (loading) OR "No scans yet" text + `Link` to `/` (`scans.length===0`) OR [search input + `FilterSelect` (Status)] + `NoResultsState` (filtered-empty) OR `<table>` with `SortableTh` header cells (Scan Date/Resources/Savings/Duration/Status/View Details) and animated rows (each row clickable via `onClick={() => viewScan(scan._id)}`), plus an explicit "View Details" button that stops propagation) + `TablePagination`.

DATA FLOW

On mount, `GET /api/scans` populates `scans`. Search/status-filter/sort/pagination are all computed client-side from the single fetched array — no additional network calls for those interactions. Clicking anywhere on a row (or its "View Details" button) calls `viewScan(scan._id)`, which does `navigate(`/scan-history/${scanId}`)` to open `ScanDetail`. Running a scan from the Navbar reloads the full list via `refetch` afterward, so the new scan appears (typically sorted to the top under the default date-desc sort).

EXAMPLE EXECUTION

User navigates to `/scan-history` → `GET /api/scans` → array of scan summaries → table renders sorted by date desc by default → user selects "Completed" in the Status `FilterSelect` → `filteredAndSorted` recomputed client-side to only completed scans, `currentPage` reset to 1 → user clicks a row → `navigate("/scan-history/<id>")` → `ScanDetail` mounts for that id.

IMPORTANT NOTES

- The "View Details" button inside a row calls `event.stopPropagation()` on its containing `<td>` so the click isn't double-handled by the row's own `onClick` (both ultimately call the same handler for the same id, so this is a defensive no-op rather than avoiding an actual bug).
- Sorting is 3-state per column: asc → desc → null. With `sortConfig` null, the `if (sortConfig)` guard in `filteredAndSorted` skips sorting entirely, leaving the filtered array in its original fetch order (not re-sorted to any particular default).
- Duration is computed via `getScanDurationMs(scan) ?? -1` as the sort key, so scans without a computable duration sort as if duration were negative (i.e., first when ascending).

 `client/src/pages/ScheduleScan.jsx`

PURPOSE

Lets the user set up, view, edit, enable/disable, or delete a single recurring scan schedule (hourly/daily/weekly) so CloudLens automatically re-scans the connected AWS account without manual triggering.

ROUTE

`/schedules`, `ProtectedRoute`-wrapped.

WHEN THIS FILE IS CALLED

When the user navigates to `/schedules`.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/Navbar.jsx`
- `components/PageTransition.jsx`
- `components/Card.jsx`
- `components/ErrorBanner.jsx`
- `components/Badge.jsx`
- `components/ToggleSwitch.jsx`

- `hooks/useAsyncData.js`
- `hooks/useScanTrigger.js`
- `services/schedule.service.js` — `getSchedule`, `createSchedule`, `updateSchedule`, `deleteSchedule`
- `utils/format.js` — `formatDateTime`
- `utils/badge.js` — `getScheduleStatusBadgeStyle`
- `utils/toast.jsx` — `showSuccessToast`, `showErrorToast`

STATE MANAGEMENT

- `useAsyncData(getSchedule, [], { errorMessage: "Failed to load scan schedule." })` → `data` aliased `schedule` (default `null` — "no schedule yet" is a valid, common state), `setData`, `loading`, `error`. Fires on mount.
- `useScanTrigger()` → `scanning`, `scanError`, `runScan`, `stepIndex`, `scanPhase`; no `onSuccess`, so completing a manual scan does not refetch the schedule.
- `useState frequency` (string, defaults to `FREQUENCY_OPTIONS[1].value = "daily"`) — the frequency `<select>`'s value, used both for creating a new schedule and editing an existing one.
- `useState creating` (bool) — in-flight flag for "Create Schedule".
- `useState saving` (bool) — in-flight flag for saving a frequency change.
- `useState toggling` (bool) — in-flight flag for the enable/disable `ToggleSwitch`.
- `useState deleting` (bool) — in-flight flag for "Delete Schedule".
- `useEffect([schedule])` — when `schedule` loads/changes, syncs local `frequency` to `schedule.frequency`.

API CALLS MADE

- `getSchedule()` → `GET /api/schedules` (client-side service unwraps the array to `schedules[0] || null`) — on mount.
- `createSchedule({ frequency, enabled: true })` → `POST /api/schedules` — from `handleCreate` when there's no existing schedule.
- `updateSchedule(schedule._id, { frequency })` → `PATCH /api/schedules/:id` — from `handleSaveFrequency` (only if `frequency` differs from `schedule.frequency`).
- `updateSchedule(schedule._id, { enabled: !schedule.enabled })` → `PATCH /api/schedules/:id` — from `handleToggleEnabled`.
- `deleteSchedule(schedule._id)` → `DELETE /api/schedules/:id` — from `handleDelete`; on success `setData(null)` resets the page to the "no schedule" state.

COMPONENT TREE

`PageTransition` > `Navbar` + heading + `ErrorBanner` + `Card` (`max-w-xl`) with three mutually-exclusive states: (1) loading text, (2) no-schedule form (frequency select + "Create Schedule" button), or (3) existing-schedule panel (title + status `Badge`, enable/disable `ToggleSwitch` row, frequency select + Save button, Last Run/Next Run definition list via `formatDateTime`, "Delete Schedule" button).

DATA FLOW

On mount, `GET /api/schedules` returns either an array with one schedule or an empty array; the service unwraps it so `schedule` is either an object or `null`, driving which of the three `Card` states renders. If `null`, the user picks a frequency and clicks Create → `POST /api/schedules` → server returns the new schedule → `setData(created)` switches the Card to the existing-schedule view. If a schedule exists, toggling enable/disable or changing+saving frequency each independently `PATCH` the schedule and replace `schedule` with the server's response (each shows a success/error toast). Deleting calls `DELETE` then explicitly sets data back to `null`, reverting to the creation form.

EXAMPLE EXECUTION

User visits `/schedules` with no existing schedule → `GET /api/schedules` → `{data: []}` → `schedule=null` → "No schedule set up yet" form shown → user selects "Weekly", clicks "Create Schedule" → `POST /api/schedules {frequency:"weekly", enabled:true}` → `201 {_id, frequency:"weekly", enabled:true, lastRun:null, nextRun:"2026-08-08T..."}` → `setData(created)` → Card switches to the existing-schedule view showing an "Enabled" badge and the computed `nextRun` date.

IMPORTANT NOTES

- The backend's schedule model conceptually supports multiple schedules per user via the array-shaped `GET` response, but the client service layer (`schedule.service.js`) and this page only ever work with a single schedule (`schedules[0]`) — creating a second schedule is not exposed in the UI even though `POST /api/schedules` would allow it.
- The Save-frequency button is disabled both while saving and when `frequency` already equals `schedule.frequency`, the same disabled-when-unchanged pattern used in `NotificationSettings`' email save.
- No confirmation dialog before "Delete Schedule" — clicking it immediately calls the `DELETE` endpoint.

PURPOSE

Account creation form (full name, email, password, confirm password) that registers a new user and redirects to `/login` on success (does not auto-login).

ROUTE

`/signup`, **not** wrapped in `ProtectedRoute` (top-level sibling route, reachable while unauthenticated).

WHEN THIS FILE IS CALLED

User navigates to `/signup` directly, or clicks "Sign Up" from Login's footer link.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `components/ErrorBanner.jsx`
- `components/PageTransition.jsx`
- `services/auth.service.js` — `signup`

STATE MANAGEMENT

- `useState fullName` (string, "").
- `useState email` (string, "").
- `useState password` (string, "").
- `useState confirmPassword` (string, "").
- `useState error` (string|null).
- `useState loading` (bool, false).
- No `useEffect`.

API CALLS MADE

- `signup(fullName, email, password)` → `POST /api/auth/signup` — called from `handleSubmit` only after a client-side check that `password === confirmPassword` passes. On success, `navigate("/login")` (note: `confirmPassword` is never sent to the backend — it's purely a client-side match check). On failure, `error` is set from `err.response?.data?.message` (falls back to "Failed to create account").

COMPONENT TREE

`PageTransition` (centered) > `motion.div` card > header (cloud icon, "Create your account") + `ErrorBanner` + `<form>` (Name, Email, Password, Confirm Password inputs, submit button with spinner) + footer link to `/login`.

DATA FLOW

User fills all four fields → submits → `handleSubmit` first checks `password===confirmPassword` client-side (returns early with a local `error` if mismatched, never reaching the network call) → if they match, calls `signup()` → `POST /api/auth/signup {fullName, email, password}` → backend creates the user → on success, `navigate("/login")` sends the user to the login form to sign in with their new credentials.

EXAMPLE EXECUTION

User enters `name="Jane Doe"`, `email="jane@x.com"`, `password="Secret123"`, `confirmPassword="Secret123"` → submits → passwords match, `loading=true` → `POST /api/auth/signup` → `201` → `navigate("/login")` → Login page renders; the user must now log in separately.

IMPORTANT NOTES

- Password-match validation is entirely client-side; the backend is never told what `confirmPassword` was.
- No password-strength/format validation client-side beyond native `required` / `type="password"` attributes (the backend's `utils/validators.js` may enforce rules server-side — not visible from this file).
- Unlike `Login`, `Signup` does **not** store a token or navigate to `"/` — the user always has to explicitly log in afterward via a separate request.

PURPOSE

The application's route table and top-level layout shell — defines every URL path in the SPA, gates the authenticated section behind `ProtectedRoute`, wraps route transitions in Framer Motion's `AnimatePresence` for animated page swaps, and sets up the global toast portal and the react-loading-skeleton theme colors.

ROUTE

N/A — this file *defines* all routes; it is not itself routed.

WHEN THIS FILE IS CALLED

On every render of the React tree: once when `main.jsx` mounts `<App />` into the DOM, and its internal `<Routes>` re-renders on every client-side navigation.

WHO CALLS THIS FILE

- `client/src/main.jsx` (renders `<App />` into the DOM root)

WHICH FILES IT DEPENDS ON

- `components/AppToaster.jsx` — global `react-hot-toast` container, rendered once outside the router
- `components/ProtectedRoute.jsx` — auth guard wrapping the protected route group
- All 11 page components: `pages/Analytics.jsx`, `pages/AuditLogs.jsx`, `pages/ConnectAws.jsx`, `pages/Dashboard.jsx`, `pages/Login.jsx`, `pages/NotificationSettings.jsx`, `pages/Recommendations.jsx`, `pages/ScanDetail.jsx`, `pages/ScanHistory.jsx`, `pages/ScheduleScan.jsx`, `pages/Signup.jsx` — each mapped to exactly one `<Route>`.
- External: `react-router-dom` (`BrowserRouter`, `Routes`, `Route`, `useLocation`), `framer-motion` (`AnimatePresence`), `react-loading-skeleton` (`SkeletonTheme`).

STATE MANAGEMENT

No `useState` in `App.jsx` itself. The inner `AnimatedRoutes` component calls `useLocation()` (react-router-dom) to read the current pathname, used both to key `<Routes location={location} key={location.pathname}>` (forcing a full remount/animation cycle per route change) and as the `location` prop to `<Routes>`. No `useEffect`.

API CALLS MADE

None directly — `App.jsx` has no service imports; all data fetching happens inside the individual page components once mounted.

COMPONENT TREE

`SkeletonTheme` (sets `--skeleton-base-color` / `--skeleton-highlight-color` as base/highlight colors) > `AppToaster` (toast portal, sibling to the router, always mounted) + `BrowserRouter` > `AnimatedRoutes` > `AnimatePresence(mode="wait")` > `Routes(location, key=pathname)`: one `<Route element={}<ProtectedRoute />>` wrapping 9 nested child routes, plus two top-level unguarded routes (`/login`, `/signup`).

Full route table:

PATH	COMPONENT	PROTECTEDROUTE?
/	Dashboard	yes
/scan-history	ScanHistory	yes
/scan-history/:scanId	ScanDetail	yes
/recommendations	Recommendations	yes
/connect-aws	ConnectAws	yes
/schedules	ScheduleScan	yes
/notifications	NotificationSettings	yes
/analytics	Analytics	yes
/audit-logs	AuditLogs	yes
/login	Login	no

PATH	COMPONENT	PROTECTEDROUTE?
/signup	Signup	no

DATA FLOW

React Router matches the current URL against this table on every navigation; for any path under the `ProtectedRoute` element, `ProtectedRoute.jsx` checks `localStorage` for a "token" — if present it renders `<Outlet />` (the matched child route's element), otherwise it issues `<Navigate to="/login" replace />`, bouncing the user to the login form before any protected page ever mounts. Keying `<Routes>` on `location.pathname` combined with `AnimatePresence mode="wait"` means each navigation is treated as the old route's component unmounting (playing its `PageTransition` exit animation) before the new route's component mounts (playing its entrance animation), rather than React reusing the same component instance across routes.

EXAMPLE EXECUTION

Unauthenticated user's browser hits `/` → `BrowserRouter` matches `/` inside the `ProtectedRoute` group → `ProtectedRoute` reads `localStorage.token` → not present → `<Navigate to="/login" replace />` → URL becomes `/login` → `Routes` re-matches → `Login` renders (unguarded route) → user logs in, token stored → `navigate("/")` → `Routes` re-matches `/` → `ProtectedRoute` now sees a token → renders `<Outlet/>` → `Dashboard` mounts.

IMPORTANT NOTES

- `ProtectedRoute` wraps the *entire* group of 9 routes via nested routing (a single `<Route element={}<ProtectedRoute/></>` with children), rather than wrapping each route individually — any future route added inside that block automatically inherits the auth guard.
- `AppToaster` is rendered outside `BrowserRouter` / `AnimatePresence` entirely, so toast notifications persist across route-change animations rather than resetting per page.
- There is no catch-all/404 route defined — an unmatched path renders nothing inside `<Routes>` rather than showing a not-found page.
- No logout route/button/logic is visible in this file (logout, if implemented, must live inside `Navbar.jsx` or another component outside this file set).

 `client/src/main.jsx`

PURPOSE

The Vite/React entry point — mounts the React application into the DOM's `#root` element wrapped in `StrictMode`, and globally imports the app's base CSS plus the react-loading-skeleton stylesheet.

ROUTE

N/A — not a route; this is the JS entry point loaded before any routing exists.

WHEN THIS FILE IS CALLED

Once, when the browser loads the built JS bundle (index.html's `<script type="module" src="/src/main.jsx">`), before React Router or any route matching occurs.

WHO CALLS THIS FILE

Nothing in the source tree imports `main.jsx` (`importedBy: []` in the dependency graph) — it is loaded directly by index.html/Vite as the module entry point, not imported by application code.

WHICH FILES IT DEPENDS ON

- `client/src/App.jsx` — the component tree being mounted
- `client/src/index.css` — global Tailwind/base styles
- External: `react` (`StrictMode`), `react-dom/client` (`createRoot`), `react-loading-skeleton/dist/skeleton.css` (base styles for the `Skeleton` loading placeholders used throughout the pages)

STATE MANAGEMENT

None — this file contains no components, hooks, or state; it is a single imperative `createRoot(...).render(...)` call.

API CALLS MADE

None.

COMPONENT TREE

`StrictMode` > `App` (the entire application; see `App.jsx`'s route table above).

DATA FLOW

On page load, the browser executes `main.jsx`, which locates the DOM node with `id="root"` (in `index.html`), creates a React root on it via `createRoot`, and renders `<StrictMode><App /></StrictMode>` into it. From that point forward, all data flow is governed by `App.jsx`'s routing and each page's own fetch logic.

EXAMPLE EXECUTION

Browser loads `index.html` → loads `/src/main.jsx` as a module → `createRoot(document.getElementById('root'))` → `.render(<StrictMode><App /></StrictMode>)` → `App` mounts → `BrowserRouter` reads the current URL and renders the matching route's page component.

IMPORTANT NOTES

- `StrictMode` is enabled, which in development causes React to intentionally double-invoke certain functions (component bodies, some effects) to help surface side-effect bugs — this does not affect production builds' actual runtime behavior/output.
 - If `document.getElementById('root')` is missing from `index.html`, `createRoot(null)` would throw at startup — not a concern under normal Vite scaffolding, but worth noting as an implicit dependency on `index.html`'s markup.
-

FRONTEND COMPONENTS (A–M)

Reusable UI building blocks, alphabetically first half.

 client/src/components/AIInsightsCard.jsx

PURPOSE

Renders the "AI Insights" card on the Dashboard — a `Card` shell showing a list of short rule-based sentences (not actually LLM-generated, per the file's own comment) about the latest S3 scan, each prefixed with a severity icon/color. Handles its own loading skeleton and empty state.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
insights	array of <code>{ _id?, severity, message }</code>	No	<code>[]</code>	List of insight objects to render as list items
loading	boolean	No	<code>false</code>	Switches the body to a skeleton placeholder

WHEN THIS FILE IS CALLED

Rendered unconditionally on `client/src/pages/Dashboard.jsx` (line 115) as `<AIInsightsCard insights={dashboardData.aiInsights} loading={isLoading} />`, directly below the `HeroSection`.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `client/src/components/Card.jsx` — provides the glass card shell, hover/entrance motion props are passed through to it

INTERNAL BEHAVIOR

- Defines a static `SEVERITY_META` map (HIGH/MEDIUM/LOW/INFO → icon component + Tailwind text-color class) and falls back to `INFO` for unrecognized severities.
- Defines Framer Motion `listVariants` / `itemVariants` for a staggered list entrance (0.07s stagger, 0.1s delay, fade+slide-in-from-left per item).
- Renders a `Card` with `hover` enabled, an entrance animation (`opacity 0→1, y 16→0`), and a `whileHover` lift/scale.
- Draws a decorative blurred gradient circle (`aria-hidden`) in the top-right corner.
- Renders a header row: gradient icon badge (`FaWandMagicSparkles`) + "AI Insights" title + subtitle.
- Conditional body, three branches: (a) `loading` → renders 4 `Skeleton` bars using fixed widths `SKELETON_ROW_WIDTHS = [95, 85, 90, 70]`; (b) `!insights.length` → renders "No insights yet — run a scan to generate them."; (c) otherwise → renders a `motion.ul` with staggered `motion.li` items, each showing the severity icon and `insight.message`, keyed by `insight._id || index`.

DATA FLOW

`insights` and `loading` are pure props from Dashboard's fetched `dashboardData.aiInsights` (part of the `GET /api/dashboard` response) and its combined loading state; the component fires no callbacks — it is read-only/presentational.

EXAMPLE EXECUTION

`<AIInsightsCard insights={[{_id: "1", severity: "HIGH", message: "Bucket X allows public read access"}]} loading={false} />` renders the card with a single list item: a red `FaTriangleExclamation` icon followed by the message text.

IMPORTANT NOTES

The header/body text says "AI Insights" and "AI-powered" styling (wand icon, gradient), but the source comment explicitly clarifies the insights are rule-based, generated server-side in `server/src/services/insight/s3.insight.js`, not from an LLM — naming is aspirational/marketing rather than literal.

PURPOSE

A small pill-shaped button (icon + label) used for row/footer actions like "Stop Instance", "Ignore", "Resolve" in recommendation tables and drawers. Shows a spinning `FaSpinner` and a `loadingLabel` while an async action is in flight, and disables itself during that time.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE			
variant	"danger" \	"warning" \	"success" \	"primary"	Yes (no default; undefined variant produces <code>undefined</code> in the class string)	—	Selects background/hover/focus-ring color from <code>ACTION_VARIANTS</code>
icon	component (react-icons)	Yes	—	Icon rendered when not loading			
label	string	Yes	—	Text rendered when not loading			
loadingLabel	string	No (required if <code>loading</code> can be true)	—	Text rendered while <code>loading</code>			
loading	boolean	No	— (falsy/undefined treated as not loading)	Disables the button and swaps icon/label for a spinner + <code>loadingLabel</code>			
onClick	function	No	—	Click handler, ignored while <code>loading</code> is truthy since the button is <code>disabled</code>			

WHEN THIS FILE IS CALLED

Rendered for each actionable recommendation row: in `RecommendationTable.jsx` for "Stop Instance" (only when `item.state` is not already stopped), "Ignore", and "Resolve" actions; in `ResourceDetailsDrawer.jsx` for the same three actions in the drawer footer; also imported by `client/src/pages/ScanDetail.jsx`.

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/components/ResourceDetailsDrawer.jsx`
- `client/src/pages/ScanDetail.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — only external packages (`framer-motion` for the `motion.button` wrapper, `react-icons/fa6` for `FaSpinner`).

INTERNAL BEHAVIOR

- Looks up the variant's Tailwind classes from the static `ACTION_VARIANTS` map.
- Renders a `motion.button` that is `disabled={loading}`, with `whileHover` / `whileTap` scale animations that are turned off (`undefined`) while loading so a disabled button doesn't visually react to hover/tap.
- Conditionally renders `FaSpinner` (with `animate-spin`) + `loadingLabel` when `loading` is truthy, otherwise the passed `Icon` + `label` .

DATA FLOW

Purely controlled by props from the parent row/drawer state; `onClick` is the only callback fired, invoked by the parent to trigger the corresponding recommendation action (stop/ignore/resolve), typically wired to a hook like `useRecommendationActions` .

EXAMPLE EXECUTION

`<ActionButton variant="danger" icon={FaCircleStop} label="Stop Instance" loadingLabel="Stopping..." loading={false} onClick={handleStop} />` renders a red pill button reading "Stop Instance" with a stop icon; while `loading` becomes `true` it shows a spinning icon and "Stopping...", and the button is unclickable.

IMPORTANT NOTES

There is no default for `variant`, so passing an unrecognized/omitted variant produces `undefined` in the template literal (`${ACTION_VARIANTS[variant]}` → `"undefined"` string appended to `className`), which would silently drop all variant-specific styling rather than throwing.

 `client/src/components/AnimatedCounter.jsx`

PURPOSE

Animates a numeric display value from 0 (on mount) or its previous value up to a new `value`, used for stat numbers like resource counts and dollar amounts (HeroSection, SummaryCard) so they visibly count up rather than jump.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
value	number	No	<code>0</code>	Target numeric value to animate to
format	function <code>(v: number) => string</code>	No	<code>(v) => Math.round(v).toLocaleString()</code>	Formats the interpolated value each frame (e.g. currency)
duration	number (seconds)	No	<code>1</code>	Animation duration passed to Framer Motion's <code>animate()</code>
className	string	No	<code>undefined</code>	Passed through to the rendered <code>motion.span</code>

WHEN THIS FILE IS CALLED

Used by `HeroSection.jsx` for "Total Resources", "Monthly Cost", and "Estimated Savings" stat blocks (each only rendered when not in the parent's loading state), and by `SummaryCard.jsx` for its stat tiles.

WHO CALLS THIS FILE

- `client/src/components/HeroSection.jsx`
- `client/src/components/SummaryCard.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `react` (`useEffect`, `useRef`) and `framer-motion` (`motion`, `useMotionValue`, `animate`).

INTERNAL BEHAVIOR

- Creates a Framer Motion `motionValue` initialized to `0` and a `spanRef` DOM ref.
- On mount and whenever `value` changes (effect dependency array is `[value]`, with `format` / `duration` deliberately excluded via an `eslint-disable` comment), calls `animate(motionValue, value, { duration, ease: "easeOut", onUpdate })`.
- `onUpdate` writes the formatted latest value directly into `spanRef.current.textContent` on every animation frame — bypassing React's render cycle for performance, rather than calling `setState`.
- Returns `controls.stop` as the effect cleanup, canceling any in-flight animation if `value` changes again or the component unmounts.
- Initial render (before the effect runs) shows `format(0)` as static text inside the `motion.span`.

DATA FLOW

`value` prop drives the animation target; there is no callback output. The rendered text is a side effect of the Framer Motion `onUpdate` tick directly mutating the DOM node referenced by `spanRef`, not of React state/props re-rendering the span's children.

EXAMPLE EXECUTION

`<AnimatedCounter value={1240.5} format={formatCurrency} duration={1} />` starts at `$0.00` and animates the displayed text up to `$1240.50` over 1 second with an ease-out curve.

IMPORTANT NOTES

Because the counter writes to `textContent` imperatively, React is unaware of the current displayed text between renders — this is intentional to avoid a re-render per animation frame, but means the displayed value can be inspected only via the live DOM, not via React state. Only `value` re-triggers the animation; changing `format` or `duration` alone will not restart it (per the eslint-disable on the exhaustive-deps rule).

 `client/src/components/AppToaster.jsx`

PURPOSE

Hosts and positions the app's global toast notification container. It is a thin wrapper around `react-hot-toast`'s `Toaster`, rendered once near the app root so any code can call `toast.custom(...)` and have it appear bottom-right.

PROPS

This component takes no props.

WHEN THIS FILE IS CALLED

Rendered once, unconditionally, in `client/src/App.jsx` alongside the router — it is not scoped to any particular page/condition.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `react-hot-toast` (`Toaster`).

INTERNAL BEHAVIOR

1. Renders `<Toaster position="bottom-right" gutter={10} />` with no `toastOptions`, deliberately, per the file's comment, because all toasts are custom JSX built via `toast.custom()` in `utils/toast.jsx` and don't need react-hot-toast's built-in styling.

DATA FLOW

No props in, no callbacks out — purely a mount point. Actual toast content/state is pushed into `react-hot-toast`'s internal store by calls elsewhere in the app (e.g. `utils/toast.jsx`), and this component subscribes to and renders that store.

EXAMPLE EXECUTION

Once mounted, a call elsewhere in the app like `toast.custom(<MyToast />)` causes a toast to animate in at the bottom-right of the viewport with 10px gutter spacing between stacked toasts.

IMPORTANT NOTES

Because `toastOptions` is intentionally omitted, any code that relies on react-hot-toast's default styling (rather than fully custom JSX via `toast.custom()`) would render unstyled/plain — this component assumes all callers use the custom-JSX pattern.

 `client/src/components/Badge.jsx`

PURPOSE

A soft pill-shaped status badge with a leading colored dot, used throughout the app for severity, category, and status labels (e.g. "HIGH", "OPEN", "COMPLETED"). Purely presentational; the actual colors come from caller-supplied class strings.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
children	node	No	—	Badge label text/content
badgeClass	string	No	<code>""</code>	Tailwind classes for the pill's background/text/border color
dotClass	string	No	<code>""</code>	Tailwind classes for the leading dot's background color

WHEN THIS FILE IS CALLED

Rendered wherever a status/severity/category chip is needed: `RecommendationTable.jsx` (category, severity, and status columns, e.g. `<Badge {...getSeverityBadgeStyle(item.severity)}>{item.severity}</Badge>`), `ResourceDetailsDrawer.jsx` (status/severity/category chips in the drawer header), and directly in the pages `AuditLogs.jsx` , `ScanHistory.jsx` , and `ScheduleScan.jsx` for their respective status columns.

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/components/ResourceDetailsDrawer.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ScanHistory.jsx`
- `client/src/pages/ScheduleScan.jsx`

WHICH FILES IT DEPENDS ON

No local or external imports — a self-contained presentational component.

INTERNAL BEHAVIOR

- Renders an outer `<span>` with `badgeClass` merged into a base pill style (rounded-full, border, padding, uppercase text).
- Renders an inner `<span>` (the dot) sized `h-1.5 w-1.5 rounded-full` with `dotClass` for its color.
- Renders `children` after the dot.

DATA FLOW

`badgeClass` / `dotClass` are typically produced by helper functions in `client/src/utils/badge.js` (e.g. `getSeverityBadgeStyle(severity)` returns `{ badgeClass, dotClass }` , spread as props: `<Badge {...getSeverityBadgeStyle(item.severity)}>`). No callbacks; purely presentational output.

EXAMPLE EXECUTION

`<Badge {...getSeverityBadgeStyle("HIGH")}>HIGH</Badge>` — `getSeverityBadgeStyle("HIGH")` returns `{ badgeClass: "bg-red-50 text-red-700 border-red-200", dotClass: "bg-red-500" }` , so it renders a red-tinted pill with a red dot and the text "HIGH".

IMPORTANT NOTES

The component itself carries no color logic — all color decisions live in `client/src/utils/badge.js` 's `get*BadgeStyle` functions, which is the reuse pattern: add a new badge type by adding a new style-lookup function there, not by modifying `Badge.jsx` .

client/src/components/Card.jsx

PURPOSE

The shared "glass" card shell (blurred translucent white surface, rounded corners, soft border/shadow) that underlies nearly every card-like block in the app — summary cards, chart cards, table containers, the AI insights card. Centralizes the visual look so it only needs tuning in one place.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
children	node	No	—	Card content
className	string	No	<code>""</code>	Extra/override Tailwind classes appended last
padding	string	No	<code>"p-6"</code>	Padding utility class, overridable per-instance
hover	boolean	No	<code>false</code>	Adds a shadow-grow transition on hover
...motionProps	any (rest)	No	—	Forwarded directly to the underlying <code>motion.div</code> (e.g. <code>initial</code> , <code>animate</code> , <code>whileHover</code> , <code>transition</code> , <code>custom</code>)

WHEN THIS FILE IS CALLED

Used as the base shell by `AIInsightsCard.jsx` , `ChartCard.jsx` , `RecommendationTable.jsx` , `ServiceOverviewGrid.jsx` , `SummaryCard.jsx` , and directly by the pages `AuditLogs.jsx` , `NotificationSettings.jsx` , `ScanDetail.jsx` , `ScanHistory.jsx` , and `ScheduleScan.jsx` for their content panels.

WHO CALLS THIS FILE

- client/src/components/AIInsightsCard.jsx
- client/src/components/ChartCard.jsx
- client/src/components/RecommendationTable.jsx
- client/src/components/ServiceOverviewGrid.jsx
- client/src/components/SummaryCard.jsx
- client/src/pages/AuditLogs.jsx
- client/src/pages/NotificationSettings.jsx
- client/src/pages/ScanDetail.jsx
- client/src/pages/ScanHistory.jsx
- client/src/pages/ScheduleScan.jsx

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `framer-motion` (`motion`).

INTERNAL BEHAVIOR

1. Renders a single `motion.div` whose `className` interpolates: fixed glass styles (`rounded-2xl border border-white/60 bg-white/70 ... backdrop-blur-xl`), the caller's `padding` , a conditional hover-shadow-transition block when `hover` is true, and the caller's `className` appended last (so callers can override).
2. Spreads all remaining `motionProps` onto the `motion.div` , letting each caller define its own entrance/hover/tap animation without Card needing to know about them.

DATA FLOW

Pure pass-through container: `children` renders inside, `motionProps` configure Framer Motion behavior, no internal state or callbacks.

EXAMPLE EXECUTION

`<Card hover initial={{opacity:0,y:16}} animate={{opacity:1,y:0}} className="relative overflow-hidden">...</Card>` (as used by `AIInsightsCard`) renders a glass card that fades/slides in on mount and grows its shadow on hover.

IMPORTANT NOTES

Because `className` is appended after the conditional hover classes in the template literal, a caller's `className` can override earlier utility classes if they conflict (standard Tailwind cascade/specificity caveats apply, not a JS bug). All animation behavior is opt-in via passthrough props — Card itself defines no default motion.

client/src/components/ChartCard.jsx

PURPOSE

A `Card` -based wrapper specifically for dashboard charts: adds a title/subtitle header and a staggered entrance animation keyed by an `index` (each chart in a grid animates in slightly after the previous one), so individual chart components (`CostBreakdownDonutChart` , etc.) only need to supply their Recharts content as `children` .

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
title	string	No	—	Chart heading text
subtitle	string	No	—	Optional secondary line under the title, only rendered if truthy
index	number	No	0	Position used to stagger the entrance delay ($0.15 + index * 0.08$ s)
children	node	No	—	The chart content (Recharts <code>ResponsiveContainer</code> , skeleton, or empty state)

WHEN THIS FILE IS CALLED

Wraps every dashboard chart: `CostBreakdownDonutChart.jsx` , `ResourceDistributionBarChart.jsx` , `ResourceHealthPieChart.jsx` , `SavingsOpportunityBarChart.jsx` , and `TrendChart.jsx` , each passed a distinct `index` in the Dashboard/Analytics grid layout so they cascade in visually.

WHO CALLS THIS FILE

- client/src/components/CostBreakdownDonutChart.jsx
- client/src/components/ResourceDistributionBarChart.jsx
- client/src/components/ResourceHealthPieChart.jsx
- client/src/components/SavingsOpportunityBarChart.jsx
- client/src/components/TrendChart.jsx

WHICH FILES IT DEPENDS ON

- client/src/components/Card.jsx — provides the underlying glass shell and motion prop passthrough

INTERNAL BEHAVIOR

1. Defines cardVariants with a visible variant that is a function of custom (the index), computing transition.delay = 0.15 + index * 0.08 .
2. Renders Card with hover , custom={index} , initial="hidden" , animate="visible" , variants={cardVariants} , and a separate whileHover spring transition — the comment notes the hover spring is defined via the top-level transition prop specifically so it doesn't inherit the entrance variant's delay .
3. Renders a header block with title (h2) and, only if subtitle is truthy, a p subtitle underneath.
4. Renders children (the actual chart) below the header.

DATA FLOW

title / subtitle / index configure the static header and animation timing; children is the actual chart markup supplied by the parent chart component. No callbacks; purely presentational/animation wrapper.

EXAMPLE EXECUTION

<ChartCard title="Cost Breakdown" subtitle="Monthly cost by service · Total \$1,204.50" index={1}>...</ChartCard> renders a card whose entrance animation is delayed 0.15 + 1*0.08 = 0.23s , with the title and subtitle text above the passed-in Recharts PieChart .

IMPORTANT NOTES

The staggering only works correctly if each sibling ChartCard in a grid is given a distinct, deliberately-ordered index — reusing the same index on multiple charts collapses their entrance timing together.

client/src/components/ChartEmptyState.jsx

PURPOSE

A minimal centered "no data" placeholder shown inside a chart's fixed-height area when there is nothing meaningful to plot (e.g. total cost is \$0, or a distribution array is empty).

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
message	string	No (no default; renders empty if omitted)	—	The empty-state text to display
height	number	No	320	Pixel height applied via inline style to match the real chart's footprint

WHEN THIS FILE IS CALLED

Rendered by each of the five chart components (CostBreakdownDonutChart.jsx , ResourceDistributionBarChart.jsx , ResourceHealthPieChart.jsx , SavingsOpportunityBarChart.jsx , TrendChart.jsx) in place of the Recharts chart when their computed dataset is empty/zero, e.g. in CostBreakdownDonutChart.jsx : total === 0 ? <ChartEmptyState message="No cost data yet. Run a scan to see your cost breakdown." /> :

WHO CALLS THIS FILE

- client/src/components/CostBreakdownDonutChart.jsx
- client/src/components/ResourceDistributionBarChart.jsx
- client/src/components/ResourceHealthPieChart.jsx

- `client/src/components/SavingsOpportunityBarChart.jsx`
- `client/src/components/TrendChart.jsx`

WHICH FILES IT DEPENDS ON

No imports at all — a fully self-contained presentational component.

INTERNAL BEHAVIOR

1. Renders a flex column container sized to `height` (inline style) that centers its content both horizontally and vertically.
2. Renders a single `<p>` with the `message` text in muted slate styling.

DATA FLOW

Pure presentational: `message` / `height` in, static markup out. No state, no callbacks.

EXAMPLE EXECUTION

`<ChartEmptyState message="No cost data yet. Run a scan to see your cost breakdown." />` renders a 320px-tall centered block with that sentence in light gray text, replacing where a donut chart would otherwise be.

IMPORTANT NOTES

`height` should typically match the height of the real chart it substitutes for (each chart component's own default is 320px) so swapping between loading/empty/data states doesn't cause layout jump.

`client/src/components/ChartSkeleton.jsx`

PURPOSE

A loading placeholder shown inside a `ChartCard` while chart data is being fetched. Its `variant` selects a skeleton shape (pie, horizontal bars, or vertical bars) that roughly matches the real chart it will be replaced by, so the loading state doesn't look generic.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE		
variant	"bar" \	"pie" \	"horizontal-bar"	No	"bar"	Selects which skeleton shape to render

WHEN THIS FILE IS CALLED

Rendered by each chart component while its `loading` prop is true: `variant="pie"` in `CostBreakdownDonutChart.jsx` and (presumably) `ResourceHealthPieChart.jsx`; `variant="horizontal-bar"` or the default `"bar"` in the bar-chart components (`ResourceDistributionBarChart.jsx`, `SavingsOpportunityBarChart.jsx`, `TrendChart.jsx`), based on each chart's own orientation.

WHO CALLS THIS FILE

- `client/src/components/CostBreakdownDonutChart.jsx`
- `client/src/components/ResourceDistributionBarChart.jsx`
- `client/src/components/ResourceHealthPieChart.jsx`
- `client/src/components/SavingsOpportunityBarChart.jsx`
- `client/src/components/TrendChart.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `react-loading-skeleton` (`Skeleton`).

INTERNAL BEHAVIOR

1. Renders a fixed `h-[320px]` flex column container.
2. Three-way conditional on `variant`: `"pie"` renders a single circular `Skeleton` (180x180); `"horizontal-bar"` renders 5 horizontal `Skeleton` bars with fixed widths `[90, 70, 100, 55, 80]` (%); otherwise (default `"bar"`) renders 5 vertical bars using the module-level `BAR_HEIGHTS = [55, 90, 40, 100, 70]` array, each 32px wide.
3. Below the chart-shape skeleton, always renders a row of three small `Skeleton` blocks (64x12) to stand in for a legend.

DATA FLOW

No props besides `variant` control the static skeleton shapes; no state, no callbacks.

EXAMPLE EXECUTION

`<ChartSkeleton variant="pie" />` renders a centered 180px skeleton circle plus a 3-item legend-placeholder row, standing in for `CostBreakdownDonutChart` while `loading` is true.

IMPORTANT NOTES

The bar heights/widths (`BAR_HEIGHTS` , the horizontal-bar width array) are hardcoded decorative values, not derived from any real data — they exist purely to look chart-like during loading.

`client/src/components/ChartTooltip.jsx`

PURPOSE

A custom Recharts tooltip renderer styled to match the dashboard's translucent "glass" aesthetic, replacing Recharts' plain default tooltip box. Passed as the `content` prop to Recharts' `<Tooltip>` in each chart.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
active	boolean	No (supplied by Recharts)	—	Whether the tooltip should currently be shown
payload	array	No (supplied by Recharts)	—	Array of hovered data-point entries (<code>{name, value, color, fill, payload}</code>)
label	string	No (supplied by Recharts)	—	Category/axis label for the hovered point, if any
formatter	function (value, payload) => string	No	—	Optional value formatter (e.g. currency); falls back to raw <code>entry.value</code> if omitted

WHEN THIS FILE IS CALLED

Passed as `<Tooltip content={<ChartTooltip formatter={...} />} />` inside each of the five dashboard/analytics chart components, only rendering its content when Recharts sets `active` true (i.e., while the user hovers a data point).

WHO CALLS THIS FILE

- `client/src/components/CostBreakdownDonutChart.jsx`
- `client/src/components/ResourceDistributionBarChart.jsx`
- `client/src/components/ResourceHealthPieChart.jsx`
- `client/src/components/SavingsOpportunityBarChart.jsx`
- `client/src/components/TrendChart.jsx`

WHICH FILES IT DEPENDS ON

No imports at all — fully self-contained.

INTERNAL BEHAVIOR

- Guard clause: returns `null` if `!active || !payload?.length`, so nothing renders when not hovering a data point.
- Renders a glass-styled box; if `label` is present, shows it as a bold header line.
- Maps over `payload`, rendering one row per series/entry: a colored dot (`entry.color || entry.fill`), the series `name`, and the value — run through `formatter(entry.value, entry.payload)` if a formatter was supplied, otherwise the raw `entry.value`.

DATA FLOW

Entirely driven by props Recharts injects when a user hovers the chart (`active`, `payload`, `label`); `formatter` is caller-supplied to control value display (e.g. `formatCurrency`). No callbacks fired — read-only rendering.

EXAMPLE EXECUTION

Hovering a donut slice in `CostBreakdownDonutChart` triggers Recharts to render `<ChartTooltip active payload={{name: "EC2", value: 340.12, color: "#3b82f6", payload: {...}}} formatter={formatCurrency} />`, showing a glass box with a blue dot, "EC2:", and "\$340.12".

IMPORTANT NOTES

Falls back to `entry.color || entry.fill` for the dot color since different Recharts series types (bars vs. pie cells) populate different fields; omitting a `formatter` for numeric values that need formatting (e.g. currency) will display raw unformatted numbers.

 `client/src/components/CopyButton.jsx`

PURPOSE

A small pill button that copies a given string to the clipboard via the Clipboard API and briefly flashes a "Copied" confirmation state. Used in the resource details drawer's console-actions row (e.g. copy ARN, copy bucket name).

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
value	string	Yes	—	The text written to the clipboard on click
label	string	Yes	—	Button text shown when not in the "copied" state

WHEN THIS FILE IS CALLED

Rendered in `ResourceDetailsDrawer.jsx` 's "AWS Console Actions" section for S3 resources, e.g. `<CopyButton value={getS3BucketArn(resource.resourceId)} label="Copy ARN" />` and `<CopyButton value={resource.resourceId} label="Copy Bucket Name" />`.

WHO CALLS THIS FILE

- `client/src/components/ResourceDetailsDrawer.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends on external `react` (`useState`) and `react-icons/fa6` (`FaRegCopy` , `FaCheck`); `framer-motion` for the button's hover/tap animation.

INTERNAL BEHAVIOR

- Holds local state `copied` (boolean, default `false`).
- `handleCopy` is an async click handler: calls `navigator.clipboard.writeText(value)` ; on success sets `copied` to `true` then, after 1500ms via `setTimeout` , resets it to `false` ; on failure, the `catch` block is empty (silently swallows the error — e.g. Clipboard API unavailable or permission denied) per the inline comment, with no recovery/error UI.
- Renders a `motion.button` with hover/tap scale animation; icon and label swap based on `copied` : `FaCheck` (emerald) + "Copied" when true, `FaRegCopy` + `label` when false.

DATA FLOW

`value` is written to the OS clipboard as a side effect of the click; `label` is static display text. No callback props — the component owns its own transient "copied" UI state and does not notify the parent of success/failure.

EXAMPLE EXECUTION

Clicking `<CopyButton value="arn:aws:s3:::my-bucket" label="Copy ARN" />` copies the ARN string to the clipboard, and the button immediately shows a green checkmark and "Copied" for 1.5 seconds before reverting to "Copy ARN".

IMPORTANT NOTES

Clipboard write failures are silently ignored (empty `catch` block) — there is no user-visible error state if `navigator.clipboard` is unavailable (e.g. non-HTTPS context) or permission is denied, so a failed copy looks identical to a successful click from the button alone (the "Copied" flash still only fires on success, but no failure feedback is given either).

PURPOSE

Renders the Dashboard's "Cost Breakdown" donut chart, showing monthly cost per AWS service as pie slices (Recharts `PieChart`), with the smallest services folded into an "Other" slice once there are more than `MAX_SLICES` (6) services. Handles its own loading skeleton and zero-cost empty state.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
costBreakdown	array of <code>{ service, cost }</code>	No	<code>[]</code>	Per-service monthly cost entries from the dashboard API
index	number	No (no default)	—	Forwarded to <code>ChartCard</code> to stagger its entrance animation
loading	boolean	No	<code>false</code>	Shows a pie-shaped skeleton instead of the chart

WHEN THIS FILE IS CALLED

Rendered on `client/src/pages/Dashboard.jsx` (line 350) as `<CostBreakdownDonutChart costBreakdown={dashboardData.costBreakdown} index={1} loading={isLoading} />`, inside the 2-column chart grid alongside the other three chart components.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `client/src/components/ChartCard.jsx` — provides the titled/animated card shell
- `client/src/components/ChartEmptyState.jsx` — shown when total cost is 0
- `client/src/components/ChartSkeleton.jsx` — shown while `loading`
- `client/src/components/ChartTooltip.jsx` — custom hover tooltip content
- `client/src/utils/chartColors.js` — `CATEGORICAL_PALETTE` for slice fill colors
- `client/src/utils/format.js` — `formatCurrency` for the subtitle total and tooltip values

INTERNAL BEHAVIOR

- `foldIntoSlices(costBreakdown)` : maps each `{service, cost}` entry to `{name, value}`, sorts descending by value; if there are $\leq \text{MAX_SLICES}$ (6) entries returns them as-is, otherwise keeps the top 5 and sums the remainder into one `{name: "Other", value: otherTotal}` slice.
- `chartData` is memoized via `useMemo` on `costBreakdown`; `total` is the sum of all slice values, recomputed every render (not memoized).
- Wraps everything in `ChartCard` with `title="Cost Breakdown"` and a dynamic `subtitle` : while `loading`, just "Monthly cost by service"; otherwise appends `· Total {formatCurrency(total)}`.
- Three-way conditional body: `loading` → `ChartSkeleton variant="pie"`; `total === 0` → `ChartEmptyState` with a "run a scan" message; otherwise → a Recharts `PieChart` inside a 320px-tall `ResponsiveContainer`.
- The `Pie` uses `innerRadius={68} / outerRadius={110}` (a true donut, not a full pie), `paddingAngle={chartData.length > 1 ? 3 : 0}` (no gap when there's only one slice), and an 800ms ease-out entrance animation.
- Each `Cell` is colored via `CATEGORICAL_PALETTE[i % CATEGORICAL_PALETTE.length]` (cycles the 8-color palette if more slices exist) with a 2px white stroke separating slices.
- Renders a `Tooltip` using `ChartTooltip` with `formatter={(value) => formatCurrency(value)}`, and a bottom `Legend` with circular icons.

DATA FLOW

`costBreakdown` (from `GET /api/dashboard`'s `costBreakdown` field) flows through `foldIntoSlices` into Recharts-ready `{name, value}` pairs; `loading` gates which of three visual states renders. No callbacks fired — the chart is read-only, hover state is entirely internal to Recharts/ `ChartTooltip`.

EXAMPLE EXECUTION

Given `costBreakdown = [{service: "EC2", cost: 300}, {service: "S3", cost: 120}, {service: "RDS", cost: 40}]` and `loading=false`, renders a donut with 3 colored slices (blue EC2, emerald S3, amber RDS per `CATEGORICAL_PALETTE` order) and subtitle "Monthly cost by service · Total \$460.00". Hovering a slice shows a glass tooltip with the service name and formatted currency value.

IMPORTANT NOTES

`total` is computed from `chartData` (post-folding) rather than the raw `costBreakdown`, but since folding only regroups entries without changing the sum, the totals are equivalent. The "Other" slice label is a fixed string "Other" — if an actual AWS service were ever literally named "Other" it would visually merge with the folded bucket, though this is a purely theoretical edge case given AWS's real service names.

 `client/src/components/ErrorBanner.jsx`

PURPOSE

A shared inline error message banner (red-tinted box) used across nearly every page to display a fetch/action error, replacing what the file's comment says was previously a repeated `{error && <div>...}` pattern at each call site. Renders nothing when there is no message.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE	
message	string \	falsy	No	—	Error text; component renders <code>null</code> if falsy
className	string	No	<code>""</code>	Extra classes appended to the banner	

WHEN THIS FILE IS CALLED

Rendered near the top of essentially every page, right after `PageTransition` / `Navbar`: `Analytics.jsx`, `AuditLogs.jsx`, `ConnectAws.jsx`, `Dashboard.jsx`, `Login.jsx`, `NotificationSettings.jsx`, `Recommendations.jsx`, `ScanDetail.jsx`, `ScanHistory.jsx`, `ScheduleScan.jsx`, `Signup.jsx`. Typical usage combines two error sources, e.g. Dashboard's `<ErrorBanner message={error || scanError} className="mt-6" />`.

WHO CALLS THIS FILE

- `client/src/pages/Analytics.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ConnectAws.jsx`
- `client/src/pages/Dashboard.jsx`
- `client/src/pages/Login.jsx`
- `client/src/pages/NotificationSettings.jsx`
- `client/src/pages/Recommendations.jsx`
- `client/src/pages/ScanDetail.jsx`
- `client/src/pages/ScanHistory.jsx`
- `client/src/pages/ScheduleScan.jsx`
- `client/src/pages/Signup.jsx`

WHICH FILES IT DEPENDS ON

No imports at all — fully self-contained.

INTERNAL BEHAVIOR

- Guard clause: returns `null` immediately if `message` is falsy (so pages can pass `error || scanError` unconditionally without an extra wrapping check).
- Otherwise renders a `<div role="alert">` with red border/background (light mode) and corresponding dark-mode variants, containing the raw `message` text.

DATA FLOW

`message` is typically a string pulled from a page-level data-fetching hook's error state (e.g. `useAsyncData`'s `error`) or a scan-trigger hook's `scanError`; no callbacks — purely a display component.

EXAMPLE EXECUTION

`<ErrorBanner message="Failed to load dashboard data. Is the backend server running?" className="mt-6" />` renders a red alert box with that text; `<ErrorBanner message={null} />` renders nothing.

IMPORTANT NOTES

Uses `role="alert"` for screen-reader accessibility (announces the message assertively when it appears). The component's own comment notes it added dark-mode color variants that were "previously missing everywhere," implying this was a later consolidation of duplicated inline error markup.

 `client/src/components/FilterSelect.jsx`

PURPOSE

A shared table-toolbar dropdown: a native `<select>` with a leading `label` (used as `aria-label`, not visible text) and a trailing filter icon overlay. Used by every filterable table in the app (recommendations, scan history, audit logs) to keep filter-dropdown styling consistent.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
label	string	Yes	—	Used as the <code>aria-label</code> on the <code><select></code> (not rendered as visible text)
value	string	Yes	—	Currently selected option value (controlled)
onChange	function <code>(value: string) => void</code>	Yes	—	Called with the new raw string value on selection change
options	array of <code>{ value, label }</code>	Yes	—	Options rendered inside the <code><select></code>

WHEN THIS FILE IS CALLED

Rendered in table toolbars: `RecommendationTable.jsx` (e.g. status/severity filters), and directly in `AuditLogs.jsx` and `ScanHistory.jsx` (e.g. `<FilterSelect label="Status" value={statusFilter} onChange={...} />` in `ScanHistory.jsx`).

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ScanHistory.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `react-icons/fa6` (`FaFilter`).

INTERNAL BEHAVIOR

1. Renders a `relative` -positioned wrapper containing a native `<select>` (styled with `appearance-none` to hide the default OS arrow) and an absolutely-positioned `FaFilter` icon overlaid on the right (`pointer-events-none` so clicks pass through to the select).
2. `onChange` handler unwraps the native event to call the caller's `onChange(event.target.value)` with just the string value, not the event.
3. Maps `options` to `<option>` elements keyed by `option.value` .

DATA FLOW

Fully controlled component: `value` / `options` come from parent filter state; selecting a new option calls `onChange(newValue)` , and the parent is expected to update its own state (e.g. `statusFilter`) which flows back in as `value` .

EXAMPLE EXECUTION

`<FilterSelect label="Status" value={statusFilter} onChange={(v) => setStatusFilter(v)} options={[{value: "all", label: "All Statuses"}, {value: "OPEN", label: "Open"}] />` renders a dropdown; selecting "Open" calls `setStatusFilter("OPEN")` .

IMPORTANT NOTES

Since there's no visible `<label>` element, `aria-label={label}` is the only accessible name for the select — the `label` prop text itself is never shown in the UI, which is a common pattern for compact toolbar controls but relies entirely on the `aria-label` for accessibility.

PURPOSE

The large gradient "hero" banner at the top of the Dashboard page, showing the CloudLens title/tagline and four key stats (Last Scan, Total Resources, Monthly Cost, Estimated Savings) using `AnimatedCounter` for the numeric ones. Purely presentational — the file's comment notes the "Run Scan" action itself now lives in the global `Navbar`, so this component only displays stats.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE	
totalResources	number	No	—	Passed to an <code>AnimatedCounter</code> ; falsy values fall back to <code>0</code> at the call site (<code>'totalResources</code>	0')
monthlyCost	number	No	—	Same, formatted with <code>formatCurrency</code>	
estimatedSavings	number	No	—	Same, formatted with <code>formatCurrency</code>	
lastScan	object (Scan)	No	—	Passed to <code>formatScanTime</code> ; shows "No scans yet" if falsy	
loading	boolean	No	<code>false</code>	Shows <code>Skeleton</code> placeholders instead of real stat values	

WHEN THIS FILE IS CALLED

Rendered once on `client/src/pages/Dashboard.jsx` (line 105), always shown (not conditionally mounted), with `loading={isLoading}` toggling its internal skeleton state while dashboard data is being fetched.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `client/src/components/AnimatedCounter.jsx` — animates the Total Resources/Monthly Cost/Estimated Savings numbers
- `client/src/utils/format.js` — `formatCurrency`, `formatScanTime`

INTERNAL BEHAVIOR

- Defines a local `StatBlock` subcomponent (label + value) reused for all four stats.
- Renders a `motion.section` with a dark gradient background (`slate-900` via `indigo-950` to `slate-900`), fading/sliding in on mount (`y: -16 → 0`), plus two decorative blurred glow circles (`pointer-events-none` , absolutely positioned).
- Renders the "CloudLens" gradient-text `h1` and an uppercase tagline "AI Infrastructure Optimizer".
- Wraps all four `StatBlock` s in a `SkeletonTheme` (custom light-on-dark skeleton colors matching the dark hero background) and a flex-wrap row.
- Each `StatBlock` 's content is a ternary: `loading` → a `Skeleton` of a stat-appropriate size; else → the real value (`formatScanTime(lastScan)` or "No scans yet" for the first block; `AnimatedCounter` for the other three, with `format={formatCurrency}` on the cost/savings blocks).

DATA FLOW

All four stat props come directly from Dashboard's fetched `dashboardData` / `lastScan` state; `loading` mirrors Dashboard's combined loading flag. No callbacks — this is a read-only display section.

EXAMPLE EXECUTION

`<HeroSection totalResources={42} monthlyCost={1204.5} estimatedSavings={310.2} lastScan={scanObj} loading={false} />` renders the dark hero banner with "42" counting up under "Total Resources", "\$1204.50" and "\$310.20" counting up under the cost/savings labels, and a formatted scan timestamp under "Last Scan".

IMPORTANT NOTES

While `loading`, the skeleton widths are hardcoded per stat block (e.g. 130px for Last Scan, 40px for Total Resources) to roughly match expected content width, not measured dynamically. The "Run Scan" button intentionally does not live here despite this being the page's most prominent visual block — it's in `Navbar.jsx` instead, per the inline comment.

PURPOSE

A shared square, icon-only button used for the Navbar's theme toggle, notifications bell, avatar, AWS-connection indicator, and mobile hamburger menu. The file's comment says it replaced four near-identical inline buttons previously duplicated in Navbar.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
children	node	No	—	Icon (and optionally a notification dot <code>span</code>) rendered inside the button
onClick	function	No	—	Click handler
ariaLabel	string	No	—	Sets <code>aria-label</code> for accessibility
className	string	No	""	Extra classes appended to the base icon-button style
...rest	any	No	—	Spread onto the button (e.g. <code>aria-expanded</code> used by Navbar's dropdown triggers)

WHEN THIS FILE IS CALLED

Used exclusively within `Navbar.jsx` for: the dark-mode toggle, the notifications bell (with `aria-expanded={notifOpen}`), the user avatar menu trigger, the AWS-connection status trigger (only rendered when a connection exists), and the mobile hamburger/close toggle.

WHO CALLS THIS FILE

- `client/src/components/Navbar.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `framer-motion` (`motion`).

INTERNAL BEHAVIOR

- Defines a static `iconButtonClass` (9x9 rounded square, hover background, dark-mode variants, focus ring).
- Renders a `motion.button` with `whileHover={{scale: 1.08}}` / `whileTap={{scale: 0.92}}`, merging `iconButtonClass` with the caller's `className`, and spreading `...rest` (which Navbar uses to pass `aria-expanded`).

DATA FLOW

Purely a styled button wrapper; `onClick` is the only functional callback, fired by the parent (`Navbar`) to toggle its own local dropdown/menu state (e.g. `setNotifOpen`, `setAvatarOpen`).

EXAMPLE EXECUTION

```
<IconButton onClick={toggleTheme} ariaLabel="Toggle dark mode">{theme === "dark" ? <FaSun /> : <FaMoon />}</IconButton>
```

renders a square button that grows slightly on hover/shrinks on tap and calls `toggleTheme` on click.

IMPORTANT NOTES

`aria-label` is a distinct named prop (not part of `...rest`), while other ARIA attributes like `aria-expanded` are passed via the rest spread — a minor inconsistency in how ARIA props are threaded through, though both end up correctly on the DOM node.

PURPOSE

A small pill-shaped link (visually matching `CopyButton`) that opens an external URL — such as a deep link into the AWS Console — in a new tab. Used in the resource details drawer's console-actions row.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
href	string	Yes	—	External URL to open
icon	component (react-icons)	Yes	—	Icon rendered before the label

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
label	string	Yes	—	Link text

WHEN THIS FILE IS CALLED

Rendered in `ResourceDetailsDrawer.jsx` 's "AWS Console Actions" section for S3 resources, e.g. `<LinkButton href={getS3ConsoleUrl(resource.resourceId, resource.metadata?.region)} icon={FaArrowUpRightFromSquare} label="Open in AWS Console" />`, alongside similar links for metrics and permissions pages.

WHO CALLS THIS FILE

- `client/src/components/ResourceDetailsDrawer.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `framer-motion` (`motion`).

INTERNAL BEHAVIOR

1. Renders a `motion.a` with `target="_blank"` and `rel="noopener noreferrer"` (safe external-link defaults preventing `window.opener` access), styled identically to `CopyButton` 's pill (white background, slate border, hover/tap scale animation).
2. Renders `Icon` then `label` as children.

DATA FLOW

Purely a styled anchor tag; no internal state, no callbacks — clicking navigates the browser to `href` in a new tab, which is native browser behavior rather than a fired JS event.

EXAMPLE EXECUTION

`<LinkButton href="https://s3.console.aws.amazon.com/s3/buckets/my-bucket?region=us-east-1" icon={FaArrowUpRightFromSquare} label="Open in AWS Console" />` renders a pill link; clicking opens the AWS S3 console for that bucket in a new browser tab.

IMPORTANT NOTES

Visually near-identical to `CopyButton.jsx` (same padding, border, hover/tap animation classes) but semantically an anchor rather than a button — the two together read as one consistent "action row" per the file's own comment, despite being separate components with duplicated styling rather than a shared base.

 `client/src/components/Navbar.jsx`

PURPOSE

The global top navigation bar rendered on every authenticated page: brand/logo, primary nav links, dark-mode toggle, notifications dropdown, user-avatar dropdown, an AWS-connection status dropdown (with disconnect), a context-aware "Run Scan"/"Connect AWS" call-to-action button, a responsive mobile menu, and the scan-progress modal overlay.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
onScan	function	No	—	Click handler for the "Run Scan" button, invoked to start a scan
scanning	boolean	No	—	Disables/animates the Run Scan button and controls the <code>ScanProgressModal</code> 's <code>isOpen</code>
stepIndex	number	No	<code>0</code>	Forwarded to <code>ScanProgressModal</code> to indicate current scan step
scanPhase	string	No	<code>"idle"</code>	Forwarded to <code>ScanProgressModal</code> to indicate current scan phase

WHEN THIS FILE IS CALLED

Rendered at the top of nearly every authenticated page — `Analytics.jsx`, `AuditLogs.jsx`, `Dashboard.jsx`, `NotificationSettings.jsx`, `Recommendations.jsx`, `ScanDetail.jsx`, `ScanHistory.jsx`, `ScheduleScan.jsx` — each wiring its own `useScanTrigger` -derived `onScan` / `scanning` / `stepIndex` / `scanPhase`. `ScanDetail.jsx` notably renders it twice conditionally (an early-return error state with just `onScan` / `scanning`, and the full version with all four props in the main render path).

WHO CALLS THIS FILE

- `client/src/pages/Analytics.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/Dashboard.jsx`
- `client/src/pages/NotificationSettings.jsx`
- `client/src/pages/Recommendations.jsx`
- `client/src/pages/ScanDetail.jsx`
- `client/src/pages/ScanHistory.jsx`
- `client/src/pages/ScheduleScan.jsx`

WHICH FILES IT DEPENDS ON

- `client/src/components/IconButton.jsx` — all icon-only trigger buttons (theme, notifications, avatar, AWS, mobile menu)
- `client/src/components/ScanProgressModal.jsx` — overlay shown while `scanning` is true
- `client/src/hooks/useAwsConnection.js` — provides `connection`, `awsLoading`, `disconnecting`, `disconnect`
- `client/src/hooks/useClickOutside.js` — closes each of the three dropdowns (notifications, avatar, AWS) when clicking elsewhere
- `client/src/hooks/useTheme.js` — provides `theme` / `toggleTheme` for the dark-mode button

INTERNAL BEHAVIOR

1. Local state: `mobileOpen`, `notifOpen`, `avatarOpen`, `awsOpen` (all booleans, all default `false`).
2. Three refs (`notifRef`, `avatarRef`, `awsRef`) each wired to `useClickOutside` to close their respective dropdown on an outside click.
3. An effect closes the mobile menu whenever `location.pathname` changes (route navigation auto-collapses the mobile menu).
4. `handleLogout` removes the `"token"` key from `localStorage` and navigates to `/login`.
5. `navLinkClass(to, mobile)` computes active/inactive styling per link using `isPathActive` (exact match for `/`, `startsWith` for other routes).
6. `runScanButton(className)`: a `motion.button` disabled while `scanning`, with a continuously-rotating `FaBolt` icon (`animate={{rotate: 360}}`, `repeat: Infinity`) while scanning, and label toggling "Run Scan"/"Scanning..."
7. `connectAwsButton(className)`: a `MotionLink` (motion-wrapped router `Link`) to `/connect-aws`, shown instead of the scan button — per comment, whenever no AWS account is connected, since the scanner would reject a scan without one.
8. Renders brand/logo linking to `/`, a horizontal scrollable desktop nav (`NAV_LINKS`: Dashboard, Scan History, Recommendations, Analytics, Scheduled Scans, Notification Settings, Audit Logs), hidden below `lg` breakpoint.
9. Renders the dark-mode `IconButton` with an `AnimatePresence`-driven icon crossfade/rotate between `FaSun` / `FaMoon` keyed on `theme`.
10. Renders the notifications `IconButton` + dropdown; dropdown content is currently static placeholder text ("You're all caught up.") with no dynamic notification list.
11. Renders the avatar `IconButton` + dropdown with "Settings" (no `onClick` wired — a no-op button) and "Sign out" (`onClick={handleLogout}`).
12. Conditionally (`!awsLoading && connection`) renders an AWS-connection `IconButton` (cloud icon, green connected dot) + dropdown showing `connection.accountId` / `connection.region` and a "Disconnect" button wired to the hook's `disconnect`, itself showing a spinner + "Disconnecting..." while `disconnecting`.
13. Conditionally renders either `runScanButton` or `connectAwsButton` based on `connection` truthiness, once for desktop (`hidden lg:flex`) and once inside the mobile panel (`mt-2`), each time gated on `!awsLoading`.
14. Renders a mobile hamburger `IconButton` (`lg:hidden`) toggling `mobileOpen`, swapping `FaBars` / `FaXmark`.
15. `AnimatePresence`-wrapped mobile panel expands/collapses height when `mobileOpen`, listing all `NAV_LINKS` plus the scan/connect CTA.
16. Always renders `<ScanProgressModal isOpen={scanning} stepIndex={stepIndex} phase={scanPhase} />` at the end, regardless of dropdown states.

DATA FLOW

`onScan` is invoked by the Run Scan button's `onClick`; `scanning` / `stepIndex` / `scanPhase` flow straight through to `ScanProgressModal` and gate the Run Scan button's disabled/spinning state. `theme` / `connection` / etc. come from hooks, not props, making Navbar partially self-sufficient (it doesn't need auth/theme state passed down). No prop callback exists for the mobile-menu, dropdown, or logout state — those are entirely internal.

EXAMPLE EXECUTION

On `Dashboard.jsx`: `<Navbar onScan={runScan} scanning={scanning} stepIndex={stepIndex} scanPhase={scanPhase} />`. If the user has an AWS connection and clicks "Run Scan", `runScan` (from `useScanTrigger`) fires, `scanning` becomes `true`, the bolt icon starts spinning, the

button becomes disabled and reads "Scanning...", and `ScanProgressModal` opens showing progress via `stepIndex` / `scanPhase` .

IMPORTANT NOTES

The "Settings" item in the avatar dropdown has no `onClick` — it is currently a dead/no-op button. `ScanDetail.jsx` calls `<Navbar onScan={runScan} scanning={scanning} />` without `stepIndex` / `scanPhase` in its error-state early return, relying on their declared defaults (`stepIndex = 0` , `scanPhase = "idle"`). Logout clears only the `token` `localStorage` key, not any other client-side auth/user cache. The notifications dropdown UI exists but has no real data source wired — it always shows the same "You're all caught up." placeholder regardless of actual notification state.

 `client/src/components/NoResultsState.jsx`

PURPOSE

A shared "your filters produced zero rows" empty state for tables, distinct from a table having zero rows outright (each table renders that case separately since the copy/CTA differs per domain, per the file's comment). Shows a message and a "Clear Filters" button.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
message	string	No (no default; renders empty if omitted)	—	Explanation text (e.g. "No scans match your filters.")
onClear	function	No	—	Called when "Clear Filters" is clicked

WHEN THIS FILE IS CALLED

Rendered when a filtered table's result set is empty: in `RecommendationTable.jsx` , and directly in `AuditLogs.jsx` and `ScanHistory.jsx` , e.g. `filteredAndSorted.length === 0 ? <NoResultsState message="No scans match your filters." onClear={clearFilters} /> : ...` .

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ScanHistory.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `framer-motion` (`motion`).

INTERNAL BEHAVIOR

1. Renders a centered flex column with vertical padding (`py-16`), containing the `message` text and a `motion.button` labeled "Clear Filters" (static text, not a prop) with hover/tap scale animation.
2. The button's `onClick` is wired directly to the `onClear` prop.

DATA FLOW

Purely presentational aside from the one callback: `onClear` is invoked with no arguments when clicked, and the parent is expected to reset its own filter state (e.g. `clearFilters` resetting status/search filters back to defaults).

EXAMPLE EXECUTION

`<NoResultsState message="No scans match your filters." onClear={clearFilters} />` renders the message plus a "Clear Filters" button; clicking it calls `clearFilters()` , which the parent page uses to reset filter state, causing the table to repopulate.

IMPORTANT NOTES

The button label "Clear Filters" is hardcoded, not a prop — every caller gets identical button text regardless of domain, only the `message` text varies per table.

PURPOSE

Wraps a page's root element so route changes get a subtle fade+vertical-slide transition instead of an abrupt swap. Relies on an `AnimatePresence` in `App.jsx` keyed on `location.pathname` to trigger the `exit` animation on route change.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
children	node	No	—	Page content
className	string	No	—	Classes applied to the wrapping <code>motion.div</code>

WHEN THIS FILE IS CALLED

Wraps the root JSX of essentially every routed page: `Analytics.jsx`, `AuditLogs.jsx`, `ConnectAws.jsx`, `Dashboard.jsx`, `Login.jsx`, `NotificationSettings.jsx`, `Recommendations.jsx`, `ScanDetail.jsx`, `ScanHistory.jsx`, `ScheduleScan.jsx`, `Signup.jsx`. `Dashboard.jsx` and `ScanDetail.jsx` each use it twice — once for an early error-state return, once for the main render path — each with a different `className`.

WHO CALLS THIS FILE

- `client/src/pages/Analytics.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ConnectAws.jsx`
- `client/src/pages/Dashboard.jsx`
- `client/src/pages/Login.jsx`
- `client/src/pages/NotificationSettings.jsx`
- `client/src/pages/Recommendations.jsx`
- `client/src/pages/ScanDetail.jsx`
- `client/src/pages/ScanHistory.jsx`
- `client/src/pages/ScheduleScan.jsx`
- `client/src/pages/Signup.jsx`

WHICH FILES IT DEPENDS ON

No local file imports — depends only on external `framer-motion` (`motion`).

INTERNAL BEHAVIOR

- Defines `pageVariants` with three named states: `initial` (opacity 0, y 12), `animate` (opacity 1, y 0, 0.35s ease-out), `exit` (opacity 0, y -8, 0.2s ease-in).
- Renders a `motion.div` using these variants as `initial="initial" animate="animate" exit="exit"`, with `className` applied directly and `children` rendered inside.

DATA FLOW

Purely a layout/animation wrapper; no state, no callbacks. The `exit` animation only actually plays if an ancestor `AnimatePresence` (in `App.jsx`) is present and the component is being unmounted as part of a keyed route swap — using `PageTransition` outside that context still applies `initial` / `animate` but `exit` would not visibly run since nothing triggers unmount-time exit animations without `AnimatePresence`.

EXAMPLE EXECUTION

Navigating from `/dashboard` to `/recommendations` causes `App.jsx`'s `AnimatePresence` (keyed on pathname) to animate the old page's `PageTransition` out (fade + slide up, 0.2s) while the new page's `PageTransition` animates in (fade + slide down-to-rest, 0.35s).

IMPORTANT NOTES

The component itself does not import `AnimatePresence` — it only defines the variants; the actual mount/unmount transition orchestration is `App.jsx`'s responsibility, so `PageTransition`'s `exit` variant has no effect if used in isolation without that ancestor.

FRONTEND COMPONENTS (N-Z)

Reusable UI building blocks, alphabetically second half — including RecommendationTable, the single most important component in the app.

CloudLens Component Dossier — Batch F3 (N-Z reusable components)

Covers: ProtectedRoute.jsx, RecommendationTable.jsx, RecommendationTableSkeleton.jsx, ResourceDetailsDrawerSkeleton.jsx, ResourceDistributionBarChart.jsx, SavingsOpportunityBarChart.jsx, ScanProgressModal.jsx, ServiceOverviewGrid.jsx, SortIcon.jsx, SortableTh.jsx, SummaryCard.jsx, TablePagination.jsx, TableSkeleton.jsx, ToastCard.jsx, ToggleSwitch.jsx, TrendChart.jsx.

All paths are relative to the CloudLens repo root (`c:\Users\AKSHAT JOHRI\OneDrive\Desktop\CloudLens`).

 `client/src/components/ProtectedRoute.jsx`

PURPOSE

Route guard for react-router-dom that gates the entire authenticated app shell behind a login check. It is used as the `element` of a parent layout `<Route>` in `App.jsx`; it renders `<Outlet />` (letting nested authenticated pages render) if a JWT token exists in `localStorage`, otherwise redirects to `/login`.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<i>(none)</i>	—	—	—	Component takes no props; it is rendered directly as a route element (<code><Route element={}<ProtectedRoute /></code>) and reads all state from <code>localStorage</code>.</code>

WHEN THIS FILE IS CALLED

Rendered exactly once in `client/src/App.jsx`, as the `element` of a parent `<Route>` that wraps every authenticated page route: `/`, `/scan-history`, `/scan-history/:scanId`, `/recommendations`, `/connect-aws`, `/schedules`, `/notifications`, `/analytics`, `/audit-logs`. It re-evaluates on every navigation that matches within that route subtree.

WHO CALLS THIS FILE

- `client/src/App.jsx`

WHICH FILES IT DEPENDS ON

- `react-router-dom` (external) — `Navigate` for the redirect, `Outlet` for the nested-route mount point. No internal component/util imports (dep_graph shows `imports: []`).

INTERNAL BEHAVIOR

- Reads `const token = localStorage.getItem("token")` synchronously on every render — no `useState` / `useEffect`, so it re-reads fresh each time the component function runs.
- Ternary: `token ? <Outlet /> : <Navigate to="/login" replace />`.
- If a token is present, `<Outlet />` renders, allowing React Router to mount whichever nested authenticated route matched the URL.
- If no token, `<Navigate to="/login" replace />` performs a client-side redirect and replaces the current history entry (so the back button does not return to the guarded page).

DATA FLOW

No data fetching or context involved. The single source of truth is the synchronous presence/absence of the `"token"` key in `localStorage`, checked fresh on every render/route change.

EXAMPLE EXECUTION

Navigating to `/scan-history` with no `token` in `localStorage` → component renders `<Navigate to="/login" replace/>` → browser URL becomes `/login`; `ScanHistory` never mounts. With a token present, `<Outlet/>` renders and React Router mounts `<ScanHistory />` in its place.

IMPORTANT NOTES

- This is a presence check only, not a validity/expiry check — `token ? ... : ...` passes for any truthy string, including an expired or malformed JWT. Real auth enforcement (e.g., handling 401s) must happen elsewhere (not in this file).
- No loading state exists; the check assumes `localStorage.getItem` is synchronous (true in browsers) so there's no flash-of-unauthenticated-content, but also no guard if `localStorage` access were to throw.
- Used exactly once in the whole app as a layout-route element, never rendered directly by any page component.

 `client/src/components/RecommendationTable.jsx`

PURPOSE

The primary interactive data table for AWS cost/security recommendations. It renders a search box, three filter dropdowns (severity/status/resource type), a sortable and paginated table with inline Stop/Ignore/Resolve row actions, and opens a `ResourceDetailsDrawer` when a row is clicked. It is reused unmodified across the Dashboard, Recommendations, and ScanDetail pages, driven entirely by a `recommendations` array prop.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>recommendations</code>	array	No	<code>[]</code>	Recommendation objects to display; expected fields include <code>_id</code> , <code>resourceId</code> , <code>resourceName</code> , <code>resourceType</code> , <code>instanceType</code> , <code>category</code> , <code>severity</code> , <code>action</code> , <code>state</code> , <code>status</code> , <code>estimatedSavings</code>
<code>onRecommendationUpdate</code>	function	Yes (used unguarded)	—	Called with the updated recommendation object after any successful action, so the parent page can patch its own list
<code>refresh</code>	function	No	—	Passed through to <code>useRecommendationActions</code> ; invoked only after a successful Stop action, letting the parent re-fetch dependent data
<code>loading</code>	boolean	No	<code>false</code>	Shows <code><RecommendationTableSkeleton /></code> instead of the toolbar/table; also forwarded as-is to <code><ResourceDetailsDrawer loading></code>

WHEN THIS FILE IS CALLED

Rendered on three pages: `Dashboard.jsx` (`recommendations={recommendations}`, `refresh={refresh}`, `loading={isLoading}`), `Recommendations.jsx` (same shape), and `ScanDetail.jsx` (same shape, scoped to one scan's recommendations). Always mounted unconditionally by these pages (not behind extra conditionals in the parent beyond passing `loading`).

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`
- `client/src/pages/Recommendations.jsx`
- `client/src/pages/ScanDetail.jsx`

WHICH FILES IT DEPENDS ON

- `./ActionButton.jsx` — pill-shaped Stop/Ignore/Resolve buttons with a loading spinner state
- `./Badge.jsx` — soft pill badges for category/severity/status
- `./Card.jsx` — glass card shell + entrance animation wrapper
- `./FilterSelect.jsx` — the three toolbar dropdowns (Severity/Status/Resource Type)
- `./NoResultsState.jsx` — "no rows match filters" empty state with a Clear Filters button
- `./RecommendationTableSkeleton.jsx` — loading placeholder
- `./ResourceDetailsDrawer.jsx` — the slide-in detail panel opened on row click (documented separately, out of scope here)
- `./SortableTh.jsx` — sortable `<th>` cells
- `./TablePagination.jsx` — pager control

- `../hooks/useRecommendationActions.js` — shared Ignore/Resolve/Stop API logic + loading-id state
- `../utils/badge.js` — `getSeverityBadgeStyle`, `getRecommendationStatusBadgeStyle`, `getCategoryBadgeStyle`
- `../utils/format.js` — `formatCurrency`, `formatActionLabel`
- `../utils/sort.js` — `compareValues` generic comparator
- External: `framer-motion` (row/container animation), `react` (`useMemo`, `useState`), `react-icons/fa6` (search/action icons)

INTERNAL BEHAVIOR

1. Local state: `selectedResource` (drawer target, `null` = closed), `search` (string), `severityFilter` / `statusFilter` / `resourceTypeFilter` (each default `"ALL"`), `sortConfig` (`null` | `{key, direction}`), `currentPage` (default `1`).
2. `resourceTypeOptions` — `useMemo` building a de-duplicated, sorted list of every `item.resourceType` present in `recommendations`, used to populate the Resource Type dropdown's options dynamically (prefixed with a synthetic `"ALL"` option at render time).
3. `filteredAndSorted` — `useMemo` that filters `recommendations` by severity/status (`item.status` || `"OPEN"` when absent)/resourceType/search (`matchesSearch` does a case-insensitive substring match against `` ${resourceName} ${resourceId} ``), then, if `sortConfig` is set, sorts a shallow copy using `compareValues(getSortValue(a,key), getSortValue(b,key))` from `utils/sort.js`, negating the comparison when `direction === "desc"`.
4. `getSortValue(item, key)` normalizes values per column: `resourceName` / `instanceType` / `category` → lowercased strings; `severity` → mapped through module-level `SEVERITY_RANK = { HIGH: 3, MEDIUM: 2, LOW: 1 }` (unknown/missing → `0`); `estimatedSavings` → numeric, default `0`; `status` → lowercased, default `"OPEN"`.
5. Pagination: `PAGE_SIZE = 8` (module constant, not configurable via props). `totalPages = Math.max(1, Math.ceil(filteredAndSorted.length / PAGE_SIZE))`. `safePage = Math.min(currentPage, totalPages)` guards against a stranded page after filtering shrinks the result set. `paginated` is the slice for `safePage`.
6. `handleSort(sortKey)` — three-state cycle: no sort or a different key → ascending; ascending on the same key → descending; descending on the same key → `null` (clears sort). Always resets `currentPage` to `1`.
7. Every filter/search `onChange` (search input, all three `FilterSelect` s) also resets `currentPage` to `1` inline.
8. `clearFilters()` resets `search` / `severityFilter` / `statusFilter` / `resourceTypeFilter` / `currentPage` together; wired to `NoResultsState`'s `onClear`.
9. Early return: `if (!loading && !recommendations.length)` renders a minimal `Card` with just a static "No recommendations yet. Run a scan to analyze your infrastructure." message — no toolbar, table, or `ResourceDetailsDrawer` is rendered in this branch at all.
10. Main render: heading, then `loading ? <RecommendationTableSkeleton/> : (...)` — the `(...)` branch is the toolbar (search input with `FaMagnifyingGlass` + 3 `FilterSelect` s) followed by either `<NoResultsState/>` (when `filteredAndSorted.length === 0`) or the actual table.
11. Table `<thead>` maps a module-level `COLUMNS` array (7 entries: resource, instanceType, category, severity, actions, savings, status — `actions` has `sortKey: null`) through `<SortableTh>`. `<tbody>` is wrapped in `<AnimatePresence initial={false}>` with `rowVariants` staggering entrance (delay = `Math.min(index, 8) * 0.03` s) and a simple fade on exit.
12. Row click (`onClick={() => setSelectedResource(item)}`) on the whole `<motion.tr>` opens the drawer; the Actions `<td>` calls `event.stopPropagation()` so clicking Stop/Ignore/Resolve does not also open the drawer.
13. Actions cell: only rendered (as buttons) when `item.status === "OPEN"`, else a static `—`. Within OPEN rows:
 - Stop button appears only when `item.action === "STOP"` and `item.state?.toLowerCase() !== "stopped"` (otherwise shows static text "Already stopped"); wired to `handleStop(item._id)`, loading flag `stoppingId === item._id`.
 - Ignore button always shown for OPEN rows, wired to `handleIgnore(item._id)`, loading flag `actionId === item._id`.
 - Resolve button always shown for OPEN rows, wired to `handleResolve(item._id)`, loading flag `actionId === item._id` (same state variable as Ignore).
1. `handleRecommendationUpdate(updated)` — the callback actually passed into `useRecommendationActions`. It calls the parent's `onRecommendationUpdate(updated)` prop, and additionally, if `updated._id` matches the currently open drawer's resource `_id`, replaces `selectedResource` with the fresh copy so the open drawer's badges/buttons don't go stale after an action.
2. `const { stoppingId, actionId, handleIgnore, handleResolve, handleStop } = useRecommendationActions(handleRecommendationUpdate, refresh)` — same handlers are reused by both the inline row buttons and the `ResourceDetailsDrawer`'s footer buttons.
3. `<TablePagination>` rendered below the table with `page={safePage}`, `totalPages`, `pageSize={PAGE_SIZE}`, `totalItems={filteredAndSorted.length}`, `onPageChange={setCurrentPage}`.
4. `<ResourceDetailsDrawer>` is always mounted (outside the loading/empty conditionals, at the bottom of the `Card`) with `resource={selectedResource}` (`null` when closed), `onClose`, `onIgnore` / `onResolve` / `onStop` wired to the same shared handlers, `actionLoadingId={actionId}`, `stoppingId={stoppingId}`, `loading={loading}` (the table's own `loading` prop, reused verbatim, not a drawer-specific flag).

DATA FLOW

The parent page owns the `recommendations` array (fetched once via a service call + `useAsyncData`) and passes it down along with `onRecommendationUpdate` / `refresh` callbacks that patch that same array/refetch dependent data. `RecommendationTable` performs all filtering, sorting, search, and pagination entirely client-side over whatever array it's given — there is no server-side filtering/pagination. Row actions call `useRecommendationActions`, which hits `recommendation.service.js` endpoints directly and, on success, hands the updated object back up through `onRecommendationUpdate` and (for Stop only) triggers `refresh`.

EXAMPLE EXECUTION

Given `recommendations=[{_id:"r1", resourceId:"i-0abc", resourceName:"web-server-1", resourceType:"EC2", instanceType:"t3.large", category:"COST", severity:"HIGH", action:"STOP", state:"running", status:"OPEN", estimatedSavings:42.50}]`, `loading=false`: renders the Card heading, toolbar (Resource Type select offers "All Types" + "EC2"), and one row — "web-server-1 / EC2 · i-0abc", "t3.large", COST badge, HIGH badge + "Stop Instance" caption (from `formatActionLabel`), Stop/Ignore/Resolve buttons (Stop visible since state is "running"), "\$42.50", OPEN badge. Clicking the row (outside the Actions cell) opens the drawer for `r1`. Clicking "Resolve" calls `handleResolve("r1")` → `resolveRecommendation("r1")` → on success, `handleRecommendationUpdate(updatedItem)` propagates to the parent and (if open) the drawer, and a success toast "Recommendation resolved." appears.

IMPORTANT NOTES

- Resolve and Ignore share the same `actionId` loading-state variable, so while one is in flight for a row (`actionId === item._id`), **both** Resolve and Ignore buttons on that row render in their loading/disabled state simultaneously — noted as observed behavior, not confirmed as intentional.
- `refresh` is invoked only after a successful **Stop** action (per `useRecommendationActions.handleStop`'s `onSuccess` argument); Ignore/Resolve never call `refresh`, only `onRecommendationUpdate`.
- The `!loading && !recommendations.length` early return skips rendering `<ResourceDetailsDrawer>` entirely — the drawer only exists once there is at least one recommendation or `loading` is true.
- `PAGE_SIZE` (8) is a module-level constant, not customizable via props.
- `COLUMNS` is a fixed 7-column definition; "Actions" intentionally has `sortKey: null`, making `SortableTh` render it as plain unsortable header text.
- All filtering/sorting/pagination is purely client-side over whatever `recommendations` array is passed in.

`client/src/components/RecommendationTableSkeleton.jsx`

PURPOSE

Static loading placeholder that approximates `RecommendationTable`'s toolbar + table shape (search box, 3 filter pills, a 6-row table) so the layout doesn't jump when real data arrives. Purely presentational — no props, no state.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<i>(none)</i>	—	—	—	Component takes no props.

WHEN THIS FILE IS CALLED

Rendered by `RecommendationTable.jsx` in place of its toolbar+table+ `NoResultsState` block whenever the `loading` prop is `true` (the `Card` wrapper and heading are still rendered by `RecommendationTable` itself, outside this component).

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`

WHICH FILES IT DEPENDS ON

- `react-loading-skeleton` (external) — `Skeleton` primitive. No internal imports (dep_graph shows `imports: []`).

INTERNAL BEHAVIOR

- Module constants: `ROWS = 6` (skeleton row count) and `COLUMN_LABELS = ["Resource", "Instance Type", "Severity", "Actions", "Savings", "Status"]` — **6 labels**, one fewer than `RecommendationTable`'s real `COLUMNS` array (7, which also includes "Category").
- Renders a static toolbar row: one wide `Skeleton` (height 42, mimicking the search box) plus three 140px-wide `Skeleton`s (mimicking the filter dropdowns).

- 3. Renders a `<table>` whose `<thead>` maps `COLUMN_LABELS` to plain `<th>` cells, and whose `<tbody>` generates `ROWS` (6) rows, each with hand-matched cell shapes: a two-line skeleton for Resource, a single-line skeleton for Instance Type, a pill skeleton for Severity, two small pill skeletons for Actions, a line for Savings, and a pill for Status.

DATA FLOW

None — entirely static markup with no props, state, or data dependencies.

EXAMPLE EXECUTION

`<RecommendationTableSkeleton />` always renders the identical 6-column × 6-row skeleton table with a toolbar above it, regardless of context.

IMPORTANT NOTES

- `COLUMN_LABELS` has 6 entries while `RecommendationTable` 's live `COLUMNS` constant has 7 (the real table also renders a "Category" column). The skeleton's header and rows are hand-matched to the 6-column set (no Category placeholder) — a structural mismatch from the real table, though visually plausible as an approximation.
- Per its own code comment, it is "not styled for dark mode, same as the table itself."

 `client/src/components/ResourceDetailsDrawerSkeleton.jsx`

PURPOSE

Loading placeholder mirroring `ResourceDetailsDrawer` 's real internal layout — header block, badge row, a 3-tile stat grid, a recommendation detail panel, a 7-row detail list, and a 3-button footer row.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<i>(none)</i>	—	—	—	Component takes no props.

WHEN THIS FILE IS CALLED

Imported by `client/src/components/ResourceDetailsDrawer.jsx` (out of scope for this dossier). Per its own code comment, it exists for "if a `loading` resource is ever passed in — e.g. a future async resource-detail fetch" — i.e., today `ResourceDetailsDrawer` receives its `resource` synchronously as a prop (already present in the parent's `recommendations` array), so this skeleton appears to be provisioned ahead of a loading path that may not be fully wired up yet.

WHO CALLS THIS FILE

- `client/src/components/ResourceDetailsDrawer.jsx`

WHICH FILES IT DEPENDS ON

- `react-loading-skeleton` (external) — `Skeleton` primitive only.

INTERNAL BEHAVIOR

1. Renders a flex-column, full-height layout matching the drawer's structure: header (label + title + subtitle skeleton lines), a scrollable body, and a footer.
2. Body: two badge-pill skeletons; a 3-column grid of 3 bordered stat tiles (each with a label skeleton + value skeleton); a bordered panel with a title skeleton, a 2-line skeleton, then a second title + line pair (mirroring a description/recommendation text block); and a bordered list of 7 rows, each with a label skeleton on the left and a value skeleton on the right (mirroring detail key/value rows).
3. Footer: 3 button-shaped skeletons (mirroring Stop/Ignore/Resolve).

DATA FLOW

None — static markup with no props/state.

EXAMPLE EXECUTION

`<ResourceDetailsDrawerSkeleton />` renders the same fixed skeleton shape every time; there is nothing to vary the output.

IMPORTANT NOTES

Per its own code comment, this exists for a hypothetical future "loading resource" state that does not appear to be wired into the drawer's current synchronous prop-drilling data flow — its actual trigger condition lives in `ResourceDetailsDrawer.jsx`, which is out of scope for this dossier and was not read in full to confirm whether/when this skeleton is actually shown.

client/src/components/ResourceDistributionBarChart.jsx

PURPOSE

Recharts vertical bar chart on the Dashboard showing resource count per AWS service (EC2/S3/RDS/IAM/etc.), one bar per service present in the latest scan. Per its code comment, it replaces an older EC2-only Running-vs-Stopped chart.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
resourceDistribution	array	No	[]	{service, count} entries from GET /api/dashboard 's resourceDistribution field
index	number	No	undefined	Forwarded to ChartCard for staggered entrance-animation delay
loading	boolean	No	false	Shows <ChartSkeleton variant="bar"/> instead of the chart

WHEN THIS FILE IS CALLED

Rendered once on `Dashboard.jsx` : `<ResourceDistributionBarChart resourceDistribution={dashboardData.resourceDistribution} index={2} loading={isLoading} />` , positioned alongside `SavingsOpportunityBarChart` in a shared grid row.

WHO CALLS THIS FILE

- client/src/pages/Dashboard.jsx

WHICH FILES IT DEPENDS ON

- ./ChartCard.jsx — title/subtitle + entrance-animation card wrapper
- ./ChartEmptyState.jsx — "no data" placeholder
- ./ChartSkeleton.jsx — loading placeholder (variant="bar")
- ./ChartTooltip.jsx — custom Recharts tooltip content
- ../utils/chartColors.js — CATEGORICAL_PALETTE (8-color array cycled by index)
- External: recharts (ResponsiveContainer , BarChart , Bar , XAxis , YAxis , CartesianGrid , Tooltip , Legend , Cell)

INTERNAL BEHAVIOR

- chartData maps resourceDistribution to {name: entry.service, count: entry.count} .
- legendPayload is built manually: because a single <Bar> with per- <Cell> fill colors doesn't auto-generate distinct Recharts legend entries, one legend entry per data point is constructed explicitly as {value: name, type: "circle", color: CATEGORICAL_PALETTE[i % CATEGORICAL_PALETTE.length]} .
- Render branches: loading → <ChartSkeleton variant="bar"/> ; else !chartData.length → <ChartEmptyState message="No scanned resources yet. Run a scan to see resource distribution."/> ; else the real BarChart with CartesianGrid (horizontal lines only), XAxis keyed on "name", YAxis with allowDecimals={false} (integer-only ticks), a custom Tooltip (ChartTooltip , slate-tinted cursor), an explicit Legend fed legendPayload , and a single <Bar dataKey="count"> containing one <Cell> per data point colored via CATEGORICAL_PALETTE[i % 8] .
- Bar entrance animation is Recharts' own (isAnimationActive , 800ms, ease-out) — independent of Framer Motion.

DATA FLOW

`Dashboard.jsx` fetches the whole dashboard payload once (via `useAsyncData / getDashboard`) and passes the pre-aggregated `resourceDistribution` slice straight through; this component performs no fetching, filtering, or aggregation — pure presentation of already-aggregated backend data.

EXAMPLE EXECUTION

`resourceDistribution=[{service:"EC2",count:12},{service:"S3",count:5},{service:"RDS",count:2}]` → renders 3 bars labeled EC2/S3/RDS colored blue/emerald/amber (first three `CATEGORICAL_PALETTE` entries), with a matching-colored legend beneath the chart and a

"Resources per service" subtitle.

IMPORTANT NOTES

- Colors are assigned purely by the array index/order of `resourceDistribution`, not a fixed per-service color map — if the backend ever reorders services between scans, the same service could render in a different color across renders.
- The color-cycling expression (`CATEGORICAL_PALETTE[i % CATEGORICAL_PALETTE.length]`) is computed twice — once building `legendPayload`, once per `<Cell>` — rather than derived from one shared array.

`client/src/components/SavingsOpportunityBarChart.jsx`

PURPOSE

Recharts horizontal bar chart on the Dashboard listing top savings opportunities by resource across every AWS service (not EC2-specific), assuming data is already sorted/capped server-side. Container height scales with the number of entries.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>savingsOpportunities</code>	array	No	<code>[]</code>	<code>{resource, savings}</code> entries, pre-sorted/capped by the backend
<code>index</code>	number	No	<code>undefined</code>	Forwarded to <code>ChartCard</code> for stagger delay
<code>loading</code>	boolean	No	<code>false</code>	Shows <code><ChartSkeleton variant="horizontal-bar"/></code>

WHEN THIS FILE IS CALLED

Rendered once on `Dashboard.jsx`: `<SavingsOpportunityBarChart savingsOpportunities={dashboardData.savingsOpportunities} index={3} loading={isLoading} />`.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `./ChartCard.jsx`, `./ChartEmptyState.jsx`, `./ChartSkeleton.jsx` (`variant="horizontal-bar"`), `./ChartTooltip.jsx`
- `../utils/chartColors.js` — `CHART_COLORS.emerald` (single fixed fill color, not the categorical palette)
- `../utils/format.js` — `formatCurrency`
- External: `recharts` (`ResponsiveContainer`, `BarChart`, `Bar`, `XAxis`, `YAxis`, `CartesianGrid`, `Tooltip`, `Legend`, `LabelList`)

INTERNAL BEHAVIOR

- `chartData` maps `savingsOpportunities` to `{name: entry.resource, savings: entry.savings}`.
- `height = Math.max(MIN_HEIGHT=240, chartData.length * ROW_HEIGHT=48)` — the chart container grows taller as more opportunities exist, so bars don't get squeezed at any row count.
- `loading` → `<ChartSkeleton variant="horizontal-bar"/>`; `!chartData.length` → `<ChartEmptyState message="No savings opportunities found – your infrastructure looks well optimized."/>`; else a `BarChart` with `layout="vertical"` (bars extend horizontally): numeric `XAxis` with `tickFormatter={formatCurrency}`, categorical `YAxis` (resource names, fixed `width={140}`), `Tooltip` via `ChartTooltip` with a `formatter` that runs values through `formatCurrency`, a single `<Bar dataKey="savings">` filled with the fixed `CHART_COLORS.emerald`, right-rounded only (`radius={[0,10,10,0]}`), plus a `<LabelList>` printing the formatted dollar value at the end of each bar.
- 800ms `ease-out` Recharts bar entrance animation.

DATA FLOW

Purely presentational over an array assumed pre-sorted and pre-capped by the backend (per its own code comment: "already service-agnostic, already sorted highest-savings-first ... and already capped ... so this component just renders what it's given"); no client-side sorting/filtering/limiting occurs here.

EXAMPLE EXECUTION

`savingsOpportunities=[{resource:"i-0abc (web-server-1)", savings:120.00},{resource:"my-bucket", savings:45.50}]` → `height = max(240, 2*48) = 240`; renders two horizontal bars (the \$120.00 bar longer, assuming pre-sorted descending order), each right-labeled with its dollar amount, X-axis ticks formatted as currency.

IMPORTANT NOTES

- Unlike `ResourceDistributionBarChart`, this chart uses one fixed color (emerald) for every bar rather than a rotating palette — appropriate since all bars represent the same metric (savings).
- Relies entirely on the backend for sort order and any capping of the list; a long or unsorted array would render as-is, with the container growing unbounded taller for large arrays and no max-height/scroll safeguard.

`client/src/components/ScanProgressModal.jsx`

PURPOSE

Full-screen modal overlay showing live progress of an AWS infrastructure scan as a checklist of steps (Fetch EC2 → Fetch S3 → Fetch Pricing → Analyze → Save Database). Portalled to `document.body` so it renders above the (backdrop-blurred) Navbar rather than being visually clipped/mispositioned by it.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>isOpen</code>	boolean	Yes	—	Controls <code>AnimatePresence</code> -driven mount/unmount of the overlay + modal
<code>stepIndex</code>	number	Yes	—	Index into <code>SCAN_STEPS</code> marking the current/most-recently-completed step
<code>phase</code>	string	Yes	—	<code>"success"</code> \ <code>"error"</code> \ anything else treated as "running"; drives icon, heading text, and progress-bar color

WHEN THIS FILE IS CALLED

Rendered once inside `Navbar.jsx`: `<ScanProgressModal isOpen={scanning} stepIndex={stepIndex} phase={scanPhase} />`. Since `Navbar` is mounted on every authenticated page, this modal can appear regardless of which page triggered the scan, driven by `Navbar`'s own `useScanTrigger` hook state.

WHO CALLS THIS FILE

- `client/src/components/Navbar.jsx`

WHICH FILES IT DEPENDS ON

- `../constants/scanSteps.js` — `SCAN_STEPS`, a 5-entry array (`ec2`, `s3`, `pricing`, `analyze`, `save`) with `key` / `label` pairs
- External: `react-dom` (`createPortal`), `framer-motion` (`AnimatePresence`, `motion`), `react-icons/fa6` (`FaCircleCheck`, `FaCircleExclamation`, `FaSpinner`)

INTERNAL BEHAVIOR

- Local `StepRow` subcomponent renders one checklist row: icon reflects `state` (`"done"` = green check, `"active"` = spinning blue spinner, `"error"` = red exclamation, `"pending"` = small gray dot) plus the step's label, dimmed gray when pending.
- `getStepState(index, stepIndex, phase)` (local pure function): `phase === "error" && index === stepIndex → "error"`; `phase === "success" || index < stepIndex → "done"`; `index === stepIndex → "active"`; else `"pending"`. Every row's visual state derives from this single global `stepIndex` / `phase` pair.
- `progressPercent`: `100` if `phase === "success"`; otherwise `((stepIndex + (phase === "error" ? 0.5 : 1)) / SCAN_STEPS.length) * 100` — the active step counts as fully complete unless it errored, in which case it counts as half-complete.
- The entire modal (dim overlay + centered dialog) is wrapped in `createPortal(..., document.body)`. Code comment explains why: `Navbar` has `backdrop-blur-xl`, and `backdrop-filter` on an ancestor creates a new CSS containing block for `position: fixed` descendants — without the portal the modal was positioned relative to the (short) `Navbar` instead of the viewport, pushing it mostly off-screen.
- `AnimatePresence` conditionally renders the overlay (0.2s fade) and modal (spring scale/translate entrance, reversed on exit) only while `isOpen` is true.
- Header icon/heading switches on `phase`: `"error"` → red exclamation + "Scan Failed"; `"success"` → green check + "Scan Complete"; otherwise → spinning blue spinner + "Scanning Infrastructure...".
- Below the header, maps `SCAN_STEPS` to `<StepRow>` using `getStepState` per row.

8. Bottom `role="progressbar"` element (with `aria-valuenow` rounded from `progressPercent`) has an animated-width `motion.div` fill — red on error, blue-to-indigo gradient otherwise.

DATA FLOW

`Navbar`'s `useScanTrigger` hook (out of scope) manages the scan lifecycle — starting the scan API call and stepping through `SCAN_STEPS` on a timer, since the backend scan is a single opaque request with no incremental-status endpoint (per the file's own comment) — and feeds `isOpen` / `stepIndex` / `phase` down as props. This component has no internal state, effects, or data fetching of its own; it is a purely controlled presentational view.

EXAMPLE EXECUTION

`isOpen=true, stepIndex=2, phase="running"` → `progressPercent = ((2+1)/5)*100 = 60` ; steps 0 ("Fetch EC2") and 1 ("Fetch S3") show green checks ("done"), step 2 ("Fetch Pricing") shows the blue spinner ("active"), steps 3–4 show gray dots ("pending"); header shows the blue spinner + "Scanning Infrastructure..."; progress bar animates to 60% width in a blue-to-indigo gradient.

IMPORTANT NOTES

- Step pacing is frontend-illustrative, not a real per-step event stream from the backend (per the file's own comment) — actual timing logic lives entirely in the out-of-scope `useScanTrigger` hook, not in this component.
- The portal-to-`document.body` is a deliberate fix for a specific `backdrop-filter` / `position:fixed` containing-block interaction with `Navbar` ; removing it would silently reintroduce the mispositioning bug.
- z-index layering: overlay is `z-[60]` , modal dialog is `z-[70]` , both above `Navbar`'s own stacking context.

 `client/src/components/ServiceOverviewGrid.jsx`

PURPOSE

Dashboard grid of per-AWS-service summary cards (EC2/S3/RDS/IAM, plus a generic fallback for any future service), each showing resource count, recommendation count, and estimated savings (when nonzero) with a service-specific icon and gradient accent.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>services</code>	array	No	<code>[]</code>	<code>{service, resourceCount, recommendationCount, estimatedSavings}</code> entries
<code>loading</code>	boolean	No	<code>false</code>	Renders 4 skeleton cards instead of real data

WHEN THIS FILE IS CALLED

Rendered once on `Dashboard.jsx` : `<ServiceOverviewGrid services={dashboardData.serviceSummary} loading={isLoading} />` , positioned above the `SummaryCard` KPI grid.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `./Card.jsx` — glass card shell + motion passthrough
- `../utils/format.js` — `formatCurrency`
- External: `react-icons/fa6` (`FaServer` , `FaBucket` , `FaDatabase` , `FaUserShield` , `FaCloud`), `react-loading-skeleton` (`Skeleton`)

INTERNAL BEHAVIOR

- `SERVICE_META` lookup keyed by service code: `EC2` → `FaServer` /"Resources"/blue-indigo gradient; `S3` → `FaBucket` /"Buckets"/emerald-teal; `RDS` → `FaDatabase` /"Instances"/indigo-purple; `IAM` → `FaUserShield` /"Users/Roles"/amber-orange.
- `DEFAULT_META` (`FaCloud` /"Resources"/slate gradient), returned by `getServiceMeta()` for any service code not in `SERVICE_META` — per the code comment, this means a future scanner (Lambda, EBS, CloudFront, DynamoDB, ...) renders a correct card automatically with no change to this file.
- `loading` branch: always renders exactly 4 `<Card>` skeletons in the grid, regardless of the real eventual service count.
- Empty branch (`!services.length` , checked only when not loading): a single centered `Card` reading "No scanned resources yet. Run a scan to see resources by service."

- Otherwise maps `services` to one `<Card>` per entry: a blurred gradient accent circle in the corner, icon + service-name header, then a `<dl>` of `resourceCount` (label from `meta.resourceLabel`), `recommendationCount`, and — only `if (entry.estimatedSavings > 0)` — a Savings row in emerald text via `formatCurrency`.
- Entrance animation: `cardVariants` staggers each card in via `custom={index}` (0.05s × index delay, opacity + y fade); `whileHover` lifts (`y:-6`, `scale:1.02`) via a spring transition.

DATA FLOW

Consumes Dashboard's already-aggregated `serviceSummary` array from the single dashboard API response; this component performs no fetching, sorting, or computation beyond the icon/gradient/label lookup.

EXAMPLE EXECUTION

`services=[{service:"EC2", resourceCount:12, recommendationCount:3, estimatedSavings:84.20}]` → one card: blue-indigo gradient accent, `FaServer` icon, "EC2" heading, "Resources: 12", "Recommendations: 3", "Savings: \$84.20" (emerald). `services=[{service:"Lambda", resourceCount:4, recommendationCount:0, estimatedSavings:0}]` → renders via `DEFAULT_META` (`FaCloud`, slate gradient, "Resources" label) since "Lambda" isn't in `SERVICE_META`; the Savings row is omitted since `estimatedSavings` is `0`.

IMPORTANT NOTES

- The loading skeleton always renders exactly 4 cards regardless of how many services will eventually load — currently matches the app's 4 known services, but is not derived from any real count.
- The Savings row is hidden entirely (not shown as `$0.00`) when `estimatedSavings` is falsy — a strict `> 0` check, not just truthiness.



```
client/src/components/SortIcon.jsx
```

PURPOSE

Tiny shared glyph indicating a table column's sort state: a neutral icon when the column is not the active sort key, an up/down arrow when it is. Used exclusively inside `SortableTh`.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>active</code>	boolean	Yes	—	Whether this column is the currently active sort column
<code>direction</code>	string	No	<code>undefined</code>	<code>"asc"</code> or <code>"desc"</code> ; determines up vs. down arrow when <code>active</code> is true

WHEN THIS FILE IS CALLED

Rendered inside `SortableTh.jsx`'s clickable header button, once per sortable column, on every render of a sortable table header.

WHO CALLS THIS FILE

- `client/src/components/SortableTh.jsx`

WHICH FILES IT DEPENDS ON

- `react-icons/fa6` (external) — `FaSort`, `FaSortUp`, `FaSortDown`. No internal imports.

INTERNAL BEHAVIOR

- `if (!active) return <FaSort className="text-slate-300" size={11} />` — neutral two-way arrow glyph in light gray.
- If `active`, renders `<FaSortUp size={11}/>` when `direction === "asc"`, else `<FaSortDown size={11}/>` — no explicit color class in the active branch (inherits the parent button's text color).

DATA FLOW

Purely a controlled, stateless icon; all sort state lives in the parent table's `sortConfig`.

EXAMPLE EXECUTION

`<SortIcon active={true} direction="asc" />` → renders `FaSortUp`. `<SortIcon active={false} />` → renders `FaSort` in `text-slate-300`.

IMPORTANT NOTES

If `active` is `true` but `direction` is `undefined` (not expected given `SortableTh`'s usage, which only sets `active` when `sortConfig.key` matches — implying `sortConfig.direction` is also set), the ternary falls through to `FaSortDown` by default.

client/src/components/SortableTh.jsx

PURPOSE

Shared, reusable `<th>` cell for any sortable data table — renders a clickable header (with `SortIcon`) for columns that define a `sortKey`, or plain static text for columns that don't (e.g., an "Actions" column). Exposes sort state to assistive technology via `aria-sort`.

PROPS (EXACT CONTRACT — SHARED ACROSS TABLES)

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>column</code>	object	Yes	—	<code>{key, label, sortKey}</code> ; <code>sortKey</code> of <code>null / undefined</code> makes the column unsortable
<code>sortConfig</code>	object \	null	Yes	— <code>{key, direction}</code> describing the table's current sort state, or <code>null</code> if unsorted
<code>onSort</code>	function	Yes	—	Called with <code>column.sortKey</code> when the header is clicked

WHEN THIS FILE IS CALLED

Mapped once per column inside `RecommendationTable.jsx`'s `<thead>` (over its 7-entry `COLUMNS` array) and inside `ScanHistory.jsx`'s own table header, over that page's own column definitions.

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/pages/ScanHistory.jsx`

WHICH FILES IT DEPENDS ON

- `./SortIcon.jsx` — the up/neutral/down glyph
- External: `framer-motion` (`motion`, for the button's hover/tap micro-interactions)

INTERNAL BEHAVIOR

- `active = sortConfig?.key === column.sortKey` — true only if this column's `sortKey` matches the table's active sort key.
- `ariaSort` computed: `undefined` if the column has no `sortKey` at all (no `aria-sort` attribute rendered); `"none"` if sortable but not currently active; `"ascending"` / `"descending"` if active, based on `sortConfig.direction`.
- If `column.sortKey` is truthy, renders a `<motion.button>` (`whileHover={{scale:1.05}}`, `whileTap={{scale:0.95}}`) containing the label text + `<SortIcon active direction={sortConfig?.direction}/>`, calling `onSort(column.sortKey)` on click. If `column.sortKey` is falsy, renders `column.label` as plain text — no button, not interactive, no `aria-sort` — which is how an "Actions"-type column opts out of sorting.

DATA FLOW

Fully controlled — no internal state. Reflects whatever `sortConfig` the parent table currently holds and delegates all sort-toggling logic back to the parent's `onSort` handler.

EXAMPLE EXECUTION

```
<SortableTh column={{key:"severity", label:"Severity", sortKey:"severity"}} sortConfig={{key:"severity", direction:"asc"}} onSort={fn} /> → clickable "Severity" header with an up-arrow SortIcon, aria-sort="ascending" . <SortableTh column={{key:"actions", label:"Actions", sortKey:null}} .../> → plain "Actions" text, no button, no aria-sort attribute.
```

IMPORTANT NOTES

This is the shared building block behind both `RecommendationTable`'s and `ScanHistory`'s sortable headers. Its prop contract — `column.sortKey` nullability, `sortConfig` shape `{key, direction}` — must stay in sync with each parent table's own sort-state shape and `handleSort` cycle (both known callers use an `asc → desc → null` toggle pattern).

PURPOSE

The KPI tile used extensively on the Dashboard (resource counts, costs, alert counts, EC2/IAM stat rows) — a gradient-icon card with an animated numeric counter, plus a loading skeleton state.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>title</code>	string	Yes (implicit)	—	Label text above the value
<code>value</code>	number	Yes (implicit)	—	Numeric value to animate/display; non-numbers are coerced to <code>0</code> before being passed to <code>AnimatedCounter</code>
<code>icon</code>	ReactNode	Yes (implicit)	—	Icon element rendered in the gradient badge on the right
<code>gradient</code>	string	Yes (implicit)	—	Tailwind gradient classes (e.g. <code>"from-blue-500 to-indigo-500"</code>) for both the badge and the background blur accent
<code>format</code>	function	No	<code>AnimatedCounter</code> 's own default (<code>v => Math.round(v).toLocaleString()</code>)	Formats the animated numeric value, e.g. <code>formatCurrency</code> / <code>formatPercent</code> / <code>formatBytes</code>
<code>index</code>	number	No	<code>0</code>	Stagger-animation index passed to <code>Card</code> 's <code>custom</code> prop
<code>loading</code>	boolean	No	<code>false</code>	Shows a <code>Skeleton</code> instead of <code>AnimatedCounter</code> ; also disables the hover lift

WHEN THIS FILE IS CALLED

Rendered many times (18 call sites observed) on `Dashboard.jsx` across several grouped grids: 8 top-level KPIs (`index` 0–7: Resources, Running, Stopped, Monthly Cost, Estimated Savings, High/Medium/Low Alerts), 3 category-stat cards (Security/Cost/Performance Findings), 6 EC2-stat cards (Average CPU/Network/Disk, Idle/Old Instances, Encrypted Volumes), and 6 IAM-stat cards (Users, Roles, MFA Enabled, Admin Users, Old Access Keys, High Risk Users) — each group restarts its own `index` at `0` for its own stagger sequence.

WHO CALLS THIS FILE

- `client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

- `./Card.jsx` — shell + motion passthrough
- `./AnimatedCounter.jsx` — the animated digit rollup

INTERNAL BEHAVIOR

- Wraps content in `<Card hover ...>` with `cardVariants` entrance stagger (opacity + y, delay = `index * 0.05` s) and a spring `whileHover` lift (`y:-6, scale:1.02`) — the hover lift is disabled (`undefined`) when `loading` is true.
- Renders a decorative blurred gradient circle in the top-right corner (colored by `gradient`), the `title` text, and either a `Skeleton` (80×30) or `<AnimatedCounter value={typeof value === "number" ? value : 0} format={format} />` for the big number.
- Renders `icon` inside a 56px gradient-filled rounded-square badge on the right.

DATA FLOW

`value` comes straight from Dashboard's single dashboard-API response object (e.g. `dashboardData.totalResources` , `dashboardData.ec2Stats?.avgCpu`); this component performs no fetching — it's pure display plus `AnimatedCounter`'s own internal tween from 0 to `value` whenever `value` changes.

EXAMPLE EXECUTION

`<SummaryCard index={3} title="Monthly Cost" value={1523.7} format={formatCurrency} icon={<FaDollarSign/>} gradient="from-amber-500 to-orange-500" loading={false} />` → a card titled "Monthly Cost" whose big number animates from \$0.00 up to \$1523.70 over ~1s (`AnimatedCounter`'s default duration), with an amber-orange gradient badge/accent.

IMPORTANT NOTES

- `value` is defensively coerced (`typeof value === "number" ? value : 0`) before reaching `AnimatedCounter`, so `undefined` / `null` values (e.g. `dashboardData.categoryStats?.securityFindings` before the API resolves) safely animate from/to 0 rather than throwing or rendering "NaN" — this exists alongside (not instead of) the `loading` prop, which is the primary mechanism for hiding pre-fetch state via the Skeleton branch.
- Every `SummaryCard` instance on the Dashboard page shares the same single `loading` boolean (Dashboard's overall `isLoading` flag) — there is no per-card independent loading state.

 `client/src/components/TablePagination.jsx`

PURPOSE

Shared "Showing X–Y of Z" range text plus Previous/Next pager control used by every paginated table in the app. Fully controlled — holds no internal state.

PROPS (EXACT CONTRACT — DEPENDENT ON BY 4 DIFFERENT TABLES)

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>page</code>	number	Yes	—	Current 1-indexed page number
<code>totalPages</code>	number	Yes	—	Total number of pages available (caller must pre-clamp to at least 1)
<code>pageSize</code>	number	Yes	—	Items per page, used to compute the displayed range
<code>totalItems</code>	number	Yes	—	Total item count across all pages; used for the range text and the zero-items case
<code>onPageChange</code>	function	Yes	—	Called with the new page number when Previous/Next is clicked

WHEN THIS FILE IS CALLED

Rendered at the bottom of a table's results block, only after the caller has confirmed there are rows to show — e.g. `RecommendationTable` renders it inside the branch taken when `filteredAndSorted.length !== 0` (i.e., after the `<NoResultsState/>` alternative is ruled out).

WHO CALLS THIS FILE

- `client/src/components/RecommendationTable.jsx`
- `client/src/components/ResourceTable.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ScanHistory.jsx`

WHICH FILES IT DEPENDS ON

- External only: `framer-motion` (`motion`, for button hover/tap scale), `react-icons/fa6` (`FaChevronLeft`, `FaChevronRight`). `imports: []` in `dep_graph` — no internal component/util dependencies.

INTERNAL BEHAVIOR

- Range text (always rendered, even for a single page): ``Showing ${totalItems === 0 ? 0 : (page - 1) * pageSize + 1}–${Math.min(page * pageSize, totalItems)} of ${totalItems}``.
- The Prev/Next buttons and "Page X of Y" text are rendered only `if (totalPages > 1)` — for single-page result sets, only the "Showing..." line appears, with no pager controls at all.
- Previous button: `onClick={() => onPageChange(Math.max(1, page - 1))}`, `disabled={page === 1}`; hover/tap scale animations are suppressed (`undefined`) when disabled.
- Next button: `onClick={() => onPageChange(Math.min(totalPages, page + 1))}`, `disabled={page === totalPages}`, same hover/tap suppression pattern.
- Both handlers clamp their target page via `Math.max` / `Math.min` even though the `disabled` attribute should already prevent out-of-range clicks — defensive duplication.

DATA FLOW

Purely presentational/controlled — the caller owns the actual `page` state (e.g. `RecommendationTable`'s `currentPage` / `safePage`) and passes its own setter (or equivalent) directly as `onPageChange`. This component computes no derived pagination logic beyond the display text and the clamped next/prev targets.

EXAMPLE EXECUTION

`<TablePagination page={2} totalPages={5} pageSize={8} totalItems={37} onPageChange={setPage} />` → "Showing 9–16 of 37", "Page 2 of 5", both Prev/Next enabled; clicking Next calls `onPageChange(3)`. `<TablePagination page={1} totalPages={1} pageSize={8} totalItems={4} onPageChange={setPage} />` → only "Showing 1–4 of 4" is rendered, no pager controls (since `totalPages` is not `> 1`).

IMPORTANT NOTES

- The contract requires `page` to be 1-indexed and `totalPages` to be pre-clamped to at least 1 by the caller. `TablePagination` itself does **not** defend against `totalPages < 1` or a `page` outside `[1, totalPages]` — e.g. it would render "Page 3 of 2" verbatim if a caller passed an already out-of-range `page`.
- All 4 known callers (`RecommendationTable`, `ResourceTable`, `AuditLogs`, `ScanHistory`) must each independently reproduce their own "safe page" clamping logic before passing `page` / `totalPages` in — `TablePagination` does not centralize that safety itself.

 `client/src/components/TableSkeleton.jsx`

PURPOSE

Generic, column-driven loading placeholder for simpler (non-`RecommendationTable`-shaped) data tables — reused by `ResourceTable`, `AuditLogs`, and `ScanHistory`. The column set and row count are fully caller-supplied.

PROPS (EXACT CONTRACT — DEPENDENT ON BY 3 DIFFERENT TABLES)

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>columns</code>	array of string	Yes	—	Column header labels; also determines cell count per row and (via array index) which cells get a wide vs. narrow skeleton bar
<code>rows</code>	number	No	<code>6</code>	Number of skeleton <code><tr></code> rows to render

WHEN THIS FILE IS CALLED

Rendered by each caller in place of its real `<table>` while that page/component's data is still loading — e.g. `ScanHistory.jsx`: `<TableSkeleton columns={SKELETON_COLUMNS} />`, used only for the table element itself; any surrounding toolbar/search/filter UI is not part of this component and is handled (or omitted) separately by each caller.

WHO CALLS THIS FILE

- `client/src/components/ResourceTable.jsx`
- `client/src/pages/AuditLogs.jsx`
- `client/src/pages/ScanHistory.jsx`

WHICH FILES IT DEPENDS ON

- `react-loading-skeleton` (external) — `Skeleton` primitive only. No internal imports.

INTERNAL BEHAVIOR

- Renders a `<table>` whose `<thead>` maps `columns` directly to `<th>{label}</th>` cells.
- `<tbody>` renders `rows` `<tr>` elements (`Array.from({length: rows})`), each with one `<td>` per column.
- Each cell's `Skeleton` width is `colIndex === 0 ? 140 : 80` — the first column always gets a wider (140px) bar, every other column gets a narrower (80px) bar, height fixed at 13px, regardless of what that column actually represents.

DATA FLOW

Purely presentational; `columns` is the only prop that varies shape between callers (e.g., `ScanHistory`'s own `SKELETON_COLUMNS` array vs. whatever `ResourceTable` / `AuditLogs` define) — no other data flows through it.

EXAMPLE EXECUTION

`<TableSkeleton columns={['Scan Date', 'Status', 'Resources', 'Cost', 'Duration']} rows={4} />` → a 5-column × 4-row skeleton table; the "Scan Date" column's cells are 140px wide, the remaining 4 columns' cells are 80px wide.

IMPORTANT NOTES

- The per-column width heuristic (`colIndex === 0 ? 140 : 80`) is a single hardcoded rule shared by every caller — it does not adapt to how wide any particular column's real content tends to be (e.g., a "Status" badge column and a long "Resource ID" column both get an 80px

skeleton if not first).

- Unlike `RecommendationTableSkeleton`, this component renders only the `<table>` element — callers are responsible for any surrounding toolbar/search/filter skeleton themselves (observed behavior in `ScanHistory` is to show nothing above it while loading).

`client/src/components/ToastCard.jsx`

PURPOSE

Custom-styled notification card rendered inside `react-hot-toast`'s `toast.custom()` call, replacing the library's default toast box with one matching the app's visual style (icon, message, dismiss button) across success/error/info variants.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE		
<code>t</code>	object	Yes	—	The toast instance object from <code>react-hot-toast</code> (<code>t.visible</code> drives the fade state, <code>t.id</code> is used for dismissal)		
<code>variant</code>	string	No	falls back to <code>VARIANTS.info</code>	One of <code>"success"</code> \	<code>"error"</code>	<code>"info"</code> ; selects icon + colors
<code>message</code>	string	Yes	—	The notification text		

WHEN THIS FILE IS CALLED

Not rendered directly by app pages. Invoked inside `client/src/utils/toast.jsx`'s helper functions (`showSuccessToast`, `showErrorToast`, and an info-toast helper), each of which calls `toast.custom((t) => <ToastCard t={t} variant="..." message={message} />)`. Those helpers are, in turn, called throughout the app after async actions — e.g. `useRecommendationActions`'s `handleIgnore` / `handleResolve` / `handleStop`, `NotificationSettings`' `save flow` — to surface a result to the user.

WHO CALLS THIS FILE

- `client/src/utils/toast.jsx` (per `dep_graph`; a `.jsx` utility module that wraps `react-hot-toast`'s API, not a page/component)

WHICH FILES IT DEPENDS ON

- External only: `react-hot-toast` (`toast`, for `toast.dismiss(t.id)` in the dismiss button), `react-icons/fa6` (`FaBolt`, `FaCircleCheck`, `FaCircleExclamation`, `FaXmark`). No internal imports.

INTERNAL BEHAVIOR

- `VARIANTS` map: `success` → `FaCircleCheck` /emerald icon+border; `error` → `FaCircleExclamation` /red; `info` → `FaBolt` /blue icon, slate border. `const style = VARIANTS[variant] || VARIANTS.info` — any unrecognized `variant` string silently falls back to the info styling.
- Opacity is driven by `t.visible` (`react-hot-toast`'s own enter/exit flag) via a conditional Tailwind class (`opacity-100` / `opacity-0`) combined with `transition-opacity duration-200` — this is how the toast fades in/out as `react-hot-toast` adds/removes it.
- Renders the icon, the message text (`flex-1`, so it wraps and pushes the dismiss button to the far right), and a manual dismiss button calling `toast.dismiss(t.id)`.

DATA FLOW

No internal state — visibility, variant, message, and the dismiss target are all driven by props/`react-hot-toast`'s own instance object. The toast lifecycle (auto-dismiss timing, queueing, stacking) is entirely managed by `react-hot-toast` via `toast.custom()`, not by this component.

EXAMPLE EXECUTION

`showSuccessToast("Recommendation resolved.")` (from `utils/toast.jsx`) → `react-hot-toast` calls `toast.custom(t => <ToastCard t={t} variant="success" message="Recommendation resolved." />)` → a white rounded card renders with a green check icon, "Recommendation resolved." text, and an X dismiss button, fading in over 200ms.

IMPORTANT NOTES

- This component never calls `toast()` / `toast.custom()` itself — it is the rendered *content* passed to `toast.custom()`, not the trigger. The actual toast-triggering API lives in `client/src/utils/toast.jsx`.

- `t.visible` drives this component's own opacity transition in addition to whatever position/exit animation the app's `Toaster` container (`AppToaster.jsx` , out of scope here) applies — two independent animation layers stacked on the same toast.

`client/src/components/ToggleSwitch.jsx`

PURPOSE

Small reusable on/off switch (rounded pill with a sliding knob), used for boolean settings such as notification preferences and scheduled-scan flags.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>checked</code>	boolean	Yes	—	Current on/off state; drives knob position and track color
<code>onChange</code>	function	Yes	—	Click handler (receives the native click event; no new-value argument — caller toggles its own state)
<code>disabled</code>	boolean	No	<code>undefined</code> (falsy)	Disables the button and dims it
<code>label</code>	string	No	<code>undefined</code>	Used as the button's <code>aria-label</code>

WHEN THIS FILE IS CALLED

`NotificationSettings.jsx` renders one per notification-setting row: `<ToggleSwitch checked={settings[key]} onChange={() => handleToggle(key)} disabled={togglingKey === key} label={ Toggle ${title} } />`. Also used in `ScheduleScan.jsx` (per `dep_graph importedBy`) for its own boolean field(s).

WHO CALLS THIS FILE

- `client/src/pages/NotificationSettings.jsx`
- `client/src/pages/ScheduleScan.jsx`

WHICH FILES IT DEPENDS ON

- External only: `framer-motion` (`motion`). No internal imports.

INTERNAL BEHAVIOR

1. Renders a `<motion.button role="switch" aria-checked={checked}>` as the track — fixed `h-6 w-11` (24×44px), rounded-full, colored `emerald-500` when checked, `slate-300` (`slate-600` in dark mode) when unchecked.
2. The knob is an absolutely positioned `<motion.span>` (`h-5 w-5` , inset 2px on all sides via `left-0.5 top-0.5`) whose `x` transform animates between `0` (off, left) and `20` (on, right) via a spring (`stiffness: 500, damping: 30`). Per the file's own comment, `20` is chosen specifically so `2px + 20px (knob) + 20px (travel) + 2px = 44px` total — keeping the knob visually centered with equal gaps in both states.
3. `disabled` sets the native `disabled` attribute plus `disabled:cursor-not-allowed disabled:opacity-60` ; no internal "mid-toggle" state exists — that is left entirely to the caller (e.g. `NotificationSettings` passes `disabled={togglingKey === key}` from its own state).

DATA FLOW

Fully controlled — no internal state. `checked` / `disabled` are interpreted entirely from parent-owned state; `onChange` fires as a plain click handler with no value argument, so the parent must derive the new value itself (typically by flipping its already-known current state) rather than reading it off the callback.

EXAMPLE EXECUTION

`<ToggleSwitch checked={true} onChange={() => setEnabled(false)} label="Toggle Weekly Report Email" />` → renders a green pill with the knob on the right; clicking calls the `onChange` handler, which the parent has wired to flip its own boolean.

IMPORTANT NOTES

- `onChange` receives no argument describing the new checked value — unlike a native `<input type="checkbox">`'s `onChange` , callers must derive the new value themselves rather than reading it from the event/callback.
- The 2px/20px/20px/2px math documented in the file's comment directly explains why `x: checked ? 20 : 0` (not, say, `24` or `22`) produces a visually centered knob in both states — important to preserve if track/knob sizes are ever changed independently.

PURPOSE

Shared time-series chart (line or area) used four times on the Analytics page — Monthly Cost, Estimated Savings, Resource Growth, and High Alerts trends. Per its code comment, one data point represents one historical scan, since each scan is itself a point-in-time summary snapshot.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>title</code>	string	Yes	—	Chart title, passed to <code>ChartCard</code>
<code>subtitle</code>	string	No	—	Chart subtitle, passed to <code>ChartCard</code>
<code>index</code>	number	No	—	Passed to <code>ChartCard</code> for stagger delay
<code>data</code>	array	Yes	—	Data points; each must include a <code>date</code> field (X axis) and the field named by <code>dataKey</code> . No default value (not defaulted to <code>[]</code> unlike most other chart props in this codebase)
<code>dataKey</code>	string	Yes	—	Field name in each data point used as the Y value
<code>seriesName</code>	string	Yes	—	Legend/tooltip series label
<code>color</code>	string	Yes	—	Hex color (typically from <code>utils/chartColors</code> ' <code>CHART_COLORS</code>) for the line/area stroke+fill
<code>valueFormatter</code>	function	No	<code>undefined</code>	Formats Y-axis ticks and tooltip values (e.g. <code>formatCurrency</code>)
<code>loading</code>	boolean	No	<code>false</code>	Shows <code><ChartSkeleton variant="bar"/></code>
<code>variant</code>	string	No	<code>"line"</code>	<code>"area"</code> renders <code><AreaChart></code> / <code><Area></code> ; anything else renders <code><LineChart></code> / <code><Line></code>
<code>emptyMessage</code>	string	No	<code>"No scan history yet – run a scan to start building trend data."</code>	Message shown by <code>ChartEmptyState</code> when <code>data</code> is empty

WHEN THIS FILE IS CALLED

`Analytics.jsx` renders it 4 times: Monthly Cost Trend (`index=0` , `dataKey="monthlyCost"` , `color=CHART_COLORS.blue`), Estimated Savings Trend (`index=1` , `dataKey="estimatedSavings"` , `color=CHART_COLORS.emerald`), Resources Growth (`index=2` , `dataKey="totalResources"` , `color=CHART_COLORS.indigo` , `variant="area"`), High Alerts Trend (`index=3` , `dataKey="highAlerts"` , `color=CHART_COLORS.red`) — each fed from a distinct `analytics.service.js` endpoint (`getCostTrend` / `getSavingsTrend` / `getResourceGrowth` / `getHighAlertsTrend`) via `useAsyncData` .

WHO CALLS THIS FILE

- `client/src/pages/Analytics.jsx`

WHICH FILES IT DEPENDS ON

- `./ChartCard.jsx` , `./ChartEmptyState.jsx` , `./ChartSkeleton.jsx` (`variant="bar"` — reused even though `TrendChart` itself never renders bars), `./ChartTooltip.jsx`
- `../utils/format.js` — `formatDateTime` (used for the tooltip's label)
- External: `recharts` (`ResponsiveContainer` , `LineChart` , `Line` , `AreaChart` , `Area` , `XAxis` , `YAxis` , `CartesianGrid` , `Tooltip`)

INTERNAL BEHAVIOR

- `const ChartComponent = variant === "area" ? AreaChart : LineChart` — chosen once per render, determines which Recharts wrapper component hosts the axes and series.
- `shortDate(value)` — local helper formatting the X-axis tick as e.g. "Jan 15" via `toLocaleDateString` ; returns `""` for an unparseable date instead of throwing.
- `loading` → `<ChartSkeleton variant="bar"/>` ; `!data.length` → `<ChartEmptyState message={emptyMessage}/>` ; else renders the chart: `XAxis` keyed on `"date"` with `tickFormatter={shortDate}` and `minTickGap={24}` (prevents overlapping labels on dense data);

- `YAxis` with `tickFormatter={valueFormatter}`; `Tooltip` using `ChartTooltip` with `formatter={valueFormatter}` for values and a separate `labelFormatter={value => formatDateTime(value)}` for the tooltip's header (full date/time, distinct from the short axis label).
4. Conditionally renders `<Area>` (monotone, 15% fill opacity, 2px stroke, 3px dots/5px active dots) or `<Line>` (same stroke/dot styling, no fill) based on `variant`, both keyed on `dataKey`, labeled `seriesName`, with an 800ms `ease-out` entrance animation.

DATA FLOW

Each `Analytics.jsx` call site independently fetches its own trend array (cost/savings/resource-growth/high-alerts) from a distinct `analytics.service.js` endpoint via `useAsyncData`, then passes that array plus per-chart config (`dataKey` / `color` / `valueFormatter` / `variant`) into this one shared, data-agnostic component — all domain-specific meaning comes from props, not from anything hardcoded here.

EXAMPLE EXECUTION

```
<TrendChart title="Monthly Cost Trend" subtitle="Total monthly AWS cost at each scan" index={0} data={[{date:"2026-06-01T00:00:00Z", monthlyCost:1200},{date:"2026-07-01T00:00:00Z", monthlyCost:1450}]} dataKey="monthlyCost" seriesName="Monthly Cost" color="#3b82f6" valueFormatter={formatCurrency} loading={false} />
```

→ a blue line chart with X-axis ticks "Jun 1", "Jul 1", Y-axis ticks formatted as currency, and a tooltip on hover showing the full formatted date/time plus "Monthly Cost: \$1,450.00".

IMPORTANT NOTES

- `ChartSkeleton` is always invoked with `variant="bar"` here even though `TrendChart` itself only ever renders line/area series — the loading placeholder's shape (vertical bars) does not visually match the eventual line/area chart as closely as it does for the app's actual bar-chart components.
- `data` has no default value, unlike most other chart-component array props in this codebase (which default to `[]`) — if a caller ever passed `undefined` / `null`, `!data.length` would throw rather than falling into the empty-state branch. In observed usage, `useAsyncData`'s return value on the Analytics page is always initialized as an array before render, so this has not been seen to manifest.

RESOURCE DISPLAY COMPONENTS

ResourceTable, ResourceDetailsDrawer, and ResourceHealthPieChart — the components that render a scan's findings.

`client/src/components/ResourceTable.jsx`

PURPOSE

Renders the paginated table of every AWS resource discovered in a scan (Name, Resource Type, Resource ID, State, and a per-service "Details" column) on the Scan Detail page. Generic across all four resource types via a `resourceType`-keyed switch, rather than being EC2-specific.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>resources</code>	Array	no	<code>[]</code>	The list of Resource objects for the currently-viewed scan
<code>loading</code>	boolean	no	<code>false</code>	Shows <code>TableSkeleton</code> instead of the table while the scan's resources are being fetched

WHEN THIS FILE IS CALLED

Rendered by `ScanDetail.jsx` whenever a user opens a specific scan's detail page.

WHO CALLS THIS FILE

`client/src/pages/ScanDetail.jsx`

WHICH FILES IT DEPENDS ON

- `TablePagination.jsx` — the shared page-number/prev-next control used by every paginated table in the app
- `TableSkeleton.jsx` — the shared loading placeholder

INTERNAL BEHAVIOR

- `PAGE_SIZE = 10` constant.
- `currentPage` state via `useState(1)`.
- Early return `<TableSkeleton columns={COLUMNS} />` while `loading`.
- Early return a plain "No resources found for this scan." message if `resources.length === 0`.
- Compute `totalPages = Math.max(1, Math.ceil(resources.length / PAGE_SIZE))`, clamp `currentPage` into `safePage = Math.min(currentPage, totalPages)` (defends against a stale page number after the underlying data shrinks), then slice `resources` to just the current page.
- Render each row with Framer Motion stagger-in animation (`rowVariants` , capped at a 10-row stagger delay so a full page doesn't have a long cascading entrance).
- Per-row "Details" column is computed by `getResourceDetail(resource)` , a `switch` on `resource.resourceType` :
 - `EC2` → `metadata.instanceType`
 - `RDS` → `metadata.instanceClass`
 - `S3` → `metadata.region`
 - `IAM_USER` → "Admin access" or "Standard user" based on `metadata.hasAdminAccess`
 - `IAM_ROLE` → "N attached polic(y|ies)" count
 - anything else → `"—"` fallback
- Renders `<TablePagination>` below the table with `page` , `totalPages` , `pageSize` , `totalItems` , `onPageChange={setCurrentPage}` .

DATA FLOW

`resources` array in (from the Scan Detail page's fetched scan data) → paginated client-side (no server round-trip per page) → one row per resource, service-appropriate "Details" cell → `TablePagination` drives `currentPage` back up via `onPageChange` .

EXAMPLE EXECUTION

`resources` = 14 EC2 instances + 3 S3 buckets: page 1 shows resources 1–10 (mixed EC2/S3 in whatever order the backend returned them), each EC2 row's Details column shows its instance type (e.g. `t3.micro`), each S3 row shows its region (e.g. `ap-south-1`); `TablePagination` shows "Page 1 of 2".

IMPORTANT NOTES

This table intentionally does **not** support sorting or filtering (unlike `RecommendationTable.jsx`) — it's a flat, read-only inventory view. All pagination is client-side; the full `resources` array for the scan is fetched once by the parent page, not paged from the server.

 `client/src/components/ResourceDetailsDrawer.jsx`

PURPOSE

A slide-in side panel (portalled to `document.body`) showing full detail for one resource + its recommendation — CPU/network/disk stats and Stop-Instance action for EC2, engine/storage/Multi-AZ details for RDS, MFA/policy details for IAM, AWS Console deep-links for S3 — plus the shared Resolve/Ignore/Stop action buttons. This is what opens when a user clicks a row in `RecommendationTable.jsx`.

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE	
<code>resource</code>	Object \	null	no	—	The recommendation+resource record to display; <code>null</code> /falsy closes the drawer
<code>onClose</code>	function	yes	—	Called on backdrop click, X button, or Escape key	
<code>onIgnore</code>	function	yes	—	Called with the resource's <code>_id</code> when "Ignore" is clicked	
<code>onResolve</code>	function	yes	—	Called with the resource's <code>_id</code> when "Resolve" is clicked	
<code>onStop</code>	function	yes	—	Called with the resource's <code>_id</code> when "Stop Instance" is clicked (EC2 only)	
<code>actionLoadingId</code>	string	no	—	If it matches this resource's <code>_id</code> , shows the Ignore/Resolve buttons in a loading state	
<code>stoppingId</code>	string	no	—	Same, for the Stop Instance button specifically	
<code>loading</code>	boolean	no	<code>false</code>	Shows a skeleton layout instead of real content	

WHEN THIS FILE IS CALLED

Rendered by `RecommendationTable.jsx` whenever a user clicks a recommendation row; controls its own open/closed state via the truthiness of the `resource` prop (no internal `isOpen` state).

WHO CALLS THIS FILE

`client/src/components/RecommendationTable.jsx`

WHICH FILES IT DEPENDS ON

`ActionButton.jsx`, `Badge.jsx`, `CopyButton.jsx`, `LinkButton.jsx`, `ResourceDetailsDrawerSkeleton.jsx`, `utils/awsConsole.js` (S3 console deep-link builders), `utils/badge.js` (`getResourceStateBadgeClass`, `getSeverityBadgeStyle`, `getRecommendationStatusBadgeStyle`, `getCategoryBadgeStyle`), `utils/format.js` (`formatDateTime`, `formatCurrency`, `formatPercent`, `formatBytes`, `formatActionLabel`)

INTERNAL BEHAVIOR

- `useEffect` locks page scroll (`document.body.style.overflow = "hidden"`) and binds an Escape-key listener whenever `resource` is truthy; cleans both up on close/unmount.

2. Rendered via `createPortal(..., document.body)` — specifically to escape a `backdrop-blur-xl` ancestor (`RecommendationTable` 's card wrapper), because `backdrop-filter` on an ancestor creates a new CSS containing block for `position: fixed` descendants, which was breaking this drawer's positioning before the portal was added.
3. Status/severity/category badges rendered at the top via the `badge.js` style helpers.
4. Body content branches on `resource.resourceType` :
 - `EC2` → Optimization Insights block: average CPU, network in/out, disk read/write bytes and ops, launch date, instance age, Elastic IP, security groups, EBS volumes (with per-volume encrypted/unencrypted label), plus a 3-tile current/optimized/savings cost comparison.
 - `RDS` → Database Details block: monthly cost, average CPU, engine+version, instance class, status, storage type, allocated storage, Multi-AZ, publicly accessible, backup retention, storage encrypted, availability zone, endpoint.
 - `IAM_USER` / `IAM_ROLE` / `IAM_ROOT_ACCOUNT` → Security block: MFA enabled/disabled badge, Administrator Access badge if applicable, then type-specific detail lists (last login/groups/access keys for users; last used/trust policy for roles; attached + inline policies for both).
 - `S3` → AWS Console Actions block: deep links to the bucket in the S3 console, its CloudWatch metrics, and its permissions page, plus copy-to-clipboard buttons for the ARN and bucket name.
1. Always-shown footer stats: Est. Savings, Monthly Cost, Confidence.
2. Always-shown Recommendation + Reason text block.
3. Always-shown generic resource metadata block: Resource Type, Instance Type (if present), Instance State (color-coded badge), Public/Private IP, Availability Zone, Launch Time.
4. Action bar (Ignore/Resolve/Stop) only renders if `resource.status === "OPEN"` — a resolved/ignored recommendation shows no action buttons. The Stop button itself only renders if `resource.action === "STOP"` , and shows "Instance already stopped" text instead of a clickable button if `resource.state?.toLowerCase() === "stopped"` .

DATA FLOW

`resource` prop in → portal-rendered overlay + slide-in panel → `resourceType`-branched detail sections read straight from `resource.metadata.*` → Ignore/Resolve/Stop buttons call back up to the parent page's mutation handlers with `resource._id` .

EXAMPLE EXECUTION

User clicks a HIGH-severity, OPEN, EC2 STOP recommendation row → drawer slides in from the right → shows "HIGH severity" + "OPEN" + "COST" badges → Optimization Insights shows 2.3% average CPU → footer shows Est. Savings \$7.50 → action bar shows an active red "Stop Instance" button (since `state !== "stopped"`) plus Ignore/Resolve.

IMPORTANT NOTES

The drawer has **no fetch of its own** — every field it displays comes from the `resource` object already fetched by the parent page/table; it's a pure presentational detail view, not a data-fetching component.

`client/src/components/ResourceHealthPieChart.jsx`

PURPOSE

A 5-slice donut/pie chart (Critical/High/Medium/Low/Healthy) visualizing the dashboard's `severitySummary` — the single chart that answers "how healthy is my whole scanned fleet, across every service, at a glance."

PROPS

PROP	TYPE	REQUIRED	DEFAULT	PURPOSE
<code>severitySummary</code>	Object	no	<code>{}</code>	<code>{ critical, high, medium, low, healthy }</code> counts, as returned by the dashboard summary endpoint
<code>index</code>	number	no	—	Passed through to <code>ChartCard</code> for staggered entrance animation ordering among sibling charts
<code>loading</code>	boolean	no	<code>false</code>	Shows <code>ChartSkeleton</code> instead of the chart

WHEN THIS FILE IS CALLED

Rendered on the Dashboard page as one of the summary charts.

WHO CALLS THIS FILE

`client/src/pages/Dashboard.jsx`

WHICH FILES IT DEPENDS ON

`ChartCard.jsx` (title/subtitle/card shell), `ChartEmptyState.jsx` (shown when every bucket is 0), `ChartSkeleton.jsx`, `ChartTooltip.jsx` (custom tooltip renderer), `utils/chartColors.js` (`RESOURCE_HEALTH_COLORS` — the fixed color-per-severity mapping)

INTERNAL BEHAVIOR

1. Builds a 5-entry `chartData` array (Critical/High/Medium/Low/Healthy), each `{ name, value, color }`, **filtered to only entries with `value > 0`** — a bucket with zero recommendations doesn't show up as an empty pie slice or a legend entry with no wedge.
2. `total = sum of all values`; if `total === 0`, renders `ChartEmptyState` ("No scanned resources yet. Run a scan to see resource health.") instead of an empty pie.
3. Otherwise renders a Recharts `<PieChart>` with `paddingAngle={2}` (visual separation between slices), animated on mount (800ms ease-out), a custom `ChartTooltip`, and a bottom legend.

DATA FLOW

`severitySummary` counts in (already pre-aggregated server-side by `serviceAggregation.service.js`'s `buildSeveritySummary`, which itself only counts `status: "OPEN"` recommendations) → filtered/colored/shaped for Recharts → rendered pie.

EXAMPLE EXECUTION

`severitySummary = { critical: 1, high: 3, medium: 2, low: 0, healthy: 8 }` → renders a 4-slice pie (Low is dropped since its value is 0) — 1 Critical (red), 3 High (orange), 2 Medium (amber), 8 Healthy (green) — legend shows exactly those 4 labels.

IMPORTANT NOTES

This component performs **zero aggregation itself** — the critical/high/medium/low/healthy split is entirely computed server-side (including the "critical = HIGH severity + SECURITY category" derivation, since the Recommendation schema itself has no CRITICAL level). If that count is wrong, the bug is in `serviceAggregation.service.js`, not here.

FRONTEND SERVICES, HOOKS & UTILS

The Axios wrapper layer, reusable stateful hooks, and presentation-only formatting helpers.

CloudLens Frontend Dossier — Services, Hooks, Utils, Constants, API (F4)

Covers 24 files: `client/src/services/*.service.js` (10), `client/src/utils/*` (6), `client/src/hooks/*` (6), `client/src/constants/scanSteps.js` (1), `client/src/api/api.js` (1).

 `client/src/api/api.js`

PURPOSE

Creates and exports the single shared Axios instance used by every service module in the app to talk to the backend REST API. Centralizes three cross-cutting concerns in one place: the API base URL, attaching the JWT bearer token to outgoing requests, and globally handling expired/invalid sessions (401 responses) by clearing storage and redirecting to `/login`.

WHEN THIS FILE IS CALLED

Evaluated once at module-load time (its side effect is creating and configuring the `axios.create(...)` instance and registering the two interceptors). After that, its exported `api` object's methods (`api.get`, `api.post`, `api.patch`, `api.delete`) are invoked by every service file on every backend call the app makes.

WHO CALLS THIS FILE

All 10 files in `client/src/services/`: `analytics.service.js`, `auditLog.service.js`, `auth.service.js`, `awsConnection.service.js`, `dashboard.service.js`, `notification.service.js`, `recommendation.service.js`, `report.service.js`, `scan.service.js`, `schedule.service.js`.

WHICH FILES IT DEPENDS ON

None internally. External: `axios` only.

EXPORTED FUNCTIONS/VALUES

- `api` (default export)
- Purpose: Pre-configured Axios instance with base URL, auth header injection, and 401 handling baked in.
- Input: N/A (it's an object, not a function) — consumers call `api.get(url)`, `api.post(url, body)`, `api.patch(url, body)`, `api.delete(url, config)` etc.
- Output: An Axios instance; calling its HTTP methods returns a Promise resolving to an Axios response object (`{ data, status, headers, config, ... }`) or rejecting with an Axios error.
- Internal steps (module init):
 1. `axios.create({ baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api" })` — base URL is read from the Vite env var `VITE_API_URL`; if unset, falls back to `http://localhost:5000/api`.
 2. `api.interceptors.request.use((config) => {...})` — on every outgoing request, reads `localStorage.getItem("token")`. If present, sets `config.headers.Authorization = `Bearer ${token}`` before returning `config`.
 3. `api.interceptors.response.use((response) => response, (error) => {...})` — on success, passes the response through unchanged. On error, computes `isAuthEndpoint = error.config?.url?.startsWith("/auth/")`. If `error.response?.status === 401` and `!isAuthEndpoint`, it calls `localStorage.removeItem("token")` and `window.location.href = "/login"`. In all error cases it ends with `return Promise.reject(error)`, so the error still propagates to the calling code's `catch` / `await` regardless of whether a redirect was triggered.
- Backend endpoint called: N/A (this is the transport itself, not a resource call).

- Error handling: Non-auth 401s trigger an automatic session-clear + hard redirect to `/login` ; the rejected promise is still returned, so callers' own try/catch or `.catch()` still fire. Auth-endpoint 401s (e.g. bad login credentials) are left untouched for the calling page to handle/display itself.
- Return values: N/A.

DATA FLOW

A page component calls a `*.service.js` function, which calls `api.get/post/patch/delete(path, ...)` . The request interceptor stamps on the `Authorization: Bearer <token>` header (token read fresh from `localStorage` on every request — not cached in a variable). The request goes to `baseURL + path` . On the way back, the response interceptor either passes a success straight through or, for a 401 on any non-`/auth/*` path, wipes the stored token and hard-navigates the browser to `/login` before still rejecting the promise so the original caller's error handling also runs.

EXAMPLE EXECUTION

```
// localStorage has token = "eyJhbGciOi..."
api.get("/dashboard")
// request interceptor -> headers.Authorization = "Bearer eyJhbGciOi..."
// -> GET http://localhost:5000/api/dashboard
// -> 200: response passed through unchanged, resolves with { data: { success: true, data: {...} }, status: 200, ... }
```

```
// Token has expired server-side
api.get("/dashboard")
// -> 401 { }
// isAuthenticated = "/dashboard".startsWith("/auth/") === false
// -> localStorage.removeItem("token"); window.location.href = "/login"
// -> promise still rejects; caller's catch still executes (briefly, before navigation completes)
```

IMPORTANT NOTES

- Base URL env var is exactly `VITE_API_URL` ; fallback is the literal string `http://localhost:5000/api` .
- Auth header name is exactly `Authorization` , value format `Bearer <token>` ; token is read from `localStorage` key `"token"` on every request (no in-memory caching).
- 401 handling excludes any request whose `config.url` starts with `/auth/` — this is a prefix check on the *relative* URL passed to `api.get/post` (e.g. `/auth/login`), not the full resolved URL.
- The redirect is a hard `window.location.href` assignment (full page reload), not a React Router `navigate()` — this discards all in-memory React state/app state, not just auth state.
- Because `Promise.reject(error)` runs unconditionally after the redirect is kicked off, any code following the rejected call (e.g. in a `.catch()` in the calling service/hook) still executes for a brief moment before the browser actually navigates away — this is a minor race, not guarded against.
- No request timeout is configured, and there is no token-refresh flow — a single non-renewing JWT is used until it expires or a 401 forces re-login.
- No `Content-Type` handling is explicitly set on the shared instance (Axios defaults apply per-request; `report.service.js` overrides `responseType: "blob"` for the PDF download).

 `client/src/services/analytics.service.js`

PURPOSE

Wraps the four analytics-trend REST endpoints (cost trend, savings trend, resource growth, high-alerts trend) that back the Analytics page's charts.

WHEN THIS FILE IS CALLED

When `client/src/pages/Analytics.jsx` loads/refreshes its trend charts (typically via `useAsyncData`).

WHO CALLS THIS FILE

`client/src/pages/Analytics.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — the shared Axios instance used to issue all four GET requests.

EXPORTED FUNCTIONS/VALUES

- `getCostTrend()`
- Purpose: Fetch historical cost-over-time data points.
- Input: none.
- Output: `response.data.data` (the unwrapped payload of the standard `{ success, data }` API envelope) — an array of cost-trend data points.
- Internal steps: 1) `await api.get("/analytics/cost-trend")` . 2) return `response.data.data` .
- Backend endpoint: `GET /api/analytics/cost-trend` .
- Error handling: none in this file — a rejected promise propagates unchanged to the caller (e.g. `useAsyncData` 's `refetch` , which catches it and sets an error message).
- Return values: array of cost-trend points (shape defined server-side; not transformed here).
- `getSavingsTrend()`
- Purpose: Fetch historical savings-over-time data points.
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.get("/analytics/savings-trend")` then return `response.data.data` .
- Backend endpoint: `GET /api/analytics/savings-trend` .
- Error handling: none (propagates).
- `getResourceGrowth()`
- Purpose: Fetch resource-count-growth-over-time data points.
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.get("/analytics/resource-growth")` then return `response.data.data` .
- Backend endpoint: `GET /api/analytics/resource-growth` .
- Error handling: none (propagates).
- `getHighAlertsTrend()`
- Purpose: Fetch the count of HIGH-severity recommendations over time.
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.get("/analytics/high-alerts-trend")` then return `response.data.data` .
- Backend endpoint: `GET /api/analytics/high-alerts-trend` .
- Error handling: none (propagates).

DATA FLOW

Analytics.jsx calls one or more of these functions (typically through `useAsyncData`) on mount; each issues a GET through the shared `api` instance (which attaches the JWT), unwraps `response.data.data` , and returns it directly to the caller for chart rendering.

EXAMPLE EXECUTION

```
const points = await getCostTrend();
// -> GET /api/analytics/cost-trend
// -> response.data = { success: true, data: [{ date: "2026-07-01", cost: 412.3 }, ...] }
// -> points = [{ date: "2026-07-01", cost: 412.3 }, ...]
```

IMPORTANT NOTES

All four functions share an identical shape (GET + unwrap `.data.data`); none catch errors themselves, so callers are responsible for error handling (typically via `useAsyncData`).

 `client/src/services/auditLog.service.js`

PURPOSE

Wraps the single endpoint that lists audit log entries (user logins, AWS connect/disconnect, scan start/complete, recommendation resolve/ignore, etc.) for the Audit Logs page.

WHEN THIS FILE IS CALLED

When `client/src/pages/AuditLogs.jsx` loads its table data.

WHO CALLS THIS FILE

`client/src/pages/AuditLogs.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getAuditLogs()`
- Purpose: Fetch all audit log entries for the current user's account.
- Input: none.
- Output: `response.data.data` — an array of audit log entries.
- Internal steps: 1) `await api.get("/audit-logs")` . 2) return `response.data.data` .
- Backend endpoint: `GET /api/audit-logs` .
- Error handling: none in this file; propagates to caller (typically `useAsyncData` , which catches and surfaces a generic error message).
- Return values: array of audit log objects (fields consumed elsewhere include action, timestamp, etc., per `formatAuditActionLabel` in `format.js` and `getAuditActionBadgeStyle` in `badge.js`).

DATA FLOW

AuditLogs.jsx calls `getAuditLogs()` (via `useAsyncData`) on mount; the GET is issued through `api` , response unwrapped to the raw array, and returned for table rendering, filtering, and sorting.

EXAMPLE EXECUTION

```
const logs = await getAuditLogs();
// -> GET /api/audit-logs
// -> response.data = { success: true, data: [{ action: "SCAN_COMPLETED", createdAt: "...", ... }, ...] }
// -> logs = [{ action: "SCAN_COMPLETED", createdAt: "...", ... }, ...]
```

IMPORTANT NOTES

Smallest service file in the app (7 lines) — a single unparameterized GET with no filtering/pagination params sent to the backend; any filtering/sorting seen on the Audit Logs page happens client-side.

 `client/src/services/auth.service.js`

PURPOSE

Wraps the two unauthenticated auth endpoints — login and signup — used by the Login and Signup pages.

WHEN THIS FILE IS CALLED

On form submission on `client/src/pages/Login.jsx` and `client/src/pages/Signup.jsx` .

WHO CALLS THIS FILE

`client/src/pages/Login.jsx` , `client/src/pages/Signup.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `login(email, password)`
- Purpose: Authenticate a user and obtain a JWT.
- Input: `email` (string), `password` (string) — positional, not an object.
- Output: `response.data` — the *full* response envelope (not unwrapped to `.data.data` like other services), presumably `{ success, data: { token, user } }` or similar, since the caller needs the raw token to store in `localStorage` .

- Internal steps: 1) `await api.post("/auth/login", { email, password })` . 2) return `response.data` .
- Backend endpoint: `POST /api/auth/login` .
- Error handling: none in this file; a rejected promise (e.g. 401 for bad credentials) propagates to the Login page's own try/catch, which is expected to display the error — note that per `api.js` 's interceptor, a 401 from `/auth/login` is explicitly excluded from the auto-redirect/token-clear logic since `isAuthEndpoint` is true.
- Return values: full envelope object, e.g. `{ success: true, data: { token: "...", user: {...} } }` (exact shape defined server-side).
- `signup(fullName, email, password)`
- Purpose: Register a new user account.
- Input: `fullName` (string), `password` (string), `email` (string) — positional.
- Output: `response.data` — full response envelope, same rationale as `login` .
- Internal steps: 1) `await api.post("/auth/signup", { fullName, email, password })` . 2) return `response.data` .
- Backend endpoint: `POST /api/auth/signup` .
- Error handling: none in this file; propagates to Signup page's try/catch (also exempt from the interceptor's 401 auto-redirect since the URL starts with `/auth/`).
- Return values: full envelope object.

DATA FLOW

Login/Signup page collects form input, calls `login / signup` , which POSTs credentials through `api` (no Authorization header needed/sent since no token exists yet — though the interceptor would still check `localStorage` and add one if a stale token happened to be present). On success the page itself is responsible for storing `response.data.data.token` into `localStorage` under the `"token"` key (the key the request interceptor reads) and navigating to the dashboard.

EXAMPLE EXECUTION

```
const result = await login("akshatjohri03@gmail.com", "hunter2");
// -> POST /api/auth/login { email, password }
// -> result = { success: true, data: { token: "eyJhbGciOi...", user: { fullName: "...", email: "..." } } }
```

IMPORTANT NOTES

Unlike every other service file, `login / signup` return the *whole* `response.data` envelope rather than unwrapping to `.data.data` — this is because the page needs access to the token field, which lives alongside (not nested inside) other envelope-level data depending on backend shape. Both endpoints are exempt from the Axios response interceptor's 401-redirect behavior by virtue of their `/auth/` prefix.

`client/src/services/awsConnection.service.js`

PURPOSE

Wraps the four AWS-account-connection endpoints: reading the current connection, testing candidate credentials, connecting an account, and disconnecting it. Backs the ConnectAws page and the `useAwsConnection` hook (used by Navbar).

WHEN THIS FILE IS CALLED

When ConnectAws.jsx loads/submit its form, and whenever `useAwsConnection` mounts (Navbar) or the user clicks disconnect.

WHO CALLS THIS FILE

`client/src/hooks/useAwsConnection.js` , `client/src/pages/ConnectAws.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getAwsConnection()`
- Purpose: Fetch the current user's connected AWS account info (if any).
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.get("/aws/account")` → return `response.data.data` .
- Backend endpoint: `GET /api/aws/account` .
- Error handling: none in this file; caller (`useAwsConnection`) catches and logs via `console.error` .

- Return values: connection object (e.g. region/alias/masked key info) or `null` /falsy if not connected (server-dependent).
- `testAwsConnection({ region, accessKeyId, secretAccessKey })`
- Purpose: Validate AWS credentials without persisting them (dry-run check, e.g. STS `GetCallerIdentity`).
- Input: object `{ region, accessKeyId, secretAccessKey }` .
- Output: `response.data.data` .
- Internal steps: `await api.post("/aws/test-connection", { region, accessKeyId, secretAccessKey })` → return `response.data.data` .
- Backend endpoint: `POST /api/aws/test-connection` .
- Error handling: none in this file; propagates to `ConnectAws.jsx`'s own try/catch (form displays the error).
- `connectAwsAccount({ region, accessKeyId, secretAccessKey, accountAlias })`
- Purpose: Persist AWS credentials, linking an AWS account to the current user.
- Input: object `{ region, accessKeyId, secretAccessKey, accountAlias }` .
- Output: `response.data.data` .
- Internal steps: `await api.post("/aws/connect", {...})` → return `response.data.data` .
- Backend endpoint: `POST /api/aws/connect` .
- Error handling: none in this file; propagates to caller.
- `disconnectAwsAccount()`
- Purpose: Remove the stored AWS connection for the current user.
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.delete("/aws/disconnect")` → return `response.data.data` .
- Backend endpoint: `DELETE /api/aws/disconnect` .
- Error handling: none in this file; `useAwsConnection.disconnect` catches and logs via `console.error` .

DATA FLOW

`ConnectAws.jsx` collects region/access key/secret in a form, optionally calls `testAwsConnection` to validate before calling `connectAwsAccount` to persist. `useAwsConnection` (used by `Navbar`) calls `getAwsConnection` on mount to show connection status app-wide, and `disconnectAwsAccount` when the user disconnects from the `Navbar` UI.

EXAMPLE EXECUTION

```
await testAwsConnection({ region: "us-east-1", accessKeyId: "AKIA...", secretAccessKey: "..." });
// -> POST /api/aws/test-connection
// -> response.data.data = { valid: true, accountId: "123456789012" } (server-dependent shape)
```

IMPORTANT NOTES

None of the four functions catch errors locally — all rely on callers (`ConnectAws.jsx` or `useAwsConnection`) for error handling/toasts.

 `client/src/services/dashboard.service.js`

PURPOSE

Wraps the two endpoints backing the main Dashboard: fetching the aggregated dashboard payload, and triggering a new infrastructure scan.

WHEN THIS FILE IS CALLED

On `Dashboard.jsx` mount (`getDashboard`), and whenever the user clicks "Run Scan" from the `Navbar` (`scanInfrastructure` , via `useScanTrigger`).

WHO CALLS THIS FILE

`client/src/hooks/useScanTrigger.js` , `client/src/pages/Dashboard.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared `Axios` instance.

EXPORTED FUNCTIONS/VALUES

- `getDashboard()`
- Purpose: Fetch the aggregated dashboard payload (summary cards, cost breakdown, resource health, AI insights, etc.).
- Input: none. Output: `response.data.data`.
- Internal steps: `await api.get("/dashboard")` → return `response.data.data`.
- Backend endpoint: `GET /api/dashboard`.
- Error handling: none in this file; propagates to caller, typically `useAsyncData` in `Dashboard.jsx`.
- `scanInfrastructure()`
- Purpose: Trigger a synchronous full AWS infrastructure scan (EC2/S3/RDS/IAM) for the connected account.
- Input: none.
- Output: `response.data` — the full envelope (not unwrapped to `.data.data`), unlike `getDashboard`.
- Internal steps: `await api.post("/scan")` → return `response.data`.
- Backend endpoint: `POST /api/scan`.
- Error handling: none in this file; caller `useScanTrigger.runScan` catches it, extracts `error.response?.data?.message`, sets `scanError`, and shows an error toast.

DATA FLOW

`Dashboard.jsx` calls `getDashboard()` on mount to populate all dashboard widgets. The global Navbar's "Run Scan" button (via `useScanTrigger`) calls `scanInfrastructure()`, which is a single blocking POST that runs the entire backend scan pipeline server-side before resolving; on success the calling page's `onSuccess` callback (usually a `refetch` of dashboard/scan data) runs.

EXAMPLE EXECUTION

```
const dashboardData = await getDashboard();
// -> GET /api/dashboard -> response.data.data = { totalMonthlyCost, resourceHealth, insights, ... }

const result = await scanInfrastructure();
// -> POST /api/scan (long-running; resolves only once the full scan pipeline finishes server-side)
// -> result = { success: true, message: "Scan completed", data: {...} } (server-dependent)
```

IMPORTANT NOTES

`scanInfrastructure` returns the full envelope (`response.data`) while `getDashboard` unwraps to `response.data.data` — inconsistent unwrap convention within the same file, mirroring the same inconsistency seen between `auth.service.js` (full envelope) and most other services (unwrapped). `scanInfrastructure` is a single opaque, non-streaming request — the visual step-by-step progress shown during a scan (`SCAN_STEPS` in `useScanTrigger`) is purely client-side pacing, not driven by real server progress events.

 `client/src/services/notification.service.js`

PURPOSE

Wraps the two endpoints for reading and updating the current user's notification settings (e.g. weekly report email preferences).

WHEN THIS FILE IS CALLED

When `client/src/pages/NotificationSettings.jsx` loads its form and when the user saves changes.

WHO CALLS THIS FILE

`client/src/pages/NotificationSettings.jsx`.

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getNotificationSettings()`
- Purpose: Fetch the current user's notification preferences.
- Input: none. Output: `response.data.data`.

- Internal steps: `await api.get("/notifications/settings")` → return `response.data.data`.
- Backend endpoint: `GET /api/notifications/settings`.
- Error handling: none in this file; propagates to caller (typically `useAsyncData`).
- `updateNotificationSettings(updates)`
- Purpose: Persist changes to notification preferences.
- Input: `updates` (object — partial settings patch, e.g. `{ weeklyReportEnabled: true }`).
- Output: `response.data.data`.
- Internal steps: `await api.patch("/notifications/settings", updates)` → return `response.data.data`.
- Backend endpoint: `PATCH /api/notifications/settings`.
- Error handling: none in this file; propagates to NotificationSettings.jsx, which is expected to catch it and show a toast (via `utils/toast.jsx`, per the dep graph's `importedBy` list for `toast.jsx` including this page).

DATA FLOW

NotificationSettings.jsx fetches current settings on mount via `getNotificationSettings`, and on form submit calls `updateNotificationSettings` with only the changed fields; both calls go straight through the shared `api` instance with no local transformation.

EXAMPLE EXECUTION

```
const settings = await getNotificationSettings();
// -> GET /api/notifications/settings -> { weeklyReportEnabled: true, ... }

const updated = await updateNotificationSettings({ weeklyReportEnabled: false });
// -> PATCH /api/notifications/settings { weeklyReportEnabled: false }
// -> updated = { weeklyReportEnabled: false, ... }
```

IMPORTANT NOTES

Both functions are thin, uncatch wrappers — identical pattern to most other service files in this codebase.

 `client/src/services/recommendation.service.js`

PURPOSE

Wraps the recommendation-list endpoint plus the three recommendation state-transition actions (ignore, resolve, stop-instance) used across the Dashboard, Recommendations page, and the shared `useRecommendationActions` hook.

WHEN THIS FILE IS CALLED

On Recommendations.jsx/Dashboard.jsx load (`getRecommendations`), and whenever a user clicks Ignore/Resolve/Stop on a recommendation row or drawer (via `useRecommendationActions`).

WHO CALLS THIS FILE

`client/src/hooks/useRecommendationActions.js` , `client/src/pages/Dashboard.jsx` , `client/src/pages/Recommendations.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getRecommendations()`
- Purpose: Fetch all recommendations for the current user's scanned resources.
- Input: none. Output: `response.data.data`.
- Internal steps: `await api.get("/recommendations")` → return `response.data.data`.
- Backend endpoint: `GET /api/recommendations`.
- Error handling: none in this file; propagates to caller.
- `ignoreRecommendation(id)`
- Purpose: Mark a recommendation as ignored (status → `IGNORED`).

- Input: `id` (string — recommendation ID).
- Output: `response.data.data` (the updated recommendation object).
- Internal steps: `await api.patch(\ /recommendations/${id}/ignore\ )` → return `response.data.data` .
- Backend endpoint: `PATCH /api/recommendations/:id/ignore` .
- Error handling: none in this file; caught by `useRecommendationActions.performAction` , which logs via `console.error` and shows an error toast with `error.response?.data?.message` (falling back to a generic message).
- `resolveRecommendation(id)`
- Purpose: Mark a recommendation as resolved (status → resolved/closed).
- Input: `id` (string). Output: `response.data.data` .
- Internal steps: `await api.patch(\ /recommendations/${id}/resolve\ )` → return `response.data.data` .
- Backend endpoint: `PATCH /api/recommendations/:id/resolve` .
- Error handling: none in this file; same `performAction` catch as above.
- `stopRecommendation(id)`
- Purpose: Trigger the actual remediation action — stopping the underlying EC2 instance a recommendation refers to.
- Input: `id` (string — recommendation ID, not instance ID). Output: `response.data.data` .
- Internal steps: `await api.post(\ /recommendations/${id}/stop\ )` → return `response.data.data` .
- Backend endpoint: `POST /api/recommendations/:id/stop` .
- Error handling: none in this file; caught by `useRecommendationActions.handleStop` 's underlying `performAction` .

DATA FLOW

Recommendations.jsx/Dashboard.jsx load the list via `getRecommendations` . Row/drawer action buttons call `useRecommendationActions` 's handlers, which call `ignoreRecommendation` / `resolveRecommendation` / `stopRecommendation` with the target recommendation's `id` ; the returned updated recommendation is passed back up via `onRecommendationUpdate` so the caller can patch its local list in place instead of refetching everything.

EXAMPLE EXECUTION

```
const updated = await resolveRecommendation("64f0c2...");
// -> PATCH /api/recommendations/64f0c2.../resolve
// -> updated = { _id: "64f0c2...", status: "RESOLVED", ... }
```

IMPORTANT NOTES

`stopRecommendation` is the one action that causes a real side effect against the user's live AWS account (stopping an EC2 instance) rather than just a database status update — reflected in `useRecommendationActions.js` giving it its own loading-id state (`stoppingId` , separate from `actionId`) and a distinct success/error toast message ("EC2 instance stopped successfully." / "Failed to stop EC2 instance.").

 `client/src/services/report.service.js`

PURPOSE

Wraps the single endpoint that downloads a scan's PDF report, and additionally performs the client-side blob-to-file-download mechanics (since this is a binary download, not a JSON call).

WHEN THIS FILE IS CALLED

When the user clicks "Download Report" on `client/src/pages/ScanDetail.jsx` .

WHO CALLS THIS FILE

`client/src/pages/ScanDetail.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `downloadScanReport(scanId)`
- Purpose: Fetch a scan's PDF report as a binary blob and trigger a browser download of it.

- Input: `scanId` (string).
- Output: none (`undefined`) — the function's effect is the side effect of downloading a file, not a returned value.
- Internal steps:
 1. `const response = await api.get(\ /reports/scan/${scanId}\ , { responseType: "blob" })` — requests the PDF as a raw `Blob` rather than JSON.
 2. `const url = window.URL.createObjectURL(new Blob([response.data], { type: "application/pdf" })))` — wraps the blob and creates an object URL.
 3. Creates a detached `<a>` element, sets `link.href = url` and `link.download = \ cloudlens-scan-report-${scanId}.pdf\`.`
 4. Appends the link to `document.body` , calls `link.click()` to trigger the browser's download, then `link.remove()` .
 5. `window.URL.revokeObjectURL(url)` to free the object URL memory.
- Backend endpoint: `GET /api/reports/scan/:scanId` .
- Error handling: none in this file; a rejected request (e.g. network error or non-2xx) propagates to `ScanDetail.jsx`'s own try/catch, which is expected to show a toast via `utils/toast.jsx` .
- Return values: none — pure side effect.

DATA FLOW

`ScanDetail.jsx` calls `downloadScanReport(scanId)` on button click; the function fetches the PDF binary through `api` (JWT still attached by the request interceptor as normal), then performs standard browser blob-download plumbing (object URL + synthetic anchor click) to save the file client-side.

EXAMPLE EXECUTION

```
await downloadScanReport("64f0c2ab12de34f5");
// -> GET /api/reports/scan/64f0c2ab12de34f5 (responseType: "blob")
// -> browser downloads a file named "cloudlens-scan-report-64f0c2ab12de34f5.pdf"
```

IMPORTANT NOTES

The filename is generated client-side (`cloudlens-scan-report-${scanId}.pdf`) rather than read from the response's `Content-Disposition` header — the file comment explains this is because that header isn't in the backend's CORS `Access-Control-Expose-Headers` allowlist by default, and exposing it would require widening the shared `cors()` config in `app.js` for every route just to support this one download. This is the only service file that touches the DOM directly (creating/clicking/removing an `<a>` element) and the only one using `responseType: "blob"` .

 `client/src/services/scan.service.js`

PURPOSE

Wraps the two read-only scan-history endpoints: listing all past scans and fetching one scan's full detail by ID.

WHEN THIS FILE IS CALLED

On `ScanHistory.jsx` load (`getScans`) and on `ScanDetail.jsx` / `Dashboard.jsx` load (`getScanById`).

WHO CALLS THIS FILE

`client/src/pages/Dashboard.jsx` , `client/src/pages/ScanDetail.jsx` , `client/src/pages/ScanHistory.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getScans()`
- Purpose: Fetch the list of all past scans for the current user's account.
- Input: none. Output: `response.data.data` .
- Internal steps: `await api.get("/scans")` → return `response.data.data` .
- Backend endpoint: `GET /api/scans` .
- Error handling: none in this file; propagates to caller.

- `getScanById(id)`
- Purpose: Fetch full detail for one specific scan (resources found, recommendations generated, timestamps, status).
- Input: `id` (string — scan ID). Output: `response.data.data`.
- Internal steps: `await api.get(\ /scans/${id}\ )` → return `response.data.data`.
- Backend endpoint: `GET /api/scans/:id`.
- Error handling: none in this file; propagates to caller.

DATA FLOW

ScanHistory.jsx calls `getScans()` to render the scan list/table. ScanDetail.jsx calls `getScanById(id)` (id from the route params) to render one scan's full detail; Dashboard.jsx also uses `getScanById` (likely for the most recent scan, per its dep-graph usage).

EXAMPLE EXECUTION

```
const scans = await getScans();
// -> GET /api/scans -> [{ _id: "...", status: "COMPLETED", startedAt: "...", completedAt: "...", ...}]

const scan = await getScanById(scans[0]._id);
// -> GET /api/scans/<id> -> { _id: "...", resources: [...], recommendations: [...], ... }
```

IMPORTANT NOTES

Both functions are simple, uncaught GET wrappers — no filtering/pagination parameters are sent; any client-side filtering/sorting on ScanHistory.jsx happens after the full list is fetched (consistent with `auditLog.service.js`'s pattern).

 `client/src/services/schedule.service.js`

PURPOSE

Wraps the CRUD endpoints for the single recurring scan schedule a user can configure, including a client-side unwrap that converts the backend's array response into a single object-or-null for easier page-level consumption.

WHEN THIS FILE IS CALLED

When `client/src/pages/ScheduleScan.jsx` loads and when the user creates/updates/deletes their schedule.

WHO CALLS THIS FILE

`client/src/pages/ScheduleScan.jsx`.

WHICH FILES IT DEPENDS ON

`client/src/api/api.js` — shared Axios instance.

EXPORTED FUNCTIONS/VALUES

- `getSchedule()`
- Purpose: Fetch the current user's single scan schedule, if one exists.
- Input: none.
- Output: the single schedule object, or `null` if none exists.
- Internal steps:
 1. `const response = await api.get("/schedules")`.
 2. `const schedules = response.data.data || []` — defends against a missing/undefined `data` field.
 3. `return schedules[0] || null` — the backend supports (at most) one schedule per user but still returns it as a 0- or 1-item array; this unwraps that array to a single value.
- Backend endpoint: `GET /api/schedules`.
- Error handling: none in this file; propagates to caller (typically `useAsyncData`).
- Return values: a schedule object `{ _id, frequency, enabled, ... }`, or `null`.
- `createSchedule({ frequency, enabled })`
- Purpose: Create the user's recurring scan schedule.

- Input: object `{ frequency, enabled }`.
- Output: `response.data.data` (the created schedule object).
- Internal steps: `await api.post("/schedules", { frequency, enabled })` → return `response.data.data`.
- Backend endpoint: `POST /api/schedules`.
- Error handling: none in this file; propagates to `ScheduleScan.jsx` (which shows a toast per `toast.jsx` import list).
- `updateSchedule(id, updates)`
- Purpose: Modify an existing schedule (e.g. change frequency or toggle enabled).
- Input: `id` (string), `updates` (object partial patch).
- Output: `response.data.data`.
- Internal steps: `await api.patch(`/schedules/${id}`, updates)` → return `response.data.data`.
- Backend endpoint: `PATCH /api/schedules/:id`.
- Error handling: none in this file; propagates to caller.
- `deleteSchedule(id)`
- Purpose: Remove the user's schedule entirely.
- Input: `id` (string). Output: none (`undefined`) — this function does not return anything, unlike the other three.
- Internal steps: `await api.delete(`/schedules/${id}`)` — response is awaited but discarded.
- Backend endpoint: `DELETE /api/schedules/:id`.
- Error handling: none in this file; propagates to caller.

DATA FLOW

`ScheduleScan.jsx` calls `getSchedule()` on load to determine whether to show a "create" or "edit" form (based on `null` vs. an object). Submitting the form calls `createSchedule` or `updateSchedule` depending on whether a schedule already exists; deleting calls `deleteSchedule`.

EXAMPLE EXECUTION

```
const schedule = await getSchedule();
// -> GET /api/schedules -> response.data.data = [{ _id: "s1", frequency: "WEEKLY", enabled: true }]
// -> schedule = { _id: "s1", frequency: "WEEKLY", enabled: true }

// If response.data.data were [] or undefined:
// -> schedule = null
```

IMPORTANT NOTES

`getSchedule` is the only function among all 10 service files that reshapes the response beyond a plain `.data.data` unwrap (array → single object/null) — done specifically because, per the inline comment, the backend models schedules as a list even though only one per user is supported. `deleteSchedule` is the only function in this file (and one of very few across all services) that returns nothing.

 `client/src/utils/awsConsole.js`

PURPOSE

Pure helper functions that build deep-link URLs into the AWS Management Console for a given S3 bucket (objects, metrics, and permissions tabs) and construct an S3 bucket ARN string — used so the Resource Details drawer can link users straight to the relevant AWS console page for a resource.

WHEN THIS FILE IS CALLED

Whenever `ResourceDetailsDrawer.jsx` renders detail for an S3 bucket resource and needs to build "open in AWS console" links or display the bucket's ARN.

WHO CALLS THIS FILE

`client/src/components/ResourceDetailsDrawer.jsx`.

WHICH FILES IT DEPENDS ON

None (no internal or external imports) — pure string/URL logic only.

EXPORTED FUNCTIONS/VALUES

- `getS3ConsoleUrl(bucketName, region)`
- Purpose: Build a URL to the bucket's "objects" tab in the AWS S3 console.
- Input: `bucketName` (string), `region` (string, optional).
- Output: URL string.
- Internal steps: calls internal `buildS3ConsoleUrl(bucketName, region, "objects")`.
- Return values: e.g. `"https://s3.console.aws.amazon.com/s3/buckets/my-bucket?region=us-east-1&tab=objects"`.
- `getS3MetricsUrl(bucketName, region)`
- Purpose: Build a URL to the bucket's "metrics" tab.
- Input/Output: same shape as above, with `tab="metrics"`.
- `getS3PermissionsUrl(bucketName, region)`
- Purpose: Build a URL to the bucket's "permissions" tab.
- Input/Output: same shape as above, with `tab="permissions"`.
- `getS3BucketArn(bucketName)`
- Purpose: Construct the bucket's ARN string for display/copy.
- Input: `bucketName` (string).
- Output: ``arn:aws:s3:::${bucketName}`` — no region/account segment, since S3 bucket ARNs never include them (bucket names are globally unique).
- Internal steps: single template-literal concatenation, no validation.
- (internal, not exported) `buildS3ConsoleUrl(bucketName, region, tab)`
- Builds ``https://s3.console.aws.amazon.com/s3/buckets/${encodeURIComponent(bucketName)}?${params}`` where `params` is a `URLSearchParams` of `{ region: region || DEFAULT_REGION, tab }`. `DEFAULT_REGION` is the module-level constant `"us-east-1"`.

DATA FLOW

ResourceDetailsDrawer renders an S3 resource's region/name (data already fetched from the backend, e.g. via `dashboard.service.js` / `scan.service.js`) and passes them into these builders to produce clickable console links and a copyable ARN — all computed client-side with no network calls.

EXAMPLE EXECUTION

```
getS3ConsoleUrl("cloudlens-logs-bucket", "eu-west-1")
// -> "https://s3.console.aws.amazon.com/s3/buckets/cloudlens-logs-bucket?region=eu-west-1&tab=objects"

getS3ConsoleUrl("cloudlens-logs-bucket") // region omitted
// -> "https://s3.console.aws.amazon.com/s3/buckets/cloudlens-logs-bucket?region=us-east-1&tab=objects"

getS3BucketArn("cloudlens-logs-bucket")
// -> "arn:aws:s3:::cloudlens-logs-bucket"
```

IMPORTANT NOTES

S3-only — no EC2/RDS/IAM console URL builders exist in this file despite the generic-sounding filename; if the drawer needs console links for other resource types, none are provided here. `bucketName` is URI-encoded in the path segment but not otherwise validated.

 `client/src/utils/badge.js`

PURPOSE

A large collection of pure lookup functions mapping domain enum values (scan status, recommendation status, severity, resource state, category, audit action, schedule enabled/disabled) to Tailwind CSS class strings, used to color-code badges consistently throughout the dashboard.

WHEN THIS FILE IS CALLED

Whenever a component renders a status/severity/category badge — on every table row and detail view across Recommendations, ScanHistory, AuditLogs, ScheduleScan, and the Resource Details drawer.

WHO CALLS THIS FILE

```
client/src/components/RecommendationTable.jsx , client/src/components/ResourceDetailsDrawer.jsx ,
client/src/pages/AuditLogs.jsx , client/src/pages/ScanDetail.jsx , client/src/pages/ScanHistory.jsx ,
client/src/pages/ScheduleScan.jsx .
```

WHICH FILES IT DEPENDS ON

None (no imports) — pure lookup/switch logic.

EXPORTED FUNCTIONS/VALUES

There are two families of exports: "flat" class-string functions (older/simpler) and "style" object functions (`{ badgeClass, dotClass }` , used by the newer `<Badge>` component with a colored dot).

- `getScanStatusBadgeClass(status)` → string. Switches on `"COMPLETED"` (emerald), `"RUNNING"` (blue), `"FAILED"` (red), default (slate).
- `getRecommendationStatusBadgeClass(status)` → string. Switches on `"OPEN"` (blue), `"IGNORED"` (amber), default (emerald — i.e. resolved/other statuses read as "success" green).
- `getSeverityBadgeClass(severity)` → string. Switches on `"HIGH"` (red), `"MEDIUM"` (amber), default (emerald, i.e. LOW).
- `getResourceStateBadgeClass(state)` → string. Lower-cases `state` first (`state?.toLowerCase()`), switches on `"running"` (emerald), `"stopped"` (red), default (slate).
- `getSeverityBadgeStyle(severity)` → `{ badgeClass, dotClass }` . Same HIGH/MEDIUM/default tiers as `getSeverityBadgeClass` but returns soft-fill+border classes plus a separate dot color class.
- `getRecommendationStatusBadgeStyle(status)` → `{ badgeClass, dotClass }` . Same OPEN/IGNORED/default tiers as the flat version, richer styling.
- `getCategoryBadgeStyle(category)` → `{ badgeClass, dotClass }` . Switches on `"SECURITY"` (red), `"PERFORMANCE"` (indigo), `"RELIABILITY"` (teal), `"OPERATIONAL_EXCELLENCE"` (purple), `"COST"` /default (amber).
- `getScheduleStatusBadgeStyle(enabled)` → `{ badgeClass, dotClass }` . Ternary, not switch: `enabled` truthy → emerald; falsy → slate.
- `getAuditActionBadgeStyle(action)` → `{ badgeClass, dotClass }` . Switches on specific `AuditLog.action` values: `"AWS_DISCONNECTED"` / `"EC2_STOPPED"` → red; `"RECOMMENDATION_IGNORED"` → amber; `"AWS_CONNECTED"` / `"SCAN_COMPLETED"` / `"RECOMMENDATION_RESOLVED"` → emerald; `"USER_LOGIN"` / `"SCAN_STARTED"` → blue; default → slate.
- `getScanStatusBadgeStyle(status)` → `{ badgeClass, dotClass }` . Same COMPLETED/RUNNING/FAILED/default tiers as `getScanStatusBadgeClass` , richer styling.

For each: Input is the raw enum string (or boolean for the schedule variant); Output is either a Tailwind class string or a `{ badgeClass, dotClass }` object; there is no error handling — unrecognized/ `undefined` inputs simply fall through to each function's `default` branch (never throws).

DATA FLOW

A component receives a status/severity/category value from backend data (e.g. a `Recommendation.status` field from `recommendation.service.js`) and passes it directly into the relevant lookup function to get back class names, which are applied to a `<span>` / `<Badge>` element — no async behavior, no state.

EXAMPLE EXECUTION

```
getSeverityBadgeStyle("HIGH")
// -> { badgeClass: "bg-red-50 text-red-700 border-red-200", dotClass: "bg-red-500" }

getScanStatusBadgeClass("FAILED")
// -> "bg-red-100 text-red-700"

getResourceStateBadgeClass("STOPPED") // note: uppercase input
// -> "bg-red-100 text-red-700" (lower-cased internally to "stopped" before the switch)
```

IMPORTANT NOTES

Per the file's own header comment, the flat `*BadgeClass` functions and the richer `*BadgeStyle` functions intentionally overlap in coverage (e.g. both `getScanStatusBadgeClass` and `getScanStatusBadgeStyle` exist) — the style variants were added for a redesigned `<Badge>` component without removing the older flat-class functions, so existing callers of the originals keep working unchanged. Color semantics are consistent across all functions: success/positive states always use emerald, warnings/ignored always amber, danger/failure always red, and neutral/unknown always slate — never a mix of e.g. green vs. emerald for the same meaning.

PURPOSE

Central palette of hex color constants for all Recharts-based charts in the app, mirroring the Tailwind gradient colors used elsewhere (SummaryCard, HeroSection) but as real hex values, since SVG `fill` / `stroke` attributes need actual color values rather than Tailwind class names.

WHEN THIS FILE IS CALLED

Whenever a chart component (donut, bar, pie, trend line) needs to assign a color to a data series/slice/bar.

WHO CALLS THIS FILE

```
client/src/components/CostBreakdownDonutChart.jsx , client/src/components/ResourceDistributionBarChart.jsx ,
client/src/components/ResourceHealthPieChart.jsx , client/src/components/SavingsOpportunityBarChart.jsx ,
client/src/pages/Analytics.jsx .
```

WHICH FILES IT DEPENDS ON

None (no imports) — pure constant definitions.

EXPORTED FUNCTIONS/VALUES

- CHART_COLORS** (object) — base named-color hex map: `blue #3b82f6` , `indigo #6366f1` , `emerald #10b981` , `green #22c55e` , `amber #f59e0b` , `orange #f97316` , `rose #f43f5e` , `red #ef4444` , `teal #14b8a6` , `slate #64748b` .
- SEVERITY_COLORS** (object) — `{ HIGH: CHART_COLORS.red, MEDIUM: CHART_COLORS.amber, LOW: CHART_COLORS.emerald }` , keyed by `Recommendation.severity` .
- RESOURCE_STATE_COLORS** (object) — `{ running: CHART_COLORS.emerald, stopped: CHART_COLORS.rose }` , keyed by lowercase EC2 instance state.
- RESOURCE_HEALTH_COLORS** (object) — 5-tier palette for the Dashboard's ResourceHealthPieChart: `{ CRITICAL: "#991b1b", HIGH: CHART_COLORS.red, MEDIUM: CHART_COLORS.amber, LOW: CHART_COLORS.blue, HEALTHY: CHART_COLORS.emerald }` . `CRITICAL` is a hardcoded darker red literal (`#991b1b`), one shade darker than `CHART_COLORS.red` , rather than derived from `CHART_COLORS` .
- CATEGORICAL_PALETTE** (array) — ordered list of 8 colors for charts with arbitrary/unbounded categories: `[blue, emerald, amber, rose, indigo, teal, orange, slate]` (values, not names), intended to be indexed by category position.

None of these are functions — all are static exported objects/arrays consumed directly (e.g. `SEVERITY_COLORS[rec.severity]`) by chart components.

DATA FLOW

A chart component maps its dataset to Recharts `<Cell fill={...}>` or `<Bar fill={...}>` props by looking up each data point's category/severity/state in the relevant exported map (or indexing into `CATEGORICAL_PALETTE` by position for uncategorized data) — pure, synchronous, no side effects.

EXAMPLE EXECUTION

```
SEVERITY_COLORS["HIGH"] // -> "#ef4444"
RESOURCE_HEALTH_COLORS["CRITICAL"] // -> "#991b1b"
CATEGORICAL_PALETTE[2] // -> "#f59e0b" (amber)
```

IMPORTANT NOTES

`RESOURCE_HEALTH_COLORS.CRITICAL` (`#991b1b`) is the one color in the file not derived from `CHART_COLORS` — a standalone literal chosen to read as "more severe than plain HIGH" at a glance, per the inline comment. All other domain-specific maps (`SEVERITY_COLORS` , `RESOURCE_STATE_COLORS` , rest of `RESOURCE_HEALTH_COLORS`) are built by referencing `CHART_COLORS` fields rather than duplicating hex literals, so a future palette-wide color change only needs to happen in one place.

PURPOSE

Collection of pure formatting/label-lookup helpers used throughout the app: currency, recommendation action labels, audit action labels, percentages, byte sizes, dates/times, and scan duration. The single most widely-imported util file (13 importers).

WHEN THIS FILE IS CALLED

Wherever raw backend values (numbers, enum strings, ISO timestamps) need to be rendered as human-readable text — across nearly every page and several chart/table components.

WHO CALLS THIS FILE

`client/src/components/CostBreakdownDonutChart.jsx`, `HeroSection.jsx`, `RecommendationTable.jsx`, `ResourceDetailsDrawer.jsx`, `SavingsOpportunityBarChart.jsx`, `ServiceOverviewGrid.jsx`, `TrendChart.jsx`; `client/src/pages/Analytics.jsx`, `AuditLogs.jsx`, `Dashboard.jsx`, `ScanDetail.jsx`, `ScanHistory.jsx`, `ScheduleScan.jsx`.

WHICH FILES IT DEPENDS ON

None (no imports) — pure functions and lookup tables only.

EXPORTED FUNCTIONS/VALUES

- `formatCurrency(value)`
 - Purpose: Render a number as a USD currency string.
 - Input: `value` (number, or null/undefined).
 - Output: string, e.g. `"$412.30"`.
 - Internal steps: `` ${Number(value || 0).toFixed(2)} `` — `nullish/falsy values coerce to 0` first.
 - Note: always renders `$0.00` for `0`, `null`, `undefined`, `NaN` -producing input, or any other falsy value — cannot distinguish "no data" from "actually zero."
- `formatActionLabel(action)`
 - Purpose: Convert a `Recommendation.action` enum value (e.g. `"STOP"`, `"ENABLE_MFA"`) into a human label.
 - Input: `action` (string).
 - Output: string, e.g. `"Stop Instance"`.
 - Internal steps: looks up `action` in the module-level `ACTION_LABELS` map (12 entries covering EC2/S3/RDS actions like `STOP`, `DOWNSIZE`, `UPSIZE`, `REVIEW`, and IAM actions like `ENABLE_MFA`, `DELETE_UNUSED_KEY`, `REVIEW_PERMISSIONS`, `ROTATE_ACCESS_KEY`, `REMOVE_ADMIN_ACCESS`, `DELETE_UNUSED_ROLE`, `TAG_RESOURCE`, and `NONE`); if not found, falls back to a generic transform: lowercase the string, split on `_`, capitalize each word's first letter, and join with spaces.
 - Return values: mapped label, or a Title-Cased fallback derived from the raw enum string; `undefined` / `null` input becomes `""` before the fallback transform (via `(action || "")`).
- `formatAuditActionLabel(action)`
 - Purpose: Convert an `AuditLog.action` enum value into a human label.
 - Input: `action` (string). Output: string, e.g. `"AWS Connected"`.
 - Internal steps: same pattern as `formatActionLabel` but against a separate `AUDIT_ACTION_LABELS` map (8 entries: `USER_LOGIN`, `AWS_CONNECTED`, `AWS_DISCONNECTED`, `SCAN_STARTED`, `SCAN_COMPLETED`, `RECOMMENDATION_IGNORED`, `RECOMMENDATION_RESOLVED`, `EC2_STOPPED`), with the same lowercase/split/capitalize fallback for unmapped values.
 - Note: kept as a separate map/function from `formatActionLabel` specifically so values like `AWS_CONNECTED` render as `"AWS Connected"` / `EC2_STOPPED` as `"EC2 Stopped"` rather than the fallback transform's `"Aws Connected"` / `"Ec2 Stopped"`.
- `formatPercent(value)`
 - Purpose: Render a number as a percentage string with one decimal place.
 - Input: `value` (number). Output: string, e.g. `"42.5%"`.
 - Internal steps: `` ${Number(value || 0).toFixed(1)}% ``.
- `formatBytes(value)`
 - Purpose: Convert a raw byte count into a human-readable size (B/KB/MB/GB/TB).
 - Input: `value` (number of bytes).
 - Output: string, e.g. `"4.2 MB"` or `"0 B"`.
 - Internal steps: 1) `bytes = Number(value || 0)`; if `bytes <= 0` return `"0 B"` immediately. 2) `exponent = Math.min(Math.floor(Math.log(bytes) / Math.log(1024)), BYTE_UNITS.length - 1)` where `BYTE_UNITS = ["B", "KB", "MB", "GB", "TB"]`. 3) `scaled = bytes / 1024 ** exponent`. 4) returns `scaled.toFixed(...)` with 0 decimals if `scaled >= 10` or `exponent === 0` (i.e. plain bytes), else 1 decimal, followed by the unit.
- `formatDateTime(value)`
 - Purpose: Render an ISO date/timestamp as a locale-formatted date+time string.

- Input: `value` (ISO string, Date, or falsy).
- Output: string, e.g. `"8/1/2026, 3:45:00 PM"` (via `Date.prototype.toLocaleString()`), or `"N/A"`.
- Internal steps: returns `"N/A"` if `!value`; otherwise builds `new Date(value)` and returns `"N/A"` if `Number.isNaN(date.getTime())`, else `date.toLocaleString()`.
- **`formatScanTime(scan)`**
- Purpose: Pick the most relevant timestamp off a scan object and format it.
- Input: `scan` (object with optional `completedAt` / `startedAt` / `createdAt`).
- Output: string (delegates to `formatDateTime`).
- Internal steps: `formatDateTime(scan.completedAt || scan.startedAt || scan.createdAt)` — prefers completion time, falls back to start time, then creation time.
- **`getScanDurationMs(scan)`**
- Purpose: Compute a scan's duration in milliseconds from its start/completion timestamps (there is no stored `duration` field on the backend `Scan` model).
- Input: `scan` (object with `startedAt` / `completedAt`).
- Output: number (ms) or `null`.
- Internal steps: returns `null` if `!scan?.startedAt || !scan?.completedAt`; otherwise `ms = new Date(scan.completedAt).getTime() - new Date(scan.startedAt).getTime()`, returns `null` if `Number.isNaN(ms) || ms < 0`, else `ms`.
- **`formatDuration(ms)`**
- Purpose: Render a millisecond duration as `"Xm Ys"` or `"Ys"`.
- Input: `ms` (number or `null`). Output: string or `null`.
- Internal steps: returns `null` if `ms == null` (catches both `null` and `undefined`); else `totalSeconds = Math.round(ms / 1000)`, `minutes = Math.floor(totalSeconds / 60)`, `seconds = totalSeconds % 60`; returns `` ${minutes}m ${seconds}s `` if `minutes > 0`, else `` ${seconds}s ``.
- **`formatScanDuration(scan)`**
- Purpose: One-call convenience combining the two functions above, with a running-scan fallback.
- Input: `scan` (object). Output: string.
- Internal steps: `formatDuration(getScanDurationMs(scan)) || (scan?.status === "RUNNING" ? "In progress" : "-")` — if duration can't be computed (e.g. scan still running or malformed timestamps), shows `"In progress"` for a `RUNNING` scan or an em dash `"-"` otherwise.

DATA FLOW

Pages/components pass raw values straight from service-layer responses (numbers, ISO date strings, enum strings) into these functions immediately before rendering into JSX text nodes — entirely synchronous, no state, no side effects, no network calls.

EXAMPLE EXECUTION

```
formatCurrency(412.3) // -> "$412.30"
formatActionLabel("ENABLE_MFA") // -> "Enable MFA" (explicit map hit)
formatActionLabel("SOME_NEW_ACTION") // -> "Some New Action" (fallback transform)
formatBytes(4500000) // -> "4.3 MB"
formatScanDuration({ status: "COMPLETED", startedAt: "2026-08-01T10:00:00Z", completedAt: "2026-08-01T10:02:15Z" })
// -> "2m 15s"
formatScanDuration({ status: "RUNNING", startedAt: "2026-08-01T10:00:00Z" })
// -> "In progress"
```

IMPORTANT NOTES

`formatCurrency` and `formatPercent` cannot distinguish "missing/no data" from a genuine `0` — both render `$0.00 / 0.0%` for either case, since they coerce falsy input to `0` before formatting. `formatActionLabel` and `formatAuditActionLabel` are separate functions/maps by design (not a shared generic labeler) specifically to hardcode acronym casing (`AWS`, `EC2`) that the generic snake_case-to-Title-Case fallback would otherwise get wrong. Scan duration is always derived client-side from two timestamps rather than being stored/returned by the backend.

PURPOSE

A single generic comparator function used to sort table rows by an arbitrary field, used by sortable-column table headers.

WHEN THIS FILE IS CALLED

Whenever a user clicks a sortable column header in the Recommendation table or Scan History table.

WHO CALLS THIS FILE

client/src/components/RecommendationTable.jsx , client/src/pages/ScanHistory.jsx .

WHICH FILES IT DEPENDS ON

None (no imports).

EXPORTED FUNCTIONS/VALUES

- `compareValues(valueA, valueB)`
- Purpose: Generic ascending comparator for `Array.prototype.sort()` , usable across mixed field types (numbers, strings, dates-as-strings).
- Input: `valueA` , `valueB` — any two values of the same field from two different rows.
- Output: number — negative if `valueA` sorts before `valueB` , positive if after, `0` if equal (standard JS comparator contract).
- Internal steps: 1) if both `typeof valueA === "number"` and `typeof valueB === "number"` , returns `valueA - valueB` (true numeric comparison). 2) Otherwise, returns `String(valueA).localeCompare(String(valueB))` (locale-aware string comparison, used for text fields and also as the fallback for dates/mixed types that aren't both numbers).
- Return values: number (comparator result) only — never throws, since `String(...)` coerces anything (including `null` / `undefined` , which become `"null"` / `"undefined"`).

DATA FLOW

A table component's column-header click handler calls `.sort((a, b) => compareValues(a[field], b[field]))` (optionally reversing the result for descending order) over its in-memory row array — entirely client-side, no backend involvement, since sorting happens on data already fetched.

EXAMPLE EXECUTION

```
compareValues(120, 45) // -> 75 (positive: 120 sorts after 45 ascending)
compareValues("Charlie", "Alice") // -> positive (locale compare: "Charlie" after "Alice")
compareValues(null, "text") // -> String(null).localeCompare("text") -> "null".localeCompare("text")
```

IMPORTANT NOTES

Only true numeric fields (`typeof === "number"`) get numeric comparison — a numeric value stored as a string (e.g. `"120"`) falls through to `localeCompare` , which does lexicographic (not numeric) ordering and could sort `"120"` before `"45"` . Dates are not special-cased at all: if a caller sorts by an ISO date *string* field, it works correctly under `localeCompare` only because ISO-8601 strings happen to sort correctly lexicographically; a caller sorting by `Date` objects or epoch numbers-as-strings would not get date-aware ordering beyond that lexicographic coincidence.

PURPOSE

Three thin wrapper functions around `react-hot-toast`'s `toast.custom()` API that render the app's own `ToastCard` component instead of `react-hot-toast`'s built-in styled toast — done specifically to sidestep a dark-mode CSS specificity bug.

WHEN THIS FILE IS CALLED

Anywhere the app needs to show a success/error/info toast notification after an async action — recommendation actions, scan triggering, notification settings save, report download, schedule save.

WHO CALLS THIS FILE

`client/src/hooks/useRecommendationActions.js` , `client/src/hooks/useScanTrigger.js` ,
`client/src/pages/NotificationSettings.jsx` , `client/src/pages/ScanDetail.jsx` , `client/src/pages/ScheduleScan.jsx` .

WHICH FILES IT DEPENDS ON

`client/src/components/ToastCard.jsx` — the actual toast UI component rendered inside `toast.custom()` . External: `react-hot-toast` .

EXPORTED FUNCTIONS/VALUES

- `showSuccessToast(message)`
 - Purpose: Show a success-styled toast.
 - Input: `message` (string). Output: whatever `toast.custom()` returns (a toast ID string, per react-hot-toast's API — not typically consumed by callers here).
 - Internal steps: `toast.custom((t) => <ToastCard t={t} variant="success" message={message} />)` .
- `showErrorToast(message)`
 - Purpose: Show an error-styled toast.
 - Input: `message` (string). Output: toast ID (react-hot-toast).
 - Internal steps: `toast.custom((t) => <ToastCard t={t} variant="error" message={message} />)` .
- `showInfoToast(message)`
 - Purpose: Show an info-styled toast.
 - Input: `message` (string). Output: toast ID (react-hot-toast).
 - Internal steps: `toast.custom((t) => <ToastCard t={t} variant="info" message={message} />)` .

DATA FLOW

A hook or page catches an error (or reaches a success branch) after an async service call, extracts/derives a human-readable message (often `error.response?.data?.message` or a hardcoded string), and passes it to one of these three functions; `toast.custom()` hands the render function a toast context object `t` and mounts `<ToastCard>` into react-hot-toast's toast container (rendered by `AppToaster.jsx` at the app root), which is what the user actually sees.

EXAMPLE EXECUTION

```
showSuccessToast("Recommendation resolved.")
// -> react-hot-toast's toast.custom() renders <ToastCard t={...} variant="success" message="Recommendation resolved." />
// in the app-level toast container mounted by <AppToaster/>
```

IMPORTANT NOTES

The file's own header comment explains why `toast.custom()` is used instead of `toast.success()` / `toast.error()` : react-hot-toast's built-in success/error wrappers style their message via internally injected CSS-in-JS (goober) that loads *after* Tailwind's stylesheet, so on equal CSS specificity the injected styles silently win — this previously caused toast text to render nearly black-on-black in dark mode regardless of `dark:` Tailwind classes passed via `toastOptions` . Using `toast.custom()` to render the app's own `ToastCard` JSX sidesteps the library's styling entirely, since no goober-generated CSS is involved. This is a `.jsx` file (not `.js`) because it contains JSX (the `<ToastCard .../>` render functions).

 `client/src/hooks/useAsyncData.js`

PURPOSE

Generic reusable hook that fetches data on mount (and whenever its `deps` array changes), tracking `loading` / `error` state uniformly — replacing what the file's own comment says "every page in this app was doing by hand" before this hook existed. The most widely-used hook in the app (8 page importers).

WHEN THIS FILE IS CALLED

Called at the top of nearly every page component (Analytics, AuditLogs, Dashboard, NotificationSettings, Recommendations, ScanDetail, ScanHistory, ScheduleScan) to load that page's primary data.

WHO CALLS THIS FILE

`client/src/pages/Analytics.jsx`, `AuditLogs.jsx`, `Dashboard.jsx`, `NotificationSettings.jsx`, `Recommendations.jsx`, `ScanDetail.jsx`, `ScanHistory.jsx`, `ScheduleScan.jsx`.

WHICH FILES IT DEPENDS ON

None internally. External: `react` (`useCallback`, `useEffect`, `useState`).

EXPORTED FUNCTIONS/VALUES

- `useAsyncData(fetcher, deps = [], { initialValue = null, errorMessage = "Failed to load data." } = {})`
- Purpose: Fetch async data, expose it plus loading/error state, and provide a stable `refetch` function.
- Input:
 - `fetcher` — an async function (typically a `*.service.js` call, e.g. `() => getDashboard()`) that returns the desired data or throws.
 - `deps` — dependency array (default `[]`) controlling when the fetch re-runs, passed straight into the internal `useCallback` / `useEffect`.
 - Options object: `initialValue` (default `null`, the `data` state's starting value) and `errorMessage` (default `"Failed to load data."`, shown on failure).
- Output/Return value: `{ data, setData, loading, error, setError, refetch }`
- `data` — the fetcher's resolved value, or `initialValue` before the first load completes.
- `setData` — raw React state setter, exposed so a page can optimistically patch `data` locally (e.g. after a mutation) without a full refetch.
- `loading` — boolean, `true` initially and during every `refetch()` call, until it resolves/rejects.
- `error` — `null` normally, or the `errorMessage` string if the last fetch attempt failed.
- `setError` — raw React state setter, exposed so a page can clear or manually set the error.
- `refetch` — stable (memoized via `useCallback`) async function that re-runs `fetcher` and updates state; can be called manually (e.g. after a successful mutation elsewhere on the page).
- Internal steps:
 1. `refetch` (wrapped in `useCallback(..., deps)`): sets `error` to `null`, calls `await fetcher()`, stores the result via `setData(result)`; on thrown error, `console.error(err)` and `setError(errorMessage)`; in a `finally` block, `setLoading(false)` unconditionally.
 2. A `useEffect(..., deps)` calls `refetch()` on mount and whenever any value in `deps` changes.
- Backend endpoint called: N/A — delegates entirely to whatever `fetcher` does.
- Error handling: Catches synchronously/asynchronously thrown errors from `fetcher`, logs the raw error via `console.error`, and sets `error` to the human-readable `errorMessage` string (the raw error detail is not exposed in returned state, only in the console).
- Return values: see shape above; note `loading` is never reset to `true` again automatically except via a manual `refetch()` call or a `deps` change re-running the effect.

DATA FLOW

A page calls `useAsyncData(() => getDashboard())` (or similar). On mount, the internal `useEffect` invokes `refetch`, which calls the passed-in fetcher (itself calling into a `*.service.js` function, which calls `api.*` through the shared Axios instance). The result populates `data`; the page renders based on `loading / error / data`. If the page later needs to reload (e.g. after a scan completes), it calls the returned `refetch()` directly instead of waiting for a `deps` change.

EXAMPLE EXECUTION

```
const { data: dashboard, loading, error, refetch } = useAsyncData(
  () => getDashboard(),
  [],
  { errorMessage: "Failed to load dashboard." }
);
// on mount: loading=true, data=null
// after resolve: loading=false, data={ totalMonthlyCost: 1240.5, ... }, error=null
// after a failed retry: loading=false, error="Failed to load dashboard." (data left as whatever it was before, not reset)
```

IMPORTANT NOTES

Both the `useCallback` and `useEffect` disable the `react-hooks/exhaustive-deps` ESLint rule and spread the caller-supplied `deps` array directly as their dependency list — this means the hook trusts the caller to pass a complete and correctly-memoized `deps` array; if `fetcher` itself is a fresh closure on every render (common, since callers pass inline arrow functions) but `deps` doesn't change, `refetch`'s `useCallback` will still be a *stale* closure over an old `fetcher` reference after the first render, though since `deps` is what actually triggers re-running (not `fetcher` identity), this doesn't cause bugs in typical usage where the fetcher's logic itself doesn't change between renders. On error, `data` is

left untouched (not reset to `initialValue` or `null`) — the previous successful result stays visible/stale under the hood while `error` is set, which callers may or may not account for in their rendering.

`client/src/hooks/useAwsConnection.js`

PURPOSE

Encapsulates fetching and displaying the current user's AWS connection status, plus a `disconnect` action — shared by the Navbar (and any other future component) so each consumer gets its own independent fetch rather than a global store/context.

WHEN THIS FILE IS CALLED

On mount of any component that uses it (currently only `Navbar.jsx`, which mounts on every page since it's rendered by every page component per the dep graph).

WHO CALLS THIS FILE

`client/src/components/Navbar.jsx`.

WHICH FILES IT DEPENDS ON

`client/src/services/awsConnection.service.js` — for `getAwsConnection` and `disconnectAwsAccount`. External: `react` (`useEffect`, `useState`).

EXPORTED FUNCTIONS/VALUES

- `useAwsConnection()`
- Purpose: Load and expose the current AWS connection state, and provide a way to disconnect it.
- Input: none (no parameters).
- Output/Return value: `{ connection, loading, disconnecting, disconnect }`
- `connection` — the connection object from `getAwsConnection()`, or `null` initially/if disconnected/on error.
- `loading` — boolean, `true` until the initial `getAwsConnection()` call settles (success or failure).
- `disconnecting` — boolean, `true` while a `disconnect()` call is in flight.
- `disconnect` — async function; calls `disconnectAwsAccount()` and, on success, sets `connection` to `null`.
- Internal steps:
 1. On mount, a `useEffect` with an empty dependency array declares a local `cancelled` flag, then calls `getAwsConnection().then(...).catch(...).finally(...)`.
 2. On success: `if (!cancelled) setConnection(data)`.
 3. On failure: `console.error(error)` (no `error` state is exposed to the caller — only console-logged).
 4. In `finally`: `if (!cancelled) setLoading(false)`.
 5. The effect's cleanup function sets `cancelled = true` — guards against setting state after the component using this hook has unmounted (e.g. fast route navigation away from a page while the request is still in flight).
 6. `disconnect` (not wrapped in `useEffect`, callable anytime): sets `disconnecting=true`, calls `await disconnectAwsAccount()`, on success sets `connection` to `null`; on failure, `console.error(error)` (again not exposed as state); `finally` sets `disconnecting=false`.
- Backend endpoints called (via the service layer): `GET /api/aws/account` (on mount), `DELETE /api/aws/disconnect` (on `disconnect()`).
- Error handling: Both the mount fetch and `disconnect()` swallow errors into `console.error` only — there is no `error` field in the returned object, so a failed load or failed disconnect is invisible to the consuming component beyond the browser console; the UI simply stays in its prior state (`connection` unchanged on a failed disconnect, `connection` stays `null`/unset on a failed initial load).
- Return values: see shape above.

DATA FLOW

Navbar mounts `useAwsConnection()`, which immediately fires `getAwsConnection()` through the service layer (→ `api.get("/aws/account")`, JWT attached by `api.js`'s request interceptor) to determine whether to show "Connect AWS" or connected-account UI. If the user clicks a disconnect control, Navbar calls the returned `disconnect()`, which calls `disconnectAwsAccount()` (→ `api.delete("/aws/disconnect")`) and clears local `connection` state on success, updating the Navbar UI without a full page reload.

EXAMPLE EXECUTION

```
const { connection, loading, disconnecting, disconnect } = useAwsConnection();
// on mount: loading=true, connection=null
// after GET /api/aws/account resolves: loading=false, connection={ region: "us-east-1", accountAlias: "prod", ... }

await disconnect();
// disconnecting=true -> DELETE /api/aws/disconnect -> disconnecting=false, connection=null
```

IMPORTANT NOTES

No polling — this hook fetches connection status exactly once per mount (empty `useEffect` dependency array `[]`); it does not re-fetch on an interval or re-check after other parts of the app change the connection (e.g. after `ConnectAws.jsx` successfully connects an account elsewhere, a mounted `Navbar`'s `useAwsConnection` instance will not automatically learn about it — only a fresh mount, e.g. from a route change that remounts `Navbar`, would refetch). No caching/sharing across instances — each component calling this hook gets its own independent `GET /api/aws/account` request and its own local state; two components using it simultaneously issue two separate network calls rather than sharing one cached result, per the file's own header comment ("each caller gets its own fetch (no global store)"). Uses a `cancelled` closure flag (not `AbortController`) to prevent post-unmount state updates — the underlying HTTP request itself is not aborted, only the resulting `setState` calls are suppressed.

 `client/src/hooks/useClickOutside.js`

PURPOSE

Small reusable hook that invokes a handler callback when a `mousedown` event occurs outside a given DOM ref's element — used to close dropdown/menu UI.

WHEN THIS FILE IS CALLED

Whenever a component with a dismissible dropdown/menu (currently the `Navbar`'s user/account menu) is mounted, for as long as that component stays mounted.

WHO CALLS THIS FILE

`client/src/components/Navbar.jsx`.

WHICH FILES IT DEPENDS ON

None internally. External: `react` (`useEffect`).

EXPORTED FUNCTIONS/VALUES

- `useClickOutside(ref, handler)`
- Purpose: Detect clicks outside a referenced element and invoke a callback.
- Input: `ref` — a React ref object (e.g. from `useRef()`) attached to the DOM element that should be considered "inside"; `handler` — callback function invoked with the triggering `MouseEvent` when a click lands outside `ref.current`.
- Output/Return value: none (`undefined`) — this hook only registers a side-effecting event listener; it returns nothing for the caller to use directly.
- Internal steps:
 1. `useEffect` (deps: `[ref, handler]`) defines `listener = (event) => { if (!ref.current || ref.current.contains(event.target)) return; handler(event); }` — no-ops if the ref isn't attached yet, or if the click target is inside (or is) the referenced element.
 2. `document.addEventListener("mousedown", listener)`.
 3. Cleanup function: `document.removeEventListener("mousedown", listener)`, run on unmount or whenever `ref` / `handler` change (re-registering the listener each time).
- Error handling: none — no try/catch; `handler` is expected to be a safe callback (e.g. `() => setMenuOpen(false)`).
- Return values: none.

DATA FLOW

Purely a DOM-event/UI-state concern with no backend interaction: `Navbar` attaches a ref to its dropdown container and passes a "close menu" callback as `handler`; any `mousedown` anywhere in the document that isn't inside that container triggers the callback, closing the menu.

EXAMPLE EXECUTION

```
const menuRef = useRef(null);
useClickOutside(menuRef, () => setMenuOpen(false));
// user clicks inside the referenced <div ref={menuRef}> -> listener no-ops (menu stays open)
// user clicks anywhere else in the document -> setMenuOpen(false) runs (menu closes)
```

IMPORTANT NOTES

Listens on `mousedown` (not `click`) — closes the menu on the initial press rather than waiting for a full click (press+release) to complete, which is the conventional choice for this pattern since it feels more responsive and avoids conflicts with click handlers inside the outside area. Because `handler` is in the effect's dependency array, passing an inline arrow function as `handler` on every render (rather than a memoized one) causes the listener to be removed and re-added on every render — functionally harmless here since it's a cheap operation, but worth noting for anyone reusing this hook in a hot-render-path component.

`client/src/hooks/useRecommendationActions.js`

PURPOSE

Centralizes the ignore/resolve/stop action logic (API call + optimistic-ish update callback + toast notification + per-row loading state) shared by both `RecommendationTable.jsx`'s row buttons and `ResourceDetailsDrawer.jsx`'s footer buttons, so this logic exists in exactly one place rather than being duplicated across those two UIs.

WHEN THIS FILE IS CALLED

Whenever a user clicks Ignore, Resolve, or Stop on a recommendation, from either the recommendations table or the resource detail drawer.

WHO CALLS THIS FILE

`client/src/components/RecommendationTable.jsx`.

WHICH FILES IT DEPENDS ON

`client/src/services/recommendation.service.js` (`ignoreRecommendation` , `resolveRecommendation` , `stopRecommendation`);
`client/src/utils/toast.jsx` (`showSuccessToast` , `showErrorToast`). External: `react` (`useState`).

EXPORTED FUNCTIONS/VALUES

- `useRecommendationActions(onRecommendationUpdate, refresh)`
- Purpose: Provide ready-to-wire handlers for the three recommendation actions, with shared loading-state and toast behavior.
- Input: `onRecommendationUpdate` — callback invoked with the updated recommendation object after a successful ignore/resolve/stop (used by the caller to patch its local list); `refresh` — optional async callback run only after a successful **stop** action (e.g. to refetch dependent data like resource status).
- Output/Return value: `{ stoppingId, actionId, handleIgnore, handleResolve, handleStop }`
- `stoppingId` — the recommendation `id` currently being stopped, or `null` (used to show a per-row spinner only on the Stop button of the affected row).
- `actionId` — the recommendation `id` currently being ignored or resolved, or `null` (shared between both actions — see Important notes).
- `handleIgnore(id)` — calls `ignoreRecommendation`.
- `handleResolve(id)` — calls `resolveRecommendation`.
- `handleStop(id)` — calls `stopRecommendation`, then `refresh` on success.
- Internal steps (shared `performAction(id, actionFn, setLoadingId, { successMessage, errorMessage }, onSuccess)` helper, not exported):
 1. `setLoadingId(id)` — sets either `actionId` or `stoppingId` depending on which setter was passed in.
 2. `const updated = await actionFn(id)` — calls the relevant service function.
 3. `onRecommendationUpdate(updated)` — always invoked with the fresh recommendation.
 4. `if (onSuccess) await onSuccess()` — only supplied for `handleStop` (bound to `refresh`).
 5. `showSuccessToast(successMessage)`.
 6. On thrown error: `console.error(errorMessage, error)`, then `showErrorToast(error.response?.data?.message || errorMessage)` — prefers the backend's own error message if present.
 7. `finally: setLoadingId(null)` — always clears the loading id regardless of outcome.

- Backend endpoints called (via service layer): `PATCH /api/recommendations/:id/ignore` (`handleIgnore`), `PATCH /api/recommendations/:id/resolve` (`handleResolve`), `POST /api/recommendations/:id/stop` (`handleStop`).
- Error handling: Catches any error from the service call, logs it with a context-specific message, and shows an error toast using the backend's message when available (`error.response?.data?.message`) or a hardcoded fallback per action ("Failed to ignore recommendation.", "Failed to resolve recommendation.", "Failed to stop EC2 instance."). The loading flag is always cleared via `finally` , so the UI never gets stuck showing a spinner after a failure.
- Return values: see shape above.

DATA FLOW

A user clicks an action button in `RecommendationTable` (or, indirectly, `ResourceDetailsDrawer`); the corresponding handler calls the shared `performAction` , which sets the correct per-action loading id (so only the clicked row shows a spinner), calls the recommendation service function (→ `api` → backend), then calls `onRecommendationUpdate` so the parent component can splice the updated recommendation into its in-memory list without a full refetch, then (for stop only) runs `refresh` for any dependent data, then shows a toast reflecting success or failure.

EXAMPLE EXECUTION

```
const { stoppingId, actionId, handleIgnore, handleResolve, handleStop } =
  useRecommendationActions(updateLocalList, refetchResourceDetails);

await handleStop("64f0c2ab12de34f5");
// stoppingId = "64f0c2ab12de34f5" (while in-flight)
// -> POST /api/recommendations/64f0c2ab12de34f5/stop
// -> updated recommendation returned -> updateLocalList(updated)
// -> await refetchResourceDetails()
// -> showSuccessToast("EC2 instance stopped successfully.")
// -> stoppingId = null
```

IMPORTANT NOTES

`actionId` is shared between `handleIgnore` and `handleResolve` (both pass `setActionId` as `setLoadingId`) — if a user could trigger both actions on different rows in rapid succession, the single `actionId` state would only reflect the most recently started action, potentially showing a loading spinner on the wrong row or clearing it prematurely for the other; `stoppingId` is kept separate specifically because Stop is the one action with a real-world side effect (actually stopping an EC2 instance) rather than just a status change, so the file's author gave it isolated state. Error messages prefer the backend's `error.response?.data?.message` over the hardcoded fallback, meaning server-side validation/business-logic errors (e.g. "Instance is already stopped") surface directly to the user when present.

 `client/src/hooks/useScanTrigger.js`

PURPOSE

Drives the "Run Scan" button's full UX: triggers the actual backend scan, and independently paces a purely client-side, illustrative step-by-step progress readout (since the backend scan endpoint is one opaque blocking request with no incremental status API). Lives at hook level because the trigger button itself moved into the global Navbar (reachable from every page) rather than being local to just the Dashboard.

WHEN THIS FILE IS CALLED

Whenever the user clicks "Run Scan" from the Navbar, on any page.

WHO CALLS THIS FILE

`client/src/pages/Analytics.jsx` , `AuditLogs.jsx` , `Dashboard.jsx` , `NotificationSettings.jsx` , `Recommendations.jsx` , `ScanDetail.jsx` , `ScanHistory.jsx` , `ScheduleScan.jsx` (i.e. essentially every page, since each page presumably renders `Navbar` , which is where the actual scan-trigger UI lives — though the dep graph lists the pages, not `Navbar.jsx` , as direct importers, meaning each page instantiates its own `useScanTrigger` and presumably passes state/handlers down into `Navbar` as props, rather than `Navbar` calling the hook itself).

WHICH FILES IT DEPENDS ON

`client/src/services/dashboard.service.js` (`scanInfrastructure`); `client/src/constants/scanSteps.js` (`SCAN_STEPS`); `client/src/utils/toast.jsx` (`showSuccessToast` , `showErrorToast` , `showInfoToast`). External: `react` (`useRef` , `useState`).

EXPORTED FUNCTIONS/VALUES

- `useScanTrigger(onSuccess)`

- Purpose: Trigger a backend infrastructure scan while driving a client-paced step progress UI, and notify the caller on success so it can refresh its own data.
- Input: `onSuccess` — optional async callback invoked after the scan completes successfully (e.g. a page's `refetch`).
- Output/Return value: `{ scanning, scanError, runScan, stepIndex, scanPhase }`
- `scanning` — boolean, `true` for the full duration of `runScan` (from click until the function's `finally` block).
- `scanError` — `null`, or the extracted error message string if the last scan failed.
- `runScan` — the async function to call on button click.
- `stepIndex` — number, current index into `SCAN_STEPS` for the progress UI (0-based).
- `scanPhase` — one of `"idle"`, `"running"`, `"success"`, `"error"` — drives which visual state the progress modal shows.
- Internal steps (`runScan`):
 1. Sets `scanning=true`, `scanError=null`, `scanPhase="running"`, `stepIndex=0`; shows `showInfoToast("Infrastructure scan started.")`.
 2. Starts `timerRef.current = window.setInterval(...)` at `STEP_INTERVAL_MS = 900` ms, incrementing `stepIndex` by 1 each tick but capping at `MAX_AUTO_STEP = SCAN_STEPS.length - 2` (i.e. auto-advance stops one step short of the final "Save Database" step — that step only completes once the real request resolves, so the UI can never claim completion before the backend actually confirms it).
 3. `await scanInfrastructure()` — the actual blocking `POST /api/scan` call.
 4. On success: `window.clearInterval(timerRef.current)`, `setStepIndex(SCAN_STEPS.length - 1)` (jumps straight to the final step), `setScanPhase("success")`, calls `await onSuccess()` if provided, shows `showSuccessToast("Infrastructure scan completed successfully.")`, then `await wait(COMplete_HOLD_MS)` (800ms) — holds the "success" UI state visible briefly before the modal/indicator would close.
 5. On error: `window.clearInterval(timerRef.current)`, `console.error("Scan failed:", error)`, derives `message = error.response?.data?.message || "Scan failed. Check backend logs and AWS credentials."`, sets `scanError=message`, `scanPhase="error"`, shows `showErrorToast(message)`, then `await wait(ERROR_HOLD_MS)` (1500ms) — holds the error UI state visible before closing.
 6. `finally`: `setScanning(false)`, `setScanPhase("idle")` — always resets scanning/phase regardless of outcome, after the relevant hold delay has elapsed.
- Backend endpoint called (via service layer): `POST /api/scan`.
- Error handling: Catches the rejected `scanInfrastructure()` promise, extracts `error.response?.data?.message` (falling back to a generic message mentioning backend logs and AWS credentials), stores it in `scanError`, and shows an error toast; the step-progress interval is always cleared on both success and failure paths so it never keeps ticking after the request settles.
- Return values: see shape above.

DATA FLOW

Some page-level component (or a shared layout wrapping Navbar) calls `useScanTrigger(refetchPageData)`; clicking "Run Scan" calls `runScan()`, which kicks off a local `setInterval` to animate through `SCAN_STEPS` (`ec2` → `s3` → `pricing` → `analyze`, stopping short of `save`) purely for visual feedback while the real `POST /api/scan` request is in flight server-side (running the actual EC2/S3/RDS/IAM scan pipeline). When the request resolves, the interval is cleared, the UI jumps to the final "Save Database" step, `onSuccess` runs (typically re-fetching the page's data so newly-scanned resources/recommendations appear), and a success toast shows; the whole `scanning` / `scanPhase` state resets to idle after a short hold delay.

EXAMPLE EXECUTION

```
const { scanning, scanError, runScan, stepIndex, scanPhase } = useScanTrigger(refetchDashboard);

await runScan();
// scanPhase: "running" -> stepIndex ticks 0,1,2 every 900ms (capped at SCAN_STEPS.length - 2 = 3)
// -> POST /api/scan resolves after (say) 4200ms
// -> stepIndex = 4 (SCAN_STEPS.length - 1, "Save Database"), scanPhase = "success"
// -> await refetchDashboard()
// -> showSuccessToast("Infrastructure scan completed successfully.")
// -> 800ms hold -> scanning=false, scanPhase="idle"
```

IMPORTANT NOTES

The step progress is explicitly **not** driven by real backend events — per the file's own header comment, the backend scan is "a single opaque request with no incremental status API," so `SCAN_STEPS` (from `constants/scanSteps.js`) is animated on a fixed client-side timer (`STEP_INTERVAL_MS = 900` ms) that has no actual correlation to which AWS service the backend is currently scanning; a scan that finishes in 2

seconds vs. 10 seconds will show the same simulated step pacing up to the cap, then jump straight to the final step whenever the real request actually resolves. `MAX_AUTO_STEP = SCAN_STEPS.length - 2` guarantees the auto-advancing timer can never reach (let alone pass) the final step on its own — only the real response completing can advance to `SCAN_STEPS.length - 1`. The hold delays (`COMPLETE_HOLD_MS = 800` , `ERROR_HOLD_MS = 1500`) exist purely for UX (giving the user time to read the final success/error state) and add real latency to when `scanning` finally becomes `false` after the network request itself has already settled.

`client/src/hooks/useTheme.js`

PURPOSE

Manages light/dark theme state by toggling a `dark` class on `<html>` and persisting the choice to `localStorage`, with the OS `prefers-color-scheme` media query as the initial default when no explicit choice has been stored yet.

WHEN THIS FILE IS CALLED

On mount of `Navbar.jsx` (rendered on every page), and whenever the user clicks the theme toggle control.

WHO CALLS THIS FILE

`client/src/components/Navbar.jsx`.

WHICH FILES IT DEPENDS ON

None internally. External: `react` (`useEffect` , `useState`).

EXPORTED FUNCTIONS/VALUES

- `useTheme()`
- Purpose: Provide the current theme and a way to toggle it, keeping the DOM/localStorage in sync.
- Input: none.
- Output/Return value: `{ theme, toggleTheme }`
- `theme` — `"light"` or `"dark"`.
- `toggleTheme` — function; flips `theme` between `"dark"` and `"light"`.
- Internal steps:
 1. `useState(getInitialTheme)` — lazy initializer calling the module-level `getInitialTheme()`: reads `window.localStorage.getItem("cloudlens-theme")` (constant `STORAGE_KEY = "cloudlens-theme"`); if it's exactly `"light"` or `"dark"`, uses that; otherwise falls back to `window.matchMedia("(prefers-color-scheme: dark)").matches ? "dark" : "light"`.
 2. A `useEffect` (deps: `[theme]`) runs on every `theme` change: `document.documentElement.classList.toggle("dark", theme === "dark")` (adds/removes the `dark` class on `<html>`, which Tailwind's `dark:` variant selectors key off of), then `window.localStorage.setItem(STORAGE_KEY, theme)`.
 3. `toggleTheme = () => setTheme((prev) => (prev === "dark" ? "light" : "dark"))`.
- Backend endpoint called: none — entirely client-side/browser-storage state.
- Error handling: none — no try/catch around `localStorage` / `matchMedia` access (would throw in environments where these APIs are unavailable/blocked, though this isn't handled).
- Return values: see shape above.

DATA FLOW

Navbar (mounted on every page) calls `useTheme()`; each mount independently reads the current theme from `localStorage` / OS preference and applies the `dark` class to `<html>` via its own `useEffect`. Since the actual "source of truth" is the DOM class and `localStorage` (not React context/state shared across components), multiple independent `useTheme()` instances across route changes stay consistent with each other without needing a shared provider — each new mount just re-derives the same value from the same persistent storage.

EXAMPLE EXECUTION

```
// First-ever visit, no stored preference, OS is in dark mode:
const { theme, toggleTheme } = useTheme();
// theme = "dark" (from prefers-color-scheme) -> <html class="dark"> applied, localStorage["cloudlens-theme"] = "dark"

toggleTheme();
// theme = "light" -> <html> loses "dark" class, localStorage["cloudlens-theme"] = "light"
```

IMPORTANT NOTES

No shared context/provider — per the file's own header comment, this is a deliberate design choice: "each page mounts its own `<Navbar/>` (and therefore its own `useTheme` call), but since the actual state lives in the DOM (`.dark` on `<html>`) and `localStorage` rather than React state, the theme stays in sync across route changes without needing a shared context/provider." This means if two `useTheme()` instances were ever mounted simultaneously on the same page (not the current usage pattern, but structurally possible), toggling one would update `localStorage` /the DOM class but not the other instance's own `theme` state until it re-renders for an unrelated reason — there's no cross-instance event/subscription mechanism.

 `client/src/constants/scanSteps.js`

PURPOSE

Defines the ordered list of illustrative "steps" shown in the scan progress UI (modal), mirroring the backend's scan pipeline stages (`server/src/services/scanner`) as a friendly, human-readable readout — purely presentational data, not driven by real backend step events.

WHEN THIS FILE IS CALLED

Imported once at module load by the two consumers; its exported constant is read whenever the scan progress UI needs to render or step through the list.

WHO CALLS THIS FILE

`client/src/components/ScanProgressModal.jsx` , `client/src/hooks/useScanTrigger.js` .

WHICH FILES IT DEPENDS ON

None (no imports).

EXPORTED FUNCTIONS/VALUES

- `SCAN_STEPS` (array of objects, not a function)
- Purpose: Ordered list of scan pipeline stages for progress-UI rendering.
- Value: 5 entries, each `{ key, label }` :
 1. `{ key: "ec2", label: "Fetch EC2" }`
 2. `{ key: "s3", label: "Fetch S3" }`
 3. `{ key: "pricing", label: "Fetch Pricing" }`
 4. `{ key: "analyze", label: "Analyze" }`
 5. `{ key: "save", label: "Save Database" }`
- Consumed by `useScanTrigger.js` for its `MAX_AUTO_STEP = SCAN_STEPS.length - 2` cap and `stepIndex` bounds, and by `ScanProgressModal.jsx` to render the actual step list/labels in the UI.

DATA FLOW

`useScanTrigger` imports `SCAN_STEPS` to know how many steps exist and to compute the auto-advance cap and the final index; `ScanProgressModal` imports the same array to render each step's label and highlight the current one based on the `stepIndex` value `useScanTrigger` produces. No network or state involved in this file itself — it is static, immutable reference data.

EXAMPLE EXECUTION

```
import { SCAN_STEPS } from "../constants/scanSteps";
SCAN_STEPS.length // -> 5
SCAN_STEPS[3] // -> { key: "analyze", label: "Analyze" }
```

IMPORTANT NOTES

Per its own header comment, this list exists because "the backend scan is a single opaque request with no incremental status API, so steps are paced on the frontend rather than driven by real per-step events from the server" — i.e. this file's ordering/timing has no live connection to which AWS service (EC2/S3/RDS/IAM) the backend scanner is actually processing at a given moment; it is purely a UX simulation mirroring the *conceptual* pipeline order, notably omitting RDS and IAM from the visible steps even though the backend scanner service does scan those resource types (per the backend `scanner.service.js` dependency graph, which orchestrates `ec2.scanner.js` , `iam.scanner.js` , `rds.scanner.js` , and `s3.scanner.js`).

PART 4



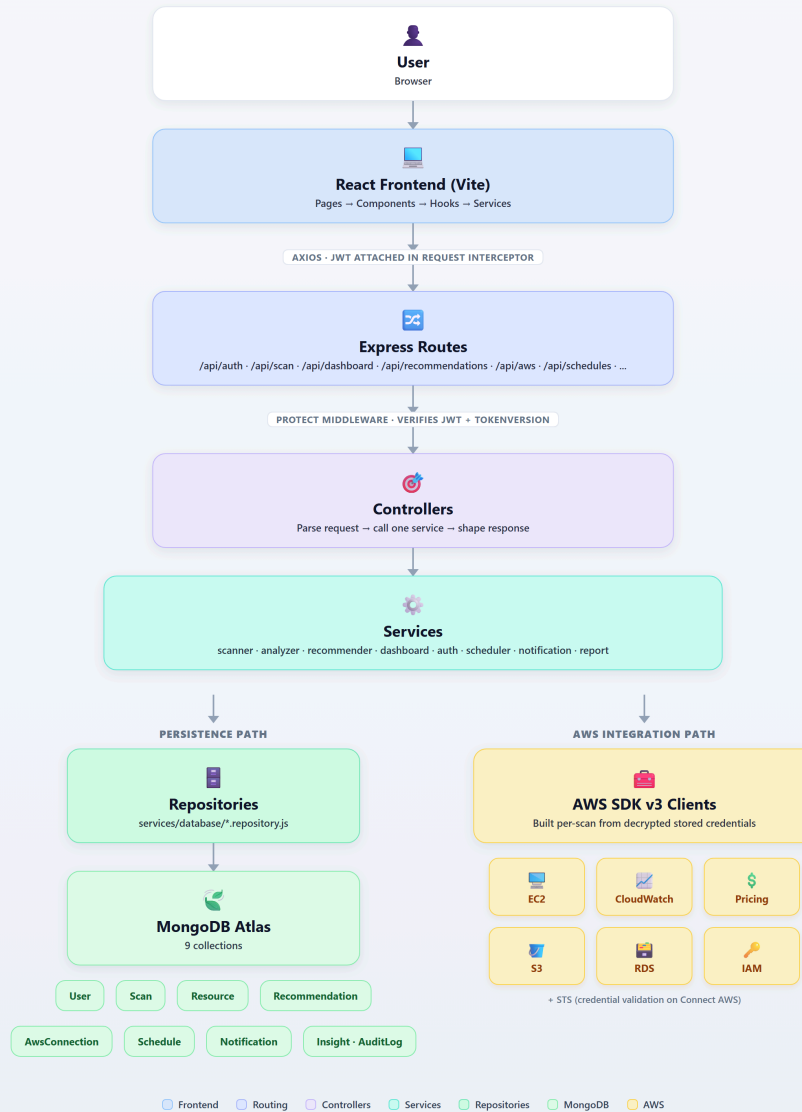
Architecture Diagram

One picture of the whole system.

Part 4 — Architecture Diagram

CloudLens — System Architecture

MERN stack · request flow from browser to AWS and back



Every layer below Controllers is reached through exactly one path — routes never call services directly, and only Repositories import Mongoose models.

The full request path: User → React Frontend → Axios (JWT attached) → Express Routes → protect middleware → Controllers → Services, which then split into two paths — persistence (Repositories → MongoDB Atlas's 9 collections) and AWS integration (AWS SDK v3 clients → EC2 / CloudWatch / Pricing / S3 / RDS / IAM, plus STS for credential validation).

PART 5



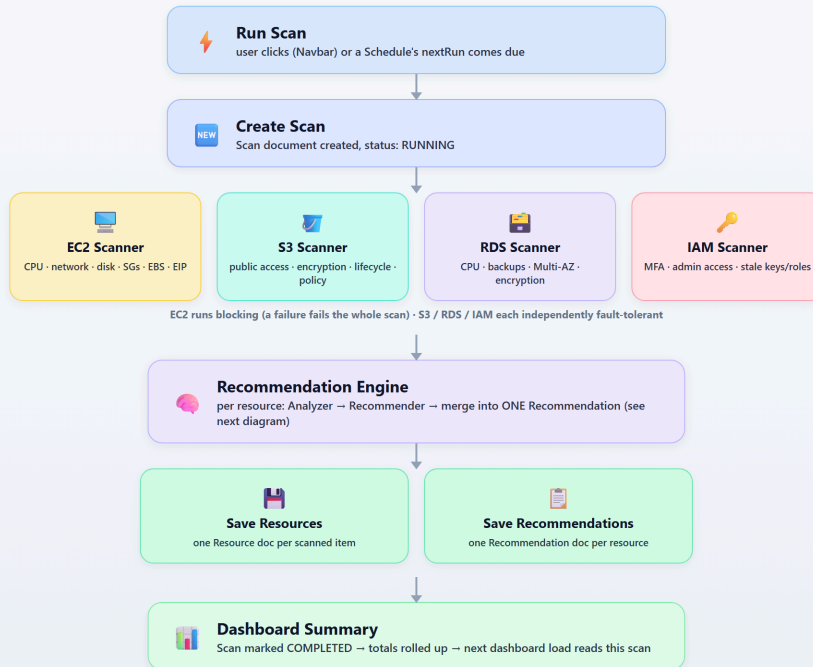
Scan Flow

What happens between clicking “Run Scan” and the dashboard updating.

Part 5 — Scan Flow

CloudLens — Scan Execution Flow

One POST /api/scan → four resource-type scanners → one merged recommendation per resource



One POST /api/scan fans out to four independent per-service scanners. EC2 runs blocking — a failure there fails the whole scan. S3, RDS, and IAM are each individually fault-tolerant, so a permissions gap in any one of them just zeroes out that service's contribution instead of aborting the scan.

PART 6



Recommendation Engine Flow

How a raw AWS API response becomes a priced, actionable finding.

Part 6 — Recommendation Engine Flow

CloudLens — Recommendation Engine Flow

Per resource, per check — how a raw AWS API response becomes a priced, actionable finding



WORKED EXAMPLE — EC2, STOPPED INSTANCE

Resource: `state: "stopped"` → Analyzer: `isRunning = false` → Metrics: `averageCPU = 0` (no CloudWatch data) → Business Rules: `isRunning` guard short-circuits to `severity: LOW, action: NONE` → Recommendation: "already stopped, no active cost" → Estimated Savings: `$0` → saved as OPEN, LOW severity.

Per resource, per check: a raw AWS response is normalized by a mapper, reduced to booleans/derived values by an analyzer, combined with CloudWatch metrics, run through threshold-based business rules, merged into one recommendation (worst severity wins), priced against the AWS Price List API, and saved as an OPEN Recommendation document.

PART 7



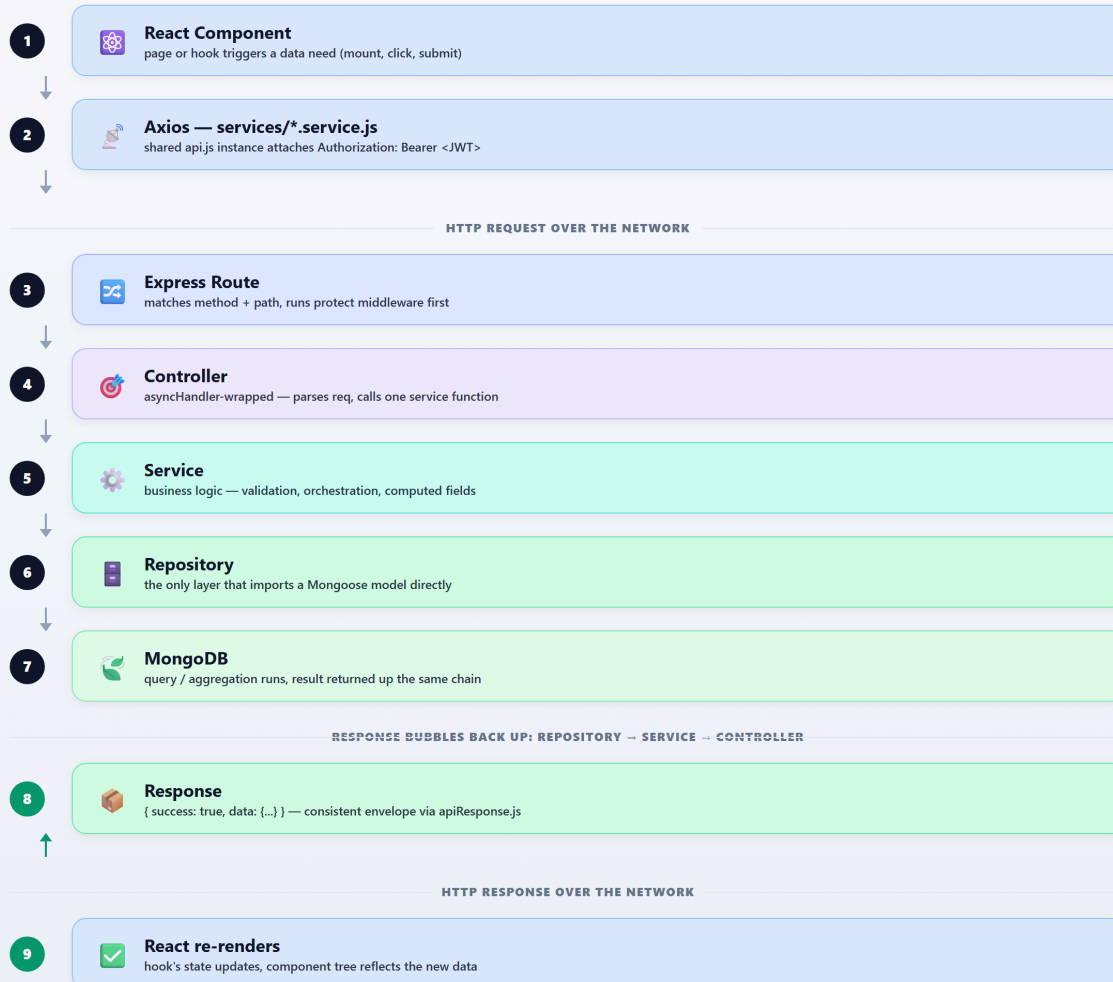
API Flow

The request/response path every endpoint in the app follows.

Part 7 — API Flow

CloudLens — API Request/Response Flow

Every REST call in the app follows this exact path, down and back up



Every REST call in the app, request and response, through the exact same nine stages — no layer is ever skipped, and only the Repository stage is allowed to import a Mongoose model.

PART 8



Database Documentation

Every collection: fields, relationships, indexes, and a real example document.

Part 8 — Database Documentation

CloudLens uses **MongoDB** (via Mongoose) as its only datastore. There are **9 collections**, all defined under `server/src/models/`. Every model uses Mongoose's automatic `_id` (ObjectId) as its primary key. Every write path goes through a **repository** (`server/src/services/database/*.repository.js`) — no controller or scanner ever calls `Model.find()` / `Model.save()` directly. That indirection is what lets the recommendation-status audit (OPEN/RESOLVED/IGNORED filtering) live in one place instead of being re-implemented at every call site.

Entity-Relationship Overview

User (1)	————< (1) AwsConnection	one AWS account connected per user
User (1)	————< (1) Schedule	one scan schedule per user
User (1)	————< (1) Notification	one notification preference doc per user
User (1)	————< (N) AuditLog	every security-relevant action a user takes
User (1)	————< (N) Scan	every scan run belongs to exactly one user
Scan (1)	————< (N) Resource	every resource discovered in that scan
Scan (1)	————< (N) Recommendation	every recommendation generated in that scan
Scan (1)	————< (N) Insight	fleet-wide S3 insights generated in that scan

Nothing uses Mongoose `ref` + `.populate()` in practice for the Scan→Resource/Recommendation relationship — `scanId` is stored as a plain indexed foreign key and repositories query by it directly (`Resource.find({ scanId })`). This is a deliberate simplicity choice: the app never needs a resource embedded inside a scan document, only "give me everything for this scan," which a direct query answers without the overhead of `.populate()`.

1. User

File: `server/src/models/User.js` Collection: `users`

PURPOSE

The account a human logs into. Everything else in the system (AWS connection, scans, schedules, notifications, audit logs) is scoped to a `userId` that points back here. CloudLens is multi-tenant at the data layer — every repository query is filtered by the authenticated user's ID, so one user can never see another's resources.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
<code>fullName</code>	String	yes	—	Display name, shown in the Navbar
<code>email</code>	String	yes	—	<code>unique</code> , <code>trim</code> , <code>lowercase</code> — the login identifier
<code>password</code>	String	yes	—	bcrypt hash, never the plaintext password
<code>tokenVersion</code>	Number	no	<code>0</code>	Incremented on "logout from all devices"; every JWT embeds the version it was issued with, and <code>protect</code> middleware rejects any token whose version doesn't match the current value
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	<code>timestamps: true</code>

RELATIONSHIPS

Referenced by `userId` from `AwsConnection`, `Schedule`, `Notification`, `AuditLog`, `Scan`, `Resource`, `Recommendation`, `Insight` — every other collection in the system.

INDEXES

`email` has a unique index (from `unique: true`). No compound indexes.

SECURITY NOTES

`toJSON` is overridden to strip `password` from every serialized response — so even if a controller forgets to `.select("-password")` or accidentally does `res.json(user)` with a raw Mongoose doc, the hash can never leak in an API response.

EXAMPLE DOCUMENT

```
{
  "_id": "665f1a2b3c4d5e6f7a8b9c0d",
  "fullName": "Akshat Johri",
  "email": "akshat@example.com",
  "password": "$2b$10$abcdefghijklmnopqrstuv...",
  "tokenVersion": 2,
  "createdAt": "2026-01-10T08:12:00.000Z",
  "updatedAt": "2026-07-20T14:03:11.000Z"
}
```

2. AwsConnection

File: `server/src/models/AwsConnection.js` Collection: `awsconnections`

PURPOSE

Stores exactly one set of AWS credentials per user — the credentials the scanner uses to call EC2/RDS/S3/IAM/CloudWatch/Pricing on the user's behalf.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES		
<code>userId</code>	ObjectId → User	yes	—	<code>unique: true</code> — enforces one connection per user at the DB level		
<code>accountId</code>	String	yes	—	The AWS account ID, resolved via STS <code>GetCallerIdentity</code> at connect time		
<code>accountAlias</code>	String	no	—	Optional friendly label		
<code>region</code>	String	yes	—	Default region used for all SDK clients		
<code>accessKeyId</code>	String	yes	—	Stored in plaintext — this half of an AWS credential pair is not secret on its own (it's an identifier, not a secret)		
<code>encryptedSecretKey</code>	String	yes	—	The AWS secret access key, encrypted at rest via <code>utils/encryption.js</code> (AES) before being saved — never stored in plaintext		
<code>status</code>	String enum	no	<code>CONNECTED</code>	<code>`CONNECTED \</code>	<code>DISCONNECTED</code>	<code>INVALID`</code>
<code>connectedAt</code>	Date	no	<code>Date.now</code>			
<code>lastValidatedAt</code>	Date	no	—	Updated whenever the connection is re-validated (e.g. before a scan)		

RELATIONSHIPS

One-to-one with `User` (`unique: true` on `userId`). Consumed by `services/aws/*.service.js` (via `factories/awsClient.factory.js`) to build authenticated AWS SDK clients for every scan.

INDEXES

Unique index on `userId` .

SECURITY NOTES

`toJSON` strips `encryptedSecretKey` on every serialization — defense in depth so the encrypted blob (not just the plaintext) never leaves the server in an API response, even though it's already encrypted.

EXAMPLE DOCUMENT

```
{
  "_id": "665f1b3c4d5e6f7a8b9c0d1e",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "accountId": "123456789012",
  "accountAlias": "prod-account",
  "region": "ap-south-1",
  "accessKeyId": "AKIAxxxxxxxxxxxxxx",
  "encryptedSecretKey": "a1b2c3d4:e5f6...(iv:ciphertext hex)",
  "status": "CONNECTED",
  "connectedAt": "2026-06-01T10:00:00.000Z",
  "lastValidatedAt": "2026-07-31T09:15:00.000Z"
}
```

3. Scan

File: `server/src/models/Scan.js` Collection: `scans`

PURPOSE

One document per scan run. This is the "root" of every scan's results — every `Resource`, `Recommendation`, and `Insight` created during that run carries this scan's `_id` as `scanId`. The dashboard always reads from the **latest** `Scan` for a user (`Scan.findOne({ userId }).sort({ createdAt: -1 })`).

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES		
<code>userId</code>	ObjectId → User	yes	—	indexed		
<code>startedAt</code>	Date	no	<code>Date.now</code>			
<code>completedAt</code>	Date	no	—	Set when the scan finishes (success or failure)		
<code>status</code>	String enum	no	<code>RUNNING</code>	<code>`RUNNING \</code>	COMPLETED	FAILED`
<code>totalResources</code>	Number	no	<code>0</code>	Sum across all 4 scanners		
<code>estimatedSavings</code>	Number	no	<code>0</code>	Sum of every recommendation's <code>estimatedSavings</code> from this scan		
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	<code>timestamps: true</code>		

RELATIONSHIPS

Parent of `Resource`, `Recommendation`, and `Insight` via `scanId`. Child of `User` via `userId`.

INDEXES

`userId` indexed (single-field).

EXAMPLE DOCUMENT

```
{
  "_id": "665f20000000000000000001",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "startedAt": "2026-07-31T09:00:00.000Z",
  "completedAt": "2026-07-31T09:01:42.000Z",
  "status": "COMPLETED",
  "totalResources": 14,
  "estimatedSavings": 87.32,
  "createdAt": "2026-07-31T09:00:00.000Z",
  "updatedAt": "2026-07-31T09:01:42.000Z"
}
```

4. Resource

File: `server/src/models/Resource.js` Collection: `resources`

PURPOSE

One document per AWS resource discovered by a scan (an EC2 instance, an RDS instance, an S3 bucket, an IAM user/role/root account). This is the "what exists" record — separate from `Recommendation`, which is the "what should you do about it" record. A resource always exists even if it has zero actionable recommendations.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES					
<code>userId</code>	ObjectId → User	yes	—	indexed					
<code>scanId</code>	ObjectId → Scan	yes	—	which scan discovered this resource					
<code>resourceType</code>	String	yes	—	"EC2" \	"RDS"	"S3"	"IAM_USER"	"IAM_ROLE"	"IAM_ROOT_" (not a scherr enum — enf convention i scanner)
				\	\	\	\	\	
<code>resourceId</code>	String	yes	—	The AWS-native identifier (instance ID, bucket name, DB instance identifier, IAM user/role name)					
<code>name</code>	String	no	—	Human-friendly name (often same as <code>resourceId</code>)					
<code>state</code>	String	no	—	EC2: <code>running/stopped/...</code> ; RDS: <code>available/stopped/...</code> ; S3/IAM: not really state-bearing, left unset					
<code>metadata</code>	Object (<code>{}</code>)	no	<code>{}</code>	Schema-less bag — every scanner stuffs its own shape in here (CPU/network/disk stats for EC2, engine/storage details for RDS, encryption/lifecycle details for S3, MFA/policy details for IAM). This is why every frontend component that reads <code>resource.metadata</code> branches on <code>resourceType</code> first (see <code>ResourceTable.jsx</code> , <code>ResourceDetailsDrawer.jsx</code>)					

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—	

RELATIONSHIPS

Child of `Scan` (`scanId`) and `User` (`userId`). Not directly linked to `Recommendation` by a foreign key — the two are correlated at query time by matching `resourceId` (+ `userId`), since a resource and its recommendation are always written in the same scan pass.

INDEXES

`userId` indexed. No compound index on (`scanId`, `resourceType`) — queries filter in application code after a single `find({ scanId, userId })` .

EXAMPLE DOCUMENT (EC2)

```
{
  "_id": "665f200000000000000000010",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "scanId": "665f20000000000000000001",
  "resourceType": "EC2",
  "resourceId": "i-0abcd1234efgh5678",
  "name": "web-server-1",
  "state": "running",
  "metadata": {
    "instanceType": "t3.micro",
    "availabilityZone": "ap-south-1a",
    "publicIp": "13.234.1.1",
    "privateIp": "10.0.1.5",
    "launchTime": "2025-11-01T00:00:00.000Z",
    "averageCpu": 2.3,
    "networkIn": 12000,
    "networkOut": 8000,
    "diskReadBytes": 1000,
    "diskWriteBytes": 500,
    "diskReadOps": 3,
    "diskWriteOps": 2,
    "instanceAgeDays": 273,
    "securityGroups": [{ "groupId": "sg-01", "groupName": "default" }],
    "volumeCount": 1,
    "volumeIds": ["vol-0123"],
    "volumeTypes": ["gp2"],
    "encrypted": [false],
    "elasticIp": "NONE",
    "costComparison": { "currentMonthlyCost": 7.5, "optimizedMonthlyCost": 7.5, "monthlySavings": 0 }
  }
}
```

5. Recommendation

File: `server/src/models/Recommendation.js` **Collection:** `recommendations`

PURPOSE

The core "actionable finding" record — one per resource per scan (the scanners merge every sub-check for a resource into a single Recommendation document; see Part 6). This is what drives the dashboard's severity counts, the Recommendations table, savings totals, and the Ignore/Resolve workflow.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
<code>userId</code>	ObjectId → User	yes	—	indexed

FIELD	TYPE	REQUIRED	DEFAULT	NOTES				
<code>scanId</code>	ObjectId → Scan	yes	—					
<code>resourceId</code>	String	yes	—	Matches a <code>Resource.resourceId</code> from the same scan				
<code>category</code>	String enum	no	<code>COST</code>	<code>`COST \</code>	<code>SECURITY \</code>	<code>PERFORMANCE \</code>	<code>RELIABILITY \</code>	<code>C</code>
<code>severity</code>	String enum	yes	—	<code>`LOW \</code>	<code>MEDIUM \</code>	<code>HIGH</code> (there is no <code>CRITICAL</code> in the schema – the dashboard derives "critical" as HIGH + <code>SECURITY</code> category at read time, see <code>serviceAggregation.service.js</code>)		
<code>action</code>	String enum	yes	—	<code>NONE, STOP, DOWNSIZE, UPSIZE, REVIEW, ENABLE_MFA, DELETE_UNUSED_KEY, REVIEW_PERMISSIONS, ROTATE_ACCESS_KEY, REMOVE_ADMIN_ACCESS, DELETE_UNUSED_ROLE, TAG_RESOURCE</code>				
<code>confidence</code>	Number	no	<code>100</code>	0–100 score; EC2's CPU-based recommendation computes this via a weighted signal blend (see <code>ec2.analyzer.js</code>), everything else uses fixed per-check values				
<code>recommendation</code>	String	yes	—	Human-readable remediation text				
<code>reason</code>	String	yes	—	Human-readable justification (often the concatenation of every sub-check's own reason, pipe-separated)				
<code>hourlyPrice</code>	Number	no	<code>0</code>	From the AWS Price List API, only meaningful for EC2/RDS				
<code>monthlyCost</code>	Number	no	<code>0</code>	What this resource currently costs per month (0 for IAM; storage-only for a stopped RDS instance; 0 for a stopped EC2 instance)				

FIELD	TYPE	REQUIRED	DEFAULT	NOTES		
<code>estimatedSavings</code>	Number	no	<code>0</code>	What could be saved per month if the recommendation is followed		
<code>status</code>	String enum	no	<code>OPEN</code>	<code>`OPEN \</code>	<code>RESOLVED \</code>	<code>IGNORED`</code> — user-controlled via the Resolve/Ignore buttons
<code>createdAt</code> / <code>updatedAt</code>	Date	auto	—			

RELATIONSHIPS

Child of `Scan` and `User`. Correlated to `Resource` by matching `resourceId`.

INDEXES

`userId` indexed. No index on `scanId` or `status` despite both being extremely common filter fields — see the audit note in Part 9 (a candidate future improvement, not yet implemented, since Phase 3 of the audit scoped only Critical/High fixes).

STATUS LIFECYCLE (IMPORTANT, AND EASY TO GET WRONG — SEE THE AUDIT FINDINGS)

- `monthlyCost` sums must **never** filter by `status` — a `RESOLVED` recommendation doesn't mean AWS stopped billing for the resource, it just means the finding is marked handled.
- `estimatedSavings` / **severity counts** / **category counts** / **savings-opportunity lists** must filter to `status: "OPEN"` — a resolved or ignored recommendation is no longer outstanding work and shouldn't keep inflating "how many issues need attention."
- Historical/trend data** (`analytics.repository.js`) is deliberately left **unfiltered** — a trend chart shows what was true *at scan time*, not retroactively edited by a later Resolve/Ignore action.

EXAMPLE DOCUMENT

```
{
  "_id": "665f20000000000000000020",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "scanId": "665f20000000000000000001",
  "resourceId": "i-0abcd1234efgh5678",
  "category": "COST",
  "severity": "HIGH",
  "action": "STOP",
  "confidence": 88,
  "recommendation": "Instance is underutilized. Consider stopping or downsizing it.",
  "reason": "Average CPU utilization is only 2.30%",
  "hourlyPrice": 0.0104,
  "monthlyCost": 7.5,
  "estimatedSavings": 7.5,
  "status": "OPEN",
  "createdAt": "2026-07-31T09:00:12.000Z",
  "updatedAt": "2026-07-31T09:00:12.000Z"
}
```

6. Insight

File: `server/src/models/Insight.js` **Collection:** `insights`

PURPOSE

Fleet-wide, rule-based (not per-resource) observations — currently only generated by the S3 scanner (`services/insight/s3.insight.js`), e.g. "40% of your buckets have no lifecycle policy." Distinct from the dashboard's own `aiInsights` (built live from `serviceAggregation.service.js` 's `INSIGHT_RULES`) — this collection is scan-scoped and persisted, the dashboard's `aiInsights` is computed on read and not stored.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES				
userId	ObjectId → User	yes	—	indexed				
scanId	ObjectId → Scan	yes	—					
category	String enum	no	GENERAL	`SECURITY \	COST \	STORAGE \	LIFECYCLE \	GENERAL`
severity	String enum	no	INFO	`INFO \	LOW \	MEDIUM \	HIGH`	
message	String	yes	—					

RELATIONSHIPS

Child of `Scan` and `User` .

INDEXES

`userId` indexed.

EXAMPLE DOCUMENT

```
{
  "_id": "665f20000000000000000030",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "scanId": "665f20000000000000000001",
  "category": "LIFECYCLE",
  "severity": "MEDIUM",
  "message": "3 of 5 S3 buckets have no lifecycle policy configured.",
  "createdAt": "2026-07-31T09:01:00.000Z",
  "updatedAt": "2026-07-31T09:01:00.000Z"
}
```

7. Schedule

File: `server/src/models/Schedule.js` Collection: `schedules`

PURPOSE

One recurring-scan configuration per user. Read every 60 seconds by `scheduler.service.js` 's `node-cron` job — any schedule whose `nextRun` is due gets a scan triggered automatically.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES		
userId	ObjectId → User	yes	—	<code>unique: true</code> — one schedule per user		
frequency	String enum	yes	—	`hourly \	daily \	weekly`
enabled	Boolean	no	true	Toggled off without deleting the schedule		
lastRun	Date	no	null			
nextRun	Date	yes	—	Computed by <code>utils/schedule.util.js</code> from <code>frequency</code>		
createdAt	Date	no	Date.now	(not Mongoose <code>timestamps: true</code> — manually defaulted, and there is no <code>updatedAt</code>)		

RELATIONSHIPS

One-to-one with `User` .

INDEXES

Unique index on `userId` .

CONCURRENCY NOTE

`scheduler.service.js` keeps an in-memory `Set` of schedule IDs currently mid-scan so a slow scan can't be picked up twice by the next 60-second tick (see Part 2's entry for that file).

EXAMPLE DOCUMENT

```
{
  "_id": "665f20000000000000000040",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "frequency": "daily",
  "enabled": true,
  "lastRun": "2026-07-31T02:00:00.000Z",
  "nextRun": "2026-08-01T02:00:00.000Z",
  "createdAt": "2026-05-01T00:00:00.000Z"
}
```

8. Notification

File: `server/src/models/Notification.js` Collection: `notifications`

PURPOSE

Per-user email notification preferences, consumed by `services/notification/*.service.js` to decide whether to send a "scan completed" email, a "high severity found" email, or the weekly digest.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
<code>userId</code>	ObjectId → User	yes	—	<code>unique: true</code>
<code>email</code>	String	yes	—	Destination address (may differ from the login email if the user changes it)
<code>scanCompleted</code>	Boolean	no	<code>true</code>	
<code>highSeverity</code>	Boolean	no	<code>true</code>	
<code>weeklyReport</code>	Boolean	no	<code>true</code>	

RELATIONSHIPS

One-to-one with `User`.

INDEXES

Unique index on `userId`.

EXAMPLE DOCUMENT

```
{
  "_id": "665f20000000000000000050",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "email": "akshat@example.com",
  "scanCompleted": true,
  "highSeverity": true,
  "weeklyReport": false
}
```

9. AuditLog

File: `server/src/models/AuditLog.js` Collection: `auditlogs`

PURPOSE

An append-only trail of security/account-relevant events — logins, AWS connect/disconnect, scan start/complete, recommendation resolve/ignore, EC2 stop actions. Powers the Audit Logs page.

FIELDS

FIELD	TYPE	REQUIRED	DEFAULT	NOTES
userId	ObjectId → User	yes	—	indexed
action	String enum	yes	—	Values come from constants/auditActions.js 's AUDIT_ACTION_VALUES (e.g. USER_LOGIN , AWS_CONNECTED , AWS_DISCONNECTED , SCAN_STARTED , SCAN_COMPLETED , EC2_STOPPED , RECOMMENDATION_RESOLVED , RECOMMENDATION_IGNORED)
description	String	yes	—	Human-readable detail
createdAt	Date	auto	—	timestamps: { createdAt: true, updatedAt: false } — deliberately no updatedAt , since an audit record must never be mutated after creation

RELATIONSHIPS

Child of User .

INDEXES

Compound index: { userId: 1, createdAt: -1 } — the only compound index in the whole schema, purpose-built for "this user's audit log, newest first," which is exactly how the Audit Logs page queries it.

EXAMPLE DOCUMENT

```
{
  "_id": "665f2000000000000000000060",
  "userId": "665f1a2b3c4d5e6f7a8b9c0d",
  "action": "SCAN_COMPLETED",
  "description": "Scan completed: 14 resources scanned, $87.32/mo in potential savings identified.",
  "createdAt": "2026-07-31T09:01:42.000Z"
}
```

PART 9



How to Explain This Project

Interview-ready talking points for every major subsystem, plus the strongest specific stories worth having ready.

Part 9 — How to Explain This Project (Interview Notes)

The 30-second pitch

"CloudLens is a full-stack MERN app that connects to a user's AWS account and scans EC2, RDS, S3, and IAM for cost waste and security misconfigurations. It's built as a layered pipeline — AWS SDK wrapper → mapper → analyzer → recommender → scanner — so every one of the four resource types goes through the same shape of pipeline, just with resource-specific checks. Everything gets priced against the live AWS Price List API, merged into one recommendation per resource, and shown on a dashboard with severity/category tracking, a resolve/ignore workflow, scheduled recurring scans, and email notifications."

Overall Architecture

What to say: "It's a strictly layered backend — routes are just path+middleware+controller, controllers call exactly one service function and shape the HTTP response, services hold the business logic, and only the repository layer touches Mongoose models directly. That last rule is the one that matters most: it means a rule like 'savings totals must only count OPEN recommendations, not resolved ones' only has to be correct in one place, not re-verified at every call site that happens to query the Recommendation collection."

Why this design, if asked: Cutting across that vertical stack, the scanning pipeline itself is organized *by concern* rather than by resource type — there's an `aws/` folder, an `analyzer/` folder, a `recommender/` folder, a `scanner/` folder, each containing one file per resource type. Adding a fifth scanned AWS service means adding one file to each of those folders, not inventing a new subsystem.

Trade-off to be honest about if pushed: State management on the frontend is deliberately simple — no Redux/Context, just per-page hooks and localStorage for the JWT. That's a reasonable choice for an app this size, but it means auth state isn't reactive across tabs/components without a page reload, and if the app grew a lot more shared cross-page state, that would need revisiting.

Scanning

What to say: "A scan is one Express endpoint (`POST /api/scan`) that fans out to four independent per-service scanners. Each scanner follows the same four-stage pipeline: an AWS SDK wrapper fetches raw data, a mapper normalizes it, an analyzer turns it into booleans and derived metrics, and a recommender turns those into severity/action findings. Every resource gets exactly one merged Recommendation document, even though it might have gone through 6 or 7 independent checks internally — the merge picks the worst severity and combines the flagged messages."

A specific technical detail worth having ready: "EC2 is the only one of the four scanners that isn't individually fault-tolerant — if EC2 scanning throws (say, a missing IAM permission), the whole scan fails and the Scan document is stuck at RUNNING. S3, RDS, and IAM are each wrapped in their own try/catch, so a permissions gap there just zeroes out that service's contribution and the scan continues. That's a real, known asymmetry I'd fix by giving EC2 the same treatment — it's a good example of a bug that's easy to introduce when you add fault-tolerance to some but not all of a set of parallel operations, and a good thing to point out if asked 'what would you change.'"

A subtle bug worth telling as a war story: "There was a real bug where a *stopped* EC2 instance would get a 'stop this instance to save \$X/month' recommendation — because CloudWatch has zero datapoints for a stopped instance, so the average-CPU calculation naturally returns 0, which is indistinguishable from 'running but completely idle' unless you separately check the instance's actual state. The fix was threading an explicit `isRunning` boolean through every CPU-based check so it short-circuits to 'no active cost, no action' instead of running the idle-detection logic on a lie. RDS has the same bug pattern but a different fix nuance — a stopped RDS instance still accrues *storage* cost (unlike EC2), so only the compute portion gets zeroed, not the whole monthlyCost."

Recommendation Engine

What to say: "Every check is a pure function — no side effects, no I/O — that takes normalized data and returns `{severity, action, confidence, recommendation, reason}`. That makes them trivial to unit test in isolation, which is actually how several of these bugs were verified fixed: a standalone script calling the function directly with known inputs and asserting the exact output, no database or AWS mocking needed."

How severity gets decided, concretely: each resource type has a merge function that takes the worst severity across every check that ran, and generally either concatenates every flagged check's message (EC2/RDS/S3) or picks the single worst check's own specific remediation action (IAM — because IAM has real distinct actions like `ENABLE_MFA` / `ROTATE_ACCESS_KEY` / `REMOVE_ADMIN_ACCESS`, where EC2/RDS/S3 mostly just say generic `REVIEW`).

Where "Critical" comes from, if asked (a good detail — shows attention to schema design): "The Recommendation schema only has LOW/MEDIUM/HIGH severity — there's no CRITICAL level in the enum. 'Critical' on the dashboard is *derived* at read time: a HIGH-severity finding

in the SECURITY category counts as critical, a HIGH finding anywhere else counts as just 'high'. That's a deliberate choice to avoid touching every scanner's logic just to add a severity tier — the category field already existed for a different purpose (IAM findings are always SECURITY, everything else defaults to COST) and reusing it avoided a schema migration."

Dashboard

What to say: "The dashboard never talks to AWS live — everything it shows comes from the *last completed scan*'s Resource and Recommendation documents. That's a deliberate choice: it keeps every page load fast and cheap (a MongoDB aggregation instead of N AWS API round-trips), at the cost of the dashboard being only as fresh as the last scan."

The filtering rule that's easy to get wrong (great interview answer for 'tell me about a subtle bug'): "Recommendation has a status field — OPEN, RESOLVED, IGNORED. The rule is: real infrastructure spend (`monthlyCost` sums) must never be filtered by status, because resolving a tracking record in my app doesn't change what AWS is actually billing. But 'outstanding work' numbers — severity counts, savings totals, category counts — *must* filter to OPEN only, because a resolved or ignored finding isn't work that still needs doing. And historical trend data is deliberately left completely unfiltered, because a trend chart should show what was objectively true at scan time, not get retroactively rewritten by a later Resolve click. Getting this distinction wrong in either direction produces a dashboard that's subtly, confidently lying to you — either overstating outstanding risk, or making your infrastructure look cheaper than it actually is."

Authentication

What to say: "JWT-based, stateless in the normal case, with one specific mechanism for handling 'log out of everywhere': the User model has a `tokenVersion` integer, every issued JWT embeds the version it was issued with, and the `protect` middleware checks that the token's version matches the user's *current* version on every request. Bumping the version instantly invalidates every previously-issued token without needing a server-side token blacklist or Redis session store."

A real security fix worth mentioning if asked about hardening: "CORS was originally wide open — any origin could call the API. Combined with the JWT living in localStorage (readable by any JS on the page), that's a standard 'one XSS away from token theft' gap. I locked it down to an env-driven allowlist of known frontend origins. I didn't move the token out of localStorage into an httpOnly cookie in the same pass — that's a bigger architectural change (CSRF protection, cookie domain handling) that I scoped out deliberately rather than mixing into an otherwise-contained CORS fix."

AWS Integration

What to say: "AWS SDK v3, one client per service (EC2/RDS/S3/IAM/CloudWatch/Pricing/STS), built fresh per-scan from the user's stored credentials. The interesting part is the error-handling philosophy in the AWS wrapper layer: almost every call is individually try/caught with a *specific, deliberate* fallback for the 'I can't determine this' case, and the fallback is chosen to avoid a false alarm, not to avoid a missed finding. For example, if fetching a bucket's replication config fails for a permissions reason, that resolves to 'no replication configured' rather than raising an error — because we'd rather silently under-report a minor cost optimization than crash a whole bucket's scan over one signal we couldn't read."

One place that intentionally goes the other way, if you want to show you understand the exception too: " `isBucketPublic` is the one function where 'no config found' resolves to the *alarming* answer — a bucket with literally no Block Public Access configuration at all really is public by AWS's own default behavior, so returning 'public' there isn't a conservative choice, it's just factually correct."

Cost Optimization

What to say: "Pricing comes from AWS's live Price List API (`GetProducts`) wherever it's reliably queryable — EC2 and S3. RDS storage pricing is a hardcoded fallback rate instead, because RDS storage pricing splits across storage-type/engine/deployment-option combinations in a way that's much less reliably filterable through that API; querying it live wasn't worth the fragility for a secondary cost component. Every calculator has a similar documented fallback constant for when the live lookup fails or the region can't be mapped, so a resource never just silently reports \$0."

MongoDB

What to say: "Nine collections, no `.populate()` /ref-based joins anywhere in practice — `scanId` is stored as a plain indexed foreign key and every child collection is queried directly by it, because the app never needs a resource embedded inside a scan document, only 'give me everything for this scan.' The one place with real aggregation complexity is the dashboard summary, which uses a `$facet` pipeline to compute several different severity-bucket counts in a single round trip instead of four separate queries."

A gap worth naming honestly if asked about what you'd improve: "There's no compound index on (`scanId`, `userId`) on the Recommendation or Resource collections despite that being the most common query shape — everything currently relies on the single-field `userId` index. That's a reasonable target for a follow-up pass, especially as scan history grows."

React

What to say: "Every table in the app — Scan History, Recommendations, Audit Logs, the resource table on Scan Detail — follows the exact same pagination pattern: a `PAGE_SIZE` constant, a `currentPage` state, a clamped `safePage` derived value, and a shared `TablePagination` component. That consistency wasn't an accident — when I added pagination to the one table that was missing it, I copied the existing pattern exactly rather than inventing a new one, specifically so the app doesn't end up with three different pagination behaviors a user has to relearn."

Animation/UX detail worth mentioning: "Framer Motion drives page transitions and staggered list entrances, capped so a long list (50+ rows) doesn't have a multi-second cascading animation — the stagger delay is clamped to the first ~10 items, everything after that animates in together."

Node / Express

What to say: "Every controller is wrapped in a shared `asyncHandler` so a rejected promise anywhere in the request chain reaches Express's centralized error middleware instead of crashing the process or leaving a hanging request. Errors are two kinds: `ApiError` instances that carry their own intended HTTP status code (thrown deliberately, e.g. 'no AWS account connected' → 400), and everything else, which the error handler defaults to 500 — so a genuinely unexpected bug never accidentally leaks as a 200 or a misleading 4xx."

Common Interview Questions and Best Answers

"Walk me through what happens when a user clicks Run Scan." → Give the Part 10 walkthrough at whatever depth they want: click → `useScanTrigger` → `POST /api/scan` → `protect` middleware → controller → `scanInfrastructure` → four scanners (EC2 blocking, S3/RDS/IAM fault-tolerant) → Resource + Recommendation documents saved → Scan marked COMPLETED → response → frontend refetches dashboard data → charts re-render. Mention the illustrative (not real) progress-step animation on the frontend as a nice detail showing you understand the UI is decoupled from actual backend progress.

"How would you scale this to a large AWS account with hundreds of EC2 instances?" → "The scanners process resources sequentially in a for-loop, not in parallel — that's the first bottleneck I'd fix, batching instances through `Promise.all` with a concurrency cap so it doesn't overwhelm CloudWatch/Pricing API rate limits. IAM is actually the chattier bottleneck in practice — analyzing one user alone can be 8+ separate IAM API calls, so a large user base would hit IAM's comparatively low rate limits before EC2 becomes the problem."

"What would you do differently about the schema?" → "I'd add a real `CRITICAL` severity level to the Recommendation enum instead of deriving it at read time from severity+category — the current approach works but means the 'critical' concept only exists inside one aggregation function, not in the data model itself, which makes it easy to recompute inconsistently if a second place ever needs the same logic."

"How do you make sure a Resolve/Ignore action doesn't lie about actual AWS spend?" → Explain the OPEN-filtering rule from the Dashboard section above — this is one of the best "I understand data correctness, not just CRUD" answers available in this project.

"Tell me about a bug you found and fixed." → The stopped-instance-phantom-savings bug (Scanning section above) is the strongest one: a real, subtle, root-caused bug (an empty-array average silently returning 0), with a clean fix (an explicit boolean guard threaded through every affected function), verified with a standalone reproducible test — a complete, well-structured story.

"Why MongoDB instead of a relational database?" → "The core entities — a scan's resources and recommendations — are naturally document-shaped and don't need cross-collection joins in the hot path (everything's queried by `scanId` / `userId`, not joined). The `metadata` field on Resource is deliberately schema-less because each of the four resource types has a completely different shape of interesting data (CPU stats vs. engine version vs. bucket encryption vs. IAM policy attachments) — modeling that in a relational schema would mean either a sparse wide table or an EAV pattern, both worse than just letting Mongoose accept an arbitrary object there."

"What's the single riskiest piece of this codebase, security-wise?" → "The AWS secret access key is encrypted at rest before being stored, and it's stripped from every JSON serialization via a Mongoose `toJSON` transform — so even a careless `res.json(connection)` can't leak it. But the JWT itself lives in localStorage, which is a real, known trade-off: it's simple and works, but it's readable by any JS running on the page, so an XSS vulnerability anywhere in the frontend would mean session theft. Locking down CORS was the first hardening step; moving to an httpOnly cookie would be the next one if I were prioritizing security work."

PART 10



Code Execution: Run Scan Walkthrough

Function-by-function, from the click to the dashboard updating.

Part 10 — Code Execution: What Happens When a User Clicks "Run Scan"

This is a complete, function-by-function trace of the single most important user action in CloudLens, from the click to the dashboard updating with new numbers. Every file and function named here is documented in full in Part 2.

Step 0 — Where the button lives

The "Run Scan" button is **not** a single global component instance — `Navbar.jsx` is rendered separately inside *each* top-level page (`Dashboard.jsx`, `ScanHistory.jsx`, etc.), and each page wires its own `useScanTrigger(onSuccess)` hook to it, passing a page-specific `onSuccess` callback (e.g. Dashboard's is `refetch`, which re-runs `fetchDashboardBundle`). This is why running a scan from the Dashboard refreshes the Dashboard's own data specifically — the trigger hook and the refresh callback are colocated per-page, not a single app-wide event.

`Navbar` itself doesn't know how to run a scan — it just renders a button (`runScanButton`) that calls whatever `onScan` prop it was given, and shows a spinning-bolt icon + "Scanning..." label while `scanning` is true. It also renders `<ScanProgressModal isOpen={scanning} stepIndex={stepIndex} phase={scanPhase} />`, an overlay that shows the illustrative step list from `constants/scanSteps.js` (Fetch EC2 → Fetch S3 → Fetch Pricing → Analyze → Save Database).

Step 1 — Click

```
// Dashboard.jsx
const { scanning, scanError, runScan, stepIndex, scanPhase } = useScanTrigger(refetch);
...
<Navbar onScan={runScan} scanning={scanning} stepIndex={stepIndex} scanPhase={scanPhase} />
```

The user clicks the button → `onClick={onScan}` fires → `runScan()` (from `hooks/useScanTrigger.js`) executes.

Step 2 — `useScanTrigger`'s `runScan()` (client/src/hooks/useScanTrigger.js)

1. `setScanning(true)`, clear any previous error, `setScanPhase("running")`, `setStepIndex(0)`.
2. `showInfoToast("Infrastructure scan started.")` — an immediate toast, before any network call resolves.
3. Starts a `setInterval` (every 900ms) that auto-advances `stepIndex` — **purely cosmetic**, driving `ScanProgressModal`'s illustrative step list. The backend has no incremental-progress API; this interval stops one step short of the last step (`MAX_AUTO_STEP = SCAN_STEPS.length - 2`) so the UI never claims "Save Database" is done before the real request actually resolves.
4. `await scanInfrastructure()` — the real network call (see Step 3).
5. On success: clear the interval, jump `stepIndex` to the final step, `setScanPhase("success")`, call the page's `onSuccess` callback (`await onSuccess()` — for Dashboard, this is `refetch`), show a success toast, hold for 800ms (`COMPLETE_HOLD_MS`) so the "success" state is visible before the modal closes.
6. On failure: clear the interval, extract `error.response?.data?.message` (falls back to a generic message), `setScanError`, `setScanPhase("error")`, show an error toast, hold for 1500ms.
7. `finally`: `setScanning(false)`, `setScanPhase("idle")` — this is what closes `ScanProgressModal` (`isOpen={scanning}`).

Step 3 — `scanInfrastructure()` (client/src/services/dashboard.service.js)

```
export const scanInfrastructure = async () => {
  const response = await api.post("/scan");
  return response.data;
};
```

One line: `POST /api/scan` (no request body needed — the backend identifies the user from the JWT, not from anything in the request). Uses the shared `api.js` Axios instance, whose request interceptor has already attached `Authorization: Bearer <token>` from `localStorage`.

Step 4 — Express receives the request

`server/src/app.js` → `app.use("/api/scan", protect, scanRoutes)` → `protect` middleware (`middleware/auth.middleware.js`) verifies the JWT signature, checks the embedded `tokenVersion` against the current `User.tokenVersion` (rejects if they've mismatched, e.g. after a "log out everywhere"), and attaches `req.user = { userId, ... }`. `server/src/routes/scan.routes.js` matches `POST /` (mounted at `/api/scan`) to the `scan` controller.

Step 5 — `scan` controller (`server/src/controllers/scan.controller.js`)

```
const scan = asyncHandler(async (req, res) => {
  await recordAuditLog(req.user.userId, AUDIT_ACTIONS.SCAN_STARTED, "Infrastructure scan started");
  const result = await scanInfrastructure(req.user.userId);
  await recordAuditLog(req.user.userId, AUDIT_ACTIONS.SCAN_COMPLETED, `Scan completed - ${result.totalResources} resources scanned, $${result.estimatedSavings}`);
  sendSuccess(res, result);
});
```

1. Writes an `AuditLog` entry (`SCAN_STARTED`) **before** the scan even begins.
2. Calls `scanInfrastructure(req.user.userId)` — the real orchestrator (see Step 6). This is the one line that does all the actual work; everything else in this controller is bookkeeping.
3. Writes a second `AuditLog` entry (`SCAN_COMPLETED`) with the final resource/savings counts, **only reached if `scanInfrastructure` didn't throw**.
4. `sendSuccess(res, result)` — the shared response-shaping utility, sends `{ success: true, data: { scanId, totalResources, estimatedSavings } }` with a 200 status.

If `scanInfrastructure` throws (see Part 2's note on EC2 being the one non-fault-tolerant scanner), `asyncHandler` catches it, the `SCAN_COMPLETED` audit log is **never written** (only `SCAN_STARTED` exists for that attempt), and the error propagates to `errorHandler.middleware.js`, which sends an error JSON response — which is exactly what `useScanTrigger`'s catch block reads `error.response?.data?.message` from.

Step 6 — `scanInfrastructure(userId)` (`server/src/services/scanner/scanner.service.js`)

The real orchestrator — full detail in Part 2, summarized here:

1. Looks up the user's `AwsConnection`. None found → throws `ApiError(400, "Please connect an AWS account before running a scan.")` immediately (the Navbar actually prevents this case entirely by showing a "Connect AWS" button instead of "Run Scan" when there's no connection — see `Navbar.jsx`'s `connection ? runScanButton(...) : connectAwsButton(...)`).
2. `createScan(userId)` — a new `Scan` document, `status: "RUNNING"`.
3. `getAwsClientsForUser(userId)` (`factories/awsClient.factory.js`) — re-fetches the `AwsConnection`, decrypts the stored `encryptedSecretKey` (via `utils/encryption.js`), and builds 6 authenticated AWS SDK v3 clients (EC2, CloudWatch, Pricing [always `us-east-1`], S3, RDS, IAM).
4. `await scanEC2(scan._id, userId, {...})` — runs to completion, unguarded. Internally: fetches every instance + all EBS volumes + all Elastic IPs in parallel, then for each instance sequentially fetches CPU/network/disk/pricing, runs 7 independent recommendation checks (base CPU, network, disk, age, security groups, EBS, Elastic IP), merges them into one `Recommendation`, saves both `Resource` and `Recommendation`, and returns totals.
5. `scanS3`, `scanRDS`, `scanIAM` — each individually wrapped in try/catch; a permission gap in any one of these degrades to `{totalResources: 0, estimatedSavings: 0}` for that service rather than failing the whole scan. Each follows the same fetch → analyze → recommend → merge → save shape as EC2, with S3 additionally generating fleet-wide `Insight` documents after all buckets are processed.
6. Sums `totalResources` and `estimatedSavings` across all four.
7. `completeScan(scan._id, totalResources, estimatedSavings)` — `Scan.status` → `"COMPLETED"`, sets `completedAt` and the rolled-up totals.
8. Returns `{ scanId, totalResources, estimatedSavings }` up to the controller.

Database writes that happened in this one scan, concretely, for an account with 3 EC2 instances, 2 S3 buckets, 1 RDS instance, 5 IAM users, 2 IAM roles:

- 1 `Scan` document (created, then updated)
- $3 + 2 + 1 + 5 + 2 + 1$ (root account) = 14 `Resource` documents
- $3 + 2 + 1 + 5 + 2 + 1 = 14$ `Recommendation` documents

- Some number of `Insight` documents (from S3's fleet-wide analysis, e.g. "1 of 2 buckets missing lifecycle rules")
- 2 `AuditLog` documents (`SCAN_STARTED` , `SCAN_COMPLETED`)

Additionally, if the user has a `Notification` document with `scanCompleted: true` (or `highSeverity: true` and at least one HIGH-severity recommendation was found), `scanNotification.service.js` sends an email via `email.service.js` /Nodemailer — this happens as a side effect of the scan (exact trigger point documented in Part 2's cross-cutting-services section, populated by the background research agents).

Step 7 — Response flows back

`{ success: true, data: { scanId, totalResources, estimatedSavings } }` → Axios resolves in `scanInfrastructure()` (frontend) → resolves the `await scanInfrastructure()` in `useScanTrigger`'s `runScan()` → success branch runs (Step 2.5 above) → `await onSuccess()`, i.e. Dashboard's `refetch`.

Step 8 — Dashboard refetches (client/src/pages/Dashboard.jsx)

`refetch` (from `useAsyncData`) re-runs `fetchDashboardBundle`:

```
const fetchDashboardBundle = async () => {
  const [dashboard, recommendations, scans] = await Promise.all([
    getDashboard(),           // GET /api/dashboard
    getRecommendations(),    // GET /api/recommendations
    getScans(),               // GET /api/scans
  ]);
  return { dashboard, recommendations, lastScan: scans[0] || null };
};
```

Three parallel GETs. `GET /api/dashboard` is answered by `dashboard.controller.js` → `dashboard.service.js` + `serviceAggregation.service.js`, which read **only the just-created Scan** (it's now the latest one, `Scan.findOne({userId}).sort({createdAt: -1})`) and that scan's fresh `Resource` / `Recommendation` documents, run the MongoDB `$facet` aggregation (severity/category counts, OPEN-status-filtered) plus the plain-JS service/severity/cost-breakdown/savings-opportunities/AI-insights computation, and return the assembled dashboard payload.

Step 9 — Re-render

`useAsyncData` updates `data` state → `Dashboard.jsx` re-renders → every chart/card component (`SummaryCard`, `ResourceHealthPieChart`, `CostBreakdownDonutChart`, `ResourceDistributionBarChart`, `SavingsOpportunityBarChart`, `ServiceOverviewGrid`, `AIInsightsCard`, `RecommendationTable`) receives the new props and re-renders with the fresh numbers, each with its own Framer Motion entrance animation. The whole round trip — click to updated dashboard — is bounded by however long the real AWS scan itself took (typically a few seconds for a small account, dominated by IAM if the account has many users, since IAM makes the most AWS API calls per resource of the four services).

Failure path, concretely

If the connected AWS credentials lack `ec2:DescribeInstances` permission: `scanEC2` throws inside `getEC2Instances` → propagates up through `scanInfrastructure` (uncaught) → the `Scan` document is left at `status: "RUNNING"` forever (nothing ever marks it `FAILED`) → the controller's `await scanInfrastructure(...)` rejects → `asyncHandler` forwards to `errorHandler.middleware.js` → an error JSON response goes back → `useScanTrigger`'s catch block fires, showing an error toast with the AWS SDK's error message, and the Dashboard's data is left exactly as it was before the click (no partial update). This is the one documented weak point in the scan pipeline's error handling (see Part 2's note on `scanner.service.js`).

PART 11



Folder Structure

Every folder, what it holds, and why the codebase is organized this way.

Part 11 — Folder Structure

Complete Tree

```
CloudLens/
├── server/                                Express + MongoDB backend
│   ├── src/
│   │   ├── app.js                        Express app assembly: CORS, JSON body parsing, route
│   │   │                               mounting, error handlers. Exports the app; does NOT listen.
│   │   ├── server.js                    Process entry point: connects Mongo, starts the HTTP
│   │   │                               listener, starts the scheduler cron job.
│   │   ├── config/
│   │   │   ├── aws.js                  Environment-driven setup, built once at require-time
│   │   │   │                               Builds AWS SDK v3 client factories (EC2/CloudWatch/
│   │   │   │                               Pricing/S3/RDS/IAM/STS), incl. the Pricing API's
│   │   │   │                               fixed us-east-1 region quirk.
│   │   │   ├── db.js                   MongoDB connection setup (MONGO_URI).
│   │   │   └── mailer.js               Nodemailer transport setup (SMTP or Ethereum fallback).
│   │   ├── constants/
│   │   │   ├── auditActions.js         Shared enum-like value lists, single source of truth
│   │   │   │                               The full AUDIT_ACTION_VALUES enum for Auditlog.action.
│   │   │   └── regions.js              The 4 supported AWS regions (also duplicated in the
│   │   │                               frontend's ConnectAws.jsx – a known, tracked
│   │   │                               duplication, not accidental).
│   │   ├── routes/
│   │   │   (12 files)                  Express Router definitions – path + middleware +
│   │   │                               controller only, zero business logic. One file per
│   │   │                               resource (auth, scan, scans, dashboard, recommendation,
│   │   │                               aws, schedule, notification, analytics, report,
│   │   │                               auditlog, health).
│   │   ├── controllers/
│   │   │   (12 files)                  Thin HTTP-layer glue: parse req → call one service
│   │   │                               function → shape res.json(). One per route file above.
│   │   ├── middleware/
│   │   │   (singular – auth + rate limiting)
│   │   │   ├── auth.middleware.js       `protect` – verifies JWT + tokenVersion, attaches
│   │   │   │                               req.user.
│   │   │   └── rateLimiter.middleware.js
│   │   ├── middlewares/
│   │   │   (plural – a real, tracked naming inconsistency in
│   │   │   ├── errorHandler.middleware.js this codebase; both folders coexist deliberately
│   │   │   └── notFound.middleware.js   left as-is, not a typo you should "fix")
│   │   ├── models/
│   │   │   (9 files)                  9 Mongoose schemas – see Part 8 for full documentation
│   │   │                               of every field/index/relationship.
│   │   ├── services/
│   │   │   The bulk of the backend's logic, organized by concern:
│   │   │   ├── aws/
│   │   │   │   Thin AWS SDK v3 wrappers, one file per AWS service
│   │   │   │   (7 files)
│   │   │   │   (ec2, rds, s3, iam, cloudwatch, pricing, sts).
│   │   │   │   NEVER contains analysis/recommendation logic.
│   │   │   ├── scanner/
│   │   │   │   The 4 resource-type orchestrators (ec2/rds/s3/iam
│   │   │   │   (6 files)
│   │   │   │   .scanner.js) + scanner.service.js (top-level entry
│   │   │   │   point) + ebs.scanner.js (EMPTY – dead code stub).
│   │   │   ├── analyzer/
│   │   │   │   Pure functions: raw AWS data → normalized findings
│   │   │   │   (5 files)
│   │   │   │   (cpu/ec2/rds/s3/iam.analyzer.js).
│   │   │   ├── recommender/
│   │   │   │   Pure functions: normalized findings → severity/
│   │   │   │   (4 files)
│   │   │   │   action/recommendation objects, plus each resource
│   │   │   │   type's merge-into-one-Recommendation function.
│   │   │   ├── database/
│   │   │   │   Repositories – the ONLY layer allowed to import
│   │   │   │   (10 files)
│   │   │   │   Mongoose models directly. One per collection
│   │   │   │   (+ dashboard.repository.js and
│   │   │   │   analytics.repository.js, which aggregate ACROSS
│   │   │   │   collections).
│   │   │   ├── dashboard/
│   │   │   │   dashboard.service.js (assembles the full dashboard
│   │   │   │   (2 files)
│   │   │   │   response) + serviceAggregation.service.js (the
│   │   │   │   generic, service-agnostic breakdown/insights logic).
│   │   │   ├── notification/
│   │   │   │   Email-sending orchestration: which emails to send,
│   │   │   │   (3 files)
│   │   │   │   when, respecting per-user Notification preferences.
```

- └─ email/
 - (2 files)
 Nodemailer sending + HTML templates.
- └─ report/
 - (2 files)
 PDF report generation (pdfkit) for a single scan.
- └─ scheduler/
 - (1 file)
 node-cron 60-second tick + in-flight-schedule de-duplication (scheduler.service.js).
- └─ schedule/
 - (1 file)
 CRUD for a user's Schedule document (schedule.service.js – distinct from scheduler/, which is the cron runner).
- └─ scan/
 - (1 file)
 Scan history listing/formatting (scanHistory.service.js).
- └─ recommendation/
 - (1 file)
 Resolve/Ignore/Stop-Instance business logic (recommendation.service.js).
- └─ insight/
 - (1 file)
 Fleet-wide, rule-based S3 insight generation (s3.insight.js – the only insight generator today).
- └─ awsConnection/
 - (1 file)
 Connect/disconnect AWS account business logic, credential encryption orchestration.
- └─ auth/
 - (1 file)
 Signup/login/logout-all business logic (bcrypt hashing, JWT issuance).
- └─ auditLog/
 - (1 file)
 Write-side helper for creating AuditLog entries.
- └─ analytics/
 - (1 file)
 Historical trend computation for the Analytics page.
- └─ mappers/
 - (4 files)
 Normalize AWS's raw, deeply-nested response shapes (ec2/iam/rds/s3.mapper.js) into flat objects – sits between services/aws/ and services/analyzer/.
- └─ builders/
 - (1 file)
 Builds AWS Price List API filter objects from a resource's attributes (pricing.filter.builder.js).
- └─ parser/
 - (1 file)
 Extracts a usable hourly price number out of the Price List API's verbose nested JSON response (pricing.parser.js).
- └─ factories/
 - (1 file)
 Builds a full set of authenticated AWS SDK clients for one user from their stored, decrypted credentials (awsClient.factory.js) – the bridge between AwsConnection documents and services/aws/.
- └─ utils/
 - (10 files)
 Cross-cutting helpers with no service-specific home: ApiError, apiResponse, asyncHandler, encryption (AES for the stored AWS secret key), iamPolicy.util, instanceType.util (adjacent-instance-type lookup for UPSIZE/DOWNSIZE), jwt, region.util, schedule.util, validators.
- └─ package.json
- └─ .env
 - Not committed – MONGO_URI, JWT_SECRET, ENCRYPTION_KEY, CORS_ORIGIN, SMTP_*, etc.
- └─ client/
 - React (Vite) frontend
 - └─ src/
 - └─ main.jsx
 - React root – mounts <App /> into #root.
 - └─ App.jsx
 - react-router-dom route table; wraps every authenticated route in <ProtectedRoute>.
 - └─ pages/
 - (11 files)
 One component per route – owns page-level state and data-fetching, composes components/ into a layout. Login, Signup, Dashboard, ScanHistory, ScanDetail, Recommendations, Analytics, AuditLogs, ConnectAws, ScheduleScan, NotificationSettings.
 - └─ components/
 - (39 files)
 39 reusable UI pieces, roughly three tiers: generic primitives (Badge, Card, IconButton, LinkButton, ActionButton, ToggleSwitch, CopyButton, SortIcon, SortableTh, TablePagination, TableSkeleton), chart wrappers (ChartCard, ChartSkeleton,

	ChartEmptyState, ChartTooltip + 5 actual chart components), and domain-specific composites (RecommendationTable, ResourceTable, ResourceDetailsDrawer, ScanProgressModal, ServiceOverviewGrid, AIInsightsCard, Navbar).
├ hooks/ (6 files)	Reusable stateful logic extracted from pages: useAsyncData (generic fetch-on-mount), useAwsConnection, useRecommendationActions, useScanTrigger, useTheme, useClickOutside.
├ services/ (10 files)	One file per backend resource, wraps Axios calls using the shared api.js instance – the frontend's mirror of the backend's routes/ list.
├ api/ └ api.js	The single shared Axios instance: base URL, JWT request interceptor, 401 response handling.
├ utils/ (6 files)	Presentation-only helpers: format.js (currency/date/percent/bytes formatting), badge.js (severity/status/category → Tailwind class maps), chartColors.js, sort.js, awsConsole.js (deep-link URL builders), toast.jsx.
└ constants/ └ scanSteps.js	The step list shown in ScanProgressModal.
├ package.json	
└ vite.config.js	
└ docs/	Generated developer documentation output (this guide and its accompanying diagrams).

Why This Structure

The backend's `services/` folder is deliberately **not** organized by resource type at the top level (there's no single `ec2/` folder containing everything EC2-related) — it's organized by **concern** (`scanner/`, `analyzer/`, `recommender/`, `aws/`), with each concern folder then containing one file per resource type. This means adding a fifth scanned service (say, Lambda) means adding one new file to `scanner/`, `analyzer/`, `recommender/`, `aws/`, and `mappers/` each — a consistent, predictable shape — rather than creating a new top-level folder and having to decide its internal organization from scratch.

The strict `routes → controllers → services → database repositories → models` layering (no layer skips the one below it) is what makes the status-filtering rules documented in Part 8 enforceable in exactly one place: because controllers never touch Mongoose directly, a rule like "savings totals must filter to OPEN status" only has to be correct inside the relevant repository/service function, not re-verified at every call site.

Folder Structure — Visual

